Handwriting Robot Code Theory Manual

Report methods and the current demo pipeline

D. R. Braune u23539195
University of Pretoria

Revised study edition November 2026

Purpose | What problem does this solve

Blue labels explain the role in the robot.

Method | How does it work

Indigo labels unpack mathematics and implementation.

Justification | Why use this approach

Green labels state the rationale and assumptions.

Alternatives | What else could have been used

Purple labels compare relevant methods.

Library | What could a library change

Amber labels distinguish implementation, cost and equivalence.

Evidence | Where does the claim come from

Burgundy labels identify sources, records and limits.

The report's named methods and all 35 numbered equations take priority. Current signatures and source hashes identify the implementation being explained.

Contents

1 Report-to-code reading map 2

2 The current demo pipeline 3

3 Paths and datasets 6

4 Camera acquisition 8

5 Image processing and line segmentation 14

6 Text recognition and writer identification 95

7 Glyph extraction and writer profiles 145

8 Glyph curation 181

9 Handwriting synthesis and candidate selection 191

10 Direct page retracing 220

11 Gantry mapping and control 225

12 Web orchestration and evaluation 266

13 Independent line workers 299

14 Inspection images and glyph sheets 305

15 Training and numerical differentiation 309

16 Timing and allowance accounting 413

17 Expanded measurement, verification and coefficient evidence 417

18 Glossary 428

19 Function index and source provenance 433

1 Report-to-code reading map

The report is the priority map. Its explanations are placed beside the relevant code stages, rather than collected in a detached report chapter. All 35 numbered equations have a home below. Each module opens with its input-to-output flow; its detailed stages group dependent calculations and their supporting functions. Training remains outside the live path.

Evidence | Reading the evidence

The reference report is the unsuffixed `23539195_BrauneDR_FINALREPORT.pdf` in Downloads (108 PDF pages). Published sources establish methods; captured code establishes implementation; worked arithmetic is a derivation. Trade-off explanations are not invented historical decision records. Unsupported coefficient choices and missing sources are identified explicitly.

Report equation	Explanation in code context
1	Belt travel and pitch radius (reproduce.py, page 230)
2	Linear speed to motor speed (reproduce.py, page 230)
3	Torque curve interpolation (reproduce.py, page 231)
4	Steady opposing load (reproduce.py, page 231)
5	Command lattice and rounding (reproduce.py, page 231)
6	Driver current limit (reproduce.py, page 231)
7	Minimum base resistance (reproduce.py, page 247)
8	Base current and saturation check (reproduce.py, page 247)
9	Switch pull down current (reproduce.py, page 247)
10	Measured axis scale (reproduce.py, page 241)
11	Bresenham minor step count (reproduce.py, page 252)
12	Integral rectangle sum (segment.py, page 18)
13	Background ratio and ink mask (segment.py, page 30)
14	Otsu threshold criterion (segment.py, page 30)
15	Deskew concentration (segment.py, page 61)
16	Local seam cost (segment.py, page 68)
17	Seam recurrence (segment.py, page 68)
18	Fixed canvas scale (recognise.py, page 100)
19	Convolution response (recognise.py, page 106)
20	Batch normalisation (recognise.py, page 114)
21	LSTM cell and hidden state (recognise.py, page 128)
22	CTC sequence loss (recognise.py, page 134)
23	CTC occupancy and derivative (train.py, page 381)
24	Controlled stroke width (recognise.py, page 104)
25	Forced transcript alignment (style.py, page 156)
26	Character cut cost (style.py, page 157)
27	Stored glyph coordinates (style.py, page 157)
28	Measured connectedness (style.py, page 157)
29	New glyph placement (synthesise.py, page 194)
30	Automatic anchoring strength (synthesise.py, page 194)
31	Candidate objective (synthesise.py, page 195)
32	Machine placement (reproduce.py, page 232)
33	Nominal timing budget (reproduce.py, page 255)
34	Pooled character recovery (recognise.py, page 141)
35	Geometric agreement and natural baseline (verification, page 417)

2 The current demo pipeline

Purpose | Which implementation this edition describes

The documentation snapshot is `Software/DemoCode`, captured during this revision. `START.sh` changes into that directory, sets `OPENBLAS_NUM_THREADS=4` and `OPENBLAS_CORETYPE=ARMV8`, sets `READER_WEIGHTS=paper_cnn_bilstm_ctc_joint_best.npz`, then starts `web/server.py`. The variable named CP3 in that server resolves to its own parent, DemoCode; its name does not redirect imports to CleanPipeline3. Source hashes and exact signatures are supplied with this edition. No robot module was imported or executed for this work.

Evidence | Current source versus a running board

These are facts about the captured local source, not confirmation of the ODROID's environment, files, loaded weights or successful requests. Two files were already uncommitted: the server and new lineworkers helper. Those contents are included in the documentation snapshot and left untouched. If another agent subsequently changes the tree, the hash manifest identifies exactly which version was documented. No training, inference, hardware run, deployment, commit or report edit is part of this revision.

2.1 Read a photographed page

Method | Acquisition orientation segmentation reading and naming

OpenCV obtains camera frames. Pillow opens image files and applies EXIF orientation; a configured quarter turn is a camera mounting hint. The segmenter's axis score compares empty-row evidence in two orientations. `SegmentLines(page, mergeValley=None)` then estimates illumination, deskews, filters structures/components, finds line centres, optimises seams, refines ownership and builds full-resolution crops. Its current signature does not take a timer; the server times this complete call as one stage.

The reader probes up to three sampled lines in both directions to resolve 180-degree inversion. The comparison uses mean best-frame log score, not a transcript likelihood. Chosen probe outputs are reused; remaining lines are eligible for lineworker mapping. If inversion wins, crop rotation and line order are changed together. Metadata is reordered to match the returned text.

Each surviving crop is named with the configured writer model. A weighted arithmetic average of class vectors supplies the page decision, with weight equal to the crop's count of pixels below 128. More dark pixels are treated as more evidence; the procedure is not a Bayesian multiplication of independent observations. Supplied expected text/author are used for reported evaluation, not as substitutes for the network prediction.

2.2 Models and defaults that changed

Evidence | There is one writer checkpoint in this tree

The module default reader is `paper_cnn_bilstm_ctc_scratch_best.npz`; START.sh explicitly overrides it with the joint checkpoint. Calling the module directly without that override can therefore load a different reader. The writer default is `author_shape_11w_pw170_scratch_weights.npz`. `INK_MODEL`

is loaded once and `SHAPE_MODEL=INK_MODEL`: the two server names reference the same object, not independent ink/shape checkpoints. The stale log wording does not change this assignment.

The page namer uses that shared object with stroke normalisation disabled. Candidate judging calls `ClassifyImage` with its default stroke normalisation enabled. Thus preparation differs even when weights are shared. Class mappings come from the checkpoint. Thirteen configured writer names do not prove thirteen output classes or thirteen usable profiles. Actual loaded class metadata, source populations and historical eleven-writer tests are separate evidence.

Method | Capital Forward is the current method name

Both reader and writer classes now expose `Forward`. The function references in this edition use the current case. The mathematical forward operations can remain unchanged while a caller using the old method spelling is incompatible.

2.3 Generate a new handwriting job

Purpose | Separate intent from interface compatibility

The intended endpoint flow validates text/writer, creates a GantryConfig, chooses physical x-height before wrapping, runs joint candidate synthesis, renders a preview, writes G-code and saves timing/placement metadata. The server's default candidate count is fourteen (overridable through WEBAPP_TRIES); the direct function default is six. Neither source comment's claimed tuning gain is adopted here as a new measured result.

Current synthesis always uses the uniform raster renderer and style-calibration path. It offers the best five prior-ranked variants, prefers a stored medoid anchor, suppresses joins below 0.45 and chooses cursive fallback style at 0.5. Retired controls such as JOIN_PROB_CAP, IDEAL_SOURCE, USE_STYLE_CALIBRATION and tangent-ligature switches do not exist in this tree. They are not live knobs merely because an older guide mentioned them.

Evidence | Static integration discrepancies in the captured source

The current `ApiGenerate` still accesses `cfg.ignoreRuling` and passes `ignoreRuling` into dataclass replacement, but current GantryConfig defines no such field. It also calls RenderTrajectory with `profile` and `uniformInk`, which the current renderer does not accept. `retrace.PageStrokes` passes `timer` to SegmentLines, which does not accept it. These are visible caller/callee contract mismatches, not newly executed failures. This manual explains the intended algorithms and actual local definitions without asserting that these endpoints complete successfully as captured. No compatibility fix is applied.

2.4 Place and execute motion

Method | The path and the machine have different responsibilities

Current ToMachine uniformly shrinks to bounds, places the top at origin minus margin/line offset, and removes row-centre wander while retaining scaled synthesis pitch. It has no ignoreRuling/fitToRuling branch. The configuration does not define accelMmS2. The controller's commanded step lattice is nominally 80 steps/mm; homing derives per-axis factors from measured switch travel.

WriteGcode can add connectors through endpoint joining and ring closure, so its drawn path is not always identical to the raw synthesis preview. The runner implements a limited modal dialect, converts linear targets to calibrated steps and pulses major/minor axes sequentially. A pen-confirmation failure is printed but the current runner continues. Command counters and parser coordinates are

not independent carriage measurements. A successful return is not a proof of physical contact force or exact endpoint arrival.

2.5 Independent line processes and numerical threads

Purpose | Parallelise independent work without confusing it with a new model

The new `web/lineworkers.py` stores configured reader/namer objects in parent state. When os.fork exists and enough jobs remain, Map creates an explicit fork pool, preserving returned job order. LINE_WORKERS=1 disables the pool; zero requests up to the CPU count, bounded by job count. The minimum is four jobs per Map call, not necessarily four original page lines after orientation probes have already been read.

Justification | Parallel speed depends on overhead and oversubscription

If N independent passes cost t each, an ideal pool with P equal workers would take about $\lceil N/P \rceil t$ plus setup and communication. For a serial fraction s, idealised Amdahl speedup is $1/[s + (1 - s)/P]$. These are explanatory bounds under simplified assumptions, not measured ODROID gains. Memory bandwidth, heterogeneous cores, crop transfer, pool startup and native numerical threads alter the outcome.

Workers attempt to cap OpenBLAS threads to one using environment settings and selected shared-library entry points. It is best effort, not a portable proof that every backend is capped. Fork provides copy-on-write address spaces, not zero allocation or zero IPC. The explicit fork context is distinct from Python's platform/version default. Forking a threaded service with native state requires deployment care. Exceptions in pool work fall back to serial execution of the same function; errors can still propagate if the serial function also fails [1, 2].

2.6 Dependencies and fair library comparisons

Library | First principles does not mean no libraries are present

The numerical core uses NumPy arrays and compiled linear algebra. Flask/CORS serve requests, Pillow decodes/encodes and renders UI artefacts, OpenCV acquires camera frames, gpiod accesses GPIO, and Python multiprocessing manages workers. Training also contains explicit PNG/JPEG decoders. This boundary is more precise than the old manual's blanket no-library statement.

Compare an operation with fixed inputs, parameters, dtype, padding, axes, normalisation and reduction before comparing independently trained accuracy. A library may offer compiled/fused kernels or GPU execution, while the custom code exposes intermediates and design rules. It is not defensible to attach an unmeasured universal speed multiplier or accuracy gain to the word library. Library APIs documented here were consulted for semantics; their versions are not evidence of packages installed on the ODROID.

3 Paths and datasets

Evidence | Current implementation boundary

`Software/DemoCode/paths.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Resolve datasets, personal writer pages and checkpoint locations before loading images or networks. Paths establish provenance; they do not establish which model is deployed.

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
Data = Path(__file__).resolve().parent.parent / 'CNN'
Weights = Data / 'weights'
Profiles = Data / 'NOGIT' / 'StyleProfiles10'
PersonalPages = Data / 'PersonalDataset'
Cache = Data / 'NOGIT'
DatabaseWriters = ['150', '151', '152', '153', '384', '551', '552', '588']
PersonalWriters = ['abhinav', 'dylan', 'owen', 'thiya', 'yeukita']
AllWriters = DatabaseWriters + PersonalWriters
ValFraction = 0.15
SplitSeed = 0
```

Stage: Checkpoint and log persistence

3.1 Method background: Checkpoint and log persistence

Purpose | The role in this system

Preserve weights, class mappings and evidence about a training run.

Justification | The reason for this construction

Named arrays and metadata make inference state inspectable and support interrupted work. A best checkpoint is chosen by a stated criterion rather than final epoch alone.

Alternatives | What changes if another method is used

Pickled framework checkpoints carry richer objects but add trust/version concerns. Saving only weights is simpler but cannot reproduce optimiser continuation by itself.

Library | Implementation speed and accuracy

NumPy savez and framework state_dict formats are storage mechanisms, not evidence of the host, exact run or accuracy. Reloading files does not automatically replace already-instantiated server objects. [3, 4].

3.2 PersonalWriterPages

```
def PersonalWriterPages(writer)
```

Evidence | Source and method link

`Software/DemoCode/paths.py` line 22. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 3.1.

Purpose | What this contributes

Preserve weights, class mappings and evidence about a training run.

Inputs

`writer`
: the lower case name of one photographed writer, for example "owen"

Output. A generator yielding `(imagePath, labelPath)` for each of that writer's pages, in sorted filename order.

Purpose | What it is for

Finds one writer's pages and pairs each with its transcript. Every stage that needs real handwriting with known text starts here: training, profile building, and the measurement of segmentation accuracy.

Justification | Why the order matters

Several measurement scripts pair the n-th detected line crop on a page with the n-th line of that page's transcript. If the pages were returned in whatever order the file system happened to supply, that pairing would still appear to work but would sometimes be comparing a line against the wrong label, and the resulting accuracy would be wrong in a way that no error message would reveal. Sorting makes the order a property of the filenames rather than of the file system.

Justification | Why a generator

A generator produces its values one at a time as they are asked for, rather than building the whole list first. Here that means a caller can begin processing the first page immediately, and can stop after one page without the cost of listing them all.

4 Camera acquisition

Evidence | Current implementation boundary

`Software/DemoCode/web/camera.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Acquire a still image or preview frame under a shared camera lock, then pass captured pixels to the page-processing stage. Preview lifecycle and still capture share the device.

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
SCRIPT_DIR = Path(__file__).resolve().parent
CAPTURE_DIR = paths.Cache / 'CameraCaptures'
CAMERA_INDEX = int(os.environ.get('CAMERA_INDEX', '1'))
WARMUP_FRAMES = 5
```

Stage: Camera capture and image orientation

4.1 Method background: Camera capture and image orientation

Purpose | The role in this system

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter.

Justification | The reason for this construction

An existing preview handle avoids competing camera opens; warm-up frames allow acquisition to settle. EXIF transpose applies metadata to pixels rather than assuming the viewer appearance equals array orientation.

Alternatives | What changes if another method is used

Direct V4L2 access offers lower-level controls; a CSI camera uses a different acquisition stack. Reading one immediate frame may capture an unsettled exposure/focus state.

Library | Implementation speed and accuracy

OpenCV VideoCapture and Pillow ImageOps.exif_transpose are already used. Their presence concerns acquisition/decoding and orientation, not a claim that OpenCV implements the custom segmentation. Sensor claims need actual negotiated settings. [5, 6].

4.2 _GetLock

```
def _GetLock()
```

Evidence | Source and method link

`Software/DemoCode/web/camera.py` line 39. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 4.1.

Purpose | What this contributes

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter.

Method | Current implementation

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _GetLock():
    global _previewLock
    if _previewLock is None:
        import threading
        _previewLock = threading.Lock()
    return _previewLock
```

Justification | Interpret the method rather than infer a benchmark

An existing preview handle avoids competing camera opens; warm-up frames allow acquisition to settle. EXIF transpose applies metadata to pixels rather than assuming the viewer appearance equals array orientation. [5, 6].

4.3 PreviewStart

```
def PreviewStart(index=None)
```

Evidence | Source and method link

`Software/DemoCode/web/camera.py` line 52. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 4.1.

Purpose | What this contributes

Opens (and keeps open) the camera for live preview streaming. Safe to call repeatedly – a no-op if already open. Kept separate from CaptureImage() so a short-lived capture doesn't leave the sensor powered on between uses – the camera only runs while someone's actually watching the live feed to frame the shot and let autofocus settle.

Method | Current implementation

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def PreviewStart(index=None):
    import cv2
    global _previewCap
    with _GetLock():
        if _previewCap is not None:
            return
        idx = CAMERA_INDEX if index is None else index
        cap = cv2.VideoCapture(idx)
        if not cap.isOpened():
```

```
            cap.release()
            raise RuntimeError(f"Could not open camera at index {idx} for preview.")
        _previewCap = cap
```

Justification | Interpret the method rather than infer a benchmark

An existing preview handle avoids competing camera opens; warm-up frames allow acquisition to settle. EXIF transpose applies metadata to pixels rather than assuming the viewer appearance equals array orientation. [5, 6].

4.4 PreviewStop

```
def PreviewStop()
```

Evidence | Source and method link

`Software/DemoCode/web/camera.py` line 68. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 4.1.

Purpose | What this contributes

Releases the preview camera handle. Always call this once you're done framing the shot – see the module comment's note on heat.

Method | Current implementation

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def PreviewStop():
    global _previewCap
    with _GetLock():
        if _previewCap is not None:
            _previewCap.release()
            _previewCap = None
```

Justification | Interpret the method rather than infer a benchmark

An existing preview handle avoids competing camera opens; warm-up frames allow acquisition to settle. EXIF transpose applies metadata to pixels rather than assuming the viewer appearance equals array orientation. [5, 6].

4.5 PreviewActive

```
def PreviewActive()
```

Evidence | Source and method link

`Software/DemoCode/web/camera.py` line 76. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 4.1.

Purpose | What this contributes

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter.

Method | Current implementation

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def PreviewActive():
    return _previewCap is not None
```

Justification | Interpret the method rather than infer a benchmark

An existing preview handle avoids competing camera opens; warm-up frames allow acquisition to settle. EXIF transpose applies metadata to pixels rather than assuming the viewer appearance equals array orientation. [5, 6].

4.6 PreviewFrameJpeg

```
def PreviewFrameJpeg()
```

Evidence | Source and method link

`Software/DemoCode/web/camera.py` line 83. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 4.1.

Purpose | What this contributes

Grabs one frame from the already-open preview camera and returns JPEG-encoded bytes, for an MJPEG-style streaming endpoint. Raises RuntimeError if PreviewStart() hasn't been called.

Method | Current implementation

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def PreviewFrameJpeg():
    import cv2
    with _GetLock():
        if _previewCap is None:
            raise RuntimeError("Preview not started -- call preview_start() first.")
        ok,frame = _previewCap.read()
    if not ok:
        raise RuntimeError("Failed to read a preview frame.")
    ok,buf = cv2.imencode(".jpg",frame)
    if not ok:
        raise RuntimeError("Failed to JPEG-encode preview frame.")
    return buf.tobytes()
```

Justification | Interpret the method rather than infer a benchmark

An existing preview handle avoids competing camera opens; warm-up frames allow acquisition to settle. EXIF transpose applies metadata to pixels rather than assuming the viewer appearance equals array orientation. [5, 6].

4.7 CaptureImage

```
def CaptureImage(timeout=10, index=None)
```

Evidence | Source and method link

`Software/DemoCode/web/camera.py` line 105. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 4.1.

Purpose | What this contributes

Captures one still image, returns its Path. Raises RuntimeError if no camera could be opened/read – callers (Server.py) should catch this and report it as a clean error, not a crash. if a live preview is currently open (PreviewStart()), this grabs the frame from THAT handle instead of opening a second one – most webcams only allow one process/handle to hold the device at a time, and this is also the intended flow: watch the live preview until autofocus settles on the page, then capture "the good image" from what's already showing.

Method | Current implementation

Acquire a frame and put its pixel orientation in agreement with the browser and segmenter. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def CaptureImage(timeout=10,index=None):
    import cv2

    with _GetLock():
        if _previewCap is not None:
            ok,frame = _previewCap.read()
            if not ok:
                raise RuntimeError("Failed to read a frame from the open preview.")
            outPath = CAPTURE_DIR/f"capture_{int(time.time()*1000)}.jpg"
            cv2.imwrite(str(outPath),frame)
            return outPath

    idx = CAMERA_INDEX if index is None else index
    cap = cv2.VideoCapture(idx)
    if not cap.isOpened():
        cap.release()
        raise RuntimeError(
            f"Could not open camera at index {idx}. Is it plugged in, and "
            "not already in use by another program (close any other app "
            "using the webcam, e.g. a video call)?"
        )

    deadline = time.time()+timeout
    frame = None
    try:
        for _ in range(WARMUP_FRAMES):
            if time.time()>deadline:
                break
            ok,frame = cap.read()
            if not ok:
                frame = None
    finally:
        cap.release()

    if frame is None:
        raise RuntimeError(
            f"Camera at index {idx} opened but produced no readable frame "
            "within the timeout."
        )

    outPath = CAPTURE_DIR/f"capture_{int(time.time()*1000)}.jpg"
    cv2.imwrite(str(outPath),frame)
    if not outPath.exists():
        raise RuntimeError(f"cv2.imwrite reported no error but wrote no file: {outPath}")
```

```
    return outPath
```

Justification | Interpret the method rather than infer a benchmark

An existing preview handle avoids competing camera opens; warm-up frames allow acquisition to settle. EXIF transpose applies metadata to pixels rather than assuming the viewer appearance equals array orientation. [5, 6].

5 Image processing and line segmentation

Evidence | Current implementation boundary

`Software/DemoCode/segment.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Start with pixel sums and neighbourhood operations; estimate paper and foreground; infer text scale and orientation; find separator seams; repair ink ownership; return ordered line crops. Resampling, drawing and thinning helpers support several of those stages and later glyph extraction.

5.1 Expanded reasoning: page-to-line segmentation

5.1.1 The measurement model and why normalise lighting

The input is a grey image $G(y, x)$, with row y increasing downwards and column x increasing rightwards. A pixel is an area sample, not a physical point. A useful approximate model is $G = L\,R_s + \eta$: L is slowly varying illumination, R_s surface reflectance and η sensor/readout noise. Paper and ink have different reflectance. The method estimates the local paper level B_r and divides by it. If $B_r \simeq LR_{\text{paper}}$, then $G/B_r \simeq R_s/R_{\text{paper}}$ and the multiplicative illumination approximately cancels. This algebra is the reason for using a ratio rather than a fixed grey threshold; it is not proof that strong shadows or glare are removed. Additive glare, saturation, texture and a window mostly filled with ink violate the approximation. Sauvola's local mean/variance threshold is a published alternative [7]; choosing it would change the measurement model and require independent comparison.

Method | Window size and edges.

A window must be wider than an individual stroke so that it estimates paper rather than that stroke, yet narrower than the illumination variation it is supposed to track. Too small a window follows and cancels ink; too large a window misses local shadows. Edge replication extends the nearest edge pixel outside the image. Zero padding would invent black borders. Reflection and reduced edge windows are alternatives. The selected radius is an implementation setting, not a universal imaging constant.

5.1.2 Integral image: four lookups, with an arithmetic example

For a zero-padded prefix-sum array define

Formula purpose and system use. Purpose in this explanation: Integral image: four lookups, with an arithmetic example **Source basis:** [8, 9, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$S(y, x) = \sum_{0 \le i < y} \sum_{0 \le j < x} G(i, j), \qquad S(0, x) = S(y, 0) = 0.$$

The half-open rectangle $[y_0, y_1) \times [x_0, x_1)$ has sum

Formula purpose and system use. Purpose in this explanation: Integral image: four lookups, with an arithmetic example **Source basis:** [8, 9, 3]. Exact project constants remain implementation choices unless

evidence is stated.

$$Q = S(y_1, x_1) - S(y_0, x_1) - S(y_1, x_0) + S(y_0, x_0).$$

The two strips are subtracted, and the doubly subtracted intersection is added back. This is inclusion–exclusion. Crow introduced summed-area tables for efficient area aggregation [9]. Constructing S takes $O(HW)$ additions and memory; each subsequent rectangle sum takes constant work regardless of radius. The complete image filter still takes $O(HW)$. It does not mean the whole operation takes constant time.

For $G = \left[\begin{smallmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{smallmatrix}\right]$, the padded S is $\left[\begin{smallmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 3 & 6 \\ 0 & 5 & 12 & 21 \\ 0 & 12 & 27 & 45 \end{smallmatrix}\right]$. The lower-right 2×2 sum is $45 - 6 - 12 + 1 = 28$, equal to $5 + 6 + 8 + 9$. Its mean is $28/4 = 7$. Rectangle boundaries and padding must use the same convention: an off-by-one error changes the area and its denominator. Prefix sums should accumulate in a type large enough to avoid overflow.

5.1.3 Ratio, ink/paper distinction and Otsu's criterion

The report's ratio is $R = G/\max(B_r, 1)$, with grey levels on the 0–255 scale. The floor of 1 is a numerical guard measured in grey-level units; it is not a claim that a near-black local background is reliable. For the reported example $G = 100$, $B_r = 161.43$, $R \simeq 0.6195 < 0.80$, so the pixel is ink [10]. The brightness ratio is different from an *ink-area fraction* $\sum M/(HW)$, which counts classified pixels.

For histogram probabilities $p_i = n_i/N$, split at bin t. Set

Formula purpose and system use. Purpose in this explanation: Ratio, ink/paper distinction and Otsu's criterion **Source basis:** [8, 9, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\omega_0 = \sum_{i \le t} p_i, \quad \omega_1 = 1 - \omega_0, \quad \mu_0 = \frac{\sum_{i \le t} i p_i}{\omega_0}, \quad \mu_1 = \frac{\sum_{i > t} i p_i}{\omega_1}.$$

Otsu selects t maximising $\omega_0 \omega_1 (\mu_0 - \mu_1)^2$. This is between-class variance; total variance equals within-class plus between-class variance, so maximising one minimises the other for a fixed histogram [8]. Empty classes are excluded. "Optimal" here means optimal for that histogram objective, not for true character recovery. The project bins ratios into 256 bins over 0–1.2, giving bin width $1.2/256 = 0.0046875$. Bin count and range are numerical resolution choices.

Constructed example: four samples at 0.4 and six at 1.0 have $\omega_0 = 0.4$, $\omega_1 = 0.6$, and between-class variance $0.4(0.6)(0.4 - 1)^2 = 0.0864$. Any threshold strictly between the two levels gives the same partition. Thus an Otsu threshold need not be unique. On a unimodal almost-blank image it can separate paper noise rather than ink. Inspecting the resulting mask remains necessary.

5.1.4 Orientation, deskew, run limits and the line-pitch estimate

A quarter-turn and a small-angle deskew solve different problems. A 90-degree turn changes which image axis follows the lines. A fine rotation corrects a small slope. The report requires the nearly-empty-row score to improve by a factor 1.35 before accepting the quarter-turn. If scores are 0.20 and 0.28, the ratio is 1.4 and passes; 0.20 and 0.25 do not. The margin expresses conservatism when evidence is weak. Neither 1.35 nor this score distinguishes upright text from upside-down text in general.

For each trial angle θ, the row projection is $p_\theta(y) = \sum_x M_\theta(y, x)$ and the objective is $E(\theta) = \sum_y p_\theta(y)^2$ [11]. For fixed total ink, concentrating it gives a larger squared sum: $(8, 0)$ scores 64 whereas $(4, 4)$ scores 32. Horizontal rows therefore tend to win. The 17-point grid from -4 to 4 degrees in half-degree steps is a search range and resolution choice. If the true optimum lies inside

the grid and the objective is well behaved, nearest-grid angular error is at most 0.25 degrees. That does not bound estimation error on curved text. Rotating sparse coordinates during the search saves resampling; the chosen full grey image must then be rotated and binarised consistently.

The line pitch P is a typical centre-to-centre distance in working-image pixels. It is not x-height. Pitch-normalised guards adapt to image scale. The report's horizontal and vertical run limits are $1.25P$ and $1.4P$; at $P = 80$ pixels these are 100 and 112 pixels. Long straight ruling is more likely to exceed them than an ordinary stroke. Long underlines and vertical letter strokes can nevertheless be removed. These are heuristics, and estimating P again after mark removal reduces circular dependence.

For a smoothed row profile, the 6%-of-maximum candidate floor rejects weak peaks. A maximum of 200 implies a candidate floor of 12, in the same units as the profile. If one dark line dominates, faint genuine lines may fall below the floor. Sorting peaks by strength and suppressing others within $0.6P$ avoids treating one tall text body as multiple lines. At $P = 80$, the separation is 48 pixels. Increasing it merges tightly spaced real lines; decreasing it admits duplicate centres. Median peak spacing resists isolated bad gaps. Autocorrelation $A(k) = \sum_y p(y)p(y+k)$ instead seeks a repeated spacing; strong ruling or harmonics can select the wrong multiple. Component heights supply a fallback scale when peak evidence is sparse. None of these fallbacks independently verifies the transcript's line count.

5.1.5 Seam cost, closing and dynamic programming, fully unpacked

The seam is one row s_x for every column x, constrained to the corridor between neighbouring line centres. For the report's cost

Formula purpose and system use. Purpose in this explanation: Seam cost, closing and dynamic programming, fully unpacked **Source basis:** [8, 9, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$C(y, x) = 4M_c(y, x) + 0.012 \left(\frac{y - y_m}{h_g} \right)^2,$$

M_c is the closed binary mask; y_m is the corridor midpoint and h_g its half-height. Both terms are dimensionless. The quadratic prior mildly favours the gap's middle; the ink term strongly discourages stroke crossings. Within $|y - y_m| \le h_g$, the prior is at most 0.012 while one ink pixel costs 4. This 333-to-1 local ratio explains priority, not an experimentally proved optimum. Total route cost depends on every column, so a long detour can outweigh one local penalty.

Closing is dilation followed by erosion with a chosen structuring element. A square radius r has side $2r + 1$. The report uses $r = \max(2, \lfloor 0.07P \rfloor)$; $P = 80$ gives $r = 5$ and side 11. It discourages a seam from slipping through tiny holes in a stroke. Too large a radius joins neighbouring lines and destroys real passages. The maximum here is a *lower bound* of 2 on radius, not a maximum permitted radius. The phrase "max closing radius" in the supplied list should not be read as an upper cap unless a particular implementation adds one.

For allowed vertical changes $d \in \mathcal{D}$, accumulated cost is

Formula purpose and system use. Purpose in this explanation: Seam cost, closing and dynamic programming, fully unpacked **Source basis:** [8, 9, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$D(y, x) = C(y, x) + \min_{d \in \mathcal{D}} [D(y + d, x - 1) + 0.02|d|].$$

The 0.02 term prices vertical movement and reduces jagged seams. Displacement is measured in working-image rows. Restricting $\mathcal{D}$ constrains slope; its exact contents belong to the implementation. Store each minimising predecessor, choose the lowest-cost final state, and backtrack. Without the

predecessors, the minimum cost alone is not a path. This is Bellman's optimal substructure principle on an acyclic graph [12, 13].

Constructed arithmetic, for $\mathcal{D} = \{-1, 0, 1\}$: if predecessor costs at rows $y-1, y, y+1$ are $0.10, 0.16, 0.11$, the transition candidates are $0.12, 0.16, 0.13$. A blank pixel with prior 0.003 then has $D = 0.123$. An ink pixel at that location has $D = 4.123$. Greedily choosing the cheapest pixel in each column can fail because it ignores a later blocked corridor. Dynamic programming compares complete prefixes. Its optimum is global only within this corridor, step set and cost function, not over all conceivable line separations. With W columns, H_g corridor rows and q transitions, time is $O(WH_gq)$; backtracking storage is $O(WH_g)$.

5.1.6 Component ownership and the 15% substantial-band rule

Seam bands assign space, not semantic identity. Let $a_{C,n}$ be the ink pixels of connected component C inside band n. A substantial band obeys $a_{C,n} \geq 0.15 \sum_j a_{C,j}$. With overlaps $(80, 15, 5)$ and total 100, the first two qualify; the third does not. The topmost substantial band is a reference owner for eligible corrections, not an instruction to transfer the whole component blindly. With threshold 0.15, at most six disjoint bands can qualify because seven would require at least 105% of the component. This bound follows from the definition; it does not justify choosing 0.15.

Eight-connectivity includes diagonal neighbours. Run-based labelling stores contiguous horizontal ink intervals and unions runs touching the row above, including diagonal overlap [14, 15]. It avoids recursively visiting every pixel in Python. Raw components are revisited because closing can create artificial connections. Dot-to-stem, ring and residual-fragment rules use size/proximity guards to avoid moving a genuine detached feature to another line. A dot's meaning cannot be recovered from proximity alone. The report's 462 reassigned pixels describe one recorded example; they are not 462 verified corrections and not an accuracy percentage.

5.1.7 Contrast, darkness exponent and solid-crop rejection

Contrast stretching maps chosen dark/light endpoints to a wider output range. For darkness $D = 1 - G/255$ in $[0, 1]$, the transformation $G' = 255(1 - D^{0.55})$ strengthens intermediate darkness: $D = 0.25$ becomes approximately 0.467. Applying the same exponent directly to brightness would have a different effect. Always identify which variable is exponentiated. The coefficient 0.55 is a project contrast heuristic, not a gamma correction derived from a known camera response.

The supplied list also asks about a dark-pixel fraction exceeding 15% and three times the page median. In the inspected `DropSolidCrops`, this applies to whole candidate line crops, not individual pixel rows: darkness is grey level below 128, and the median is taken across candidate crops. For median 0.04, the operative threshold is $\max(0.15, 0.12) = 0.15$; for median 0.08 it is 0.24. Source comments report 668 crops with densest genuine crop 12.37%, largest genuine-to-page-median ratio 2.18, and one artefact at 37.5% and 7.2 times its median. That supplies documented local intent, not independently rerun validation or a literature optimum. If every crop would be rejected, the inspected function returns the original list. These tests target unusually dense artefacts and can also reject heavy handwriting. A factor of three is a relative guard, not a three-standard-deviation statistical test. The 15% in this test has a different denominator and purpose from the 15% ownership test.

5.1.8 Complete algorithm, alternatives and verification meaning

The conceptual order is: decode greyscale; reduce the working resolution; estimate background and ink; choose quarter-turn and fine skew; estimate pitch; remove eligible long marks; re-estimate pitch and centres; close a mask for seam costs; compute and backtrack boundaries; assign initial bands; refine ownership using raw and closed components; crop each line from the grey image; exclude neighbouring

ink; apply the chosen crop preparation; sort by page order. The crop used for classification must preserve the mapping back to the original pixels used for glyph extraction.

Straight projection cuts are simpler but cannot bend around ascenders; connected-component clustering is natural for print but cursive joins and detached dots complicate it; learned segmentation needs labelled boundaries and another trained model. Seams offer explicit, inspectable geometry. That is a defensible method trade-off, not proof of superiority on every page.

Verification asks whether sums, rotations, DP recurrence and ownership obey their definitions. Validation asks whether the resulting lines contain the intended handwriting under the photographed conditions. Equal crop counts can conceal a missing line and a duplicated line. A useful seam plot shows the raw mask, closed mask, cost scale, centre corridor, final path and ownership colours. A path in a low-cost valley explains the optimisation; only comparison with labelled ink establishes correct separation. No old-segmenter comparison, new lighting test or sweep was performed for this edition.

Current function reference

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
InputHeight, InputWidth = (64, 640)
PEAK_MIN_PITCH_XH = 1.5
MIN_LINE_PITCH = 15
LINE_MERGE_VALLEY = 1.0
AXIS_TURN_MARGIN = 1.35
```

Stage: Integral images and local means

Report equation 12 Integral rectangle sum

Method | Formula

$$Q = S(y_1, x_1) - S(y_0, x_1) - S(y_1, x_0) + S(y_0, x_0).$$

Purpose | Why the system needs it

Obtain a window total from four prefix-sum entries using inclusion and exclusion. The full table still costs image-sized work; constant window work does not mean constant full-image processing time.

Evidence | Current location and source

Integral, BoxSum and BoxMean. [9].

5.2 Method background: Integral images and local means

Purpose | The role in this system

Obtain local sums and paper-background estimates without rescanning every window.

Justification | The reason for this construction

Rectangle inclusion and exclusion gives a fixed number of lookups per window after an image-sized prefix sum. It is inspectable and suits many large flat windows.

Alternatives | What changes if another method is used

A direct window loop is simpler but repeats work. A separable sliding box filter has comparable linear image cost. A Gaussian weights the centre more and changes the estimated background.

Library | Implementation speed and accuracy

SciPy ndimage.uniform_filter or OpenCV integral/boxFilter could replace parts. Match nearest-edge padding, window width and floating dtype; SciPy defaults to reflected borders. No accuracy gain follows from a matched arithmetic substitution; memory traffic and compiled loops determine speed. [9, 16, 17].

5.3 Integral

```
def Integral(img)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 12. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.2.

Purpose | What this contributes

Running total of the image, cell (y,x) holds everything above and left of it, so any rectangle sum is four lookups.

Inputs

`img` a two-dimensional array of numbers, normally a greyscale image

Output. An array one row and one column larger than the input, in which each entry holds the total of everything above and to the left of it.

Purpose | What it is for

This is the foundation of the toolkit. On its own it computes nothing useful; what it provides is the ability to ask for the total of any rectangle in the image at a cost that does not depend on the rectangle's size. Every local average, every shape-growing operation and every local ink count in this file uses it.

Method | Background: what a cumulative sum is

If a list of numbers is replaced by a list in which each entry is the total so far, then the sum of any stretch of the original list can be found by subtracting two entries of the new list. For the list 2, 3, 1, 4 the running totals are 2, 5, 6, 10, and the sum of the middle two numbers is 6 minus 2, which is 4. No addition is needed at query time, however long the stretch.

A summed-area table, also called an integral image, is the same idea in two dimensions [9]. Each cell holds the total of the whole rectangle from the top-left corner of the image down to that cell:

Formula purpose and system use. Build a padded prefix sum for later local measurements. **Source basis:** [9, 16, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathrm{ii}[y, x] = \sum_{i<y} \sum_{j<x} \mathrm{img}[i, j] \tag{5.1}$$

Source: [9]

where

$\mathrm{ii}[y, x]$
: the value stored at row y, column x of the table

$\mathrm{img}[i, j]$
: the original pixel at row i, column j

y, x the row and column being computed, counted from zero

Method | How a rectangle total is then obtained

To get the total inside a rectangle, take the table value at its bottom-right corner, which includes the rectangle but also everything above and to the left of it. Subtract the value at the top-right corner, which removes the band above. Subtract the value at the bottom-left corner, which removes the band to the left. Those two subtractions each removed the top-left block, so it has been taken away twice and must be added back once:

Formula purpose and system use. Recover one rectangle sum by inclusion and exclusion rather than rescanning pixels. **Source basis:** [9, 16, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$S = \mathrm{ii}[y_2, x_2] - \mathrm{ii}[y_1, x_2] - \mathrm{ii}[y_2, x_1] + \mathrm{ii}[y_1, x_1] \tag{5.2}$$

Source: [9]

where

S the total of all pixels inside the rectangle

y_1, x_1
: the top-left corner of the rectangle

y_2, x_2
: one row and one column past the bottom-right corner, so the rectangle stops just before them

Justification | Why the extra row and column of zeros

The formula above needs to read the table at the row above the rectangle and the column to its left. For a rectangle touching the top or left edge of the image those positions would be outside the array. Storing the table one larger in each direction, with the first row and column filled with zeros, means those reads land on genuine zeros instead and no special case is needed anywhere in the file.

Justification | Why the result is stored as floating point

The entry in the bottom-right corner is the sum of every pixel in the image. For a 3000 by 2200 photograph with values up to 255 that is about 1.7 billion, which exceeds the range of a 32-bit signed integer. An overflow would not raise an error; it would wrap around to a negative number and quietly corrupt every rectangle total computed from that part of the table. Using 64-bit floating point avoids this with room to spare.

Library | Cost

Two passes over the image to build, then four lookups per query regardless of window size. For the background estimate used by ink detection, with a radius of 40 pixels, the direct method would need 6561 additions per pixel; this needs four.

5.4 BoxSum

```
def BoxSum(img, ry, rx)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 18. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.2.

Purpose | What this contributes

Obtain local sums and paper-background estimates without rescanning every window.

Inputs

`img` a two-dimensional array

`ry` the window radius in rows, so the window spans 2*ry+1 rows

`rx` the window radius in columns

Output. An array the same shape as the input, where each entry is the total of the input inside the window centred on that position.

Purpose | What it is for

Applies the rectangle query of the summed-area table at every position at once. This is the function that actually gets used; `Integral` exists to make it possible.

Method | How it works

The four corner positions of every pixel's window are built as whole arrays of indices rather than being computed one pixel at a time, and the table is then read at all of them simultaneously. The result is the same four-term expression as before, evaluated for the entire image in a handful of array operations.

Doing it this way rather than with a loop matters because the loop would run in the Python interpreter, which is roughly two orders of magnitude slower than the same arithmetic performed inside numpy's compiled code. For a multi-megapixel photograph that is the difference between a fraction of a second and several minutes.

Method | What happens at the edges

A window centred near the border of the image hangs off the edge. The corner indices are clipped to stay inside the array, so the window is effectively cut short and the total covers fewer pixels than it would in the middle of the image. That is not corrected here, because the correction depends on what the caller wants: `BoxMean` divides by the number of pixels actually covered, while `Erode` compares against that same count. Both get what they need from `BoxCount`.

5.5 BoxCount

```
def BoxCount(ry, rx, h, w)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 27. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.2.

Purpose | What this contributes

Obtain local sums and paper-background estimates without rescanning every window.

Inputs

`shape or array`
: used for its dimensions

`ry`
: the window radius in rows

`rx`
: the window radius in columns

Output. An integer array giving, for each position, how many pixels its window actually covered once clipped to the image.

Purpose | What it is for

Window sums near the border of an image cover fewer pixels than those in the middle. This counts how many were covered, which is what makes the border behaviour of the operations above it correct rather than approximately correct.

Method | How it works

The count is the clipped height of the window multiplied by its clipped width. Because those two depend only on the row and only on the column respectively, each is computed as a short one-dimensional array and the two are then multiplied together, which is far less work than computing a two-dimensional count directly. A quantity that factorises this way is called separable, and the same idea appears again in `Resize` and `Blur`.

Purpose | Why it matters: two different uses

- `BoxMean` divides the window sum by this count instead of by the nominal window area. Without that, the average near the border would be computed as though the missing pixels were zero, so the estimated background would fall towards black near every edge and ink detection would misbehave in a band around the whole page.
- `Erode` asks whether every pixel in the window is set, which it tests by comparing the window sum against this count. Comparing against the nominal area instead would make the test impossible to satisfy anywhere near the border, and a band around the page would be eroded away.

5.6 BoxMean

```
def BoxMean(a, r)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 34. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.2.

Purpose | What this contributes

Obtain local sums and paper-background estimates without rescanning every window.

Inputs

`img` a two-dimensional array

`r` a single radius used for both rows and columns

Output. A floating point array the same shape as the input, giving the average value in the window centred on each position.

Purpose | What it is for

This is how the pipeline estimates the local background brightness of a photograph. Comparing a pixel against the average of its own surroundings, rather than against one number for the whole page, is what allows ink to be found correctly on a page that is brighter on one side, or has a shadow across it, or was photographed with the flash off to one side.

Method | How it works

Formula purpose and system use. Estimate local paper brightness as a square-window mean. **Source basis:** [9, 16, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{mean}[y, x] = \frac{\text{BoxSum}[y, x]}{\text{BoxCount}[y, x]} \tag{5.3}$$

where

$\text{mean}[y, x]$
: the local average at this position

$\text{BoxSum}[y, x]$
: the total inside the window, from the summed-area table

$\text{BoxCount}[y, x]$
: how many pixels that window covered

Justification | Why a flat window rather than a smooth one

A gaussian-weighted average, which weights nearby pixels more than distant ones, would be slightly smoother. A flat window is used because it can be computed from the summed-area table in constant time per pixel whatever the radius, where a gaussian cannot. For a background estimate the difference is not visible: what is wanted is the general level over a region, and the fine detail of how that region is weighted does not affect which pixels end up classified as ink.

Stage: Binary morphology

5.7 Method background: Binary morphology

Purpose | The role in this system

Repair narrow gaps or identify occupied neighbourhoods while controlling mask geometry.

Justification | The reason for this construction

Boolean dilation, erosion and closing express explicit geometric neighbourhood rules before seams or skeleton operations.

Alternatives | What changes if another method is used

Opening removes small foreground features; closing fills gaps. A disk preserves a different distance notion from a square. Learned cleanup needs labelled examples and adds model dependencies.

Library | Implementation speed and accuracy

SciPy binary_closing or OpenCV morphologyEx offers compiled morphology. Specify the same footprint and border treatment: this project erosion uses the number of in-image neighbours, so default outside-zero erosion need not agree. A different footprint changes segmentation, not just speed. [18, 19].

5.8 Dilate

```
def Dilate(mask, ky, kx)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 41. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.7.

Purpose | What this contributes

Repair narrow gaps or identify occupied neighbourhoods while controlling mask geometry.

Inputs

`m` a mask: an array of true and false values

`r` the radius to grow by, in pixels

Output. A mask in which a position is true if any true pixel of the input lay within its window.

Method | Background: what morphology is

Morphology is the family of operations that change the shapes in a mask rather than the values in an image. The two basic operations are growing a shape outwards and shrinking it inwards, and almost everything else is built by combining them. The window they look through is called the structuring element; here it is always a square.

Purpose | What it is for

Growing shapes is used in this pipeline to make separate pieces of ink touch, so that they can be treated as one object. The clearest example is a letter such as i or j, which is written as two separate marks: growing both slightly makes them touch, after which the grouping stage treats the letter and its dot as a single piece and cannot assign them to different lines of writing.

Method | How it works

A position should be turned on if any true pixel lies within its window. Asking whether any pixel in a region is true is the same as asking whether the number of true pixels is greater than zero, and that number is exactly what a window sum provides:

Formula purpose and system use. Mark positions with any foreground neighbour inside the chosen footprint. **Source basis:** [18, 19]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{dilate}(m)[p] \;=\; \big(\,\text{BoxSum}(m, r, r)[p] \;>\; 0\,\big) \tag{5.4}$$

where

- m the input mask, with true counted as 1 and false as 0
- p the position being computed
- r the radius of the square structuring element

Expressing it as a window sum means dilation inherits the constant cost per pixel of the summed-area table, so growing by twenty pixels costs the same as growing by two.

5.9 Erode

```
def Erode(mask, ky, kx)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 45. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.7.

Purpose | What this contributes

Repair narrow gaps or identify occupied neighbourhoods while controlling mask geometry.

Inputs

- `m` a mask
- `r` the radius to shrink by

Output. A mask in which a position is true only if every pixel its window covered was true in the input.

Purpose | What it is for

Shrinking shapes. On its own it is used to remove small specks and to thin a shape; its main use here is as the second half of a closing, described next.

Evidence | How it works, and the one subtlety

Where dilation asks whether *any* pixel in the window is set, erosion asks whether *every* one is. That is true exactly when the number of set pixels equals the number of pixels the window covered:

Formula purpose and system use. Keep positions whose in-image neighbourhood is all foreground. **Source basis:** [18, 19]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{erode}(m)[p] \;=\; \big(\,\text{BoxSum}(m, r, r)[p] \;=\; \text{BoxCount}(r, r)[p]\,\big) \tag{5.5}$$

where

m the input mask

p the position being computed

$\text{BoxCount}(r, r)[p]$
how many pixels the window at p actually covered after clipping

The comparison must be against the covered count and not against the full window area of (2r+1) squared. Near the border the window is clipped, so its sum can never reach the full area, and a test against the area would report false for every position within r pixels of the edge. The visible result would be a band eroded away around the whole page, and since erosion is half of a closing, that band would then be removed from every mask the pipeline closes.

5.10 Close

```
def Close(mask, ky, kx)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 50. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.7.

Purpose | What this contributes

Repair narrow gaps or identify occupied neighbourhoods while controlling mask geometry.

Inputs

`m` a mask

`r` the radius

Output. A mask with gaps narrower than the structuring element filled in, and the original thickness of the shapes preserved.

Purpose | What it is for

Repairing ink that should be continuous but is not. Thresholding a photograph leaves small holes inside strokes where the paper texture shows through, and a stroke drawn with a drying pen can have genuine breaks in it. Both are repaired by a closing.

Justification | How it works, and why the two steps are needed

A closing is a dilation followed by an erosion by the same radius. The dilation joins anything separated by less than the window, which fills the gaps, but it also makes every stroke fatter. The erosion then removes a layer of the same thickness, which restores the original width. What does not come back is the gap: once two sides have been joined, eroding the joined shape thins it but does not split it again.

Formula purpose and system use. Bridge small mask gaps through dilation followed by erosion. **Source basis:** [18, 19]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{close}(m, r) \;=\; \text{erode}\big(\text{dilate}(m, r),\, r\big) \tag{5.6}$$

where

m the input mask

r the radius, which sets the largest gap that will be bridged

So a closing bridges a gap narrower than the kernel while leaving a thin stroke thin, which a plain dilation would not. That property is why it, and not dilation, is used before thinning: thinning is sensitive to both holes and thickness, so the operation applied beforehand must fix the first without changing the second.

5.11 AnyWithinRadius

```
def AnyWithinRadius(m, r, axis)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 54. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.7.

Purpose | What this contributes

Repair narrow gaps or identify occupied neighbourhoods while controlling mask geometry.

Inputs

`mask`
: a mask

`r`
: the radius

`axis`
: which axis to work along: 0 for down the rows, 1 for across the columns

Output. A mask that is true wherever any element within r positions along that single axis was true.

Purpose | What it is for

A dilation along one direction only. It is used where the tolerance being applied has a direction to it, and growing in both directions would be wrong.

There are two such places. When a printed rule has been detected, it is widened slightly along its own length, so that a rule which is a fraction of a degree off horizontal is still removed over its whole length rather than in broken pieces; widening it vertically as well would eat into the writing above and below it. And when the paper region has been found, it is shrunk inwards by a margin in each axis separately, to keep the slightly soft edge of the sheet from being read as ink.

5.12 LongRunMask

```
def LongRunMask(m, n, axis)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 61. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.7.

Purpose | What this contributes

Repair narrow gaps or identify occupied neighbourhoods while controlling mask geometry.

Inputs

`m` a mask

`n` the minimum length, in pixels, that a run must reach

`axis`
the direction along which the run is measured

Output. A mask that is true at every pixel belonging to an unbroken run of at least n set pixels along that axis.

Purpose | What it is for

Telling printed lines apart from handwriting. This is the test that removes the ruling from a lined page and the dark edge of a binder, both of which would otherwise dominate every measurement the segmenter makes.

Justification | Why run length is the right test

A printed rule is drawn by a machine: it is straight, continuous, and spans the page. A pen stroke changes direction constantly, so the number of pixels it continues in any one direction without a break is small. At the sizes this pipeline works with, the threshold is set to between 1.25 and 1.4 times the spacing between lines of writing, which is roughly 60 to 80 pixels; almost no handwritten stroke runs that far in a straight line, and a rule runs much further.

Method | How it works

Two passes of cumulative sums. The first finds where a long run begins. A stretch of n pixels starting at position i is entirely set exactly when the number of set pixels inside it equals n, and that count comes from one subtraction of the cumulative sum:

Formula purpose and system use. Compute interval ink totals efficiently along a selected axis. **Source basis:** [18, 19]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{starts}[i] \;=\; \left(c[i+n] - c[i] \;=\; n \right) \tag{5.7}$$

where

starts$[i]$
true when a run of length n begins at position i

c the cumulative sum of the mask along the chosen axis, with a leading zero

n the minimum run length

i the candidate starting position

The second pass turns starting positions into coverage. A run of length n that covers position j must begin somewhere between n-1 places before j and j itself, and it cannot begin so late that it would

run off the end. A second cumulative sum over the starts then answers "is there any start in that range" with another single subtraction:

Formula purpose and system use. Retain ink supported by a sufficiently long contiguous run. **Source basis:** [18, 19]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{covered}[j] \;=\; \big(\, cs[h_j + 1] - cs[l_j] > 0 \,\big), \qquad l_j = \max(0,\, j - n + 1),\; h_j = \min(j,\, L - n) \tag{5.8}$$

where

covered[j]
: true when position j lies inside some long run

cs : the cumulative sum of the starts array

l_j, h_j
: the first and last positions at which a run covering j could begin, clamped to the valid range

L : the length of the axis being measured

Justification | Why it is done with cumulative sums

The cost is proportional to the number of pixels and does not depend on n. Scanning each window directly would cost n times as much, and n here is around 80.

Method | How one function handles both directions

The axis of interest is moved to the front of the array, the fixed code runs down it, and the axis is moved back afterwards. Moving an axis in numpy only changes how the array is interpreted, not the data in memory, so this costs nothing and avoids writing the same logic twice.

> *A simpler equivalent exists.* `RowRunLength`*, described next, already gives every pixel the length of its own run, which makes this function the same as asking whether that length is at least n, with a transpose for the other axis. That form was checked against this one on 4800 randomly generated masks of every shape, with n from 1 to 40 and both axes, and never disagreed; on a page-sized array it also ran about twice as fast, because it makes one labelling pass and one count instead of building two cumulative-sum arrays and several arrays of clamped indices.*

5.13 RowRunLength

```
def RowRunLength(mask)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 74. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.7.

Purpose | What this contributes

Repair narrow gaps or identify occupied neighbourhoods while controlling mask geometry.

Inputs

`mask`
: a two-dimensional mask; runs are measured across the rows

Output. An integer array in which every set pixel holds the length of the unbroken horizontal run it belongs to, and every clear pixel holds zero.

Purpose | What it is for

Measuring how far ink continues horizontally through each pixel. It is used to find printed rules, and in `BinarizeLine` to delete them from a line crop.

Method | How it works, in three vectorised steps

First, the places where a run begins are found as the positions where the row steps from clear to set, which is a comparison between the row and a copy of itself shifted by one.

Second, a cumulative sum over those starting positions numbers the runs consecutively: the first run becomes 1, the second 2, and so on. The numbering is then cleared wherever there is no ink, so that only ink carries a run number.

Third, counting how many pixels carry each number gives the length of each run, and indexing those counts by the run number image hands every pixel the length of its own run.

Justification | Why the numbering does not leak between rows

The cumulative sum is taken over the whole array flattened into one long sequence, which would be wrong if a run could continue from the end of one row into the start of the next. It cannot: the comparison that finds starts is padded at both ends of every row, so each row begins and ends with a boundary and the last run of a row is always closed before the next row's first run begins.

Method | Background: what counting occurrences means here

The counting step uses a standard operation that, given a list of non-negative whole numbers, returns how many times each value occurs. Applied to an image of run numbers it gives the size of every run in one pass, which is why no loop over runs is needed.

Stage: Otsu and ink decisions

Report equation 13 Background ratio and ink mask

Method | Formula

$R = G/\max(B_r, 1)$ and $M = \mathbf{1}[R < T]$.

Purpose | Why the system needs it

Reduce multiplicative illumination changes before thresholding. The floor prevents zero division; glare, clipping and ink-filled windows can still invalidate the paper-background approximation.

Evidence | Current location and source

InkMask and SegmentLines crop normalisation. [3, 8].

Report equation 14 Otsu threshold criterion

Method | Formula

$T = \arg\max_t \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2$.

Purpose | Why the system needs it

Separate the histogram into two maximally distinct classes. Histogram optimality is not semantic optimality for true ink, and empty classes require guards. Bins, range, tie handling and comparison polarity matter.

Evidence | Current location and source

Otsu, InkMask and BinarizeLine. [8, 17].

Method | Normalisation and affine terminology

Min–max scaling uses $(x - x_{\min})/(x_{\max} - x_{\min})$ to map a range; z-score scaling uses $(x - \mu)/\sigma$ to centre and scale a distribution. Neither is identical to illumination division, stroke-width normalisation or geometric x-height normalisation. Each removes a different nuisance variable. The denominator needs a defined constant-image convention. These are direct definitions, not evidence that all are active inference stages.

5.14 Method background: Otsu and ink decisions

Purpose | The role in this system

Separate handwriting from paper using a reproducible histogram criterion.

Justification | The reason for this construction

Otsu maximises between-class variance; dividing by estimated paper brightness first reduces multiplicative lighting changes. The ratio pipeline is a project construction around a published threshold rule.

Alternatives | What changes if another method is used

Fixed grey thresholds are cheap but lighting-sensitive. Local mean or Sauvola thresholds adapt to local statistics but introduce their own windows and constants. None guarantees preservation of faint strokes.

Library | Implementation speed and accuracy

OpenCV threshold with OTSU and skimage.filters.threshold_otsu implement the criterion, but quantisation, float ranges, ties and strict versus inclusive comparison must match. OpenCV adaptiveThreshold is a different method, not the same ratio calculation. [8, 7, 17].

5.15 Otsu

```
def Otsu(v, hi=None)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 89. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.14.

Purpose | What this contributes

Separate handwriting from paper using a reproducible histogram criterion.

Inputs

`v` an array of pixel values, either whole numbers or fractions

`hi` when the input is fractional, the largest value worth considering; defaults to 1.2, which suits the ratio image produced by `InkMask` because nothing above that carries ink

Output. A threshold. For whole-number input it is a brightness level from 0 to 255; for fractional input it is a value in the input's own units. Pixels below the threshold are ink.

Purpose | What it is for

Deciding, for a given image, which brightness separates ink from paper. The point of computing it rather than fixing it is that no single number works for two photographs: one taken in daylight and one under a lamp have completely different brightness ranges, and a fixed threshold would classify the whole of one of them as paper.

Method | Background: what a histogram is

A histogram counts how many pixels fall at each brightness level, ignoring where they are. For a page of handwriting it usually has two humps: a large one at the bright end for the paper and a small one at the dark end for the ink, with a valley between them. The threshold should sit in that valley.

Method | The idea: measure the quality of every candidate threshold

Otsu's method [8] turns the choice into something measurable. Any candidate threshold divides the pixels into two groups. A good threshold is one where the two groups are as different from each other as possible, which is measured by the variance *between* the groups:

Formula purpose and system use. Compute between-class separation for each candidate threshold. **Source basis:** [8, 7, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\sigma_B^2(t) \;=\; \omega_B(t)\,\omega_F(t)\,\left[\mu_B(t) - \mu_F(t)\right]^2 \tag{5.9}$$

Source: [8]

where

$\sigma_B^2(t)$
: the between-class variance, the quantity being maximised

t : the candidate threshold

$\omega_B(t)$
: the fraction of all pixels at or below t, that is the share in the darker group

$\omega_F(t)$
: the fraction above t, so the two fractions add to 1

$\mu_B(t)$
: the mean brightness of the darker group

$\mu_F(t)$
: the mean brightness of the brighter group

Reading the formula: the squared bracket is how far apart the two group means are, so it rewards a threshold that genuinely separates dark from bright. The product of the two fractions in front is largest when the groups are of comparable size and falls to zero when either is empty, which prevents the degenerate answer of putting the threshold at one extreme and calling almost everything one group.

Justification | Why maximising this is the right objective

The total variance of the image is a fixed property of the image and does not depend on the threshold. It can be split exactly into the variance within the two groups plus the variance between them. Since the total is fixed, making the between-group part as large as possible is the same as making the within-group part as small as possible, that is, choosing the threshold that makes each group internally as uniform as it can be. The between-group form is used because it needs only running totals, where the within-group form would need a second pass over the data for every candidate.

Method | How all 256 candidates are evaluated at once

Writing p for the histogram scaled so that its entries sum to one, the two quantities needed are themselves cumulative sums, and the criterion rearranges into a form built entirely from them:

Formula purpose and system use. Choose the threshold index attaining maximum separation in the current histogram. **Source basis:** [8, 7, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\sigma_B^2(t) \;=\; \frac{\left[\mu_T\,\omega(t) - \mu(t)\right]^2}{\omega(t)\,\left(1-\omega(t)\right)} \tag{5.10}$$

Source: [8]

where

$\omega(t)$ the cumulative sum of p up to t, the share of pixels at or below t

$\mu(t)$ the cumulative sum of p weighted by brightness, the contribution of those pixels to the mean

μ_T the mean brightness of the whole image, which is the final value of the weighted cumulative sum

Both cumulative sums are computed once, the expression is evaluated for every t simultaneously as array arithmetic, and the largest entry is the answer. No loop over thresholds is needed.

Evidence | The guard in the denominator

At the two extreme thresholds one group is empty, so the product in the denominator is exactly zero. Dividing by it would give infinity, and since the method picks the largest value those positions would always win. The denominator is therefore floored at a very small positive number, which leaves every genuine candidate unchanged and makes the degenerate ones lose.

Justification | Why whole numbers and fractions are handled separately

A whole-number image has exactly 256 possible values, so its histogram can be counted exactly and the answer returned as a brightness level. A fractional image, such as the ratio of each pixel to its local background, has no natural set of levels, so its values are clipped at `hi` and sorted into 256 bins, and the answer is returned as the value at the centre of the chosen bin rather than as a bin number.

5.16 BinarizeLine

```
def BinarizeLine(gray, stripRules=True, closeGaps=True)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 108. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.14.

Purpose | What this contributes

A grey line crop to a clean ink mask, threshold, close pinholes, drop specks, and remove long thin runs, which are printed rules.

Inputs

`gray`
: a greyscale image of one line of writing

Output. A mask of the ink in that crop, with small holes closed, specks removed and printed rules deleted.

Purpose | What it is for

The standard way of turning a grey line crop into ink. The glyph extractor uses it before tracing strokes, and the synthesiser uses it to measure its own rendered output with the same code that measured the writer's real pages, which is what makes the two comparable.

Justification | The four steps and why each is needed

Threshold. `Otsu` chooses the level, so nothing about the brightness of this particular photograph is assumed.

Close small holes. Thresholding a photograph leaves single-pixel holes inside strokes where the paper texture shows through. These matter more than their size suggests, because the next stage after binarising is usually thinning, and thinning preserves the topology of the shape it is given: a hole in a stroke forces the resulting centreline to split and run round both sides of it, producing a small loop that was never written.

Remove specks. Pixels are grouped into connected components and any group too small to be a pen mark is deleted as dust or camera noise. The decision is made for a whole group at once, so a letter is never partly removed.

Delete printed rules. Horizontal runs longer than a limit are removed, on the reasoning given under `LongRunMask`: a pen stroke does not run level for that distance, and a printed rule does nothing else.

5.17 InkMask

```
def InkMask(gray)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 427. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.14.

Purpose | What this contributes

Separate handwriting from paper using a reproducible histogram criterion.

Inputs

`gray`
: the page as a 2-D array of grey values, 0 for black and 255 for white, held as floating point numbers so the division below does not round

Output. a pair. The first item is a boolean array the same shape as the page, true wherever the pixel is judged to be ink. The second is the **ratio image**, the page divided by its own local background, which later stages reuse to measure how dark each piece of ink is.

Purpose | What it is for

This is the first real decision in the whole pipeline: for every pixel of the photograph, is this pen or is this paper? Everything downstream, from finding the lines to cutting out letters, works on the answer. Getting it wrong in one direction loses strokes; getting it wrong in the other fills the page with shadows and paper texture that then get mistaken for writing.

Justification | Background: why a single threshold is not enough

The obvious method is a global threshold: pick one grey value, call everything darker than it ink. That works for a scanner, where the lamp lights the page evenly. It fails badly for a photograph taken with a phone, because the illumination is not even. One corner of the page sits under a lamp and the opposite corner sits in shadow, and the page is often slightly curved so the middle is brighter than the edges. The result is that paper in the bright corner can be lighter than 220 while paper in the shadowed corner is darker than 150, and real pen ink in the bright corner is around 90. There is no single number that separates ink from paper everywhere at once: any threshold low enough to reject the shadowed paper also rejects the pale pen strokes, and any threshold high enough to catch the pale strokes also catches the shadow.

Method | The idea: divide out the illumination

The fix is to stop asking "is this pixel dark?" and ask instead "is this pixel darker than the paper immediately around it?". The illumination varies smoothly across the page, so over a small neighbourhood it is almost constant. If the brightness of a pixel is modelled as the true reflectance of the page at that point multiplied by the local illumination, then dividing by an estimate of that local illumination cancels it and leaves the reflectance, which is the thing that actually distinguishes pen from paper.

Formula purpose and system use. Estimate the local illumination scale from a windowed paper level. **Source basis:** [8, 7, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{ratio}[y, x] \;=\; \frac{\text{gray}[y, x]}{\max\left(\text{BoxMean}(\text{gray}, r)[y, x],\, 1\right)} \tag{5.11}$$

where

$\text{ratio}[y, x]$
: the illumination-corrected value at this pixel; near 1 for paper, well below 1 for ink

$\text{gray}[y, x]$
: the measured grey value at this pixel

$\text{BoxMean}(\text{gray}, r)$
: the average grey value over a square window of radius r centred on each pixel, which serves as the estimate of the local illumination; see the entry for BoxMean

r
: the radius of that window, in pixels

$\max(\cdot, 1)$
: a guard so that a window which happens to average to zero cannot cause a division by zero

A ratio of 1 means the pixel is exactly as bright as its surroundings, which is what paper looks like. A ratio of 0.4 means the pixel is well under half as bright as the paper around it, which is what a pen

stroke looks like. The key property is that this is now true in the bright corner and the shadowed corner alike, because the shadow divides out of both the numerator and the denominator.

Method | Choosing the window size, and the trap in it

The radius has to satisfy two opposing requirements. It must be large enough that the window is mostly paper even when it is centred on a stroke, because the denominator is supposed to estimate the paper brightness, not the ink. If the radius were the width of a pen stroke, then a window centred inside a thick stroke would be entirely ink, the estimated background would be as dark as the ink itself, the ratio would come out near 1, and the stroke would be declared paper. That failure has a name in this file: it is the "solid ink" case handled below. The radius must also be small enough that the illumination really is roughly constant across the window, which rules out making it enormous.

Formula purpose and system use. Divide grey intensity by its guarded background estimate. **Source basis:** [8, 7, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$r \;=\; \mathrm{clip}\Big(\mathrm{round}\Big(\frac{H}{41}\Big)\,,\; 8,\; 40\Big) \tag{5.12}$$

where

r the background window radius used above

H the height of the page image in pixels

$\mathrm{clip}(v, a, b)$
: v forced into the range from a to b: the smaller of b and the larger of a and v

41 a divisor chosen so that the window comes out a few times the thickness of a pen stroke at the resolutions this pipeline works at

8, 40
: hard floor and ceiling, so a very small or very large input image still gets a sensible window

Scaling the radius with the image height rather than fixing it is what lets the same code handle a 1000-pixel-tall photograph and a 4000-pixel-tall one. A stroke in the larger photograph is four times as many pixels wide, so the window that must exceed it has to grow in step.

Method | Turning the ratio into a yes or no

The ratio image still has continuous values and a decision is needed. Otsu's method supplies the threshold, and it is unusually well suited here. Otsu's method assumes the values fall into two groups and picks the cut that separates them best in a precise statistical sense; see the entry for Otsu for the derivation. After the division the ratio image genuinely does have two groups, because the illumination that was smearing them together has been removed: a tall narrow spike just below 1 for the paper, and a much smaller spread of low values for the ink. A pixel is ink when its ratio is below the threshold Otsu returns.

Formula purpose and system use. Choose the foreground decision from the ratio distribution. **Source basis:** [8, 7, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathrm{mask}[y, x] \;=\; [\,\mathrm{ratio}[y, x] \;<\; t_{\mathrm{Otsu}}(\mathrm{ratio})\,] \tag{5.13}$$

where

$\mathrm{mask}[y, x]$
: true when this pixel is taken to be ink

$t_{\text{Otsu}}(\text{ratio})$
: the threshold Otsu's method returns for the ratio image, that is, the value that best splits its two groups

$[\,\cdot\,]$: square brackets around a condition mean 1 or true when the condition holds and 0 or false when it does not

Note that the threshold is computed from the ratio image, not from the original grey image. This is the whole point of the two-step design: a global method like Otsu's, which cannot adapt to position, is made to work on a photograph by first removing the position-dependent part of the signal.

Method | The solid-ink correction

One class of object defeats the division: a region of darkness much larger than the background window. The spine of a ring binder, a deep shadow along a fold, or the black edge of the desk behind the page are all far wider than 40 pixels. Inside such a region the window used for the denominator is entirely dark, so the estimated background is dark too, the ratio comes back near 1, and the interior is correctly rejected. The problem is the boundary, where the window straddles light and dark and the ratio dips, marking a band of false ink around every large dark object.

These regions are found by their defining property: they are completely filled. Writing is not. A pen stroke is thin, so any small window placed on the page contains some paper even when it contains a stroke; a window inside a binder shadow contains nothing but shadow. Averaging the mask itself measures exactly this fill fraction.

Formula purpose and system use. Record the mask/ratio quantities used by downstream filtering. **Source basis:** [8, 7, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{solid}[y, x] \;=\; [\,\text{BoxMean}(\text{mask},\,4)[y, x] \;>\; 0.999\,] \tag{5.14}$$

where

$\text{solid}[y, x]$
: true when this pixel lies in a completely filled dark region

$\text{BoxMean}(\text{mask}, 4)$
: the fraction of pixels marked as ink within a 9 by 9 window, obtained by averaging the mask treated as ones and zeros

4 : the window radius, giving a 9 by 9 window: small enough that no ordinary pen stroke can fill it, large enough that noise does not

0.999
: effectively a demand that every pixel in the window be ink; it is not written as 1.0 only to be safe against floating-point rounding in the average

The flagged cores are then grown slightly, with a second averaging test, and subtracted from the mask. Growing them is necessary because the false-ink band lies just outside the solid core, not inside it, so removing only the core would leave the band behind, which is the artefact being targeted.

Justification | Why the ratio image is returned as well

Later stages need to ask how dark a piece of ink is, in order to tell a real pen stroke from a faint printed rule or a crease. The original grey values cannot answer that, because they still contain the illumination. The ratio image can, and it is already computed, so it is passed out rather than recomputed. DropArtifactPieces uses it for precisely this test.

5.18 PaperRegionMask

```
def PaperRegionMask(g, P)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 438. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.14.

Purpose | What this contributes

The sheet inside the photograph, the biggest bright blob, found on a block-averaged copy and blown back up.

Inputs

`g` the page as a 2-D grey array in floating point

`P` the line pitch in pixels, that is, the typical distance between consecutive lines of writing; see LinePitch

Output. a boolean array the size of the page, true inside the sheet of paper and false outside it, shrunk slightly inward from the true paper edge.

Purpose | What it is for

A photograph of a page is rarely just the page. Around it there is desk, keyboard, hand, shadow and the dark gap at the edge of the sheet. None of that is writing, but much of it is darker than the paper and therefore survives into the ink mask. This function finds the outline of the sheet so that everything outside it can be discarded in one move, before any effort is spent trying to interpret it.

Method | The assumption being made

The method rests on one observation: of everything in the frame, the sheet of paper is both bright and large. The desk may be bright too, but it surrounds the page rather than forming one solid region with it, because a visible dark rim separates them, which is the shadow the sheet casts and the contrast at its cut edge. So the paper is the largest **connected** bright region, where connected means one can walk from any pixel of it to any other without leaving it.

Method | Step one: work small

The search is done on a copy of the page reduced by a factor of four in each direction, by averaging each 4 by 4 block of pixels into one. Two reasons. Speed: a sixteenth of the pixels means a sixteenth of the work, and the paper outline does not need pixel accuracy to be useful. Robustness: averaging suppresses noise and small features, so the result is dominated by the large-scale structure, which is what is being looked for. The reduction uses a block average rather than taking every fourth pixel, because taking every fourth pixel can land entirely in a gap and miss a thin dark rim completely, whereas an average over the block always registers it.

Method | Step two: find the dark surround

Formula purpose and system use. Identify suspicious darkness relative to the page background. **Source basis:** [8, 7, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathrm{dark}[y,x] \;=\; [\; c[y,x] \;<\; 0.45 \cdot \mathrm{median}(c) \;] \tag{5.15}$$

where

$\mathrm{dark}[y, x]$
: true when this block of the shrunken page is part of the dark surround

c : the shrunken, locally averaged copy of the page

$\mathrm{median}(c)$
: the middle value of all the block brightnesses when sorted; it stands in for the brightness of the paper

0.45 : how much darker than the paper a block must be before it counts as surround rather than page

The median is used rather than the mean because it is **robust**, meaning that a minority of extreme values barely moves it. In a photograph where a third of the frame is black desk, the mean brightness is dragged a long way down by the desk and no longer describes the paper at all, while the median still sits in the paper, because the paper is the single largest group of similar values. The threshold is a fraction of that median rather than an absolute grey level, which is again what makes it work under different lighting: in a dim photograph everything scales down together and the fraction is unchanged.

Method | The no-surround case

If less than 0.2 percent of the frame comes out dark, the conclusion is that there is no surround in the picture, because the page fills the whole frame or the image is a scan rather than a photograph. In that case the entire image is returned as paper. This matters because the "largest bright region" logic has nothing to find when there is no dark rim to separate regions in the first place, and forcing it to choose would pick an arbitrary piece of the page.

Method | Step three: the largest bright component

Connected-component labelling, described in the entry for LabelComponents, is run on the inverse of the dark mask, that is, on the bright blocks. It returns an array in which every bright block carries an integer label, with all the blocks of one connected region sharing a label. Counting how many blocks carry each label and taking the largest count identifies the sheet.

Formula purpose and system use. Keep a plausible page component using its spatial extent. **Source basis:** [8, 7, 17]. Exact project constants remain implementation choices unless evidence is stated.

$$\ell^{*} \;=\; \arg\max_{\ell \geq 1} \; \left| \{\, (y, x) : \mathrm{lab}[y, x] = \ell \,\} \right| \tag{5.16}$$

where

ℓ^{*} : the label belonging to the sheet of paper

$\mathrm{lab}[y, x]$
: the component label of this block, with 0 meaning it was not bright

$\arg\max$
: the value of the index at which the quantity is largest, as opposed to the largest value itself

$|\{\cdot\}|$: the number of elements in the set, here the number of blocks carrying that label

$\ell \geq 1$
: labels start at 1, so starting the search at 1 excludes the background

Method | Step four: back to full size

The chosen region is a mask over blocks, and what is needed is a mask over pixels. Each block stands for a 4 by 4 square of the original, so the block mask is expanded by replacing every entry with a 4 by 4 square of copies of itself. This is a **Kronecker product** with a 4 by 4 block of ones, which is the standard name for exactly that operation, and in practice it is nearest-neighbour upsampling:

a hard-edged enlargement with no smoothing, which is the correct choice because the value being enlarged is a yes-or-no flag and a smoothed version of it would be meaningless.

Method | Step five: shrink inward

Finally the mask is eroded, that is, every pixel within a radius of the paper boundary is removed from it, using a radius of 0.4 line pitches. The reason is that the boundary itself is the single least trustworthy part of the estimate. It was found on a coarse grid, the sheet's edge is darkened by shadow over a band several pixels wide, and the page may curl. Pulling the accepted region inward by a fraction of a line's height costs nothing, because people do not write right up to the cut edge of a sheet, and it reliably excludes the edge shadow that would otherwise be interpreted as a long horizontal stroke.

> *Expressing the shrink in line pitches rather than pixels is a habit worth noticing, and it recurs throughout the segmenter. The pitch is the natural unit of the problem: it is set by the size of the writing, so a quantity measured in pitches means the same thing on a photograph taken close up as on one taken from further away.*

Stage: Connected components and run labelling

5.19 Method background: Connected components and run labelling

Purpose | The role in this system

Identify whole ink pieces and measure their areas or bounds for ownership and artefact filtering.

Justification | The reason for this construction

Row runs reduce Python-level work on long spans; union operations connect labels across rows, including diagonal contacts.

Alternatives | What changes if another method is used

Flood fill or pixel-wise union-find is easier to visualise but can visit many pixels in Python. Four-connectivity keeps diagonal contacts apart and therefore changes what counts as one object.

Library | Implementation speed and accuracy

OpenCV connectedComponentsWithStats and SciPy ndimage.label provide compiled alternatives. Match eight-connectivity, background label and bounds conventions; numeric label IDs may differ even when the partitions agree. Faster labelling alone does not make ownership more correct. [14, 15, 20, 21].

5.20 LabelComponents

```
def LabelComponents(mask)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 142. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.19.

Purpose | What this contributes

The current routine scans horizontal runs, merges touching previous-row labels through a path-compressed Find helper, and compacts roots at the end. Half-open run endpoints plus the overlap tests include diagonal contact. Renaming Find does not alter the labelling idea; compare partitions rather than integer label IDs.

Method | Current implementation

Identify whole ink pieces and measure their areas or bounds for ownership and artefact filtering. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def LabelComponents(mask):
    H,W = mask.shape
    lab = np.zeros((H,W),np.int32)
    parent = [0]

    def Find(a):
        while parent[a]!=a:
            parent[a] = parent[parent[a]]   #flatten the chain while walking it
            a = parent[a]
        return a

    prev = []
    for y in range(H):
        d = np.diff(np.r_[0,mask[y].view(np.int8),0])   #runs start at 0 to 1, end at 1 to 0
        cur,k = [],0
        for a,b in zip(np.nonzero(d==1)[0].tolist(),np.nonzero(d==-1)[0].tolist()):
            while k<len(prev) and prev[k][1]<a:
                k+=1
            l,kk = 0,k
            while kk<len(prev) and prev[kk][0]<=b:   #<=b counts a diagonal touch
                r = Find(prev[kk][2])
                if l==0:
                    l = r
                elif r!=l:
                    parent[max(r,l)] = min(r,l)
                    l = min(r,l)
                kk+=1
            if l==0:
                parent.append(len(parent))
                l = len(parent)-1
            cur.append((a,b,l))
            lab[y,a:b] = l
        prev = cur
    roots = np.array([Find(i) for i in range(len(parent))])
    _,comp = np.unique(roots,return_inverse=True)   #renumber 1..n after the merges
    return comp[lab].astype(np.int32),int(comp.max())
```

Justification | Interpret the method rather than infer a benchmark

Row runs reduce Python-level work on long spans; union operations connect labels across rows, including diagonal contacts. [14, 15, 20, 21].

5.21 PieceBounds

```
def PieceBounds(labelAt, ys, xs, n, H, W)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 180. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.19.

Purpose | What this contributes

Identify whole ink pieces and measure their areas or bounds for ownership and artefact filtering.

Inputs

`pl` the component number of every ink pixel

`ys` the row coordinate of every ink pixel

`xs` the column coordinate of every ink pixel

`nc` how many components there are

`H` the image height

`W` the image width

Output. Five arrays indexed by component number: each object's topmost row, bottommost row, leftmost column, rightmost column and pixel count.

Purpose | What it is for

Every ownership rule in the segmenter asks geometric questions about objects: how tall is this piece, how wide is it, where does it sit, how much ink does it contain. This answers all of them for every object at once.

Justification | Why it is written as a scatter rather than a loop

The obvious implementation loops over components and, for each one, selects the pixels belonging to it and takes their minimum and maximum. That reads the whole image once per component, so on a page with eight hundred pen marks it reads the image eight hundred times.

Instead the coordinates are scattered into per-component accumulators in a single pass, using operations that take a list of destinations and a list of values and apply a minimum or a maximum at each destination. The pixel count comes from the same occurrence-counting operation used by `RowRunLength`. The result is one pass over the ink rather than one pass per object.

Stage: Blur and elastic displacement

5.22 Method background: Blur and elastic displacement

Purpose | The role in this system

Smooth signals or create controlled training perturbations without changing labels deliberately.

Justification | The reason for this construction

Separable spatial filters and smooth displacement fields provide controllable local variation. Their strengths are settings, not naturally correct noise models.

Alternatives | What changes if another method is used

Median filtering rejects impulse noise but changes stroke geometry differently. Bilateral smoothing uses intensity similarity. Unrestricted deformation can change the character and violate its label.

Library | Implementation speed and accuracy

SciPy gaussian_filter or OpenCV GaussianBlur/remap can implement related operations. Kernel truncation, boundary mode and dtype affect results; compiled filtering may be faster, but no unconditional timing or accuracy claim is established here. [22, 23, 24].

5.23 Blur

```
def Blur(gray, sigma, mode='edge')
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 340. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.22.

Purpose | What this contributes

Smooth signals or create controlled training perturbations without changing labels deliberately.

Inputs

`img` the image to blur

`sigma`
: the standard deviation of the gaussian, in pixels, which sets how far the blurring reaches

Output. The blurred image, in the input's number format.

Purpose | What it is for

Two uses. It smooths the random displacement fields used for training augmentation, which is what turns random noise into a coherent bending of the strokes. And it softens the edge of synthesised ink, so that a drawn stroke has the gradual edge of a real pen rather than a hard step, which matters because the networks judging that output were trained on photographs of real ink.

Method | Background: what a gaussian blur is

Each output pixel is a weighted average of its neighbourhood, with weights following the bell-shaped gaussian curve: strongest at the centre and falling away smoothly with distance. Compared with a flat average it produces no visible edge at the limit of the window, because the weights reach zero gradually.

Justification | Why it is done as two one-dimensional passes

A two-dimensional gaussian factorises exactly into the product of two one-dimensional gaussians:

Formula purpose and system use. Construct normalised Gaussian smoothing weights from distance and sigma. **Source basis:** [22, 23, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$G(x,y) \;=\; \exp\!\left(-\frac{x^2+y^2}{2\sigma^2}\right) \;=\; \exp\!\left(-\frac{x^2}{2\sigma^2}\right)\cdot\exp\!\left(-\frac{y^2}{2\sigma^2}\right) \;=\; G(x)\,G(y) \tag{5.17}$$

where

$G(x,y)$
: the two-dimensional weight at an offset of x columns and y rows from the centre

σ the standard deviation, which sets the width of the bell

$G(x)$, $G(y)$
: the one-dimensional weights along each axis

So blurring down the rows and then across the columns gives exactly the same answer as a single two-dimensional pass, at a cost of 2k multiplications per pixel instead of k squared. This is the same separability argument as for `Resize`.

Method | Truncation and normalisation

The gaussian never reaches zero, so the kernel is cut off at about three standard deviations, where under one per cent of the total weight remains. The remaining weights are then divided by their sum so they total one, which keeps the overall brightness of the image unchanged; without that, blurring would darken the image by the fraction of weight that was discarded.

5.24 Elastic

```
def Elastic(gray, sigma, alpha, rng)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 364. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.22.

Purpose | What this contributes

Bends strokes locally by reading the image at positions moved by two smooth random fields.

Inputs

`img` the image to distort

`rng` a seeded random number generator, so a distortion can be reproduced

`alpha`
: how far pixels are moved, in pixels

`sigma`
: how smoothly the displacement varies, in pixels

Output. A locally bent version of the image.

Purpose | What it is for

Training augmentation. A network trained on a fixed set of examples can memorise them; showing it plausible variations of each example forces it to learn the general pattern instead. For handwriting the most useful variation is the one that looks like the same person writing the same words again, which is not a rotation or a scaling but a local bending of the strokes [22].

Method | How it works

Formula purpose and system use. Make a smooth random displacement field for controlled image deformation. **Source basis:** [22, 23, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$\begin{aligned} d_x &= \alpha \cdot \mathrm{Blur}(U(-1,1),\, \sigma) \\ d_y &= \alpha \cdot \mathrm{Blur}(U(-1,1),\, \sigma) \\ \mathrm{out}[y,x] &= \mathrm{sample}(\mathrm{img},\ y + d_y[y,x],\ x + d_x[y,x]) \end{aligned} \tag{5.18}$$

Source: [22]

where

d_x, d_y
: the displacement fields: how far the pixel at each position moves horizontally and vertically

$U(-1, 1)$
: an array of independent random numbers drawn uniformly between minus one and one, one per pixel

α
: the displacement magnitude in pixels

σ
: the blur width, which controls how smoothly the displacement varies from pixel to pixel

sample
: bilinear interpolation, since the displaced positions are fractional

Justification | Why the smoothing is the essential step

Without the blur, each pixel would move independently of its neighbours and the image would be scrambled into noise: strokes would break into disconnected dots. Blurring the displacement field means nearby pixels move by nearly the same amount, so a stroke moves as a stroke, bending and stretching while staying continuous. The width of the blur sets the scale of the bending: small gives wobbly strokes, large gives a gentle warp of the whole crop.

The two parameters interact. A large displacement with a small blur tears the writing apart; a large displacement with a large blur is nearly a rigid shift. Useful augmentation lives between those extremes.

5.25 DrawPolyline

```
def DrawPolyline(img, pts, value, width)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 377. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.22.

Purpose | What this contributes

Draws a polyline with a round pen, filling where the distance to the nearest point on a segment is within half the pen width.

Inputs

`img` the array to draw into, which is modified in place

`pts` the path as a list of (x, y) points

`value`
: the ink level to write, 0 being black

`width`
: the pen width in pixels

Output. The same array, returned for convenience when chaining calls.

Purpose | What it is for

Drawing a pen stroke into an image. It is used to render synthesised handwriting so that the networks can judge it, to redraw ink at a uniform width inside stroke normalisation, and in the measurement harness to compare two sets of geometry. Because all three use the same function, they all see ink laid down the same way.

Alternatives | The idea: fill by distance rather than by stepping along the line

The familiar way to draw a line is to step along it filling pixels. That is fast but makes a round pen awkward, because the ends and the corners between segments have to be handled separately or they come out square and notched.

Instead, every pixel near the segment is tested by how far it lies from the nearest point *on* the segment, and filled if that distance is within half the pen width. The nearest point is found by projecting onto the segment and clamping to its ends:

Formula purpose and system use. Estimate local coverage near a path segment when rasterising handwriting. **Source basis:** [22, 23, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$t \;=\; \operatorname{clamp}\!\left(\frac{(P-A)\cdot(B-A)}{\|B-A\|^2},\,0,\,1\right), \qquad Q \;=\; A + t\,(B-A) \tag{5.19}$$

where

A, B
: the two ends of the segment being drawn

P
: the pixel being tested

t
: how far along the segment the nearest point lies, 0 at A and 1 at B

Q
: the nearest point on the segment to P

$(P-A)\cdot(B-A)$
: the dot product, which measures how far along the segment's direction P lies

A pixel is then filled when the distance from P to Q is at most half the pen width.

Justification | Why the clamp gives round ends and joins for free

Without the clamp, t could fall outside the range 0 to 1 and the distance would be measured to the infinite line through A and B, so ink would extend beyond the ends of the segment. Clamping means that for a pixel beyond an end, the nearest point is the end itself, so the filled region there is a disc of radius half the pen width. That is a round cap. At a corner between two segments both discs overlap, which is a round join. No separate handling is needed for either.

Justification | Why it is affordable

Testing every pixel in the image against every segment would be hopeless. Work is restricted to the bounding box of each segment, expanded by the pen radius, since no pixel outside that box can possibly be within half a pen width of it. The cost is therefore proportional to the amount of ink drawn rather than to the size of the canvas.

A segment of zero length, which happens when a path contains a repeated point or consists of a single point, falls back to plain distance from that point, which draws a dot.

Stage: Sampling and geometric resampling

5.26 Method background: Sampling and geometric resampling

Purpose | The role in this system

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Justification | The reason for this construction

Isotropic scaling preserves aspect ratio; separable filtering and normalised edge weights make the resampler explicit. Widened shrink filters reduce aliasing.

Alternatives | What changes if another method is used

Nearest-neighbour suits labels but aliases grey strokes. Bilinear sampling is simple; bicubic or Lanczos changes ringing, sharpness and support. Independent axis stretching distorts writer shape.

Library | Implementation speed and accuracy

Pillow resize and OpenCV resize/warpAffine are available. Their kernels, centre coordinates, antialiasing and rounding need not match this implementation. They can reduce Python overhead; recognition accuracy must be evaluated after any changed pixels. [25, 26, 24].

5.27 _Weights

```
def _Weights(inSize, outSize, kernel)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 224. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

The resampling weights for one axis, the filter widens when shrinking so every input pixel still contributes.

Inputs

`inSize`
: the length of the axis before resizing

`outSize`
: the length wanted afterwards

`kernel`
: which filter to use: "linear" for a triangle, "box" for a flat window

Output. A matrix with one row per output sample and one column per input sample; each row holds the weight of every input sample in that output sample, and each row sums to one.

Method | Background: what resampling actually is

Changing the size of an image means asking for brightness values at positions where there were none. In an output image half the width, output pixel 1 sits between input pixels 2 and 3, so its value has

to be built from the inputs around it. Choosing how to build it is choosing a filter, and a filter is described by a weight for each nearby input sample.

Purpose | What it is for

Every size change in the pipeline goes through one resampler, so there is exactly one piece of code to get right: reducing a page for the skew search, fitting a crop to the networks' fixed input size, and rendering glyphs. This function builds the weights and `Resize` applies them.

Method | Where an output sample sits

Formula purpose and system use. Position output-pixel centres relative to source samples during scaling. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$s = \frac{\text{inSize}}{\text{outSize}}, \qquad c_i = \left(i + \tfrac{1}{2}\right) s \tag{5.20}$$

Source: [25]

where

s the scale factor; greater than one means the image is being shrunk

c_i the centre of output sample i, expressed in input coordinates

i the index of the output sample, counting from zero

The halves matter. A pixel is not a point at an integer coordinate; it is a small square whose centre lies at an integer plus a half. Treating pixels as points displaces the whole image by half a pixel, which shows as a slight softening and as a systematic shift that accumulates if an image is resized more than once.

Justification | Why the filter widens when shrinking

This is the part that is easy to get wrong and expensive to get wrong. The width of the filter is multiplied by the scale factor whenever that factor exceeds one:

Formula purpose and system use. Construct triangular interpolation weights at the appropriate shrink support. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$f = \max(1, s), \qquad \text{support} = k_0 \, f \tag{5.21}$$

Source: [25]

where

f the widening factor: one when magnifying, equal to the scale factor when minifying

s the scale factor defined above

k_0 the base support of the chosen kernel: one for the triangle, one half for the box

support
: how far the filter reaches from its centre, measured in input samples

If the filter did not widen, shrinking by a factor of four would consult one input sample in every four and ignore the rest. Thin strokes falling between the consulted samples would disappear entirely, and fine texture would turn into coarse false patterns, an effect called aliasing. Widening the filter in

step with the reduction means every input sample contributes to some output sample [25]. The page reduction used by the skew search shrinks by exactly such a factor, so this is load bearing rather than theoretical.

Method | The two kernels

Formula purpose and system use. Normalise each output row so a constant input remains constant. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$w_{\text{linear}}(t) = \begin{cases} 1 - |t| & |t| < 1 \\ 0 & \text{otherwise} \end{cases} \qquad w_{\text{box}}(t) = \begin{cases} 1 & |t| \leq \frac{1}{2} \\ 0 & \text{otherwise} \end{cases} \tag{5.22}$$

Source: [25]

where

t the distance from the centre of the output sample's window to an input sample, measured in filter widths

$w(t)$ the weight that input sample receives

The triangle weights nearby samples more than distant ones and falls smoothly to zero, which gives a smooth result. The box weights everything inside the window equally, which is exactly a local average and is the right choice when reducing by a whole number, because then each output pixel covers a whole number of input pixels.

Justification | Edges, and why weights are accumulated rather than assigned

A window near the border reaches past the end of the axis. Those out-of-range positions are folded back onto the first or last sample, so several positions in one window can refer to the same input sample. Their weights must therefore be added together rather than written one over another: assigning would keep only the last and discard the rest, leaving the edge of the image too dark.

Each row is finally divided by its own total so that it sums to one. That is what guarantees a uniform area keeps its brightness after resizing, and in particular that the white padding around a fitted crop does not come back grey.

5.28 Resize

```
def Resize(arr, newW, newH, kernel='linear')
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 243. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`img` the image to resize

`newW`
the width wanted

`newH`
: the height wanted

`kernel`
: which filter to use

Output. The resampled image at the requested size, in the same number format as the input.

Purpose | What it is for

The single resampling routine used by the whole pipeline.

Justification | Background: what separable means, and why it is worth it

Resizing in two dimensions could be done with a two-dimensional filter, where each output pixel is a weighted sum over a square of input pixels. For a filter k samples across that is k squared multiplications per output pixel.

A filter is called separable when its two-dimensional weight is the product of two one-dimensional weights, one per axis. Both kernels used here are separable, so the same result is obtained by resizing along the rows and then along the columns, at a cost of 2k multiplications per pixel instead of k squared [25]. At the filter widths used when reducing a page that is roughly an order of magnitude less work.

Method | How it works

`_Weights` is called once per axis and the two matrices are applied in turn as matrix products, so the arithmetic happens inside numpy's compiled routines rather than in interpreted loops.

5.29 Halve

```
def Halve(a)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 266. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`img` the image to reduce

Output. The image at half the width and half the height.

Purpose | What it is for

A fast reduction for stages that do not need full resolution: finding the sheet of paper inside the photograph, and the coarse stage of the skew search.

Justification | Why it is a separate function

At the exact ratio of two to one a box filter reduces to averaging each two-by-two block, which can be done by reshaping the array and taking a mean, with no weights built at all. It is both the cheapest

correct reduction at that ratio and exactly what the general resampler would compute, so nothing is lost by using it.

Why reduce at all: every operation in the toolkit costs time proportional to the number of pixels, so halving each axis quarters the work. For a decision that only needs to locate something to within a few pixels, the reduction costs nothing in accuracy.

5.30 FitFixedCanvas

```
def FitFixedCanvas(gray, height, width, padValue=255)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 274. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Puts a crop on the fixed classifier canvas, scaled by one factor for both axes and padded with paper.

Inputs

`gray`
: a line crop

`height`
: the canvas height the networks expect, 64

`width`
: the canvas width the networks expect, 640

`padValue`
: the value filling the unused area; 255 means white paper

Output. An image of exactly the requested size containing the crop, scaled to fit, centred vertically and flush to the left, on a paper-coloured background.

Purpose | What it is for

The current pipeline prepares crops on the trained fixed canvas. Convolution, adaptive pooling and recurrence do not by themselves require every source image to have identical dimensions, and this implementation does not justify the canvas by an image-flattened final layer. The fixed preparation instead supplies a consistent input distribution and sequence budget for these saved models.

Method | The central rule: one scale factor, never two

Formula purpose and system use. Choose one scale fitting both source dimensions inside the trained canvas. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$\sigma = \min\left(\frac{H_{\text{canvas}}}{H_{\text{crop}}}, \frac{W_{\text{canvas}}}{W_{\text{crop}}}\right) \tag{5.23}$$

where

σ the single scale factor applied to both axes

H_{canvas}, W_{canvas}
: the fixed canvas size the networks were trained on

H_{crop}, W_{crop}
: the size of the crop being fitted

Taking the smaller of the two ratios means the crop is scaled until whichever axis runs out of room first exactly fills the canvas, leaving the other axis short; the leftover space is filled with the paper colour.

The alternative, scaling each axis independently so the crop fills the canvas exactly, would change the ratio of width to height. That ratio is precisely what both networks read: the reader distinguishes letters partly by their proportions, and the writer namer treats proportion as a characteristic of a hand. A stretched crop is, to these networks, a different person's handwriting.

Justification | Why centred vertically but flush to the left

It matches how the training crops were produced. Writing starts at the left margin, so the left edge carries meaning and is preserved. The vertical position of a line of writing inside its crop carries none, so centring removes a difference the network would otherwise have to learn to ignore.

Evidence | A small implementation guard

The scaled size is rounded to whole pixels, and rounding can produce a result one pixel larger than the canvas. The final copy therefore limits both the source and the destination, so an overshoot is trimmed instead of raising an error.

5.31 _Sample

```
def _Sample(a, sx, sy, fill, edge='area')
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 288. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`img` the source image

`sy` for each output pixel, the row to read from, which may fall between rows

`sx` the corresponding column, which may fall between columns

Output. The sampled values, one per requested position.

Purpose | What it is for

Rotating or bending an image asks for brightness at positions falling between pixels. This supplies them, and is the shared sampler behind every geometric transform in the file.

Method | Background: bilinear interpolation

The value at a position between four pixels is their weighted average, each weight given by how close the position is to that pixel. It is computed one axis at a time: blend the upper pair horizontally, blend the lower pair horizontally, then blend those two results vertically.

Formula purpose and system use. Interpolate four neighbouring grey samples at a fractional source coordinate. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$\begin{aligned} v_{\text{top}} &= p_{00}\,(1 - f_x) + p_{01}\,f_x \\ v_{\text{bot}} &= p_{10}\,(1 - f_x) + p_{11}\,f_x \\ v &= v_{\text{top}}\,(1 - f_y) + v_{\text{bot}}\,f_y \end{aligned} \tag{5.24}$$

Source: [25]

where

p_{00}, p_{01}
: the two source pixels above the position, left and right

p_{10}, p_{11}
: the two below it

f_x
: how far the position lies between the left and right columns: 0 at the left pixel, 1 at the right

f_y
: the same fraction between the upper and lower rows

v
: the interpolated value returned

Justification | Why interpolate at all

The cheaper alternative is to take whichever single pixel the position is nearest. On a rotated image the result is a visible staircase along every edge, because neighbouring output pixels jump between different source pixels. Blending removes it, and that matters here beyond appearance: the ink masks and skeletons computed downstream are sensitive to jagged edges, which produce spurious spurs when thinned.

5.32 _Like

```
def _Like(out, like)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 303. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`out` a floating point result

`ref` the array whose number format should be matched

Output. The result converted back to the reference's format, rounded and clipped to its range.

Purpose | What it is for

Interpolation must be done in floating point, but the pipeline passes images around as whole numbers from 0 to 255, because that is what a photograph is and what the networks were trained on. This performs the conversion back.

Method | The two things it gets right

It rounds rather than truncating. Truncation would bias every transformed image slightly darker, since 254.7 would become 254 rather than 255, and the bias would accumulate through successive transforms.

It clips to the valid range. A blend can overshoot slightly past 255 or below 0, and converting such a value to an 8-bit whole number without clipping makes it wrap around to the opposite extreme, turning a bright pixel black. A handful of such pixels is enough to disturb a threshold or a component count.

5.33 Rotate

```
def Rotate(gray, deg, fill=255)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 309. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`img` the image to rotate

`deg` the angle in degrees

`fill`
: the value used where no source pixel exists

Output. The rotated image, on a canvas the same size as the input.

Purpose | What it is for

Straightening a photographed page. The page is never square to the camera, and a tilt of even one degree means a horizontal cut through the image crosses more than one line of writing, which makes every later stage harder.

Method | Background: forward and inverse mapping

There are two ways to transform an image. The obvious one, called forward mapping, takes each input pixel, works out where it should land, and writes it there. The problem is that the destinations are fractional and unevenly spread, so some output pixels receive two input pixels and others receive none, leaving a scatter of holes.

The method used here is inverse mapping: for each *output* pixel, work out where it came from in the input and read that position. Every output pixel is then visited exactly once, so holes are impossible [25]. The source position is fractional, which is what `_Sample` is for.

Method | The mapping

Formula purpose and system use. Map a rotated output position back to its source sample location. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$\begin{aligned} s_x &= \cos\theta\,(x - c_x) + \sin\theta\,(y - c_y) + c_x \\ s_y &= -\sin\theta\,(x - c_x) + \cos\theta\,(y - c_y) + c_y \end{aligned} \tag{5.25}$$

Source: [25]

where

x, y the output pixel being filled

s_x, s_y
: the position in the input it is read from

c_x, c_y
: the centre of the image, which the rotation is about

θ the rotation angle in radians

Note that the signs are those of a rotation by minus theta: because the mapping runs backwards from output to input, it must be the inverse of the rotation being applied.

Alternatives | Filling rather than clamping

A source position outside the image is filled with the paper value rather than clamped to the nearest edge pixel. Clamping would smear whatever lies on the boundary across the whole corner, producing streaks that the ink detector would then have to cope with.

Justification | The edge convention, and why it was determined experimentally

Deciding whether a source position lies inside the image is less obvious than it looks. If a pixel is a point at an integer coordinate, the image spans 0 to size minus 1. If a pixel is a unit square centred on its index, the image spans minus a half to size minus a half. This implementation uses the second, area, convention.

> *The convention was not guessed. Replacing a library rotation with this one was verified by comparing the two on a test page: with the area convention the two agreed to a maximum difference of one grey level out of 255, while the other plausible convention differed by up to 245 along the edges. A difference of that size is not a rounding detail, it is a row of pixels in the wrong place.*

5.34 Warp

```
def Warp(gray, coeffs, fill=255)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 322. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`img` the source image

`coeffs`
: six numbers for an affine transform, or eight for a perspective one

`fill`
: the background value

Output. The transformed image.

Purpose | What it is for

One function covering every straight-line-preserving transform the pipeline needs: scaling, shearing, rotation, and the keystone effect of a page photographed from an angle. It is used mainly to generate training variations of a crop.

Method | Background: affine and perspective

An affine transform is any combination of scaling, rotation, shear and translation. It keeps parallel lines parallel, so a rectangle becomes a parallelogram. A perspective transform, also called a homography, additionally allows parallel lines to converge, so a rectangle becomes any four-sided shape. That is what happens when a flat page is photographed obliquely: the far edge appears shorter than the near one.

Method | The mapping

Formula purpose and system use. Apply the inverse projective coordinate mapping used by a resampled image. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$s_x = \frac{a\,x + b\,y + c}{g\,x + h\,y + 1}, \qquad s_y = \frac{d\,x + e\,y + f}{g\,x + h\,y + 1} \tag{5.26}$$

Source: [25]

where

x, y the output pixel being filled

s_x, s_y
: the source position it is read from

a, b, d, e
: the linear part: scaling, rotation and shear

c, f the translation

g, h the perspective part; both zero gives a purely affine transform, since the divisor is then 1 everywhere

The divisor is what distinguishes the two cases. When g and h are zero it equals one at every pixel and the mapping is linear; otherwise it varies across the image, so equal steps in the output correspond to unequal steps in the input, which is exactly the foreshortening that perspective produces.

Method | The half-pixel convention

The coefficients act on the *centre* of the output pixel, so a half is added to the output coordinate before the mapping is applied and subtracted from the result afterwards. This follows from the same pixels-are-squares reasoning as in `_Weights`.

> *Like the rotation convention, this was settled by measurement rather than argument: with the half-pixel offsets in place the output matched the library implementation it replaced on zero pixels of difference across a test page, and without them the entire image was displaced by half a pixel.*

Stage: Line pitch and row profiles

5.35 Method background: Line pitch and row profiles

Purpose | The role in this system

Locate line centres and choose scale-aware segmentation distances before character recognition.

Justification | The reason for this construction

Projection peaks, a letter-height floor, median gaps and autocorrelation supply interpretable estimates when line layouts vary.

Alternatives | What changes if another method is used

Straight minima work for clean separate rows but cut overlapping strokes. Hough line estimates favour straighter structures. A learned line detector needs annotated regions and another model.

Library | Implementation speed and accuracy

SciPy find_peaks/correlate can supply primitives, not the complete floor/fallback/merging policy. Prominence and distance settings must be matched. Better peak detection is not automatically better ink ownership. [11, 27].

5.36 CoreBand

```
def CoreBand(ink)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 130. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.35.

Purpose | What this contributes

Locate line centres and choose scale-aware segmentation distances before character recognition.

Inputs

`ink` a mask of one line of writing

Output. A pair of row indices marking the top and the baseline of the x-height band, or nothing at all if there is no ink or the band is under three rows tall.

Method | Background: what the x-height is

In any piece of writing, most lower case letters have a body of roughly the same height, such as a, c, e, m and o. The height of that body, measured up from the line the writing sits on, is called the x-height, after the letter x. Some letters reach above it, such as b, d and l, and these are called ascenders; some reach below the line, such as g, p and y, and these are called descenders.

Purpose | What it is for, and why it is so important

The x-height is the unit of measurement for the entire pipeline. Every stored letter shape is divided by it, so that a letter extracted from a photograph at one resolution can be drawn at any physical

size. Every proportion in a style profile is expressed as a multiple of it: how far ascenders reach, how far descenders drop, how wide the gaps between words are, how thick the pen is. Without a reliable x-height none of those numbers would be comparable between two writers or between two photographs of the same writer.

Method | How it works

Counting the ink in each row of the crop gives a one-dimensional summary called a projection profile. Rows that cross the bodies of the letters contain ink from nearly every letter on the line; rows that contain only ascenders or only descenders contain ink from the few letters that have them. The band is therefore the set of rows whose ink count is a large fraction of the maximum:

Formula purpose and system use. Estimate a lower-case body band from rows exceeding a fraction of peak ink. **Source basis:** [11, 27]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathcal{B} \;=\; \Big\{\, y \;:\; \mathrm{prof}[y] \;\geq\; 0.45 \max_{y'} \mathrm{prof}[y'] \,\Big\} \tag{5.27}$$

where

$\mathcal{B}$ the set of rows making up the band; its first and last members are returned

$\mathrm{prof}[y]$
: the number of ink pixels in row y

0.45 the fraction of the peak that a row must reach to count as part of the band

Method | Where the 0.45 comes from

This fraction thresholds the row-ink profile to estimate the letter-body band. A clear gap between body and ascender densities can make the result insensitive within an interval, but sparse text, capitals and unusual writing can remove that gap. Lower values admit more rows; higher values can collapse the band. Its exact value is a project heuristic, not an independently established measurement-error threshold.

Method | The rejection case

A band under three rows tall is reported as a failure rather than returned. That is what happens when a crop contains a stray mark rather than a line of writing, and returning a nonsensical x-height would propagate into every measurement taken from that crop.

5.37 _LetterHeight

```
def _LetterHeight(mask)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 464. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.35.

Purpose | What this contributes

Connected-piece sizes provide a typical letter scale when row-peak spacing might describe ascenders and descenders of one line instead of separate lines. It is a fallback measurement, not a recognised character label.

Method | Current implementation

Locate line centres and choose scale-aware segmentation distances before character recognition. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _LetterHeight(mask):
    lab,nc = LabelComponents(mask)
    if nc<=0:
        return None
    ys,xs = np.nonzero(mask)
    pl = lab[ys,xs]
    top,bot = np.full(nc+1,mask.shape[0]),np.zeros(nc+1,int)
    np.minimum.at(top,pl,ys)
    np.maximum.at(bot,pl,ys)
    area = np.bincount(pl,minlength=nc+1)
    hts = bot-top+1
    sel = (area>=8)&(hts>=3)&(hts<=mask.shape[0])
    if not sel[1:].any():
        return None
    return float(np.median(hts[1:][sel[1:]]))
```

Justification | Interpret the method rather than infer a benchmark

Projection peaks, a letter-height floor, median gaps and autocorrelation supply interpretable estimates when line layouts vary. [11, 27].

5.38 LinePitch

```
def LinePitch(mask)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 484. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.35.

Purpose | What this contributes

The current pitch floor is PEAK_MIN_PITCH_XH times estimated letter height. Candidate peaks exceed six percent of the smoothed maximum. Median adjacent spacing is accepted only if above the floor; otherwise autocorrelation, a 2.2-times-letter-height fallback and finally 48 pixels are tried. The retired TRUST_ALL_PEAK_GAPS bypass is absent.

Method | Current implementation

Locate line centres and choose scale-aware segmentation distances before character recognition. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def LinePitch(mask):
    W = mask.shape[1]
    prof = mask[:,int(0.05*W):int(0.97*W)].sum(1).astype(float)
    k = np.exp(-0.5*(np.arange(-12,13)/4.0)**2)
    sm = np.convolve(prof,k/k.sum(),"same")

    h = _LetterHeight(mask)
    floor = (PEAK_MIN_PITCH_XH*h) if h else 0.0   #a gap below this is within one line

    peaks = [i for i in range(1,len(sm)-1) if sm[i]>=sm[i-1] and sm[i]>sm[i+1] and sm[i]>0.06*sm.max
    ()]
```

```
    if len(peaks)>=2:
        p = int(np.median(np.diff(sorted(peaks))))
        if p>=floor:
            return p,sm

    a = sm-sm.mean()
    ac = np.correlate(a,a,"full")[len(a)-1:]
    seg = ac[15:min(200,len(ac)//3)]
    idx = next((i for i in range(1,len(seg)-1)
                if seg[i]>=0.7*seg.max() and seg[i]>=seg[i-1] and seg[i]>=seg[i+1]), None)
    if idx is not None and 15+idx>=floor:
        return 15+idx,sm

    if h:
        return max(MIN_LINE_PITCH,int(round(2.2*h))),sm
    return 48,sm
```

Justification | Interpret the method rather than infer a benchmark

Projection peaks, a letter-height floor, median gaps and autocorrelation supply interpretable estimates when line layouts vary. [11, 27].

5.39 LineCentres

```
def LineCentres(mask, H, mergeValley=None)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 632. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.35.

Purpose | What this contributes

Strongest-first peak suppression enforces a separation of 0.6 pitch. The optional mergeValley parameter controls repeated merging of neighbours with shallow intervening valleys. At the module default 1.0 the merging loop does not run. It is distinct from seam optimisation.

Method | Current implementation

Locate line centres and choose scale-aware segmentation distances before character recognition. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def LineCentres(mask,H,mergeValley=None):
    P,sm = LinePitch(mask)
    peaks = [i for i in range(1,H-1) if sm[i]>=sm[i-1] and sm[i]>sm[i+1] and sm[i]>0.06*sm.max()]
    centres = []
    for c in sorted(peaks,key=lambda i:-sm[i]):
        if all(abs(c-o)>=0.6*P for o in centres):
            centres.append(c)
    centres.sort()

    thr = LINE_MERGE_VALLEY if mergeValley is None else float(mergeValley)
    while len(centres)>1 and thr<1.0:
        ratios = [(float(sm[a:b+1].min())/max(float(min(sm[a],sm[b])),1e-9),i)
                  for i,(a,b) in enumerate(zip(centres,centres[1:]))]
        worst,i = max(ratios)
        if worst<=thr:
            break
        centres.pop(i+1 if sm[centres[i]]>=sm[centres[i+1]] else i)
```

```
    return P,centres
```

Justification | Interpret the method rather than infer a benchmark

Projection peaks, a letter-height floor, median gaps and autocorrelation supply interpretable estimates when line layouts vary. [11, 27].

Stage: Orientation and skew estimation

Report equation 15 Deskew concentration

Method | Formula

$\theta^* = \arg\max_\theta \sum_y p_\theta(y)^2$ with $p_\theta(y) = \sum_x M_\theta(y, x)$.

Purpose | Why the system needs it

Prefer rotations concentrating horizontal text into fewer rows. The selected angle grid is a project search budget, not a general guarantee on curved or perspective-distorted writing.

Evidence | Current location and source

SkewScore and Deskew; do not confuse with slant estimation. [11].

5.40 Method background: Orientation and skew estimation

Purpose | The role in this system

Align the page line axis while preserving the writer slant needed for reproduction.

Justification | The reason for this construction

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up.

Alternatives | What changes if another method is used

Hough estimates and connected-component baselines can estimate skew; perspective rectification handles projective distortion that one rotation cannot. No formal evidence makes the selected angle grid universal.

Library | Implementation speed and accuracy

OpenCV warpAffine or Pillow rotate can apply a chosen angle, but neither substitutes for the project score and search. Interpolation and expansion affect the classifier input. [11, 25, 24, 6].

5.41 EstimateSlantDeg

```
def EstimateSlantDeg(ink)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 405. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

Shears the ink by each trial angle and keeps the angle whose column profile is sharpest.

Inputs

`ink` a mask of one line of writing

Output. The slant in degrees, positive for the usual forward lean. Returns zero when there are fewer than fifty ink pixels.

Purpose | What it is for

Two things. The glyph extractor removes the slant before storing a letter, so that stored letters are upright and can be compared with one another; and the synthesiser re-applies the writer's own slant when drawing, because how far somebody leans their writing is one of the most visible features of their hand.

Method | Background: what shearing is

Shearing slides each row of an image sideways by an amount proportional to its height, which tilts upright strokes without changing the height of anything. It is the operation that turns upright writing into slanted writing, so undoing it is how slanted writing is made upright.

Method | The method: try angles and score the result

For each candidate angle the ink is sheared by that amount and the ink in each column is counted, giving a column profile. The angle whose profile is the *sharpest* is the answer [28]:

Formula purpose and system use. Select the shear that makes stroke columns most concentrated. **Source basis:** [11, 25, 24, 6]. Exact project constants remain implementation choices unless evidence is stated.

$$x' \;=\; x - \tan(\theta)\,(y - \bar{y}), \qquad \text{score}(\theta) \;=\; \sum_{j} \text{prof}_{\theta}[j]^2 \tag{5.28}$$

Source: [28]

where

x, y the coordinates of an ink pixel

x' its column after shearing by the trial angle

$\bar{y}$ the mean row of all the ink, which is the height the shear pivots about

θ the trial angle

$\text{prof}_{\theta}[j]$
: how many ink pixels land in column j after that shear

$\text{score}(\theta)$
: the sharpness being maximised

Justification | Why squaring the profile measures sharpness

This is the key step and it is worth being precise about. The total amount of ink does not change when the image is sheared, so the sum of the profile is the same for every angle; only its *distribution*

changes. For a fixed total, the sum of squares is largest when the quantity is concentrated in a few entries and smallest when it is spread evenly. Ten pixels in one column score a hundred; one pixel in each of ten columns scores ten.

So maximising the sum of squares finds the angle at which the ink stacks into the fewest, tallest columns, which is the angle at which the vertical strokes of the writing are truly vertical. The same reasoning underlies `SkewScore`, applied to rows instead of columns.

Justification | Why coordinates are sheared rather than the image

The score needs only the column each ink pixel lands in, not an image. Shearing the coordinates and counting them is therefore enough, and it avoids interpolating a whole image once per candidate angle. With sixty candidate angles that is the difference between a measurement that is free and one that dominates the runtime.

Method | Resolution and the sign

The search runs from minus 45 to plus 45 degrees in steps of 1.5 degrees, which bounds how precisely the answer can be reported. The result is negated before being returned so that a positive number means the ordinary forward lean, which makes the deslanting step read naturally as subtracting the tangent of the slant times the height.

Fewer than fifty ink pixels is treated as too little to measure and zero is returned, since on that much ink the profile is dominated by where the individual pixels happen to fall.

5.42 SkewScore

```
def SkewScore(mask, angleDeg)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 544. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

The sum of the squared ink per row at a trial angle, which peaks when the page is level.

Inputs

`mask`
: the ink mask of the page

`angleDeg`
: the trial rotation angle in degrees

Output. a single number: how sharply the ink piles into rows when the page is considered at this angle. Larger is better aligned.

Purpose | What it is for

A photographed page is never perfectly square to the camera, so the lines of writing run slightly uphill or downhill. Every later stage assumes they are horizontal: the row profile is used to find lines, horizontal seams are used to separate them, and a crop is a horizontal strip. A tilt of even two degrees smears the profile enough to merge neighbouring lines. This function provides the measurement that lets the tilt be found: given a candidate angle, it scores how level the page would look at that angle.

Method | The principle: concentration

Consider the row profile again, the count of ink pixels per row. When the page is level, every letter on a line contributes to the same few rows, so those rows have very large counts and the rows between lines have almost none. When the page is tilted, the left end of a line sits higher than the right end, so the same ink is spread over many more rows, each with a smaller count. The total amount of ink is identical either way, since rotating the page does not create or destroy pixels. What changes is how **concentrated** the profile is. So the correct angle is the one that concentrates the profile most, and all that is needed is a number that measures concentration.

Justification | Why the sum of squares measures concentration

Squaring, then summing, does it. Squaring rewards large values disproportionately: doubling a count quadruples its contribution. So for a fixed total, the sum of squares is largest when the total is piled into as few entries as possible and smallest when it is spread evenly.

Formula purpose and system use. Project rotated foreground points into candidate image rows. **Source basis:** [11, 25, 24, 6]. Exact project constants remain implementation choices unless evidence is stated.

$$S(\theta) \;=\; \sum_{k} \mathrm{prof}_{\theta}[k]^{2} \tag{5.29}$$

where

$S(\theta)$
: the skew score at trial angle theta

$\mathrm{prof}_{\theta}[k]$
: the number of ink pixels that fall into row index k once the page is considered rotated by theta

k
: the row index in the rotated frame

A numerical illustration makes the behaviour concrete. Suppose 600 ink pixels belong to one line. Level, they might fall into 4 rows of 150, giving a score of $4{\cdot}150^2 = 90\{,\}000$. Tilted, the same 600 might spread over 20 rows of 30, giving $20{\cdot}30^2 = 18\{,\}000$. The totals match at 600 but the scores differ fivefold, and the larger score correctly identifies the level case. The sum of squares is used in the same spirit elsewhere in the project, where EstimateSlantDeg uses it to find the slant of letters rather than the skew of lines.

Method | Scoring an angle without rotating the image

Rotating the whole image for each candidate angle would mean interpolating every pixel, seventeen times over for the seventeen angles tried. That is unnecessary. Only the row index each ink pixel would land in is needed, not an image, and that index follows from one line of trigonometry. The standard rotation of a point about a centre gives the rotated row as

Formula purpose and system use. Measure row-profile concentration to compare deskew angles. **Source basis:** [11, 25, 24, 6]. Exact project constants remain implementation choices unless evidence is stated.

$$k \;=\; \mathrm{round}\Big((y - c_y)\cos\theta \;-\; (x - c_x)\sin\theta \;+\; c_y\Big) \tag{5.30}$$

where

k
: the row the pixel lands in after rotation

y, x
: the pixel's original row and column

c_y, c_x
: the centre of rotation, taken as the centre of the image

θ
: the rotation angle in radians, converted from the degrees supplied

cos, sin
: the cosine and sine functions, which express how much of each original axis contributes to the new vertical position

round
: rounding to the nearest whole number, since rows are counted in whole pixels

Only the row is computed; the column is never needed, because the profile sums over columns anyway. Counting how many pixels land in each row is then a single tally of the k values, so the cost of scoring one angle is proportional to the number of ink pixels rather than to the number of pixels in the image. Since ink is a small fraction of a page, that is a large saving on top of avoiding the interpolation.

Evidence | A subtle trap: never clip the row index

Rotation moves some pixels outside the original image rectangle, so k can come out negative or beyond the image height. The obvious response, forcing k into the valid range, is badly wrong, and the code carries a comment saying so. Clipping piles every out-of-range pixel into the first or last row. That creates an enormous artificial count in one row, and because the score squares the counts, this fake spike scores higher than any genuine line of text. The effect grows with the angle, since larger angles push more pixels out of range, so the search would reliably select the most extreme angle offered. The code instead shifts the indices so the smallest becomes zero, which keeps every pixel in its own distinct row and lets the profile grow as tall as it needs to be.

> *This is a good example of why a plausible-looking defensive measure can be worse than none. Clipping an index is usually the safe thing to do; here it silently converts a correct measurement into one that is guaranteed to give the wrong answer.*

5.43 Deskew

```
def Deskew(page)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 560. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

The page is reduced by a two-to-one box average; a further reduction supplies the trial mask. Seventeen trial angles from minus four to plus four degrees are scored, then both full and working grey images are rotated and the mask rebuilt. This function neither accepts nor records a timer.

Method | Current implementation

Align the page line axis while preserving the writer slant needed for reproduction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Deskew(page):
    full = np.asarray(page,dtype=np.uint8)
    if full.ndim==3:
        full = np.rint(full[...,:3].mean(2)).astype(np.uint8)
```

```
    halfA = Halve(full)

    mask,_ = InkMask(halfA.astype(np.float64))
    q = Halve(mask.astype(np.uint8)*255)>127
    best = max(np.arange(-4,4.01,0.5),key=lambda a: SkewScore(q,a))
    fullA = Rotate(full,best,fill=255)
    halfA = Rotate(halfA,best,fill=255)
    g = halfA.astype(np.float64)
    mask,ratio = InkMask(g)
    return fullA,halfA,mask,ratio,best
```

Justification | Interpret the method rather than infer a benchmark

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up. [11, 25, 24, 6].

5.44 _SheetCrop

```
def _SheetCrop(g)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 944. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

This helper supplies a central paper-oriented crop for the orientation score. It changes evidence supplied to axis estimation, not the final physical page boundary by itself.

Method | Current implementation

Align the page line axis while preserving the writer slant needed for reproduction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _SheetCrop(g):
    bright = g>0.60*np.percentile(g,95)
    lab,n = LabelComponents(bright)
    if n==0:
        return g
    i = np.bincount(lab.ravel())[1:].argmax()+1
    yy,xx = np.nonzero(lab==i)
    return g[yy.min():yy.max()+1,xx.min():xx.max()+1]
```

Justification | Interpret the method rather than infer a benchmark

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up. [11, 25, 24, 6].

5.45 LineAxisScore

```
def LineAxisScore(page)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 957. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

The implemented score is the fraction of row sums below 0.35 times their mean after downsampling and sheet cropping. It is not the coefficient-of-variation score described in some older comments. Empty/very small masks produce zero.

Method | Current implementation

Align the page line axis while preserving the writer slant needed for reproduction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def LineAxisScore(page):
    g = np.asarray(page)[::4,::4].astype(np.float64)
    g = _SheetCrop(g)
    if g.size<100 or min(g.shape)<8:
        return 0.0
    mask,_ratio = InkMask(g)
    rows = mask.sum(axis=1).astype(np.float64)
    m = rows.mean()
    return float((rows<0.35*m).mean()) if m>0 else 0.0
```

Justification | Interpret the method rather than infer a benchmark

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up. [11, 25, 24, 6].

5.46 UprightPage

```
def UprightPage(page)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 974. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

Scores the original and a ninety-degree rotation and accepts the turn only if the rotated score exceeds 1.35 times the guarded original score. It resolves the writing axis, not a one-hundred-eighty-degree inversion.

Method | Current implementation

Align the page line axis while preserving the writer slant needed for reproduction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def UprightPage(page):
    a = np.asarray(page)
    s0 = LineAxisScore(a)
    s90 = LineAxisScore(np.rot90(a))
    if s90>AXIS_TURN_MARGIN*max(s0,1e-9):
        return np.ascontiguousarray(np.rot90(a)),90,s0,s90
    return a,0,s0,s90
```

Justification | Interpret the method rather than infer a benchmark

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up. [11, 25, 24, 6].

Stage: Minimum cost seams

Report equation 16 Local seam cost

Method | Formula

$C(y, x) = 4M_c(y, x) + 0.012[(y - y_m)/h_g]^2$.

Purpose | Why the system needs it

Avoid closed-mask ink while weakly attracting the boundary toward the middle of its corridor. The dimensionless weights encode project priorities; the literature does not establish these exact coefficients.

Evidence | Current location and source

SeamCuts. [29, 3].

Report equation 17 Seam recurrence

Method | Formula

$D(y, x) = C(y, x) + \min_d[D(y + d, x - 1) + 0.02|d|]$.

Purpose | Why the system needs it

Compare complete legal prefixes rather than greedily selecting one local pixel. Backtracking recovers the route; the optimum is limited to the chosen corridor and transitions. SeamCuts explicitly calls step 7 despite the LeastCostSeam default 3.

Evidence | Current location and source

LeastCostSeam and SeamCuts. [12, 3].

5.47 Method background: Minimum cost seams

Purpose | The role in this system

Find curved boundaries between lines without relying on a completely blank horizontal row.

Justification | The reason for this construction

Dynamic programming minimises accumulated ink, location and vertical-step costs inside a corridor, with saved predecessors recovering the actual path.

Alternatives | What changes if another method is used

Straight cuts are cheaper but cannot bend around ascenders. Graph cuts or shortest-path packages offer more general models with added setup and constraints. Learned segmentation needs labelled boundary examples.

Library | Implementation speed and accuracy

A SciPy shortest-path representation could encode the grid, but materialising a graph is unnecessary overhead for this acyclic recurrence. Compiling the same recurrence may reduce loop cost; it does not validate its ink weights. [12, 29, 30].

5.48 LeastCostSeam

```
def LeastCostSeam(cost, step=3)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 515. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.47.

Purpose | What this contributes

The cheapest path left to right across a cost field, keeping the best running cost of reaching each row and backtracking at the end.

Inputs

`cost`
: a 2-D array in which every entry is the price of passing through that pixel; here, large where there is ink and small where there is not

`step`
: the largest number of rows the path may move up or down when advancing one column, default 3

Output. an array with one entry per column, giving the row the path occupies in that column: the cheapest left-to-right route across the array.

Purpose | What it is for

Two lines of handwriting have to be separated from one another. A straight horizontal cut will not do it, because the descender of a g on the upper line hangs below the ascender of an l on the lower one, and any single row that clears the g also slices the l. What is needed is a wavy cut that dodges around the ink: it should dip below a descender and rise above an ascender, as long as it keeps moving left to right. That wavy cut is called a **seam**, and finding the cheapest one is what this function does. The technique is seam carving, introduced for resizing images [30] and adapted to the separation of handwritten text lines [29].

Purpose | The problem stated precisely

A seam is a choice of one row for each column, with the restriction that consecutive choices differ by no more than `step` rows, which is what makes the result a connected curve rather than a scatter of disconnected points. Its cost is the sum of the cost array along it. The task is to find the seam of least total cost.

Formula purpose and system use. Accumulate the minimum cost of reaching a row from legal predecessor rows. **Source basis:** [12, 29, 30]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathrm{Cost}(y_1, \ldots, y_W) \;=\; \sum_{x=1}^{W} \mathrm{cost}[y_x,\, x], \qquad |y_{x+1} - y_x| \le \mathrm{step} \tag{5.31}$$

where

y_x the row the seam occupies in column x

W the number of columns

cost$[y, x]$
: the price of the pixel at row y, column x

step
: the maximum vertical move allowed between neighbouring columns

Justification | Why brute force is impossible

Counting the candidates shows why a direct search is out of the question. With h rows to choose from in the first column and up to 2·step+1 choices at each of the remaining W−1 columns, the number of seams is about h(2·step+1)^(W−1). For a modest page half a size down, with 1200 columns and step 7, that is 15 raised to the power 1199, a number with over a thousand digits. No computer will enumerate them.

Method | The idea that makes it easy: dynamic programming

Dynamic programming is a method for problems that can be solved by building up the answer one step at a time, where the best way to finish does not depend on how the start was reached [12]. That condition holds here. Suppose the cheapest way to reach row y of column x has been computed, for every row y. The cheapest way to reach a particular row of column x+1 must arrive from one of the rows of column x that is within `step` of it, and once it is known how cheaply each of those rows can be reached, the best arrival is just the smallest of those costs plus the price of the new pixel. Nothing about the earlier part of the route matters beyond its total cost. So the whole column can be solved from the previous column alone.

Formula purpose and system use. Recover the minimum-cost route through stored predecessor offsets. **Source basis:** [12, 29, 30]. Exact project constants remain implementation choices unless evidence is stated.

$$A[y, x] \;=\; \text{cost}[y, x] \;+\; \min_{|d| \le \text{step}} \Big(A[y - d,\, x - 1] \;+\; \lambda\, |d| \Big) \tag{5.32}$$

where

$A[y, x]$
: the accumulated cost: the price of the cheapest route that starts at the left edge and ends at row y of column x

cost$[y, x]$
: the price of the current pixel

d
: the vertical move made to arrive here, so the route came from row y−d of the previous column

step
: the limit on that move

λ
: a small penalty per row of vertical movement, 0.02 in this implementation

min
: the smallest value over all the choices of d

The first column needs no recursion, since a route that ends there has visited only that one pixel, so A in column 1 is just the cost in column 1. Every later column follows from its predecessor by the formula above. The work is one pass over the array, with 2·step+1 comparisons per pixel, instead of the astronomical enumeration. That is the entire gain of dynamic programming: it replaces an exponential count of paths with a linear count of pixels.

Method | The movement penalty

The term $\lambda|d|$ charges a small amount for each row of vertical movement. Without it, any two routes through equally empty space would cost the same, and the seam would wander about aimlessly in white space, picking an arbitrary one of the many equally cheap options. The penalty breaks

those ties in favour of going straight, so that the seam moves only when moving buys it something, namely avoiding ink. At 0.02 per row it is far smaller than the cost of crossing a stroke, so it never persuades the seam to cut through a letter in order to stay level; it only decides between routes that are otherwise equivalent. This is a **regularisation** term, which is the general name for a small extra cost added to express a preference among solutions that the main objective cannot distinguish.

Method | Recovering the path

The accumulated cost gives the price of the best seam but not its route, so as each cell is filled the winning move is recorded in a second array. When the pass is complete, the lowest value in the final column is the end of the best seam, and following the stored moves backwards from there reconstructs the whole path, column by column, right to left. This is **backtracking**, and it is the standard companion to dynamic programming: the forward pass computes the value of the best solution, the backward pass reads off what that solution actually was.

Method | How it is made fast in practice

Written as a loop over rows inside a loop over columns this would be slow in Python. Instead, for each column, the candidate costs from all 2·step+1 possible moves are formed as whole shifted copies of the previous column, stacked into a small array, and the minimum is taken down the stack in one operation. Shifting a column by d rows and adding the penalty handles every row's d-th candidate at once. Only the loop over columns remains in Python, because columns are genuinely sequential: column x cannot be computed before column x−1.

> *The shifted copies are filled with infinity where the shift runs off the end of the array. Infinity is the natural value for an impossible move, because taking a minimum automatically never selects it, with no need for a special case in the code.*

5.49 SeamCuts

```
def SeamCuts(mask, centres, P, H, W)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 656. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.47.

Purpose | What this contributes

A least-ink seam between each pair of line centres, so the cut goes around descenders instead of through them.

Inputs

`mask`
: the cleaned ink mask

`centres`
: the row positions of the lines, from LineCentres

`P`
: the line pitch in pixels

`H`
: the mask height

`W`
: the mask width

Output. three items: the cuts as a list of one row-per-column array each, the same cuts stacked into a single 2-D array, and L, the number of bands the cuts divide the page into.

Purpose | What it is for

With the lines located, boundaries have to be drawn between them. This function produces one boundary curve per gap, each given as the row it occupies in every column, so that the page is divided into L horizontal bands with one line of writing in each. The curves are seams rather than straight rows, for the reason given in the entry for LeastCostSeam: a straight cut between two lines of handwriting cannot avoid slicing through the descenders of the upper line or the ascenders of the lower one, because at some column they overlap vertically.

Library | The cost landscape

A seam needs a cost array, and the art of this function is in building one whose cheapest path is the cut that a person would draw. It has two terms.

Formula purpose and system use. Price ink crossings strongly while weakly preferring the inter-line midpoint. **Source basis:** [12, 29, 30]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{cost}[y, x] \;=\; 4.0 \cdot m[y, x] \;+\; 0.012 \left(\frac{y - y_{\text{mid}}}{g} \right)^2 \tag{5.33}$$

where

$\text{cost}[y, x]$
: the price of routing the cut through this pixel

$m[y, x]$
: 1 where there is ink in the closed mask, 0 where there is not

4.0
: the price of crossing a single ink pixel

y_{mid}
: the row midway between the two line centres this cut separates

g
: half the distance between those two centres, used to express the displacement in units of the gap rather than in pixels

0.012
: the strength of the pull back toward the midline

The first term is the one that does the work: ink is expensive, so the cheapest route goes around letters. The second is a weak **quadratic penalty** on straying from the middle of the gap, quadratic meaning it grows with the square of the displacement, so small excursions are nearly free while large ones become progressively more costly. Its purpose is to settle what happens in genuinely empty space. Between two words there is no ink at all and every route is equally cheap on the first term alone, so without the second term the cut would wander to an arbitrary height and arrive at the next word from a silly position. The pull keeps it near the middle of the gap, ready to deflect in either direction when the next letter arrives.

The ratio between the two coefficients is what matters, not either value alone. At 4.0 against 0.012, crossing one ink pixel costs as much as sitting about eighteen normalised units off the midline, which is far outside the gap. So the cut will always prefer a long detour through white space to cutting a stroke, and the midline term only ever acts as a tie-breaker. Expressing the displacement in units of the half-gap rather than in pixels keeps that balance the same for widely and narrowly spaced writing.

Method | Closing the mask first

The mask is closed, dilated then eroded by about 0.07 pitches, before being used as cost. The reason is a specific failure mode: handwriting has narrow gaps inside it, between the two strokes of an n or where a pen lifted, and a seam is happy to thread through such a gap, because threading it is free while going round costs ink. The cut then passes through the middle of a letter. Closing fills gaps narrower than the structuring element while leaving the outer shape of the stroke nearly unchanged, so the threading routes disappear but the cut still traces the true outline of the letter.

Closing rather than plain dilation is the point. Dilation alone would also close the gaps, but it would leave every stroke thickened by the full radius, and the cut would then trace a bloated blob standing well clear of the real ink, wasting vertical space in both crops. The erosion that follows the dilation undoes the thickening in the places where it was not needed, which is exactly the property that distinguishes a closing from a dilation.

Evidence | The step limit

The seam here is allowed 7 rows of vertical movement per column, against the default of 3. A tall narrow descender, such as the tail of a y, demands a sharp deflection: the cut must drop below the tail and climb back within the few columns the tail occupies. With a small step limit the cut simply cannot turn fast enough and is forced through the tail. The cost of the larger limit is a little more work per pixel and a little more freedom to wander, the latter being what the midline penalty is there to restrain.

Method | The outer boundaries

Cuts exist only between consecutive centres, so the first line has no cut above it and the last none below. Two straight boundaries are added at 0.75 pitches beyond the outermost centres, clipped to the page. They are straight because there is no second line on the far side to avoid, so there is nothing for a seam to dodge, and 0.75 pitches is enough to clear the ascenders of the first line and the descenders of the last. The number of bands is then one less than the number of boundaries, which is why L is computed that way.

Stage: Ink ownership and crop construction

5.50 Method background: Ink ownership and crop construction

Purpose | The role in this system

Keep dots, rings and tails with the intended line instead of blindly following a seam boundary.

Justification | The reason for this construction

Project-specific component, overlap and size guards target handwriting ambiguity that a purely spatial partition cannot settle.

Alternatives | What changes if another method is used

Whole-component majority voting is simpler but can move too much of a touching line. Nearest-centre assignment ignores stem support. A learned owner classifier needs pixel/component annotations.

Library | Implementation speed and accuracy

Connected-component and region-property libraries can accelerate measurements; there is no one-call equivalent for these exact correction rules. Their thresholds require project evidence, not an algorithm citation. [3, 20].

5.51 RemovePrintedStructure

```
def RemovePrintedStructure(mask, g)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 577. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

Keep dots, rings and tails with the intended line instead of blindly following a seam boundary.

Inputs

`mask`
: the ink mask of the page

`g`
: the grey page at the same scale, used to locate the sheet

Output. five items: the cleaned mask, the paper-region mask, the line pitch, how many ink pixels lay outside the paper, and how many were removed as printed structure.

Purpose | What it is for

A sheet of lined or gridded paper carries printing that was never written by hand: the ruled lines, the red margin line, the holes and the darkened spine where the page meets the binding. All of it is ink as far as the mask is concerned, and all of it is poison for the stages that follow. A ruled line runs the full width of the page, so it joins every letter on a line into one connected blob, which destroys the per-letter analysis that the cutter depends on, and it adds a huge horizontal stroke to every row profile. This function removes that printing while leaving the writing intact.

Method | The distinguishing property: length

The property that separates printing from writing is not darkness, since a ruled line can be as dark as a pencil stroke, and not thickness, since both are a pen's width. It is **length**. A ruled line crosses the whole page in one unbroken straight run. Handwriting contains straight pieces too, but short ones: the longest horizontal stroke in ordinary writing is the crossbar of a t or the join between two letters, neither more than a fraction of a line's height. So a threshold on run length, expressed in line pitches, separates the two reliably.

Formula purpose and system use. Distinguish pitch-scaled long horizontal and vertical structures from shorter writing. **Source basis:** [3, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{flat} \;=\; \text{LongRunMask}\big(\text{mask},\; \lfloor 1.25\,P \rfloor,\; \text{horizontal}\big) \tag{5.34}$$

where

flat
: the mask of pixels belonging to a long horizontal run

LongRunMask
: marks every pixel that lies in an unbroken run of set pixels at least the given length; see its own entry

P
: the line pitch in pixels

1.25
: the length threshold as a multiple of the pitch

$\lfloor \cdot \rfloor$ rounding down to a whole number of pixels

The threshold of 1.25 pitches sits in a wide gap. A ruled line is tens of pitches long, and the longest horizontal stroke in handwriting is well under one pitch, since a letter is about half a pitch wide. Anything between about 1 and several pitches would give the same result, so the exact figure is not delicate. The vertical direction gets 1.4 pitches, slightly more, because a tall letter like a capital L or an f with a long descender genuinely can run more than a pitch vertically, and the extra margin keeps such letters out of the deletion.

Method | Tolerating a tilt

A ruled line on a page that is still a fraction of a degree off level is not a single row of pixels. It drifts downward as it crosses the page, so any one row contains only the piece of it that happens to lie in that row, and a run-length test applied row by row sees a series of short runs rather than one long one, and deletes nothing. The fix is to dilate the mask by one pixel vertically before the test, so that each row contains the line's pixels from the rows immediately above and below as well and the run becomes unbroken again.

But dilating also thickens the writing, and a test applied to a thickened mask would mark pixels that were never ink. So the result is dilated back and then intersected with the original mask, which restricts the deletion to pixels that really were ink to begin with. In morphological terms the sequence dilate, test, dilate, intersect is a closing used as a detector rather than as a filter: the dilation makes the feature visible to the test, and the intersection confines the consequences to the original shape.

The vertical pass uses a radius of 3 rather than 1 because the structures it targets, the shadow at the spine of a notebook and the edge of the sheet, are not straight. They wobble by several pixels across the height of the page, so a tolerance of one pixel is not enough to make them read as one continuous run.

Method | Masking off the surround first

Before any of this, PaperRegionMask locates the sheet and everything outside it is deleted from the mask in one intersection. This is done first because it is the cheapest and most decisive filter available: the desk, the hand and the shadow beyond the page account for a large share of the false ink on a photograph, and removing them wholesale means the subtler tests that follow are not distracted by them.

Justification | What is returned, and why the counts

Besides the cleaned mask, the function returns the pitch and the paper mask, both already computed here and needed later, and two counts: how many ink pixels lay outside the paper and how many were deleted as printed structure. The counts feed the diagnostics dictionary that SegmentLines returns. They exist because a failure of this stage is otherwise invisible: if the paper mask is wrong, the ink-outside-paper count is enormous, and if the run-length thresholds are wrong for a given page, the removed-pixel count is far larger or smaller than expected. Reporting them makes a silent failure a visible one.

5.52 DropArtifactPieces

```
def DropArtifactPieces(mask, ratio, P, H, W)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 592. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

The current artefact policy uses a pitch-scaled closed mask, component measurements, paper location and brightness-ratio evidence before discarding whole pieces. Its geometry guards are project rules. A discarded component is not automatically proven non-writing; inspect sparse punctuation and page-edge handwriting.

Method | Current implementation

Keep dots, rings and tails with the intended line instead of blindly following a seam boundary. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def DropArtifactPieces(mask,ratio,P,H,W):
    rc = max(1,int(0.1*P))
    dil = AnyWithinRadius(AnyWithinRadius(mask,rc,0),rc,1)
    closed = ~AnyWithinRadius(AnyWithinRadius(~dil,rc,0),rc,1)
    lab,nc = LabelComponents(closed)
    ys,xs = np.nonzero(mask)
    pl = lab[ys,xs]
    area = np.bincount(pl,minlength=nc+1).astype(float)
    top,bot,xmin,xmax,_ = PieceBounds(pl,ys,xs,nc,H,W)
    tall = (bot-top+1)>2.2*P
    thin = (xmax-xmin+1)<=0.5*P
    edgeTol = max(3,int(0.5*P))   #this close to the paper boundary counts as page
    touchL,touchR = xmin<=edgeTol,xmax>=W-1-edgeTol
    touchT,touchB = top<=edgeTol,bot>=H-1-edgeTol
    thinV = (bot-top+1)<=0.5*P
    letters = (area>=0.065*P*P)&~tall
    xL = np.percentile(xmin[letters],5) if letters.any() else -np.inf
    vbar = ((xmax-xmin+1)<=0.12*P)&((bot-top+1)>=1.2*P)
    otsuT = Otsu(ratio)
    meanRatio = np.bincount(pl,weights=ratio[ys,xs],minlength=nc+1)/np.maximum(area,1)
    faint = meanRatio>0.85*otsuT
    faintVbar = faint&((xmax-xmin+1)<=0.2*P)&((bot-top+1)>=0.6*P)

    density = area/np.maximum((bot-top+1)*(xmax-xmin+1),1)
    edgeHeight = (bot-top+1)<=1.2*P
    edgeShadow = (touchT|touchB)&edgeHeight&(density<0.15)&faint&((xmax-xmin+1)>P)

    drop = (touchL|(touchR&thin)|((touchT|touchB)&thinV&thin)
            |(tall&thin)|vbar|faintVbar|edgeShadow|(xmax<xL-1.2*P))
    drop[0] = False
    nArtifacts = int(drop.sum())
    mask&=~drop[lab]
    return mask,lab,nc,area,nArtifacts
```

Justification | Interpret the method rather than infer a benchmark

Project-specific component, overlap and size guards target handwriting ambiguity that a purely spatial partition cannot settle. [3, 20].

5.53 _BandOwnership

```
def _BandOwnership(mask, lab, nc, cuts, cutsArr, L, P, H, W, centres)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 680. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

The first guess at which line owns each piece of ink, the band holding the most of it, with small pieces, cut-off tails and dots pulled to their line.

Inputs

`mask`
: the cleaned ink mask

`lab` the component labels from DropArtifactPieces

`nc` the number of components

`cuts`
: the cuts as a list of per-column row arrays

`cutsArr`
: the same cuts as one 2-D array

`L` the number of bands

`P` the line pitch

`H` the mask height

`W` the mask width

`centres`
: the row positions of the lines

Output. six items: the ownership array assigning each ink pixel to a line, a copy of it taken before the later corrections, the band index of every pixel, the area of each component, and the row and column coordinates of the ink pixels.

Purpose | What it is for

The cuts divide the page into bands, but a band is not yet an answer. The question that must be settled for every ink pixel is which line of writing it belongs to, and the band it falls in is only the first guess at that. This function makes that guess and then applies several corrections to it, producing the ownership map that the crops are cut from.

Method | Computing the band index without a loop

Each cut is a curve, so asking which band a pixel is in means asking how many of the curves lie above it. That count is the band index directly, and it can be formed by adding up one comparison per cut over the whole image at once.

Formula purpose and system use. Count a component overlap with each initial seam band. **Source basis:** [3, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{region}[y, x] \;=\; \sum_{n=1}^{L-1} \left[\; y \;\geq\; \text{cuts}_n[x] \;\right] \tag{5.35}$$

where

$\text{region}[y, x]$
: the band index of this pixel, counting from 0 at the top

$\text{cuts}_n[x]$
: the row occupied by cut n in column x

L the number of bands, so there are L−1 interior cuts

$[\,\cdot\,]$ 1 when the condition holds, 0 otherwise

The expression is read as: a pixel is below cut n, so it lies past that boundary, and summing those indicators over all the interior cuts counts the boundaries it has passed. The outer two boundaries are excluded from the sum, since nothing lies beyond them that needs numbering; instead they are used separately to discard ink above the first line and below the last.

Justification | Why the band is only a guess

If every piece of ink lay wholly inside one band, the band index would be the final answer. Two things prevent that. First, a connected piece of ink can genuinely cross a cut: the descender of a g belongs to the upper line but hangs into the lower band, and the cut goes around the ink, so the piece ends up with pixels in both. Taking the band pixel by pixel would split that g in half, delivering its bowl to one crop and its tail to another, and neither fragment looks like a g. Second, small marks like the dot of an i are separate pieces that float well above their own line's body and can easily fall on the wrong side of a cut.

Method | The piece-by-band table

Both problems need the same information: how much of each piece lies in each band. That is a two-dimensional table, pieces against bands, and it is built with one tally by packing the two indices into a single number.

Formula purpose and system use. Define which overlaps are substantial enough to influence a reference owner. **Source basis:** [3, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{key} \;=\; \ell \cdot L \;+\; b, \qquad \text{cnt}[\ell,\, b] \;=\; \big|\{\,(y,x) : \text{lab}[y,x] = \ell,\; \text{region}[y,x] = b\,\}\big| \tag{5.36}$$

where

key a single integer encoding the pair of a piece and a band

ℓ the piece's component label

b the band index

L the number of bands, used as the multiplier so that no two distinct pairs can collide

$\text{cnt}[\ell, b]$
how many pixels of piece l fall in band b

This is **index packing**, and the reason it works is place value, the same principle as writing a two-digit number: multiplying the first index by the number of possible second values guarantees that each pair maps to its own integer. A single tally over the packed keys then produces the whole table, and reshaping the result into an array of pieces by bands unpacks it. The alternative, looping over pieces and counting each one's pixels, would be far slower in Python for no gain.

Method | Assigning each piece: the topmost substantial band

A band holds a **substantial** share of a piece when it contains at least 15 percent of that piece's ink. Among the substantial bands, the piece is assigned to the topmost.

Formula purpose and system use. Reassign only eligible pieces using the stated size and spatial guards. **Source basis:** [3, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{major}(\ell) \;=\; \min\,\big\{\, b \;:\; \text{cnt}[\ell, b] \;\geq\; 0.15 \textstyle\sum_{b'} \text{cnt}[\ell, b'] \,\big\} \tag{5.37}$$

where

$\mathrm{major}(\ell)$
: the band piece l is assigned to

$\mathrm{cnt}[\ell, b]$
: the pixels of piece l in band b

$\sum_{b'} \mathrm{cnt}[\ell, b']$
: the piece's total ink

0.15 the share of its own ink a band must hold to count as substantial

min the smallest such band index, that is, the highest on the page

The 15 percent test exists to tell a genuine crossing from a trivial one. A piece whose ink is 95 percent in one band and 5 percent in the next merely touches the boundary, and treating it as spanning two lines would be wrong. A piece split 60 to 40 really does belong to both, and something must be decided.

Choosing the topmost, rather than the band with the most ink, is a deliberate asymmetry. When a piece spans two lines it is usually because a descender from the upper line reaches down, or because two lines of writing touch. In either case the upper line is the one that is incomplete without the piece, since the shape originates there, while the lower line is merely intruded upon. Giving it to the upper line keeps the crop that would otherwise be damaged intact. If no band reaches 15 percent, which happens when a piece is spread thinly over three or more bands, the fallback is the band with the most ink.

Method | Small pieces: three graded rules

Small pieces are handled separately, because for them the band is unreliable and their size tells us what kind of thing they are. The thresholds are areas measured in P^2, the square of the pitch, which is the natural unit of area here: a letter occupies a region roughly a third of a pitch by a half, so letter-sized is a few hundredths of P^2.

- **Under 0.06 $\mathbf{P^2}$, a stray mark.** It is compared with the letter-sized pieces assigned to its own band. If none sits within six pitches horizontally, there is no writing anywhere near it and it is discarded. Otherwise the median vertical centre of the nearby writing says where its line really is, and if the mark sits more than half a pitch below that it is moved down a band, or more than 0.7 pitches above it, up a band. A speck under 0.01 P^2 is discarded in that situation rather than moved, on the grounds that something that small and that far out of place is noise.
- **Between 0.06 and 0.15 $\mathbf{P^2}$, a small but real piece.** Its own band and the two neighbours are considered, and it goes to whichever has writing whose median vertical centre is closest to its own. This is a nearest-centre rule, and it is used here rather than the band because a piece this size can easily fall on the wrong side of a cut while clearly belonging, by position, to one line.
- **A minor part of a larger piece.** A band holding a small, narrow part of a piece, under 0.12 P^2 and under 35 percent of the piece, no wider than 0.55 pitches, in a band adjacent to the piece's assigned one, is treated as a cut-off tail and follows the rest of the piece. The width condition is what distinguishes a tail from a whole word: a tail is narrow, and a wide part of a piece in another band is that band's own writing.

The median is used for the position of nearby writing rather than the mean, for the usual reason: one misplaced piece among the neighbours would drag a mean, while the median reports where the bulk of the line actually is.

Method | The dot rule

The last and most specific correction handles the dots of i and j. A dot floats clear of its own letter, often nearer to the line above than to its own line's body, so neither the band nor a nearest-centre test places it reliably. What does place it is the letter underneath it.

A piece is a dot candidate when it is small, under 0.1 P^2, compact, no more than 0.45 pitches in either direction, and roughly round, its longer side no more than 2.2 times its shorter. The roundness test is what excludes a comma or an accent from being treated as a dot. The rows below the candidate are then scanned, up to 0.4 pitches down, for the first substantial piece appearing beneath it, and the dot is given to that piece's line, provided the geometry agrees: the line's writing must sit between 0.25 and 0.7 pitches below the dot, which is where a dot floats above its own line's body. That final check is what stops the rule firing when the piece found below happens to belong to a line further away.

Method | Writing the result

All of this produces, for each piece, either a band index or a marker meaning discard. The ownership array is then filled in one operation: a pixel takes its piece's assigned band if the piece was one of those the rules relocated, and its own band index otherwise. A copy is kept before the later corrections run, because those corrections need to know which pixels were already moved here, in order not to undo that work.

5.54 _DotsJoinedAbove

```
def _DotsJoinedAbove(own, mask, lab, area, region, centres, L, P, H, W, ys, xs)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 755. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

Returns a small high-sitting piece, fused to a descender above it by the gap closing, to the line its own stem belongs to.

Inputs

`own` the ownership array, modified in place

`mask`
: the cleaned ink mask

`lab` the closed-mask component labels

`area`
: the area of each closed-mask component

`region`
: the band index of every pixel

`centres`
: the line centres

`L` the number of bands

`P` the line pitch

`H` the height

`W` the width

`ys` the row coordinates of the ink pixels

`xs` their column coordinates

Output. four items describing the components of the unclosed mask: the label array, the count, the per-pixel labels and the areas. The ownership array is corrected in place.

Purpose | The problem it solves

The closing applied before labelling in DropArtifactPieces is useful but it has a side effect. If the dot of an i on one line happens to sit just below a descender from the line above, the closing can bridge the small gap between them and fuse the two into a single component. The dot then inherits whatever the fused piece was assigned, which is the upper line, and the lower line's crop loses its dot while the upper line's crop gains a stray mark.

Justification | Why a second labelling is needed

The fix requires seeing the dot as its own object, which the closed labelling can no longer do. So the mask is labelled again, this time without closing. In the unclosed labelling the dot is separate from the descender, because the gap between them is real ink-free space; it was only the closing that joined them. The function therefore works with two labellings at once, the unclosed one to identify the dot and the closed one to recognise when a piece is in fact part of a larger structure.

Method | Identifying the dot

The same three-part test as before: small area, between 0.015 and 0.1 P^2, compact, no more than half a pitch across, and roughly round. The lower area bound is new and matters here, since it excludes single-pixel noise that would otherwise pass every other condition.

Method | Finding the stem and its line

The rows immediately below the candidate are scanned for the first other substantial piece, and the band of the pixel where the two meet is read from the region array. That gives the line the stem belongs to, which is the line the dot should belong to. The dot's own current assignment is read as the most common ownership value over its pixels.

Method | The three conditions for acting

- **The stem must be exactly one band below.** A fusion of this kind can only displace a mark by one line, since the closing bridges a gap of a few pixels. A discrepancy of two bands means something else is wrong, and acting on it would make things worse.
- **The fused piece must not be a whole word.** If the closed component containing the dot has an area of 0.2 P^2 or more, it is a word, not a dot welded to a descender. A genuine floating dot stays a small component even after closing, so a large closed area is evidence that the candidate is really part of a letter, such as the bowl of an a that the threshold happened to detach.
- **The dot must sit well below the upper line's centre.** At least 0.45 pitches below it. A mark that close to the upper line's body really is part of the upper line, and the rule declines to move it.

Only the pixels currently assigned to the upper line are moved, not every pixel of the unclosed piece. This matters when a letter has broken apart in the unclosed mask, so that the candidate shares its unclosed component with ink that was already correctly assigned: moving the whole component would drag that correct ink along with it.

> *The pattern of conditions here is characteristic of the whole ownership stage. Each correction rule is narrow on purpose, requiring several independent pieces of evidence to agree before it fires. A broad rule would fix more cases and break more, and a crop that is wrong is more costly than a crop that is merely imperfect, because the recogniser will read it confidently either way.*

5.55 _RingsToStem

```
def _RingsToStem(own, raw, nr, pr, Ar, lab, area, region, centres, P, H, W, ys, xs)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 784. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

Keep dots, rings and tails with the intended line instead of blindly following a seam boundary.

Inputs

`own` the ownership array, modified in place

`raw` the unclosed component labels

`nr` the number of unclosed components

`pr` the unclosed label of each ink pixel

`Ar` the area of each unclosed component

`lab` the closed labels

`area`
: the closed areas

`region`
: the band index of every pixel

`centres`
: the line centres

`P` the line pitch

`H` the height

`W` the width

`ys` the row coordinates of the ink pixels

`xs` their column coordinates

Output. nothing is returned; the ownership array is corrected in place.

Purpose | The problem it solves

This is the mirror image of the previous correction. There, a dot on the lower line was welded to a descender from above and wrongly given to the upper line. Here the welding runs the other way: a small ring-shaped mark belonging to the lower line, such as the dot of an i written as a small circle, or the loop at the top of a letter, is fused to the stem it sits on, and the stem's line is the one it should follow. The two corrections exist separately because the evidence needed differs: finding a ring requires detecting a hole, which the dot rule does not do.

Method | Finding a candidate: walking up from the bottom

The candidate is not a whole component here but a part of one, so it cannot be found by labelling. It is found geometrically. For each component, the lowest ink pixel in each column is located, and those bottom points are examined in order from the lowest upward, since the ring sits at the top of a stem and the stem's foot is the lowest point of the structure.

From each bottom point the algorithm walks upward, measuring at every row how wide the component is within a window around the column, and tracking the widest it has been. The walk stops when the width drops below 45 percent of that maximum. That point is the junction between the blob and the stem: a ring is wide, the stem below it is narrow, so the width collapses exactly where one ends and the other begins. This is a one-dimensional version of a shape-decomposition idea that recurs in image analysis: a narrow neck is where an object naturally divides.

Method | Validating the shape

Three conditions must hold before the candidate is accepted as a ring.

- **Its width is between 0.2 and 0.45 pitches.** Narrower is a stroke, wider is a letter body.
- **It is roughly as tall as it is wide,** with a height-to-width ratio between 0.6 and 1.6. A ring is round; a stroke is not.
- **It encloses a hole.** This is the decisive test, and it is carried out by labelling the components of the inverse of the blob after padding it with a border. If the blob encloses a hole then the background splits into two components, the outside and the hole, so a count of fewer than two components means there is no hole and the candidate is a hook or a curl rather than a ring.

The padding before that test is not optional. Without a border of background all round, a blob touching the edge of the extracted window would let the outside connect to the hole along that edge, merging them into one component and hiding the hole. Padding guarantees the outside is a single region that surrounds the blob completely. Counting holes by labelling the background is a standard technique, and the number it yields is related to the **Euler number** of the shape, the classical topological measure of components minus holes.

Method | The decision

As before, the first other piece appearing below the ring identifies the line to move to, the two bands must be adjacent, the closed component must not be large enough to be a whole word, and the junction must lie no more than 0.45 pitches above the target line's centre. Only the ring's own pixels are reassigned, confined to the rows between the junction and the bottom point and to the columns the blob occupies, and only where they currently belong to the band being moved from.

Justification | Why the examination is limited

At most 60 bottom points are examined per component, and accepting one point suppresses the columns within 0.35 pitches of it. Both are cost controls. A long word has hundreds of columns and the upward walk at each is not free, while the suppression prevents the same ring being found again from every column it spans. The cap is a deliberate compromise: a word with more than 60 distinct bottom points may have a ring missed, which costs one misplaced dot, whereas examining every column adds work; no per-page timing effect is established here.

5.56 _FragmentsToNeighbour

```
def _FragmentsToNeighbour(own, ownBeforeEF, mask, centres, L, P, H, W)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 837. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

Rehomes a piece with no ink in its own x-height band to the neighbouring line it reaches into.

Inputs

`own` the ownership array, modified in place

`ownBeforeEF`
ownership as it stood before the two dot corrections, used to tell which pixels they already moved

`mask`
the cleaned ink mask

`centres`
the line centres

`L` the number of bands

`P` the line pitch

`H` the height

`W` the width

Output. nothing is returned; the ownership array is corrected in place.

Method | The principle: a line's writing lies in its x-height band

This is the most general of the corrections, and it rests on a single observation. A line of handwriting is defined by the band of rows its letter bodies occupy, around its centre. Anything genuinely belonging to that line must have some ink there, because a letter has a body. A piece assigned to a line that has no ink at all within a quarter pitch of its centre is therefore not part of that line, whatever the cut said. It is a visitor: a descender hanging down from above, or an ascender reaching up from below.

Method | How the test is made

For each band in turn, the ink currently owned by that band is labelled into connected pieces, and for each piece the number of its pixels lying within 0.28 pitches of the band's centre is counted.

Formula purpose and system use. Measure whether a residual fragment has enough support in its assigned body band. **Source basis:** [3, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{inband}(c) \;=\; \left|\{\,(y,x) \in c \;:\; |y - \text{centre}_n| \;\le\; 0.28\,P\,\}\right| \tag{5.38}$$

where

inband(c)
how many pixels of piece c lie in the band's x-height zone

c a connected piece of the ink currently owned by band n

centre_n
the row position of line n

P the line pitch

0.28 the half-height of the x-height zone as a fraction of the pitch

A piece with a count of exactly zero, and an area of at least 0.006 P^2 so that single-pixel noise is ignored, is a candidate for rehoming. The 0.28 figure is about half the x-height expressed in pitches, which follows from the factor of roughly 2.2 between pitch and x-height noted in LinePitch: half an x-height is about 0.23 pitches, and 0.28 allows a little margin for the centre estimate being slightly off.

Method | Deciding where it belongs: two routes

Having established that a fragment does not belong where it is, two independent kinds of evidence are tried for where it does.

- **Adjacency.** The fragment is dilated by two pixels and the ownership of the surrounding ink is examined. If at least three of those neighbouring pixels belong to one other band, the fragment is physically touching that line's writing, which is strong evidence that it is part of it. The move is only made if that band is adjacent and lies on the correct side: a fragment below this line's centre may only move down, one above it only up. That consistency check is what prevents a chain of bad moves.
- **Position.** If no neighbouring band reaches three pixels, the fragment is isolated and adjacency says nothing. The fallback is to ask whether the fragment's own vertical centre falls inside a neighbouring line's x-height band, within 0.2 pitches of its centre, and if so to move it there. This is weaker evidence, which is why it is used only as a fallback and with a tighter tolerance.

Method | Not undoing the previous corrections

The ownership array as it stood before the two dot corrections is passed in so that the pixels they moved can be identified, by comparing the two arrays. A small compact fragment that sits on pixels those rules already placed is skipped. The reason is direct: a dot deliberately placed above its line by the dot rule has no ink in its line's x-height band, by the very nature of being a dot, so it looks exactly like a fragment to this test and would be moved straight back. The exception is deliberately narrow, applying only to pieces under 0.04 P^2 and no more than 0.4 pitches across, so that a genuine tail loop that happens to overlap a placed dot is still free to move.

> *Every move is recorded and applied only after the scan over all the bands is complete. Applying them as they are found would mean later tests read ownership that earlier moves had already changed, so the outcome would depend on the order the bands happened to be visited. Collecting first and applying afterwards makes the result independent of that order.*

5.57 AssignOwnership

```
def AssignOwnership(mask, lab, nc, cuts, cutsArr, L, centres, P, H, W)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 873. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

Keep dots, rings and tails with the intended line instead of blindly following a seam boundary.

Inputs

`mask`
: the cleaned ink mask

`lab` the component labels

`nc` the component count

`cuts`
: the cuts as a list

`cutsArr`
: the cuts as an array

`L` the number of bands

`centres`
: the line centres

`P` the line pitch

`H` the height

`W` the width

Output. the ownership array: for every pixel, the index of the line that owns it, or -1 for background and for ink that was discarded.

Purpose | What it is for

This function contains no logic of its own. It runs the band guess and then the three corrections, in order, and returns the result. Its value is organisational, and the organisation is the point worth explaining.

Justification | Why the rules are separate functions

The initial ownership follows the seam bands; subsequent corrections are project-specific rules for candidate fragments and connections. Separate helpers make those conditions inspectable. Historical ablation claims require the corresponding run records; the existence of a helper does not establish how its contribution was measured.

Justification | Why the order is fixed

The sequence is not arbitrary. The band guess must come first because everything else corrects it. The two welding rules come next, and they are the most specific: each requires several independent conditions and fires rarely. The fragment rule comes last and is the most general, which is the right place for it, because a general rule run first would act on cases the specific rules would have handled better. The ordering principle, specific before general, is the same one that governs a cascade of fallbacks in LinePitch: apply the best-informed rule wherever it applies, and the broad rule only to what is left.

The dependency between them is explicit in the arguments. The fragment rule receives the ownership array as it stood before the welding rules ran, precisely so it can recognise and respect their decisions. That is why the band guess returns that copy, rather than each rule working from whatever it finds.

5.58 _BuildLineCrop

```
def _BuildLineCrop(n, own, norm, P, f, H, W, minInk)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 886. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

One band in, one finished crop out, trimmed to its ink, padded, contrast stretched, and None when too little is left to be a line of writing.

Inputs

`n` which line to build, as a band index

`own` the ownership array

`norm`
the illumination-corrected full-resolution page

`P` the line pitch, in half-size pixels

`f` the scale factor from half-size to full-size coordinates

`H` the half-size height

`W` the half-size width

`minInk`
the least ink a band must own to be a real line

Output. None if this band is not a usable line; otherwise a dictionary with the finished grey crop, a fixed-size copy for the recogniser, and the bounding box the crop came from.

Purpose | What it is for

All the analysis so far has been at half resolution and in terms of masks. This function turns one band's worth of ownership into the actual image the recogniser will read: a grey picture of one line of writing, cut from the full-resolution page, contrast-corrected, with the neighbouring lines removed.

Method | Rejecting a band that is not a line

Two tests reject a band outright. It must own at least minInk pixels, a threshold the caller derives from the median ink per line so that it adapts to the page, and its ink must span at least 0.2 pitches vertically. The height test is the more interesting one: a band containing a horizontal scatter of specks has plenty of ink but no vertical extent, and is a surviving artefact rather than writing. The floor of six pixels keeps the test meaningful on a very small image. The height demanded is deliberately well under a full x-height, because a short line containing only lowercase letters is still a real line.

Method | Trimming a distant stray

A band can own a legitimate line plus one mark far away, perhaps a page number the artefact rules did not catch. Cropping to all of it would produce an image mostly of blank paper with writing at one end, which is poor input for a recogniser. So the rows containing ink are examined, the gaps between them found, and the largest contiguous run of inked rows kept.

The trim is only performed if it removes more than 0.8 pitches of height. That guard is what distinguishes a paragraph-sized gap from the ordinary small gaps inside a line of writing, which occur between the body of the line and a stray ascender. Without the guard, a line with an ordinary internal gap would have its ascenders trimmed away.

Method | From half-size decisions to full-size pixels

The bounding box of the kept ink is padded by 0.15 pitches, so that the writing does not touch the edge of the crop, and then multiplied by the scale factor to give the corresponding rectangle of the full-resolution page. This is where the two resolutions meet: every decision was made cheaply at half size, and the pixels are taken at full size.

Method | Building the alpha channel

The crop must show this line's ink and not the neighbouring lines' intrusions, so a per-pixel weight is needed saying how much of each pixel to show. That weight is called **alpha**, borrowing the standard graphics term for opacity, where 1 means fully shown and 0 fully replaced by white.

It starts as the ownership mask for this line, enlarged to the crop size bilinearly. Bilinear rather than nearest-neighbour is a deliberate choice here, and for an unusual reason: the soft edge that interpolation produces is what makes the stroke boundary in the final crop look like a real pen edge rather than a staircase of hard pixels. The slight blur is the feature, not an artefact to be tolerated.

Method | Recovering ink the mask missed

The half-size mask inevitably misses some faint ink, and at full resolution that ink is visible. So a second, local threshold is computed, on the crop alone rather than the whole page, using the same divide-by-local-background method as InkMask but with a radius scaled to the crop's own height. Because the crop is one line of writing rather than a whole page, the contrast within it is far more uniform, so this local threshold can pick up strokes that the page-wide one could not.

Those recovered pixels cannot simply be added, or the neighbouring lines' ink would come back with them. Two gates restrict them.

- **The halo.** The line's own ink is dilated by about 0.2 pitches and the recovery is confined to that neighbourhood. The reasoning is that a stroke the mask missed lies next to strokes it caught, since it is part of the same writing, whereas ink from the line above or below is further away.
- **The ownership veto.** Any pixel the page-level ownership already gave to a different line is excluded, unconditionally. This gate is enlarged with nearest-neighbour rather than bilinear interpolation, precisely the opposite choice from the alpha channel, and for the opposite reason: ownership is a yes-or-no fact about which line a pixel belongs to, and a blurred version would half-assign a pixel to two lines at once.

The final alpha is the larger of the ownership alpha and the gated recovery, taken pixel by pixel, so recovery can only ever add ink and never remove it.

Method | Contrast stretching

The crop's grey values still span whatever range the photograph happened to produce, and the recogniser was trained on images with ink near black and paper near white. A **contrast stretch** maps the observed range onto the full range linearly.

Formula purpose and system use. Apply the soft ownership mask to retain line ink and suppress neighbouring ink. **Source basis:** [3, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{linear}[y, x] \;=\; \text{clip}\left((\text{region}[y, x] - b) \cdot \frac{255}{\max(w - b,\, 1)},\; 0,\; 255 \right) \tag{5.39}$$

where

$\text{linear}[y, x]$
: the stretched grey value

$\text{region}[y, x]$
: the illumination-corrected value at this pixel

b
: the black point: the value that should become 0, taken as the 40th percentile of the values under the ink

w the white point: the value that should become 255, taken as the 75th percentile of the whole crop

$\max(w - b,\, 1)$
a guard against a degenerate crop in which the two points coincide

$\mathrm{clip}(\cdot, 0, 255)$
values forced into the valid grey range

The two reference points are percentiles rather than the minimum and maximum, and the choice is what makes the stretch stable. One unusually dark pixel, from a blob of ink or a speck of dust, would set a minimum-based black point and leave all the real ink grey. The 40th percentile of the ink pixels describes the typical darkness of a stroke, which is the value that should become black. Likewise the 75th percentile of the crop describes the paper, since paper is the majority of the area.

Method | The gamma correction

A linear stretch alone leaves faint strokes faint, because a stroke at 70 percent of the way from ink to paper maps to 70 percent of the way from black to white. A **gamma correction** applies a power to the darkness, which bends the mapping.

Formula purpose and system use. Strengthen intermediate darkness before the crop enters the recogniser. **Source basis:** [3, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$d'[y,x] \;=\; \Big(\, d[y,x] \,\Big)^{\gamma}, \qquad d[y,x] \;=\; \frac{255 - \mathrm{linear}[y,x]}{255}, \qquad \gamma = 0.55 \tag{5.40}$$

where

$d[y,x]$
the darkness at this pixel, 0 for white and 1 for black: the stretched value inverted and scaled

$d'[y,x]$
the corrected darkness

γ the gamma exponent, here 0.55

With an exponent below 1 the power function lies above the straight line on the interval from 0 to 1, so every darkness is increased, and increased most where it starts small. A faint stroke at darkness 0.3 becomes 0.3 raised to the power 0.55, which is about 0.51, nearly doubling, while ink already at 0.9 becomes 0.94, barely changing. That is precisely the desired behaviour: pull the faint strokes up to where the recogniser can see them, and leave the strong ink alone. The corrected darkness is then converted back to grey and composited with white through the alpha channel.

Method | Aligning the left margin

Finally the crop is shifted horizontally so that the first column containing ink sits the same distance from the left edge in every crop. The reason is that the recogniser's convolutional layers respond to position, so a line whose writing starts 20 pixels in and one whose writing starts 80 pixels in present measurably different inputs for the same text. Standardising the left offset removes a variation that carries no information.

> *The returned dictionary contains the crop twice: once at its natural size for display and for the letter cutter, and once squeezed onto the recogniser's fixed input canvas by FitFixedCanvas. Keeping both avoids any later stage having to guess which geometry it is looking at.*

5.59 DropSolidCrops

```
def DropSolidCrops(lines, maxAbsPct=15.0, maxMedianRatio=3.0)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 986. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

Measures the fraction of crop pixels below grey level 128. A crop is dropped only if above 15 percent and three times the crop-population median. If all would be dropped, the original list is returned. These are crop fractions, not per-row ratios or the 15-percent ownership rule.

Method | Current implementation

Keep dots, rings and tails with the intended line instead of blindly following a seam boundary. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def DropSolidCrops(lines,maxAbsPct=15.0,maxMedianRatio=3.0):
    if not lines:
        return lines
    ink = [float((np.asarray(l['image'])<128).mean()*100.0) for l in lines]
    med = float(np.median(ink))
    kept = []
    for l,v in zip(lines,ink):
        l['inkPct'] = v
        if med>0 and v>maxAbsPct and v>maxMedianRatio*med:
            continue
        l['order'] = len(kept)
        kept.append(l)
    return kept if kept else lines
```

Justification | Interpret the method rather than infer a benchmark

Project-specific component, overlap and size guards target handwriting ambiguity that a purely spatial partition cannot settle. [3, 20].

5.60 SegmentLines

```
def SegmentLines(page, mergeValley=None)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 1004. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.50.

Purpose | What this contributes

Current signature is SegmentLines(page, mergeValley=None). It calls deskew, structure removal, artefact filtering, centres, seams, ownership, full-resolution crop construction and solid-crop rejection. It returns lines, debug arrays and summary information. No timer keyword exists: the server currently times this complete call as one stage.

Method | Current implementation

Keep dots, rings and tails with the intended line instead of blindly following a seam boundary. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def SegmentLines(page,mergeValley=None):
    fullA,halfA,mask,ratio,best = Deskew(page)
    g = halfA.astype(np.float64)
    H,W = mask.shape
    mask,paper,P,paperRemoved,removed = RemovePrintedStructure(mask,g)
    mask,lab,nc,area,nArtifacts = DropArtifactPieces(mask,ratio,P,H,W)
    P,centres = LineCentres(mask,H,mergeValley)
    cuts,cutsArr,L = SeamCuts(mask,centres,P,H,W)
    own = AssignOwnership(mask,lab,nc,cuts,cutsArr,L,centres,P,H,W)

    r = max(8,min(40,round(H/41)))
    bgFull = Resize(BoxMean(g,r).astype(np.float32),fullA.shape[1],fullA.shape[0],'linear')
    norm = np.clip(fullA.astype(np.float32)/np.maximum(bgFull,1.0),0,1)*255
    f = fullA.shape[1]/W   #half-size coordinate to full size
    perLine = np.bincount(own[own>=0],minlength=L)
    minInk = max(60,0.12*np.median(perLine[perLine>0]))
    lines = []
    for n in range(L):
        crop = _BuildLineCrop(n,own,norm,P,f,H,W,minInk)
        if crop is None:
            continue
        crop['order'] = len(lines)
        lines.append(crop)
    nBuilt = len(lines)
    lines = DropSolidCrops(lines)
    nDropped = nBuilt-len(lines)

    return lines,dict(cuts=cuts,ratio=ratio,paper=paper),dict(
        angle=float(best),pitch=int(P),
        paperCoverage=round(float(paper.mean()),3),
        inkOutsidePaper=paperRemoved,removedStraightPixels=removed,
        artifactPieces=nArtifacts,nLines=len(lines),dropped=nDropped)
```

Justification | Interpret the method rather than infer a benchmark

Project-specific component, overlap and size guards target handwriting ambiguity that a purely spatial partition cannot settle. [3, 20].

Stage: Skeletonisation and centreline recovery

5.61 Method background: Skeletonisation and centreline recovery

Purpose | The role in this system

Convert thick ink areas into approximate pen paths and normalise stroke width.

Justification | The reason for this construction

Zhang–Suen deletion rules preserve local connectivity while thinning in two simultaneous sub-passes. This fits the geometric path representation.

Alternatives | What changes if another method is used

Medial-axis transforms use distance information and may branch differently. Contours trace edges rather than one pen centreline. Neither recovers actual stroke order or pressure from a photograph.

Library | Implementation speed and accuracy

skimage.morphology.skeletonize(method=zhang) and OpenCV contrib thinning are related implementations. Match binary polarity, padding and iteration rules. Lookup tables/compiled loops may reduce repeated-array cost; the resulting path is still an estimate. [31, 32].

5.62 Skeletonize

```
def Skeletonize(mask)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 194. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.61.

Purpose | What this contributes

The Ring helper constructs eight shifted neighbours from a zero-padded current mask. Every sub-pass calculates deletion eligibility before applying all deletions together. The second sub-pass uses the complementary triplet tests. This is still Zhang–Suen; it is not an exact Euclidean medial axis or measured stroke order.

Method | Current implementation

Convert thick ink areas into approximate pen paths and normalise stroke width. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Skeletonize(mask):
    img = mask.astype(np.uint8).copy()

    def Ring(P):
        p = np.pad(P,1)   #the eight neighbours of every pixel, clockwise from north
        return (p[:-2,1:-1],p[:-2,2:],p[1:-1,2:],p[2:,2:],
                p[2:,1:-1],p[2:,:-2],p[1:-1,:-2],p[:-2,:-2])

    changed = True
    while changed:
        changed = False
        for step in (0,1):   #two sub-passes, alternating corners
            P2,P3,P4,P5,P6,P7,P8,P9 = Ring(img)
            r = [P2,P3,P4,P5,P6,P7,P8,P9,P2]
            B = sum(r[:8])   #how many neighbours are set
            A = sum(((r[k]==0)&(r[k+1]==1)).astype(np.uint8) for k in range(8))   #0 to 1 steps round
     the ring
            if step==0:
                c1,c2 = (P2*P4*P6)==0,(P4*P6*P8)==0
            else:
                c1,c2 = (P2*P4*P8)==0,(P2*P6*P8)==0
            go = (img==1)&(B>=2)&(B<=6)&(A==1)&c1&c2   #a==1 means deleting it cannot split the
    stroke
            if go.any():
                img[go] = 0
                changed = True
    return img.astype(bool)
```

Justification | Interpret the method rather than infer a benchmark

Zhang–Suen deletion rules preserve local connectivity while thinning in two simultaneous sub-passes. This fits the geometric path representation. [31, 32].

Stage: Douglas Peucker simplification

5.63 Method background: Douglas Peucker simplification

Purpose | The role in this system

Reduce path vertices and command count while retaining meaningful geometric detail.

Justification | The reason for this construction

Recursive farthest-deviation tests keep corners that exceed a tolerance. The current routine uses distance to the supporting line and a near-loop split.

Alternatives | What changes if another method is used

Uniform vertex skipping ignores shape. Distance-based decimation is cheaper but does not provide the same deviation test. Spline fitting changes representation and requires sampling/tolerance choices.

Library | Implementation speed and accuracy

OpenCV approxPolyDP is a related Douglas–Peucker implementation. Endpoint handling, closed paths and distance convention can differ. Compiled execution may reduce recursion overhead; equal epsilon does not by itself guarantee identical vertices. [33, 20].

5.64 PolylineLength

```
def PolylineLength(p)
```

Evidence | Source and method link

`Software/DemoCode/segment.py` line 420. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.63.

Purpose | What this contributes

Reduce path vertices and command count while retaining meaningful geometric detail.

Inputs

`p` a path as a list or array of (x, y) points

Output. The total distance walked along the path.

Purpose | What it is for

A size measure for pen strokes. Its main use is rejecting fragments: a piece cut out of a line of writing that contains far less pen path than a real example of that letter is a fragment rather than a letter, whatever its bounding box suggests.

Method | How it works

Formula purpose and system use. Sum Euclidean segment lengths for a path-length measurement. **Source basis:** [33, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$L = \sum_i \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} \tag{5.41}$$

where

L the total path length, also called the arc length

x_i, y_i
the coordinates of the i-th point on the path

Each term is the straight-line distance between two consecutive points, by Pythagoras, and the total is their sum. It is computed as whole-array arithmetic: the differences between consecutive points, then their lengths, then one sum.

Justification | Why path length and not bounding box

A fragment can have a deceptively large bounding box. Half of a letter o spans nearly the full width and height of the whole letter while containing only half the ink, so a box-based test would accept it. Path length measures how much pen movement the shape actually represents, which is what distinguishes half a letter from a whole one.

6 Text recognition and writer identification

Evidence | Current implementation boundary

`Software/DemoCode/recognise.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Load compatible arrays; fit and optionally normalise a line; form convolutional features; apply the writer or recurrent text head; decode or align CTC; score the result. Writer pooling and recurrent text decoding are parallel heads, not successive compulsory stages.

6.1 Expanded reasoning: recognition and writer identity

6.1.1 Input dimensions, scale and information loss

The inspected input is one grey channel on a 64×640 canvas. For source height h and width w, isotropic fit uses $a = \min(64/h, 640/w)$, producing rounded dimensions near (ah, aw). White padding fills the rest. Isotropic means the same scale in both axes; it preserves aspect ratio before rounding. Stretching directly to the canvas would distort letters. The report's 156×2170 example is width-limited: $a = 640/2170 \simeq 0.29493$, so the content height is about 46 pixels. That is why a large nominal canvas height does not guarantee a tall letter image. A very long line can be compressed until dots or gaps disappear.

The exact canvas size is a project architecture choice and a checkpoint compatibility constraint, not a theorem. Increasing width retains more detail but increases feature/sequence work; increasing height can preserve vertical shape but changes activation memory. A trained fixed-shape model cannot be assumed accurate under a new preparation convention. Variable-width batches are an alternative, with padding/length masking costs. Pixel range, ink polarity, placement and stroke normalisation must match training, not merely dimensions.

6.1.2 What local feature map Z actually means

For input $X_{b,c,y,x}$, weights $W_{o,c,i,j}$ and bias b_o, a stride-one layer computes

Formula purpose and system use. Purpose in this explanation: What local feature map Z actually means **Source basis:** [34, 35, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$Z_{b,o,y,x} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i,j} W_{o,c,i,j} X_{b,c,y+i-p,x+j-p}.$$

b indexes batch examples, c input channels, o output channels, (i, j) the kernel and p zero padding. Each Z_o is a learned response map, not a labelled character map. The operation used in machine learning is cross-correlation (the kernel is not flipped), conventionally called convolution [36]. Sharing the same weights at every position expresses the assumption that a useful local stroke pattern can appear anywhere.

Constructed 2×2 example: a patch $\left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$ and kernel $\left[\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right]$ with bias 0.5 give $1 - 4 + 0.5 = -2.5$. ReLU would then give zero. The kernel values in a real network are learned, so visual intuition about an edge detector does not establish what any particular saved channel detects.

Method | Shape and parameter arithmetic.

For kernel k, padding p, stride s and dilation d, $H_{\text{out}} = \lfloor (H + 2p - d(k-1) - 1)/s + 1 \rfloor$; apply the same formula to width. With $k = 3, p = 1, s = d = 1$, size is unchanged. A convolution with bias has $C_{\text{out}}(C_{\text{in}}k^2 + 1)$ parameters; $1 \to 32$ channels costs $32(9+1) = 320$, whereas $32 \to 32$ costs $32(288+1) = 9248$. More channels increase representation capacity and both parameter and activation costs. Channel counts 32, 64 and 128 are implementation design choices; powers of two are convenient, not evidence of an optimum.

The reported blocks use two, four and six convolutions, with two 2×2 spatial pools. Output shapes are $32 \times 32 \times 320$, $64 \times 16 \times 160$ and $128 \times 16 \times 160$. Taking a maximum over height yields $128 \times 1 \times 160$; transposing gives 160 ordered feature vectors, each of length 128. A frame is therefore a feature column, not a presegmented character or a video frame. The CNN/recurrent/transcription combination follows the CRNN family of Shi et al. [35]. Its use does not justify these exact widths.

Method | Receptive field.

Track receptive-field span r and source-pixel jump j. Starting at $(r, j) = (1, 1)$, a layer of kernel k and stride s gives $r' = r + (k-1)j$, $j' = sj$. For the reported stack: two 3-kernels give $r = 5$, first pool gives $(6, 2)$; four more convolutions give 22, second pool gives $(24, 4)$; six more give $r = 72, j = 4$. Thus each final spatial position can depend on about a 72-pixel input span, subject to padding. The LSTM adds longer horizontal context. A 4-pixel frame stride does not mean a character boundary is localised to four source-image pixels.

6.1.3 Batch normalisation, ReLU, sigmoid and max pooling

For one channel containing $m = BHW$ values, training normalisation uses $\mu = m^{-1}\sum_i z_i$, $v = m^{-1}\sum_i (z_i - \mu)^2$ and $\hat{z}_i = (z_i - \mu)/\sqrt{v + \epsilon}$; the learned affine output is $u_i = \gamma \hat{z}_i + \beta$ [37]. γ changes scale, β restores a useful offset, and ϵ prevents unstable division for almost-constant input. For $(1, 3)$, $\mu = 2, v = 1$, so ignoring tiny ϵ, normalised values are $(-1, 1)$. $\gamma = 2, \beta = 0.5$ produces $(-1.5, 2.5)$. Inference uses saved running statistics, not statistics of the current single crop. A running-update coefficient controls how quickly history is forgotten; it is not the optimiser learning rate. Libraries can use different variance conventions for the running estimate [38].

ReLU is $\max(0, u)$, derivative 1 for $u > 0$ and 0 for $u < 0$, with an implementation convention at zero. It makes the network nonlinear while keeping the positive branch unsaturated [39]. Without nonlinear operations, compositions of affine maps reduce to a single affine map. This observation does not mean a network containing max pooling would become linear merely by removing ReLU: max pooling is itself nonlinear.

Sigmoid $\sigma(z) = 1/(1 + e^{-z})$ has derivative $\sigma(z)(1 - \sigma(z))$. It maps real scores into $(0, 1)$; $\sigma(0) = 0.5$ and $\sigma(\log 3) = 0.75$. It is used as a gate because multiplication can attenuate or pass information. A gate is not automatically a calibrated probability of a real-world event. Saturation near 0 or 1 yields small derivatives. Tanh maps into $(-1, 1)$ and has derivative $1 - \tanh^2 z$.

Max pooling selects the largest value in each window. For $\left[\begin{smallmatrix} 1 & 4 \\ 3 & 2 \end{smallmatrix}\right]$ it returns 4. The backward pass routes the upstream derivative to the selected maximum, using a fixed tie convention. Pooling reduces work and offers limited local shift tolerance, while discarding exact position. Average pooling retains average strength; strided convolution learns a reduction. Neither is automatically preferable for small character details.

6.1.4 LSTM gates, bidirectionality and the second recurrent layer

In the gate convention used by the inspected implementation,

Formula purpose and system use. Purpose in this explanation: LSTM gates, bidirectionality and the second recurrent layer **Source basis:** [34, 35, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\begin{bmatrix} a_i \\ a_f \\ a_g \\ a_o \end{bmatrix} = W_x x_t + W_h h_{t-1} + b, \quad i_t = \sigma(a_i),\ f_t = \sigma(a_f),\ g_t = \tanh(a_g),\ o_t = \sigma(a_o),$$

Formula purpose and system use. Purpose in this explanation: LSTM gates, bidirectionality and the second recurrent layer **Source basis:** [34, 35, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t).$$

$\odot$ means elementwise multiplication. The cell c_t carries a memory value; h_t exposes a gated version. Weights are shared across frames and do not get "transferred" from frame to frame. Input x_t, hidden state and cell state do. The original LSTM motivation was learning long temporal dependencies [40]; practical gate conventions are documented in the framework reference [41].

Constructed scalar cell: $c_{t-1} = 0.8, f_t = 0.9, i_t = 0.2, g_t = 0.5, o_t = 0.7$ gives $c_t = 0.82$ and $h_t \simeq 0.472$. The forget gate retains 90% of the previous cell along this direct path; other dependencies also affect the total gradient. A BiLSTM processes the same feature sequence left-to-right and right-to-left and concatenates hidden outputs. With 256 units in each direction it outputs 512 values per frame. The second layer therefore takes 512 inputs, whereas the first takes 128. Offline line images supply both directions; a live causal system would not have future context immediately.

For input width D and hidden width h, one direction has $4h(D + h)$ weights and either $4h$ combined biases or $8h$ separately stored input/recurrent biases. Counting the latter twice as two *effective* offsets would misdescribe the affine sum. Increasing hidden width has a quadratic recurrent cost. The selected 256 and two layers are architecture settings, not uniquely implied by CTC or handwriting.

6.1.5 CTC: paths, blank, collapse and a complete small example

Let $p_t(k)$ be the softmax probability of symbol k at frame t. For a path π of T symbols, its conditional score is $\prod_t p_t(\pi_t)$. Collapse B first merges consecutive identical symbols, then removes blank. Thus $(a, a, \varnothing, b, b)$ becomes ab, whereas $(a, \varnothing, a)$ becomes aa. Blank is a special alignment symbol, not the space character. A literal space is an ordinary label. CTC sums the scores of *all* paths collapsing to the target and takes negative logarithm [34].

Constructed two-frame example, target a: at frame one probabilities for $(a, \varnothing)$ are $(0.6, 0.4)$ and at frame two $(0.7, 0.3)$. The valid paths are $(a, a), (a, \varnothing), (\varnothing, a)$. Their total is $0.42+0.18+0.28 = 0.88$, so loss is $-\log 0.88 \simeq 0.1278$. The all-blank path scores 0.12 and produces the empty string. Computing only the best path would give 0.42 and a different training loss.

For target length U with r adjacent repeated pairs, at least $U + r$ frames are needed: each repeated adjacent label needs an intervening blank. For "book", $U = 4, r = 1$, so five frames suffice in principle. This condition is necessary, not evidence the network will learn good alignment.

Method | Dynamic summation, not a separate dynamic collapse algorithm.

Expand target labels to $\ell' = (\varnothing, \ell_1, \varnothing, \ldots, \ell_U, \varnothing)$, giving $2U + 1$ states. A transition can stay, advance one state, or skip one blank when the new label is nonblank and differs from the label two states back. Forward probability $\alpha_{t,s}$ sums valid prefixes ending at s, multiplying by the current emission. Initialise only the first blank and first label; terminate by summing the last label and last blank states. Log-domain arithmetic replaces sums by log-sum-exp to avoid underflow. For values a_i, use

$m + \log \sum_i e^{a_i - m}$ with $m = \max_i a_i$. Collapse itself is just the deterministic repeat/blank removal rule [42].

Greedy inference chooses $\arg\max_k p_t(k)$ independently at each frame, then collapses. It maximises a single path, not necessarily the total score of a collapsed transcript. Beam search considers several prefixes and can sum path scores; a language model can add linguistic assumptions. The reported reader uses greedy decoding without a language model.

6.1.6 Forced alignment and reading the CTC plot

Given a known transcript, forced alignment uses the same allowed state transitions but maximises the sum of log probabilities. Write $V_{t,s} = \log p_t(\ell'_s) + \max_{r \in \mathcal{P}(s)} V_{t-1,r}$. Store predecessors and backtrack. This Viterbi-style path is a timing/location explanation conditional on the supplied transcript, not a fresh proof that the transcript is correct. If a model is weak, a legal alignment can still put cuts inside letters. Forward–backward soft occupancy and hard best-path intervals answer different questions.

A frame edge t maps approximately to canvas coordinate $640t/T$, then to source $x = (640t/T)/a$ for left-aligned fit. With $T = 160, a = 0.5$, edge 10 maps to source $x = 80$. Padding offsets, rounding and endpoint conventions must be included when different preparation is used. CNN receptive fields and CTC's flexible blank placement mean this is an approximate geometric correspondence, not a character bounding-box measurement.

In the report's CTC plot, horizontal position is feature frame, each row a class, and the shown trace can be either a greedy class or a transcript-state alignment. Repeated g labels need a blank-separated path. Blank carrying about 46% of average probability mass is compatible with correct decoding: many frames are gaps or redundant representations. Probability mass is not the fraction of blank image pixels. Compare labels, axes and the two paths before interpreting the picture as segmentation evidence.

6.1.7 Writer head, K classes, dropout and stroke normalisation

The writer head averages the 160 feature columns, applies a 96-unit hidden layer with ReLU and a final K-logit layer. K is the number of writers in the loaded mapping, not the text alphabet size. The report evaluates eleven; current path lists can contain thirteen. Checkpoint class order determines which output index means which writer. A softmax over a closed set has no automatic "unknown writer" class. Pooling loses order but summarises distributed stroke features; it does not prove writer identity is independent of text content.

With dropout probability $p = 0.2$, training uses $\tilde{h} = mh/(1-p)$ for Bernoulli keep mask m of mean 0.8. Therefore $E[\tilde{h}] = h$. Inference removes the random mask. Dropout is published regularisation [43]; 0.2 is the project's chosen strength, not a universal optimum. For a 96-unit layer the expected number kept is 76.8, not exactly 77 on every pass.

For the shape classifier, binary ink is thinned and square-dilated to controlled width: $w = \text{clip}(\text{round}(\rho h_x), 2, 9)$, $k = 2\lfloor w/2 \rfloor + 1$, then white-background image $I_s = 255[1 - \delta_k(\text{thin}(M))]$. At $h_x = 40, \rho = 0.17$, $w = 7, k = 7$; at $h_x = 20$, $w = 3, k = 3$. If $w = 2$, the odd side is 3. Thus the effective width is quantised, not exactly ρh_x. Square dilation uses a Chebyshev neighbourhood; a disk is a different metric. Reducing pen-width cues can make geometry more comparable, but preprocessing can also erase genuine writer cues and amplify segmentation errors.

The report records widths 0.06, 0.09, 0.11 and 0.17, best epochs 38, 43, 39 and 43, and displayed accuracies 98.06%, 97.42%, 98.06% and 99.35%. These support "best observed among these four settings" for that selection experiment. The surrounding prose calls it page identification while Table 4 says per-line accuracy: the unit is unresolved here and must not silently be chosen. It is not an independent post-selection qualification score. The archived run settings and held-out unit would be needed to resolve the discrepancy. No stroke-width sweep was rerun.

Current function reference

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
CHARSET = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 .,;:\'"!?()-'
CHAR_TO_IDX = {c: i + 1 for i, c in enumerate(CHARSET)}
IDX_TO_CHAR = {i: c for c, i in CHAR_TO_IDX.items()}
WEIGHTS_DIR = paths.Weights
INPUT_HEIGHT = 64
INPUT_WIDTH = 640
DEFAULT_TEXT_WEIGHTS = WEIGHTS_DIR / 'paper_cnn_bilstm_ctc_scratch_best.npz'
DEFAULT_AUTHOR_WEIGHTS = WEIGHTS_DIR / 'author_shape_11w_pw170_scratch_weights.npz'
PenWidthXh = 0.17
```

Stage: Checkpoint and log persistence

Method rationale, alternatives and library comparison: Section 3.1.

6.2 LoadStateDictAny

```
def LoadStateDictAny(checkpointPath)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 31. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 3.1.

Purpose | What this contributes

Preserve weights, class mappings and evidence about a training run.

Inputs

`checkpointPath`
: path to a saved .npz checkpoint file

Output. a pair: a dictionary mapping parameter names to arrays, and the writer name-to-slot mapping if the file carries one, otherwise nothing.

Purpose | What it is for

A trained network is just a collection of named arrays, and this function reads that collection off disk. Its value is that it is the only place in the file that knows how a checkpoint is stored, so every network below receives a plain dictionary and none of them contains any file handling.

Method | Background: what a state dictionary is

A **state dictionary** is the standard way to store a trained network: a mapping from the name of each parameter to its values. A name like `stage1.0.weight` encodes a position in the network, here the weight array of layer 0 of the block called stage1. The network's structure is not stored at all; it is written in code, and loading consists of looking up the names that code expects. That is why the

forward methods below are full of string keys: those strings are the contract between the training script that wrote the file and the code that reads it.

Method | The .npz format

An `.npz` file is a zip archive of individual arrays, each stored under a name. That suits a state dictionary exactly, and it needs no library beyond numpy, which is the constraint this project works under.

Justification | Why pickling is refused

The file is opened with object pickling disabled. **Pickling** is a mechanism for storing arbitrary Python objects, and restoring one executes code from the file, so a checkpoint able to contain pickled objects is able to run anything when loaded. Refusing it restricts the file to plain numeric arrays, which is all a set of weights needs to be. It also acts as a check on the files themselves: a checkpoint that will not load without pickling is storing something other than arrays, and that is worth discovering rather than tolerating.

Method | Keys beginning with a double underscore

Names starting with a double underscore are skipped when building the parameter dictionary, because they hold metadata rather than weights. One is read specially: a list of writer names, which is converted into a mapping from each name to its output slot.

Carrying that mapping inside the checkpoint matters more than it appears. The namer outputs one score per slot, and a slot means nothing without knowing which writer it stands for. If the correspondence were kept separately, in a configuration file or in code, then a checkpoint trained on a different set of writers, or on the same writers in a different order, would load without complaint and attribute every page to the wrong person. Storing the names with the weights makes that mismatch impossible.

Stage: Sampling and geometric resampling

Report equation 18 Fixed canvas scale

Method | Formula

$a = \min(64/h, 640/w)$, with resized dimensions near (ah, aw).

Purpose | Why the system needs it

Preserve aspect ratio while fitting the checkpoint input. Long lines can become short vertically when width determines scale. Padding is vertically centred and left aligned.

Evidence | Current location and source

FitFixedCanvas, ResizeLineImageFixed and FrameXToPixel. [25, 3].

Method rationale, alternatives and library comparison: Section 5.26.

6.3 ResizeLineImageFixed

```
def ResizeLineImageFixed(img, height=INPUT_HEIGHT, width=INPUT_WIDTH, padValue=255)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 157. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`img` the line crop as a grey image

`height`
: the target height, default 64

`width`
: the target width, default 640

`padValue`
: the value to fill unused canvas with, default 255 for white

Output. the crop fitted onto a canvas of exactly the trained size.

Purpose | What it is for

The networks accept exactly one input size, for the reasons given under INPUT_HEIGHT. A crop cut from a page is whatever size that line of writing happened to be. This function reconciles the two, by delegating to FitFixedCanvas in segment.py.

Justification | Why the aspect ratio is preserved

The important property, explained in full in the entry for FitFixedCanvas, is that a single scale factor is applied to both axes, chosen so the crop fits inside the canvas, and the remainder is padded with white rather than stretched. Stretching each axis independently to fill the canvas would change the proportions of the letters, and proportion is precisely what distinguishes some letters from one another and one writer's hand from another's. A tall narrow l stretched to fill a wide canvas becomes something no letter looks like, and the filters trained on correctly proportioned letters would not find their features in it.

> *White, not black, is the pad value, because at this point the image is still in its natural form of dark ink on light paper. The inversion happens in the next function, and padding after inversion would require black instead. Getting this the wrong way round surrounds every crop with a solid block of apparent ink.*

6.4 TensorFromResized

```
def TensorFromResized(img)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 161. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`img` the fitted grey crop

Output. a floating-point array shaped (1, 1, height, width), scaled and inverted into the range the networks were trained on.

Purpose | What it is for

An image is 8-bit integers from 0 to 255 with white high. A network expects floating-point numbers of a particular scale and sign convention. This function performs that conversion, in three steps that must match exactly what was done during training.

Formula purpose and system use. Convert white-background grey pixels to the network ink polarity and scale. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$x \;=\; \frac{\left(1 - \frac{g}{255}\right) - 0.5}{0.5} \;=\; 1 - \frac{2g}{255} \tag{6.1}$$

where

x the value handed to the network, running from -1 for paper to $+1$ for black ink

g the original grey value, 0 for black and 255 for white

$\frac{g}{255}$ the grey value scaled to the range 0 to 1

$1 - \cdot$ the inversion, making ink high and paper low

$\frac{\cdot - 0.5}{0.5}$ the standardisation, mapping the range 0 to 1 onto -1 to $+1$

Justification | Why invert

After inversion, ink is a large number and blank paper is zero before standardising. This matters because of padding: every convolution pads with zeros, and zero must therefore mean nothing rather than something. With the original convention zero means black, so the padding round the edge of every crop would read as a border of solid ink and the first filters would spend their response on an artefact of the padding.

Justification | Why standardise to a range centred on zero

Networks train better when their inputs are centred near zero and of a magnitude near one, for the same reasons batch normalisation exists. Inputs in the hundreds would make the first layer's outputs correspondingly large and would demand very small initial weights to compensate. Mapping to -1 to $+1$ puts the input on the scale the initialisation assumed.

Justification | Why the two extra axes

The array is given the shape (1, 1, height, width) to match the four-axis convention described in the chapter introduction: one image in the batch, one channel in that image. The axes are necessary even though both lengths are 1, because every layer indexes by position and would otherwise misinterpret which axis is which.

> *These three steps are not a design decision that can be revisited independently; they are a reproduction of what the training code did. A network is only correct on inputs prepared the way its training inputs were, and a mismatch here, such as a forgotten inversion, produces a network that loads without error and returns confident nonsense.*

6.5 FrameXToPixel

```
def FrameXToPixel(canvasX, rawW, origH)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 251. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.26.

Purpose | What this contributes

Fit captured lines to the trained canvas while limiting lost detail and retaining coordinate mappings.

Inputs

`canvasX`
: a horizontal position on the network's input canvas

`rawW`
: the width of the original crop in pixels

`origH`
: the height of the original crop in pixels

Output. the corresponding position in the original crop, clamped to lie within it.

Purpose | What it is for

An alignment span is measured in network columns, but a glyph has to be cut from the original crop in original pixels. This function reverses the geometric transformation that got the crop onto the canvas, so a column index becomes a pixel position.

Method | Reversing the fit

The fit applied one scale factor to both axes, chosen as the largest that lets the crop fit inside the canvas, which is the smaller of the two ratios. Undoing it is division by that same factor.

Formula purpose and system use. Undo the fixed-canvas scale to locate an alignment boundary in the source crop. **Source basis:** [25, 26, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$s = \min\left(\frac{H_{\text{in}}}{H_{\text{orig}}}, \frac{W_{\text{in}}}{W_{\text{raw}}}\right), \qquad x_{\text{orig}} = \text{clip}\left(\frac{x_{\text{canvas}}}{s}, 0, W_{\text{raw}}\right) \tag{6.2}$$

where

s the scale factor that was applied when fitting the crop

H_{in}, W_{in}
: the network's canvas height and width, 64 and 640

H_{orig}, W_{raw}
: the original crop's height and width

x_{canvas}
: the position on the canvas

x_{orig}
: the corresponding position in the original crop

$\mathrm{clip}(\cdot, 0, W_{\mathrm{raw}})$
: the result held inside the crop

Justification | Why the smaller ratio, and why no offset

Taking the smaller of the two ratios is what guarantees the crop fits on both axes; the larger would overflow one of them. There is no offset term because the fit is left aligned horizontally, so the horizontal origins coincide; its vertical placement is centred. Had the crop been centred, this conversion would need to subtract the left margin first.

Justification | Why the result is clamped

A column index can map slightly beyond the crop, because the network's columns are a coarser grid than the original pixels and the last one may extend past the real content into the padding. Clamping keeps the returned position usable as an index without the caller checking. The value is returned as a floating-point number rather than an integer because the caller may want to interpolate between pixels, and rounding here would discard precision that was correctly computed.

Stage: Skeletonisation and centreline recovery

Report equation 24 Controlled stroke width

Method | Formula

$I_s = 255[1 - \delta_k(\mathrm{thin}(M))]$, $w = \mathrm{clip}(\mathrm{round}(\rho h_x), 2, 9)$, $k = 2\lfloor w/2 \rfloor + 1$.

Purpose | Why the system needs it

Reduce pen-width cues before shape-based identification. The effective odd kernel quantises the requested relative width; thinning can change geometry. Current PenWidthXh is 0.17. Historical sweep selection is not an independent qualification of this source snapshot.

Evidence | Current location and source

StrokeNormalize and ClassifyImage. [31, 10, 3].

Method rationale, alternatives and library comparison: Section 5.61.

6.6 StrokeNormalize

```
def StrokeNormalize(gray, penWidthXh=PenWidthXh)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 171. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.61.

Purpose | What this contributes

Convert thick ink areas into approximate pen paths and normalise stroke width.

Inputs

`gray`
: a line crop as a grey image

`penWidthXh`
: the pen width to redraw at, in x-heights, defaulting to the constant above

Output. the writing redrawn at one constant pen width as ink on white, or nothing if the crop has no usable ink.

Purpose | The problem it solves

A network asked to identify a writer from a raw crop will use whatever is most reliably distinctive, and that is not necessarily the handwriting. Pen width and ink darkness are extremely easy to measure and happen to be consistent within each writer's pages, because each writer used one pen. A network can reach high accuracy on such a dataset while having learned nothing about letterforms at all. This is a **confound**: a variable correlated with the answer that is not the thing being studied.

The confound becomes a real failure precisely where the project needs the namer most. The robot draws with one pen, so every writer's reproduced output has identical stroke width. A namer relying on width therefore has nothing to go on and is being asked a question it was never really trained for. Removing pen width from the input forces the network to use shape, which is the property that does survive reproduction.

Method | The method: thin, then re-ink

The writing is reduced to its centre line and then redrawn at a chosen width. Whatever the original thickness, the output thickness is the same, so the information is genuinely destroyed rather than merely de-emphasised.

- **Binarise.** The crop is thresholded into ink and paper by BinarizeLine.
- **Measure the x-height.** CoreBand finds the band of rows holding the letter bodies, and its height is the x-height, which sets the scale for the pen width.
- **Thin.** Skeletonize reduces the ink to a one-pixel-wide centre line by the Zhang-Suen method [31], which removes boundary pixels repeatedly while preserving connectivity, so the skeleton has the same topology as the original writing.
- **Re-ink.** The skeleton is dilated by a square of the chosen width, which draws a stroke of that width centred on the centre line, and the result is inverted back to dark ink on white paper.

Justification | Why the pen width is forced to be odd

Formula purpose and system use. Convert a requested x-height-relative stroke width into the actual odd dilation window. **Source basis:** [31, 32]. Exact project constants remain implementation choices unless evidence is stated.

$$k \;=\; 2\left\lfloor \frac{w}{2} \right\rfloor + 1, \qquad w \;=\; \mathrm{clip}\big(\mathrm{round}(0.14 \cdot \mathrm{xh}),\; 2,\; 9\big) \tag{6.3}$$

where

k the structuring element size actually used, always odd

w the pen width in pixels before being made odd

xh the measured x-height in pixels

0.14 the pen width as a fraction of the x-height

$\mathrm{clip}(\cdot, 2, 9)$
: the width held between 2 and 9 pixels, so a tiny crop still gets a visible stroke and a large one does not get a blob

$\lfloor \cdot \rfloor$ rounding down

A dilation by an even-sized square has no centre cell, so the redrawn stroke is offset by half a pixel from the skeleton it was drawn around. Over a whole line that shifts the writing slightly and, worse, shifts it in a direction that depends on the rounding, so two crops of the same writing could be offset differently. Forcing the size odd puts the pen exactly on the centre line.

Method | The x-height fallback

If no usable band is found, or the band is under four pixels tall, the x-height is taken as a third of the crop height instead, with a floor of eight pixels. A crop is built around one line of writing whose body occupies roughly the middle third, so that is a reasonable estimate when measurement fails, and it keeps the function returning something usable rather than propagating a nonsensical width.

> *Two cases return nothing at all: a crop with no ink, and one whose skeleton is empty. Returning nothing rather than a blank image is deliberate, because the caller must be able to tell "normalisation failed" from "this line is blank" and fall back to the raw crop. ClassifyImage does exactly that.*

Stage: Convolution and feature sequences

Report equation 19 Convolution response

Method | Formula

$Z_{o,y,x} = b_o + \sum_{c,i,j} W_{o,c,i,j} X_{c,y+i,x+j}$ on the appropriately padded array.

Purpose | Why the system needs it

Extract local learned stroke patterns shared across positions. This is cross-correlation in common framework terminology. A feature channel is not automatically a labelled character detector.

Evidence | Current location and source

Conv2d and the train-mode convolution pair. [35, 36].

Alternatives | LeNet five and residual networks

The report mentions LeNet-5 as a historical convolution/pooling example and residual networks as a deeper architecture with shortcut paths. A residual block computes $y = x + F(x)$ (or a projection when dimensions differ); the extra route can help gradient propagation. Neither is the current twelve-convolution, two-BiLSTM reader. Changing the backbone would require compatible weights and new evaluation; citing a benchmark architecture does not establish better accuracy or speed for this photographed dataset [44, 45]. Its historical radial-basis output compares feature position with learned class representatives through a distance-based response; it is not the current affine writer head or framewise text softmax. ANN is the broad neural-network family, CNN specifies local shared spatial weights, and RNN specifies a state carried along a sequence. These names are not separate active modules merely because they appear in the literature chapter.

6.7 Method background: Convolution and feature sequences

Purpose | The role in this system

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Justification | The reason for this construction

Weight sharing reduces parameters compared with a fully connected image mapping. Patch-to-matrix multiplication reaches compiled NumPy linear algebra while exposing the computation.

Alternatives | What changes if another method is used

Fully connected images ignore locality; larger kernels have more parameters; residual backbones add skip paths and different capacity. A replacement normally requires new weights.

Library | Implementation speed and accuracy

PyTorch Conv2d offers CPU/GPU backend kernels and autodiff. Match cross-correlation, weights, stride, padding and dtype for arithmetic comparisons. The current NumPy layer stacks one matrix product per batch item; stride views can still lead to reshape copies. No fixed speed factor is asserted. [35, 36, 46, 2].

6.8 Conv2d

```
def Conv2d(x, weight, bias, stride=1, padding=1)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 41. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`x` the input, shaped (batch, channels, height, width)

`weight`
: the filters, shaped (out channels, in channels, filter height, filter width)

`bias`
: one number added per output channel, or nothing

`stride`
: how many pixels the filter advances between positions, default 1

`padding`
: how many rows and columns of zeros to add round the input before filtering, default 1

Output. the filtered result, shaped (batch, out channels, output height, output width).

Method | What a convolution is, from scratch

A **convolution** takes a small grid of numbers, called a **filter** or **kernel**, places it over a patch of the image, multiplies each filter number by the image value beneath it, and adds the products into one output number. Then it moves along and does it again, covering every position. The result is a new image, one value per position, recording how strongly the pattern in the filter was present there.

Formula purpose and system use. Compute one learned local stroke response from all input channels. **Source basis:** [35, 36, 46, 2]. Exact project constants remain implementation choices unless evidence is stated.

$$y[o,\,p,\,q] \;=\; b[o] \;+\; \sum_{c=0}^{C-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} w[o,\,c,\,u,\,v] \cdot x[c,\;p\,s+u,\;q\,s+v] \tag{6.4}$$

where

$y[o, p, q]$
: the output value for filter o at row p, column q

$b[o]$
: the bias, one learned number added to every position of output channel o

c
: which input channel, running over all C of them

u, v
: the position inside the filter

k_h, k_w
: the filter's height and width, 3 by 3 everywhere in this project

$w[o, c, u, v]$
: the learned filter weight at that position

$x[c, y, x]$
: the input value in channel c

s
: the stride, how far the filter moves between output positions

Read the triple sum as: for one output position, visit every cell of the filter in every input channel, multiply by the input underneath, and total it. The sum over channels is the part that is easy to overlook and important: a filter is not a flat grid but a stack as deep as the input has channels, so it combines evidence from every feature map below it rather than looking at one.

Justification | Why this is the right operation for handwriting

Two properties make convolution suited to images. It is **local**: each output depends only on a small patch, which matches the fact that a stroke, a corner or a loop is a local arrangement of ink. And it is **translation-equivariant**, meaning that moving the input moves the output by the same amount without changing it: the same filter detects a vertical stroke wherever on the line it occurs. A plain fully connected layer has neither property; it would have to learn what a vertical stroke looks like separately at every position, needing enormously more parameters and far more examples.

Method | What padding is for

A 3 by 3 filter cannot be centred on a pixel in the outermost row, because part of it would hang off the edge, so without intervention each convolution would shrink the image by one pixel on every side. **Padding** adds a border of zeros so the filter can reach the edge, and a padding of 1 with a 3 by 3 filter keeps the output exactly the same size as the input. With twelve convolutions stacked in this network, the unpadded alternative would remove twenty-four pixels in total across twelve convolutions, before accounting for any intervening pooling. Zero is the correct pad value because the input has been arranged so that zero means no ink; see TensorFromResized.

Formula purpose and system use. Determine output height before allocating the feature tensor. **Source basis:** [35, 36, 46, 2]. Exact project constants remain implementation choices unless evidence is stated.

$$H_{\text{out}} \;=\; \left\lfloor \frac{H + 2\,\text{pad} - k_h}{s} \right\rfloor + 1 \tag{6.5}$$

where

H_{out}
: the output height

H
: the input height

pad
: the padding added on each side

k_h
: the filter height

s
: the stride

$\lfloor \cdot \rfloor$
: rounding down, because a filter position that does not fit entirely inside the padded input is not used

Justification | The im2col trick, and why it is used

Written literally the definition is a loop over output positions inside a loop over filters, which in Python would be unusably slow. The standard remedy is **im2col**, short for image to columns [47]: rearrange the input so that every patch the filter will see becomes one column of a large matrix. The convolution then becomes a single matrix multiplication, because that is exactly what a matrix product computes, a dot product of every row of one matrix with every column of the other.

Formula purpose and system use. Re-express the same local dot products as a matrix multiplication with explicit dimensions. **Source basis:** [35, 36, 46, 2]. Exact project constants remain implementation choices unless evidence is stated.

$$Y = W_{\text{flat}} X_{\text{cols}}, \qquad W_{\text{flat}} \in \mathbb{R}^{O \times (C k_h k_w)}, \qquad X_{\text{cols}} \in \mathbb{R}^{(C k_h k_w) \times (H_{\text{out}} W_{\text{out}})} \tag{6.6}$$

where

Y
: the output, as a matrix of one row per filter and one column per output position

W_{flat}
: the filters flattened so each becomes one row of length C times the filter area

X_{cols}
: the patches, each flattened into one column in the same order

O
: the number of filters, that is, output channels

$\mathbb{R}^{m \times n}$
: the set of real-valued matrices with m rows and n columns; this notation just states each array's shape

The gain is not from doing less arithmetic, since the number of multiplications is unchanged. It is that matrix multiplication is handed to **BLAS**, a long-standing library of linear algebra routines written to use the processor's vector instructions and its cache well. A matrix product can substantially reduce interpreter overhead relative to Python loops, with the actual factor depending on shape, backend and hardware, and reformulating the problem to reach it is worth more than any amount of tuning the loops.

Method | The strided view and reshape storage

The patches are not assembled by copying. Neighbouring patches overlap heavily, a 3 by 3 filter at stride 1 sharing six of nine pixels with its neighbour, so a literal im2col would duplicate most of the input nine times over. Instead the columns are produced as a **strided view**, which needs some explanation.

A numpy array is a flat block of memory plus a set of **strides**, one per axis, each saying how many bytes to step to advance by one along that axis. Indexing is then just arithmetic on those strides.

Supplying a different shape and a different set of strides therefore produces an array that reads the same memory in a different pattern, as a view without an initial patch copy; a later reshape can still materialise storage. Here a six-axis view is constructed whose axes are batch, channel, filter row, filter column, output row and output column, so that the patches appear laid out ready to be reshaped into columns. The strides for the two output axes are the input's own row and column strides multiplied by the stride, which is what makes stepping to the next output position step the right distance through the image.

> *A strided view of overlapping patches reports a shape far larger than the memory behind it, because the same bytes appear at many positions. That is safe to read and would be meaningless to write to. The reshape that follows does force one copy, since the pattern cannot be expressed as strides alone, but it is one copy instead of nine.*

Method | The bias

One learned number per output channel is added to every position of that channel. It lets a filter shift its whole response up or down, which matters because the squashing that follows treats zero specially: without a bias, a filter could only ever decide between zero and its raw response, whereas with one it can set the level at which it starts responding at all. Adding one number to a whole channel is written as a reshape to four axes and a plain addition, relying on **broadcasting**, numpy's rule that an axis of length 1 is repeated as needed to match the other operand.

6.9 ConvBlockForward

```
def ConvBlockForward(x, sd, prefix, nLayers)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 113. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`x` the input feature maps

`sd` the loaded state dictionary of parameters

`prefix`
: the name prefix identifying this block in the checkpoint, such as stage1

`nLayers`
: how many convolution, normalise, activate groups this block contains

Output. the feature maps after that many groups have been applied.

Purpose | What it is for

Both networks are built from the same repeated unit: a convolution, then batch normalisation, then ReLU. This function applies that unit a given number of times, so the unit is written once rather than twelve times in each network.

Justification | Why that particular order

The order is not arbitrary and each adjacency has a reason. Normalisation comes after the convolution because its purpose is to fix the scale of the convolution's output, which it cannot do before that output exists. ReLU comes after normalisation because the normalisation needs to see the signed values to standardise them meaningfully; applied to an already rectified signal, with its mass piled at zero, the mean and variance would describe mostly how many units were silent. And ReLU must come before the next convolution, since the whole point of an activation is to break the chain of linear operations.

A further consequence of this order is that the convolution's bias is redundant: the normalisation immediately subtracts a mean, which removes any constant the bias added. It is kept because the checkpoint stores it and the trained values must be applied as they were learned, but it is worth knowing that it has no effect on the result.

Method | How a layer is located in the checkpoint

Within a block the layers are numbered in threes, so group i occupies positions 3i for the convolution and 3i+1 for the normalisation, the activation having no parameters and so no position of its own. The names are built by joining the prefix and those numbers, which is how a checkpoint saved by the training framework names its layers. The pattern is effectively a contract, and the next entry describes what happens when it is relied on across different files.

Justification | The three stages, and why they get deeper

- **stage1, two groups,** on the full-resolution crop, where a filter sees a 3 by 3 patch of actual pixels and so can only detect the smallest features: an edge, a stroke, the direction of a line.
- **stage2, four groups,** after one pooling, where features combine into corners, junctions and curves.
- **stage3, six groups,** after a second pooling, where a filter's receptive field covers a large part of a letter and the features become letter-like.

The number of groups increases with depth because the number of distinguishable patterns does. There are few ways to arrange ink in a 3 by 3 patch and a great many ways to arrange strokes into a letter, so the later stages need more capacity. Placing the deepest stage after two poolings is also what makes this affordable: it runs on a sixteenth of the original area, so its six groups cost less than stage1's two.

6.10 PaperCRNNNumpy

```
class PaperCRNNNumpy
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 259. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

The callable method is Forward with a capital F. Two, four and six convolution blocks are separated by two spatial pools; a height maximum yields 160 feature columns for a 64 by 640 input. Two bidirectional recurrent layers feed the text head. The constructor default reader is the scratch checkpoint; START.sh selects the joint reader through READER_WEIGHTS instead.

Method | Methods exposed by this current class

```
def __init__(self, checkpointPath=None, hiddenSize=256)
def Forward(self, x)
```

Method | Current implementation

Extract local stroke features shared across locations before sequence recognition or writer pooling. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class PaperCRNNNumpy:
    def __init__(self,checkpointPath=None,hiddenSize=256):
        self.checkpointPath = Path(checkpointPath or DEFAULT_TEXT_WEIGHTS)
        self.sd,_ = LoadStateDictAny(self.checkpointPath)
        self.hiddenSize = hiddenSize

        self.lstmLayerParams = []
        for l in range(2):
            def Get(name,layer=l):
                return self.sd[f"sequence.{name}_l{layer}"]

            fwd = dict(weightIh=Get("weight_ih"),weightHh=Get("weight_hh"),
                       biasIh=Get("bias_ih"),biasHh=Get("bias_hh"))
            bwd = dict(weightIh=self.sd[f"sequence.weight_ih_l{l}_reverse"],
                       weightHh=self.sd[f"sequence.weight_hh_l{l}_reverse"],
                       biasIh=self.sd[f"sequence.bias_ih_l{l}_reverse"],
                       biasHh=self.sd[f"sequence.bias_hh_l{l}_reverse"])
            self.lstmLayerParams.append({"fwd":fwd,"bwd":bwd})

    def Forward(self,x):
        sd = self.sd
        x = ConvBlockForward(x,sd,"stage1",2)
        x = MaxPool2d(x,2,2)
        x = ConvBlockForward(x,sd,"stage2",4)
        x = MaxPool2d(x,2,2)
        x = ConvBlockForward(x,sd,"stage3",6)
        x = AdaptiveMaxPoolHeight(x)
        x = x[:,:,0,:]
        x = np.transpose(x,(2,0,1))   #width becomes the CTC time axis
        seq = MultilayerBidirectionalLstm(x,self.lstmLayerParams,self.hiddenSize)
        logits = Linear(seq,sd["text_head.weight"],sd["text_head.bias"])
        return LogSoftmax(logits,axis=2)
```

Justification | Interpret the method rather than infer a benchmark

Weight sharing reduces parameters compared with a fully connected image mapping. Patch-to-matrix multiplication reaches compiled NumPy linear algebra while exposing the computation. [35, 36, 46, 2].

6.11 AuthorClassifierCNNNumpy

```
class AuthorClassifierCNNNumpy
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 325. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

The current method is Forward. After the convolution backbone, height maximum and width average, a 96-unit hidden affine/ReLU head produces one logit per saved writer mapping entry. Dropout is

skipped at inference. The default file is author_shape_11w_pw170_scratch_weights.npz; the server INK_MODEL and SHAPE_MODEL variables reference this same loaded object.

Method | Methods exposed by this current class

```
def __init__(self, checkpointPath=None)
def Forward(self, x)
```

Method | Current implementation

Extract local stroke features shared across locations before sequence recognition or writer pooling. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class AuthorClassifierCNNNumpy:
    def __init__(self,checkpointPath=None):
        self.checkpointPath = Path(checkpointPath or DEFAULT_AUTHOR_WEIGHTS)
        self.sd,self.authorMapping = LoadStateDictAny(self.checkpointPath)

    def Forward(self,x):
        sd = self.sd
        x = ConvBlockForward(x,sd,"stage1",2)
        x = MaxPool2d(x,2,2)
        x = ConvBlockForward(x,sd,"stage2",4)
        x = MaxPool2d(x,2,2)
        x = ConvBlockForward(x,sd,"stage3",6)
        x = AdaptiveMaxPoolHeight(x)
        x = x[:,:,0,:]
        pooled = AdaptiveAvgPoolWidth(x)[:,:,0]
        h = Linear(pooled,sd["author_head.0.weight"],sd["author_head.0.bias"])
        h = Relu(h)
        logits = Linear(h,sd["author_head.3.weight"],sd["author_head.3.bias"])
        return logits
```

Justification | Interpret the method rather than infer a benchmark

Weight sharing reduces parameters compared with a fully connected image mapping. Patch-to-matrix multiplication reaches compiled NumPy linear algebra while exposing the computation. [35, 36, 46, 2].

6.12 ClassifyImage

```
def ClassifyImage(image, model, authorMapping=None, strokeNormalise=True)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 348. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Optionally binarises, thins and redilates the input, fits the fixed canvas, calls model.Forward and returns argmax index plus the full softmax vector. The optional authorMapping argument is not used to remap outputs. Decode indices with model.authorMapping; its order is part of the checkpoint contract.

Method | Current implementation

Extract local stroke features shared across locations before sequence recognition or writer pooling. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ClassifyImage(image,model,authorMapping=None,strokeNormalise=True):
    if strokeNormalise:
        normImg = StrokeNormalize(np.asarray(image,dtype=np.uint8))
        if normImg is not None:
            image = normImg
    t = TensorFromResized(ResizeLineImageFixed(image))
    logits = model.Forward(t)
    probs = Softmax(logits,axis=1)[0]
    predIdx = int(np.argmax(probs))
    return predIdx,probs
```

Justification | Interpret the method rather than infer a benchmark

Weight sharing reduces parameters compared with a fully connected image mapping. Patch-to-matrix multiplication reaches compiled NumPy linear algebra while exposing the computation. [35, 36, 46, 2].

Stage: Batch normalisation

Report equation 20 Batch normalisation

Method | Formula

$\hat{z} = (z - \mu)/\sqrt{v + \epsilon}$ and $y = \gamma\hat{z} + \beta$.

Purpose | Why the system needs it

Control activation scale while allowing a learned affine response. Training uses current batch statistics; inference uses saved running statistics. Changing that mode changes the operation.

Evidence | Current location and source

BatchNorm2d and BatchNorm2dForwardTrain/Backward. [37, 38].

6.13 Method background: Batch normalisation

Purpose | The role in this system

Control activation scale during learning and apply stored training statistics during inference.

Justification | The reason for this construction

Learned scale and offset preserve representational freedom after normalisation; inference folds fixed statistics into an affine calculation.

Alternatives | What changes if another method is used

Layer normalisation normalises a different set of values and does not use the same batch statistics. Omitting normalisation changes learning dynamics and usually requires retraining.

Library | Implementation speed and accuracy

PyTorch BatchNorm2d distinguishes train/eval state and running statistics. Match epsilon, momentum convention and biased/unbiased variance treatment. A numerical match does not establish equal independently trained accuracy. [37, 38].

6.14 BatchNorm2d

```
def BatchNorm2d(x, weight, bias, runningMean, runningVar, eps=1e-05)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 62. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.13.

Purpose | What this contributes

Control activation scale during learning and apply stored training statistics during inference.

Inputs

`x` the input, shaped (batch, channels, height, width)

`weight`
: the learned scale, one per channel

`bias`
: the learned shift, one per channel

`runningMean`
: the mean of each channel recorded during training

`runningVar`
: the variance of each channel recorded during training

`eps` a small constant guarding the division, default 1e-5

Output. the input with each channel standardised and then rescaled, same shape as the input.

Purpose | The problem it solves

Stack a dozen layers and the scale of the numbers flowing through them drifts. Each layer multiplies by its weights, so if those weights are on average slightly larger than the scale that preserves magnitude, the values grow layer by layer; slightly smaller, and they shrink toward nothing. Worse, the scale a layer sees changes during training, because the layers beneath it are themselves being changed. A layer is therefore chasing a moving target, which forces a small learning rate and slow training. **Batch normalisation** [37] fixes the scale explicitly: it standardises each channel so that it arrives at the next layer with a known mean and spread.

Method | Background: mean, variance and standardising

The **mean** is the average value. The **variance** is the average of the squared distances from the mean, a measure of how spread out the values are, and its square root is the **standard deviation**, which is in the same units as the values themselves. To **standardise** is to subtract the mean and divide by the standard deviation, which leaves a quantity with mean 0 and standard deviation 1 regardless of what scale it started on.

Formula purpose and system use. Standardise a channel using the saved training mean and variance. **Source basis:** [37, 38]. Exact project constants remain implementation choices unless evidence is stated.

$$\hat{x}[c] = \frac{x[c] - \mu[c]}{\sqrt{\sigma^2[c] + \epsilon}} \tag{6.7}$$

where

$\hat{x}[c]$ the standardised value in channel c

$x[c]$ the incoming value

$\mu[c]$ the mean of channel c, recorded during training

$\sigma^2[c]$ the variance of channel c, also recorded during training

ϵ a small constant, 0.00001, added so that a channel with no variation cannot cause a division by zero

Justification | Why the scale and shift are put back

Forcing every channel to mean 0 and spread 1 would be too strong a constraint, because for some features the useful information is precisely that they are usually large or usually near zero. So two learned numbers per channel are applied afterwards, a scale and a shift, letting the network restore any mean and spread it finds useful, including the original one. The point is not that the statistics are fixed at 0 and 1; it is that they are now set by two parameters the network controls directly, instead of emerging from the accumulated behaviour of every layer below.

Formula purpose and system use. Restore a learned scale and offset after normalisation. **Source basis:** [37, 38]. Exact project constants remain implementation choices unless evidence is stated.

$$y[c] \;=\; \gamma[c]\,\hat{x}[c] \;+\; \beta[c] \tag{6.8}$$

where

$y[c]$ the output value in channel c

$\gamma[c]$ the learned scale for that channel, held in the argument called weight

$\beta[c]$ the learned shift, held in the argument called bias

$\hat{x}[c]$ the standardised value from above

Method | Training-time and inference-time behaviour differ

The method takes its name from how the statistics are obtained during training: from the current **batch**, the group of examples being processed together. That cannot be done when reading one crop. A single example's own mean and variance are not a useful description of anything, and worse, the answer for one image would depend on which other images happened to accompany it, which is unacceptable at inference time.

The resolution is that training also keeps a running average of the statistics across all batches, which estimates the mean and variance over the whole dataset. Those stored values are what this function uses. So the layer behaves differently in training and in use, and that difference is intended: in training it standardises by the batch, in use by fixed numbers read from the checkpoint. This is why `runningMean` and `runningVar` are arguments here while a training implementation computes them.

Method | Folding four operations into two

Written directly the layer is a subtraction, a division, a multiplication and an addition at every one of the hundreds of thousands of values in the array. But at inference time all four constants are fixed, so the whole thing is a linear function and can be rearranged into one multiplication and one addition.

Formula purpose and system use. Fold the fixed inference statistics into one scale and one shift per channel. **Source basis:** [37, 38]. Exact project constants remain implementation choices unless evidence is stated.

$$y \;=\; \underbrace{\frac{\gamma}{\sqrt{\sigma^2+\epsilon}}}_{a}\; x \;+\; \underbrace{\left(\beta - \frac{\gamma\,\mu}{\sqrt{\sigma^2+\epsilon}}\right)}_{b} \tag{6.9}$$

where

a the combined scale, computed once per channel

b the combined shift, computed once per channel

γ, β the learned scale and shift

μ, σ^2 the stored mean and variance

ϵ the guard constant

The two combined constants are computed once per channel, over an array of a few hundred numbers, and then applied to the full array in one multiply and one add. The square root, in particular, happens once per channel rather than once per pixel. This rearrangement is exact, not an approximation, and it is the whole reason the implementation is two lines.

Justification | Why it is per channel and not per pixel

There is one mean and one variance for each channel, shared by every pixel in it. That follows from what a channel is: the map of where one particular filter responded. A filter's typical response size is a property of that filter, the same wherever on the image it fires, so one statistic describes the whole map. Normalising per pixel would instead remove the differences between positions, which is the information the map exists to carry.

Stage: Nonlinear activation and affine transforms

6.15 Method background: Nonlinear activation and affine transforms

Purpose | The role in this system

Let stacked learned layers express nonlinear image and sequence relationships, or convert features into class scores.

Justification | The reason for this construction

ReLU passes positive responses; sigmoid gates bound retention factors; tanh bounds a candidate state. An affine layer combines features with learned biases.

Alternatives | What changes if another method is used

Leaky ReLU avoids a zero negative slope; GELU is smoother but costs more and changes the network. A gate needs a bounded function rather than ordinary ReLU. These are architecture decisions, not interchangeable inference switches.

Library | Implementation speed and accuracy

PyTorch supplies Linear, ReLU, Sigmoid and related layers. NumPy already executes elementwise primitives in compiled loops. Small standalone library calls may cost more overhead than they save; fused network execution has different trade-offs. [39, 48, 49].

6.16 Relu

```
def Relu(x)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 68. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.15.

Purpose | What this contributes

-Relu: keeps positives, sets negatives to zero.

Inputs

`x` any array

Output. the array with every negative value replaced by zero.

Justification | Why a network needs this at all

Convolution and the linear layer are both **linear** operations, meaning the output is a weighted sum of the inputs. Composing linear operations gives another linear operation: two matrix multiplications are equivalent to one by the product of the matrices. So a stack of a hundred linear layers computes exactly what a single one could, and depth buys nothing. Something non-linear must sit between them, and that is what an **activation function** is for.

Formula purpose and system use. Introduce a nonlinearity by preserving positive responses and suppressing negatives. **Source basis:** [39, 48, 49]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathrm{ReLU}(x) \;=\; \max(x,\,0) \;=\; \begin{cases} x & x > 0 \\ 0 & x \le 0 \end{cases} \tag{6.10}$$

where

$\mathrm{ReLU}(x)$
: the output, named for rectified linear unit

x the input value

$\max(x, 0)$
: the larger of x and zero

$\{$ case notation: the two lines give the value in each of the two situations listed on the right

Justification | Why this particular function

ReLU is nearly the simplest non-linear function there is, and that is much of its appeal, but it has three concrete advantages over the smooth S-shaped functions that preceded it. Its derivative is 1 for every positive input, so a gradient passing back through many layers is not repeatedly multiplied by a number smaller than one, which is what made deep networks with sigmoid activations so hard to train. It produces exact zeros, so a layer's output is **sparse**, with many units silent on any given input, which tends to give cleaner and more separable features. And it costs one comparison, against an exponential for the alternatives.

Method | What it means in this network

After a convolution, a positive value means the filter's pattern was found and a negative one means something like its opposite was. ReLU discards the negatives entirely, which is to say the layer reports where each feature *was* present and stays silent about where it was not. The next layer therefore builds on evidence of presence rather than on a signed score, which is the behaviour that makes stacked filters compose into detectors for larger shapes: strokes into corners, corners into loops, loops into letters.

A unit whose input is always negative outputs zero always, and its local derivative is zero there; it receives no direct gradient through that inactive response, though upstream changes can alter future inputs. This is known as a dying unit, and it is one training limitation of ReLU. It is a training-time concern, handled in train.py by careful initialisation, and has no effect on the forward pass here.

6.17 Sigmoid

```
def Sigmoid(x)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 72. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.15.

Purpose | What this contributes

Squashes any number into 0 to 1.

Inputs

`x` any array

Output. the array squashed into the range 0 to 1.

Purpose | What it is for

Where ReLU is used to introduce non-linearity, the sigmoid is used when a number has to be interpreted as a **proportion**: how much of something to let through, on a scale from none to all. In this file its only use is in the LSTM, whose gates each need exactly that.

Formula purpose and system use. Bound an LSTM gate between zero and one so it can regulate information flow. **Source basis:** [39, 48, 49]. Exact project constants remain implementation choices unless evidence is stated.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{6.11}$$

where

$\sigma(x)$ the sigmoid of x, a value strictly between 0 and 1

x the input, any real number

e the base of the natural logarithm, about 2.71828

e^{-x} the exponential of minus x: large when x is very negative, near zero when x is large and positive

Justification | Why the shape is right for a gate

Follow the formula at three points. For a large positive x the exponential is near zero, so the result approaches 1, fully open. For a large negative x the exponential is huge, so the result approaches 0, fully closed. At zero the exponential is exactly 1 and the result is exactly one half. The curve is smooth and increases everywhere, so the gate can take any intermediate position, and because it is smooth it has a derivative everywhere, which training requires. The S shape also means it saturates: once a gate is firmly open or firmly closed, moderate changes in the input barely move it, which is the stability a memory mechanism needs.

For a very large negative input the exponential overflows and numpy issues a warning while returning infinity, whose reciprocal is correctly zero. The answer is right; only the warning is spurious. The implementation is left in its direct form because the alternative formulation adds a branch to every element of every gate at every time step to suppress a warning that misrepresents nothing.

6.18 Linear

```
def Linear(x, weight, bias)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 95. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.15.

Purpose | What this contributes

Let stacked learned layers express nonlinear image and sequence relationships, or convert features into class scores.

Inputs

`x` the input, whose last axis is the feature vector

`weight`
: the weight matrix, shaped (outputs, inputs)

`bias`
: one number added per output, or nothing

Output. the input multiplied by the transposed weight matrix, with the bias added; the last axis becomes the number of outputs.

Method | What it is

The **fully connected** or **linear** layer, the most basic layer there is: every output is a weighted sum of every input, plus a constant. Where a convolution looks at a local patch and reuses one filter everywhere, a linear layer looks at the whole input vector and has an independent weight for each input-output pair.

Formula purpose and system use. Map the current feature vector into learned output scores using an affine transformation. **Source basis:** [39, 48, 49]. Exact project constants remain implementation choices unless evidence is stated.

$$y[j] \;=\; b[j] \;+\; \sum_{i=0}^{n-1} W[j,\, i]\, x[i] \tag{6.12}$$

where

$y[j]$ the j-th output

$b[j]$ the bias for that output

$W[j, i]$
: the weight connecting input i to output j

$x[i]$ the i-th input

n the number of inputs

Justification | Why the matrix is transposed

The weight array is stored with one row per output, which is the convention the training framework used, while the input has its features along its last axis. Multiplying the input by the transpose pairs each input vector with each row of weights, giving one output per row. Transposing is free in numpy, being only a reordering of the strides, so this costs nothing and keeps the stored weights in the layout the checkpoint was written with.

Method | Where it is used in each network

In the reader it is the final step, turning the LSTM's output at each column into one score per character. The weighted sum is the right operation there because any feature of the column may bear on any character. In the namer it appears twice, as a hidden layer with a ReLU between and then a final layer producing one score per writer, which is enough to combine the pooled features non-linearly before the decision.

> *The multiplication applies to the last axis and leaves every earlier axis alone, so the same call works on a single vector, on a batch of them, or on the reader's three-axis sequence of columns. That is broadcasting again, and it is why no reshaping surrounds either use.*

Stage: Pooling and sequence formation

6.19 Method background: Pooling and sequence formation

Purpose | The role in this system

Reduce spatial resolution or summarise a feature map into sequence columns and writer features.

Justification | The reason for this construction

Two spatial pools reduce work; height maximum produces one feature vector per horizontal position; writer width averaging summarises the line.

Alternatives | What changes if another method is used

Average instead of maximum retains average activation; strided convolution learns reduction; attention pooling adds trained weights. Every alternative changes the function and may require retraining.

Library | Implementation speed and accuracy

PyTorch pooling layers are equivalents only under the same axes, windows and edge rules. This project local max implementation is designed for the invoked two-by-two, stride-two case; its general signature is not proof of every library-supported configuration. [35, 50].

6.20 MaxPool2d

```
def MaxPool2d(x, kernel=2, stride=2)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 76. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.19.

Purpose | What this contributes

Reduce spatial resolution or summarise a feature map into sequence columns and writer features.

Inputs

`x` the input, shaped (batch, channels, height, width)

`kernel`
the window size, default 2

`stride`
how far the window moves, default 2

Output. the input reduced by the stride in both directions, each output value being the largest in its window.

Purpose | What it is for

Pooling shrinks the spatial size of a feature map by summarising each small window as a single number. With a 2 by 2 window and a stride of 2, the windows do not overlap and the map comes out half as tall and half as wide, a quarter of the values.

Formula purpose and system use. Retain the strongest response in each local window while reducing feature resolution. **Source basis:** [35, 50]. Exact project constants remain implementation choices unless evidence is stated.

$$y[c,\, p,\, q] \;=\; \max_{0 \le u,v < k} \; x[c,\; p\,s + u,\; q\,s + v] \tag{6.13}$$

where

$y[c, p, q]$
the pooled value in channel c at row p, column q

k the window size

s the stride

u, v the position within the window

max the largest of the values in the window

Justification | Three reasons it is done

- **Cost.** A quarter of the values means a quarter of the work in every layer that follows. Since this network pools twice, the deepest and widest convolutional block runs on a sixteenth of the original area.
- **Receptive field.** The patch of the original image that influences one value, called its **receptive field**, grows slowly if only 3 by 3 convolutions are stacked, by two pixels per layer. Halving the resolution doubles the area each later filter covers in original pixels, which is how the network gets from detecting strokes to detecting whole letters without needing very many layers.
- **Small-shift tolerance.** Taking the largest value in a window gives the same answer wherever in that window the feature occurred, so the output is unchanged by a shift of a pixel. For handwriting this is exactly right: the same letter written a pixel higher is the same letter.

Justification | Why the maximum rather than the average

After ReLU, a value is evidence that a feature is present, and zero is silence. The maximum answers "was this feature found anywhere in this window", which is the useful question about evidence. An average would dilute a strong local response by the surrounding silence, so a thin stroke correctly detected in one position would be reported as a weak response over a wider area. Averaging has its place, and this file uses it later where the question genuinely is about a typical level rather than a presence.

Method | Implementation, and the discarded remainder

The windows are formed as a strided view, by the same mechanism described under Conv2d, with two extra axes for position within the window; taking the maximum over those two axes then pools everything at once. An odd final row or column, which cannot fill a window, is dropped before the view is built. Discarding input is acceptable here because the loss is a single row or column at the edge of a padded crop, where there is no ink, while the alternative of padding would introduce a window containing invented values whose maximum could be reported as a detected feature.

6.21 AdaptiveMaxPoolHeight

```
def AdaptiveMaxPoolHeight(x)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 87. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.19.

Purpose | What this contributes

Reduce spatial resolution or summarise a feature map into sequence columns and writer features.

Inputs

`x` the input, shaped (batch, channels, height, width)

Output. the same array with its height collapsed to 1, keeping the largest value in each column of each channel.

Purpose | What it is for, and why it is the pivot of the whole network

This one line is where the reader stops being an image network and becomes a sequence network. Before it, the data is a stack of two-dimensional feature maps. After it, each column of the image is a single vector of features, and the line is a sequence of such vectors, left to right. That is the form the LSTM and CTC both require, and this is the step that produces it.

Formula purpose and system use. Collapse vertical positions into one feature vector for each horizontal frame. **Source basis:** [35, 50]. Exact project constants remain implementation choices unless evidence is stated.

$$y[c,\,0,\,q] \;=\; \max_{0 \le p < H} \; x[c,\,p,\,q] \tag{6.14}$$

where

$y[c, 0, q]$
the collapsed value for channel c at column q, the height axis now being of length 1

$x[c, p, q]$
: the feature value at row p of that column

H
: the feature map height at this point in the network, which is 16 after two poolings of a 64-pixel crop

$\max_{0 \le p < H}$
: the largest value down the whole column

Justification | Why collapsing the height is legitimate

It looks like throwing away a great deal, and it would be for a general image. For a line of handwriting it is justified by the geometry of the problem. The crop contains exactly one line of text, so vertical position within it carries no information about *which* character is present, only about which part of a character a pixel belongs to. What identifies the character at a given horizontal position is the set of features found anywhere in that column. Taking the maximum down each column asks precisely that: was this feature present anywhere at this horizontal position.

The word **adaptive** means the output size is specified rather than the window: here the output height is 1 whatever the input height, so the window is as tall as the input. That is why the function needs no parameters, and why it would behave identically on a crop of a different height, unlike the fixed pooling above.

> *A maximum, not an average, for the same reason as before: after ReLU these values are evidence of presence, and a feature found in two rows out of sixteen should be reported as found, not diluted by the fourteen silent rows. This is the choice that lets an ascender and the body of a letter in the same column both register.*

6.22 AdaptiveAvgPoolWidth

```
def AdaptiveAvgPoolWidth(x)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 91. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.19.

Purpose | What this contributes

Reduce spatial resolution or summarise a feature map into sequence columns and writer features.

Inputs

`x`
: the input, shaped (batch, channels, height, width)

Output. the array averaged down axis 2, with that axis kept at length 1.

Purpose | What it is for

The namer, unlike the reader, has to produce one answer for the whole crop rather than one per column. The identity of a writer is a property of the line as a whole, not of a position along it. So at the end of the namer's backbone the remaining axis is collapsed by averaging, turning a sequence of feature vectors into a single feature vector describing the whole line.

Formula purpose and system use. Summarise horizontal features for a whole-line writer decision. **Source basis:** [35, 50]. Exact project constants remain implementation choices unless evidence is stated.

$$y[c] = \frac{1}{N} \sum_{q=0}^{N-1} x[c, q] \tag{6.15}$$

where

$y[c]$ the pooled value for channel c

$x[c, q]$
the feature value at position q

N the number of positions being averaged over

Justification | Why averaging is right here and maximum was right before

The question has changed, and so the summary must. The reader asks whether a feature occurs at a given position, which is a question about presence, answered by a maximum. The namer asks how characteristic a feature is of this writing, which is a question about prevalence, answered by an average. A writer who makes a particular join in most of their letters should score differently from one who makes it once; a maximum would report both identically, since both have it somewhere. Averaging is what makes the feature vector describe a habit rather than an occurrence.

This construction is known as **global average pooling**, and beyond matching the question it has a practical benefit: the output size depends only on the number of channels, not on the length of the line. The classifier head that follows therefore has a fixed input size regardless of how long the crop was.

> *The function name says width and the code averages axis 2, which is the height axis in the four-axis convention. There is no inconsistency in effect, because by the time it is called the height has already been collapsed and the array has been reduced to three axes, so axis 2 is the sequence position. The name describes the intent and the index describes the array as it actually arrives.*

Stage: Softmax and log sum exp

6.23 Method background: Softmax and log sum exp

Purpose | The role in this system

Turn logits into a normalised class distribution and sum small probabilities stably.

Justification | The reason for this construction

Subtracting the maximum leaves ratios unchanged while preventing exponential overflow. Log-domain path sums avoid underflow in CTC.

Alternatives | What changes if another method is used

Direct exponentiation is simpler but can overflow. Sigmoid treats classes independently and does not express the same exclusive-class distribution. Temperature changes confidence without necessarily changing argmax.

Library | Implementation speed and accuracy

SciPy special.logsumexp and PyTorch log_softmax provide stable primitives and autodiff. Equal formulae can differ in rounding/reduction order. Calibrated real-world confidence does not follow from normalisation alone. [48, 51, 52].

6.24 LogSoftmax

```
def LogSoftmax(x, axis)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 100. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.23.

Purpose | What this contributes

Turn logits into a normalised class distribution and sum small probabilities stably.

Inputs

`x` an array of raw scores

`axis`
which axis the scores to be compared lie along

Output. the log of the softmax along that axis: values at most zero, whose exponentials sum to one.

Method | Background: turning scores into probabilities

A network's final layer produces one unbounded number per class, called a **logit**. These are not probabilities: they can be negative, and they need not sum to anything in particular. The **softmax** converts them into probabilities by exponentiating each, which makes them positive, and dividing by the total, which makes them sum to one.

Formula purpose and system use. Normalise exponentiated logits into mutually exclusive class probabilities. **Source basis:** [48, 51, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$p_k = \frac{e^{x_k}}{\sum_j e^{x_j}} \tag{6.16}$$

where

p_k the probability assigned to class k

x_k the raw score, or logit, for class k

$\sum_j e^{x_j}$
the sum of the exponentials over every class, which makes the outputs total 1

Exponentiating is what makes the conversion monotone and amplifying: a score two units higher becomes a probability about 7.4 times larger, so softmax preserves the ranking while sharpening the differences.

Justification | Why the logarithm is taken

CTC multiplies many probabilities together, one per column of a line, and a line has well over a hundred columns. A product of a hundred values near 0.1 is about 10^{-100}: representable in float64, but below the ordinary float32 range. Longer or lower-probability paths can also underflow in float64. Direct products therefore have a precision-dependent numerical risk. This is **underflow**. Working with logarithms avoids these direct tiny products, because the logarithm of a product is the sum of the logarithms, and a sum of a hundred moderate negative numbers is an ordinary number.

Formula purpose and system use. Express those class probabilities in logarithmic form for CTC path arithmetic. **Source basis:** [48, 51, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\log\Big(\prod_t p_t\Big) \;=\; \sum_t \log p_t \tag{6.17}$$

where

$\prod_t p_t$
: the product of the per-column probabilities along a path

$\sum_t \log p_t$
: the equivalent sum of their logarithms

p_t : the probability at column t

Method | The log-sum-exp trick

Computing the logarithm of the softmax naively would exponentiate first, and a logit of 800 overflows to infinity before any logarithm is taken. The remedy uses the fact that softmax is unchanged by subtracting a constant from every score, since the constant's exponential cancels between numerator and denominator. Subtracting the largest score makes the biggest exponential exactly 1 and every other smaller, so nothing can overflow.

Formula purpose and system use. Evaluate the same log normaliser without exponential overflow. **Source basis:** [48, 51, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\log p_k \;=\; (x_k - m) \;-\; \log \sum_j e^{\,x_j - m}, \qquad m \;=\; \max_j x_j \tag{6.18}$$

where

$\log p_k$
: the log probability of class k, the function's output

x_k : the raw score for class k

m : the largest score, subtracted from every score before any exponential is taken

$\log \sum_j e^{x_j - m}$
: the log-sum-exp term, which is the normalisation expressed in log space

Underflow in the other direction is also harmless here. A score far below the maximum exponentiates to zero, contributing nothing to the sum, which is the correct contribution for a class of negligible probability. The maximum term is always present and equals 1, so the sum can never be zero and its logarithm is always defined. This rearrangement is known as the **log-sum-exp trick**, and it is standard wherever probabilities are handled in log space.

> *Every output of this function is at most zero, because a probability is at most one and the logarithm of a number at most one is at most zero. Seeing negative numbers in the reader's output is therefore expected, not a fault, and a value near zero means near-certainty.*

6.25 Softmax

```
def Softmax(x, axis)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 107. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.23.

Purpose | What this contributes

Turn logits into a normalised class distribution and sum small probabilities stably.

Inputs

`x` an array of raw scores

`axis`
which axis the scores to be compared lie along

Output. probabilities along that axis: non-negative and summing to one.

Purpose | What it is for

The same conversion as above without the logarithm, used where an actual probability is wanted for a person to read rather than a log probability for further arithmetic. In this file its only use is in the namer, where the output is a confidence per writer that gets reported.

Justification | The same stabilisation, for the same reason

The maximum is subtracted before exponentiating here too. The result is mathematically identical, since the subtraction cancels, but it cannot overflow: the largest exponential is 1 rather than potentially enormous. The cost is one pass to find the maximum and one subtraction, which is negligible against the exponentials themselves.

Justification | Why the namer uses this and the reader uses the log version

The distinction follows from what happens next. The namer's output is compared and reported directly, so a probability is what is wanted. The reader's output is summed along thousands of paths by CTC, so log probabilities are what is wanted. Converting between them afterwards would be possible but would reintroduce exactly the underflow that working in log space exists to avoid.

> *A confidence from this function is a probability over the enrolled writers only, and it necessarily sums to one across them. It therefore cannot express "none of these": a crop from somebody not enrolled is still assigned to whichever enrolled writer resembles it most, with a confidence that may be high. This is a property of the output layer, not a fault in the training, and any use of the namer on an unknown writer has to account for it.*

Stage: LSTM and bidirectional recurrence

Report equation 21 LSTM cell and hidden state

Method | Formula

$c_t = f_t \odot c_{t-1} + i_t \odot g_t$ and $h_t = o_t \odot \tanh c_t$.

Purpose | Why the system needs it

Carry gated memory along feature columns and expose a bounded hidden output. The gates and candidate are computed from current features and previous hidden state; weights are shared, not passed as the state.

Evidence | Current location and source

LstmForward and LstmBackward. [40, 41].

6.26 Method background: LSTM and bidirectional recurrence

Purpose | The role in this system

Use left and right image context to disambiguate sequential character features.

Justification | The reason for this construction

Additive cell memory and multiplicative gates offer a controlled route for retaining information; reverse recurrence is available because the complete offline line is known.

Alternatives | What changes if another method is used

A plain RNN has simpler state but difficult long dependencies. GRU uses fewer gates. Attention/transformers offer another context model and different cost, data and checkpoint requirements.

Library | Implementation speed and accuracy

PyTorch LSTM provides optimised recurrent kernels and autodiff. Gate order, two bias arrays, direction reversal and tensor layout must match. Fused kernels can avoid Python frame loops; speed and final trained accuracy are separate comparisons. [40, 41, 48].

6.27 LstmForward

```
def LstmForward(x, weightIh, weightHh, biasIh, biasHh, hiddenSize)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 123. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.26.

Purpose | What this contributes

Use left and right image context to disambiguate sequential character features.

Inputs

`x` the input sequence, shaped (time, batch, features)

`weightIh`
: the weights applied to the input, holding all four gates stacked

`weightHh`
: the weights applied to the previous output, also all four gates stacked

`biasIh`
: the bias on the input side

`biasHh`
: the bias on the recurrent side

`hiddenSize`
: the width of the memory and output vectors

Output. a pair: the output at every time step, shaped (time, batch, hidden), and the final hidden and cell state.

Purpose | The problem: a column is not enough

After the convolutional backbone the line has become a sequence of feature vectors, one per column. Classifying each column on its own would be hopeless. A single narrow column of a handwritten line is often ambiguous even to a person: the left half of an a and the left half of a d are the same stroke, and which it is depends on what follows. A smudge is identifiable only from the letters around it. What is needed is a layer whose output at each column depends on the whole line, and that is what a **recurrent** layer provides.

Method | Background: what recurrence means

A **recurrent neural network** processes a sequence one step at a time, carrying a vector of numbers from each step to the next. That carried vector is the **hidden state**, and it is the network's summary of everything seen so far. At each step the layer combines the new input with the incoming hidden state to produce a new hidden state, which is also its output for that step. The same weights are used at every step, so the layer has a fixed number of parameters however long the sequence is.

Justification | Why a plain recurrent layer is not used

In a plain recurrent layer the hidden state is overwritten at every step, as a weighted combination of the old state and the new input passed through a squashing function. Information therefore survives only by being repeatedly multiplied by the recurrent weights. Over a hundred and sixty steps that product either shrinks to nothing or grows without bound, depending on whether the weights are on average below or above the scale that preserves magnitude. These are the **vanishing** and **exploding gradient** problems, and the practical consequence is that a plain recurrent layer cannot learn to connect events more than a few steps apart. For reading a line of writing that is useless, since the evidence resolving an ambiguous letter can be a whole word away.

Alternatives | The LSTM idea: a cell you add to rather than overwrite

The **long short-term memory** layer [40] solves this by keeping a second vector, the **cell state**, whose default behaviour is to persist unchanged. It is not recomputed each step; it is modified, by having some of its contents erased and some new contents added. Because the default is to carry forward, information can survive arbitrarily many steps without being multiplied by anything, which is precisely what the plain layer could not do.

What to erase, what to add and what to reveal are decided by three **gates**, each a vector of numbers between 0 and 1 produced by a sigmoid. A gate value of 0 blocks completely, 1 lets everything through, and anything between is a proportion. A fourth quantity, the candidate, supplies the contents that might be added.

Formula purpose and system use. Compute input, forget, candidate and output gate activations from the current features and previous hidden state. **Source basis:** [40, 41, 48]. Exact project constants remain implementation choices unless evidence is stated.

$$
\begin{aligned}
i_t &= \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \\
f_t &= \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \\
g_t &= \tanh\left(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}\right) \\
o_t &= \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})
\end{aligned}
\tag{6.19}
$$

Source: [40]

where

i_t the **input gate** at step t: how much of the candidate to write into the cell

f_t the **forget gate**: how much of the existing cell to keep

g_t the **candidate**, the new contents that might be written; it uses tanh rather than a sigmoid because it is a value to be stored, which may usefully be negative, not a proportion

o_t the **output gate**: how much of the cell to reveal as this step's output

x_t the input vector at step t

h_{t-1} the hidden state carried in from the previous step

W_{i*}, W_{h*}
the learned weights applied to the input and to the previous hidden state respectively

b_{i*}, b_{h*}
the corresponding learned biases

σ the sigmoid, squashing to between 0 and 1; see its own entry

tanh
the hyperbolic tangent, squashing to between -1 and 1

Every one of the four lines has the same form: a weighted sum of the new input, plus a weighted sum of the previous output, plus biases. They differ only in their weights and in the squashing applied. That uniformity is what lets the implementation compute all four in a single matrix multiplication, described below.

Method | The two state updates

Formula purpose and system use. Update retained cell memory and expose its gated hidden output. **Source basis:** [40, 41, 48]. Exact project constants remain implementation choices unless evidence is stated.

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t) \tag{6.20}$$

Source: [40]

where

c_t the cell state after step t: the long-term memory

c_{t-1} the cell state carried in from the previous step

h_t the hidden state after step t, which is also this step's output

$\odot$ elementwise multiplication: each position of one vector multiplied by the matching position of the other, as opposed to a matrix product

f_t, i_t, g_t, o_t
the gates and candidate from above

The cell update is the heart of the method and is worth reading slowly. It is an addition, not a replacement. If the forget gate is 1 and the input gate is 0 at some position, that position of the cell passes through completely untouched, for as many steps as those gate values persist. No weight matrix is applied to it along the way, so it neither shrinks nor grows. That is the path along which information travels a long distance, and it is what the name long short-term memory refers to: a short-term memory that can be made to last.

The hidden state is then a filtered view of the cell. The tanh keeps it in a bounded range, and the output gate decides how much of each position to expose. So the layer can hold something in the cell for many steps while revealing nothing about it, and then reveal it when it becomes relevant. For

reading handwriting that is directly useful: the shape of an ambiguous stroke can be held until the letter that disambiguates it arrives.

Method | How all four gates are computed at once

The four weight matrices are stored stacked into one array, four times the hidden size tall. One matrix multiplication therefore produces all four pre-activation vectors end to end, and the implementation simply slices the result into four equal blocks in the order input, forget, candidate, output. That ordering is not a choice made here; it is the convention of the framework the weights were trained with, and slicing in a different order would silently use the forget gate's weights as the input gate's. One large matrix multiplication is also considerably faster than four small ones, for the BLAS reasons given under Conv2d.

Justification | Why there are two biases

Mathematically a single bias would do, since adding two constants is adding one constant. Both are kept because the framework that trained the weights stores both, and the trained values must be applied as they were learned. Their sum is what has any effect.

Justification | Why this loop cannot be vectorised

Every other layer in this file processes its whole input at once. This one cannot, and the reason is structural rather than a shortcoming of the implementation: step t needs the hidden and cell state from step t−1, so the steps are genuinely sequential. Within a step everything is already whole-array arithmetic across the batch and the hidden positions. This is the same situation as the column loop in LeastCostSeam, and the same conclusion holds: the loop is over the one axis that is irreducibly ordered.

> *Both states start as vectors of zeros. Zero is the natural choice for "nothing seen yet", and it has the useful property that an all-zero cell with a forget gate near 1 stays empty until something is deliberately written into it.*

6.28 BidirectionalLstmForward

```
def BidirectionalLstmForward(x, paramsFwd, paramsBwd, hiddenSize)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 143. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.26.

Purpose | What this contributes

Use left and right image context to disambiguate sequential character features.

Inputs

`x` the input sequence, shaped (time, batch, features)

`paramsFwd`
: the weights and biases for the forward pass

`paramsBwd`
: the weights and biases for the backward pass

`hiddenSize`
: the width of each direction's output

Output. the two directions' outputs joined along the feature axis, so twice as wide as one direction.

Justification | Why one direction is not enough

An LSTM reading left to right has, at each column, seen everything to the left and nothing to the right. For handwriting that is half the available evidence and often the less useful half. Consider the stroke that begins both an a and a d: what distinguishes them is the ascender that comes afterwards. A left-to-right reader at the start of that letter cannot know. It will often get the answer right in the end, because the output at the later column can be correct, but its representation of the earlier columns is impoverished, and the earlier columns are where CTC may want to place the character.

Method | The construction

The remedy is to run a second, completely separate LSTM over the sequence reversed, then put its outputs back into forward order and join the two side by side. Each position then carries a summary of everything to its left, from the forward pass, and a summary of everything to its right, from the backward pass. The two have independent weights throughout; nothing is shared.

Formula purpose and system use. Join left-to-right and right-to-left context at the same image frame. **Source basis:** [40, 41, 48]. Exact project constants remain implementation choices unless evidence is stated.

$$y_t = [\, \overrightarrow{h}_t \,;\, \overleftarrow{h}_t \,] \tag{6.21}$$

where

y_t the combined output at position t, of width twice the hidden size

$\overrightarrow{h}_t$ the forward LSTM's hidden state at position t, summarising positions 1 to t

$\overleftarrow{h}_t$ the backward LSTM's hidden state at the same position, summarising positions t to the end

$[\,\cdot\,;\,\cdot\,]$
concatenation: the two vectors laid end to end, not added or combined

Concatenation rather than addition is the right choice because the two summaries answer different questions and the layer above should be free to weigh them differently. Adding them would force every feature to mean the same thing in both directions and would discard which direction it came from.

Justification | Why this is legitimate here and not always

Reading the future is only possible because the whole line is available before any output is needed. A system transcribing speech as it is spoken could not do this, since the future has not happened yet. Handwriting recognition on a captured image has no such constraint: the entire line was photographed at once. Bidirectional recurrence is standard in handwriting recognition for exactly this reason, and the architecture this network follows is built on it [35].

> *Reversing a numpy array along its first axis is a view, not a copy, so running the backward pass costs nothing extra in memory. The output of the backward pass is reversed again so that position t of both halves refers to the same column; forgetting that second reversal would align each position with its mirror image and produce fluent-looking nonsense.*

6.29 MultilayerBidirectionalLstm

```
def MultilayerBidirectionalLstm(x, layerParams, hiddenSize)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 150. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.26.

Purpose | What this contributes

Use left and right image context to disambiguate sequential character features.

Inputs

`x` the input sequence

`layerParams`
: a list of parameter sets, one per layer, each holding a forward and a backward set

`hiddenSize`
: the width of each direction in each layer

Output. the sequence after passing through every layer in turn.

Method | What stacking buys

Each layer's output sequence becomes the next layer's input sequence. The reader uses two such layers. The gain is the same as from depth anywhere in a network: the first layer works on the convolutional features directly and can form short-range relationships between neighbouring columns, such as grouping the strokes that make up one letter; the second works on the first's output, where each position already summarises a neighbourhood, and can form relationships between those summaries, such as relating a letter to the word around it.

Justification | Why the second layer's input is twice as wide

Because the first layer concatenates two directions, its output has twice the hidden size, and the second layer's input weights are shaped to accept that. This is handled implicitly here: the weights come from the checkpoint and already have the right shape, so the function needs no knowledge of the widths and the same loop would work for any number of layers.

> *Two layers is the depth the architecture specifies [35]. More would add parameters and sequential work for a task where the first layer already covers a letter and the second a word, which is the range the evidence actually spans.*

Stage: CTC decoding and forced alignment

Report equation 22 CTC sequence loss

Method | Formula

$L = -\log \sum_{\pi : B(\pi) = \ell} \prod_t p_t(\pi_t).$

Purpose | Why the system needs it

Learn complete transcriptions without character boxes by summing all valid alignment paths. Collapse merges adjacent repeats before removing blank; literal spaces remain ordinary labels.

Evidence | Current location and source

CtcLoss; DecodeCtc solves a cheaper best-path inference problem. [34, 42].

6.30 Method background: CTC decoding and forced alignment

Purpose | The role in this system

Train from complete line transcripts and recover transcript-constrained character locations for extraction.

Justification | The reason for this construction

CTC marginalises valid blank/repeat paths; greedy decoding selects a path cheaply; forced alignment maximises over paths for a supplied known transcript.

Alternatives | What changes if another method is used

Manual character boxes need annotation and unreliable cursive cuts. Beam search can improve transcript search at extra cost. Attention decoding learns a different alignment mechanism.

Library | Implementation speed and accuracy

PyTorch CTCLoss computes the training loss, not the entire greedy/forced-alignment pipeline. Its mean reduction divides by target length, unlike the project batch mean. Align lengths, blank, precision and reduction before numerical comparison. [34, 42, 53].

6.31 DecodeCtc

```
def DecodeCtc(logProbs)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 187. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.30.

Purpose | What this contributes

Train from complete line transcripts and recover transcript-constrained character locations for extraction.

Inputs

`logProbs`
: the reader's output, shaped (time, batch, classes), holding a log probability per character per column

Output. a list of strings, one per item in the batch.

Purpose | The problem CTC exists to solve

The reader produces one set of character scores per column, around 160 of them, while the line contains perhaps 40 characters. Nothing says which columns correspond to which character, and the correspondence differs for every line and every writer: a wide m covers many columns, a narrow i covers few. Training a network to produce a short sequence from a long one would ordinarily require knowing that correspondence, which would mean marking by hand where every letter begins and ends on every training line. For tens of thousands of lines that is not feasible.

Connectionist temporal classification [34] removes the requirement. It lets the network be trained on the text alone, with no indication of where the characters sit, by defining a rule that maps any

per-column output onto a shorter string, and then training the network so that the total probability of all column-outputs mapping to the correct string is high. The alignment is never specified; it is whatever the network finds convenient.

Method | The collapsing rule

The rule has two steps, applied in this order: merge runs of the same character into one, then delete the blanks.

Formula purpose and system use. Select the highest-probability class at each frame for greedy inference. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathcal{B}(\pi) \ : \ \text{merge repeats, then remove blanks} \tag{6.22}$$

Source: [34]

where

$\mathcal{B}$ the collapsing map, which takes a sequence of one label per column and returns a string

π a **path**: one label chosen for each column, including possibly the blank

`blank`
: the extra label, slot 0, which stands for "emitting nothing here"

Justification | Why the blank is necessary

The blank is the part that makes the scheme work, and its purpose is easiest to see from what goes wrong without it. If the rule were only "merge repeats", the word `letter` could never be produced, because its two consecutive t's would merge into one. The blank provides a separator: a path reading t, t, blank, t, t collapses first to t, blank, t and then, with the blank removed, to tt. So a blank between two runs of the same character means a genuine doubled letter, while its absence means one letter held across several columns.

- **a, a, a, b, b** collapses to **ab**, one a and one b.
- **a, blank, a** collapses to **aa**, a doubled letter.
- **a, a, blank, b** collapses to **ab**, the blank being harmless between different letters.

The blank also gives the network somewhere to put the columns between letters, the gaps and joins where no character is being written. In practice a trained CTC network emits blank for the large majority of columns and spikes briefly on a character where it is confident, which is why its output looks sparse.

Method | Greedy decoding, and what it approximates

What is wanted is the single most probable *string*, which requires summing over every path that collapses to it, since many paths give the same string. That is expensive. This function instead takes the most probable *path*, by picking the best-scoring character independently in each column, and collapses that. This is **greedy** or **best-path** decoding.

Formula purpose and system use. Collapse adjacent repeated path symbols and remove blanks in the correct order. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$\pi_t^* \ = \ \arg\max_k \ \log p_t(k), \qquad \hat{y} \ = \ \mathcal{B}(\pi^*) \tag{6.23}$$

Source: [34]

where

π_t^* the chosen label for column t

$\log p_t(k)$
: the reader's log probability for character k at column t

$\arg\max_k$
: the character index with the highest score

$\hat{y}$ the decoded string

$\mathcal{B}$ the collapsing map defined above

The approximation is not exact: the most probable path need not collapse to the most probable string, because a string whose probability is spread over many moderately good paths can beat one concentrated in a single excellent path. In practice the difference is small for a well-trained network, whose output is confident enough that one path dominates, and the measured accuracy of 96 to 99 percent on real crops is obtained with this decoder. A beam search would recover the difference at several times the cost.

> *Because the comparison within a column is only between scores in that same column, the logarithm makes no difference to which character wins. The log probabilities could be replaced by raw logits here and the output would be identical. They are log probabilities because the same array is used by the alignment, which does need them.*

6.32 CtcForcedAlign

```
def CtcForcedAlign(logProbs, text)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 204. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.30.

Purpose | What this contributes

Walks the one path through the scores that spells a known text, and returns the column span of every character plus the path's mean score.

Inputs

`logProbs`
: the reader's output for one line, shaped (time, classes)

`text`
: the known text of that line

Output. a list pairing each character with the span of columns it occupies, together with a confidence; or nothing if the text cannot be aligned.

Purpose | What it is for, and why the project needs it

Decoding answers "what does this line say". This answers a different question: given that the line says this, *where* is each character. That is what the glyph extractor needs. To build a library of a writer's letterforms, the pipeline must cut individual letters out of their line crops, and to do that it must

know which columns of the crop hold which letter. No separate letter-finding step is needed, because the reader already contains that information implicitly in its per-column scores; forced alignment extracts it.

Justification | Why it is called forced

The text is not being recognised; it is given. The algorithm is **forced** to produce an alignment consistent with that text, and among all such alignments it finds the most probable. So it cannot return the wrong letters, only a wrong idea of where they are. This is a much easier problem than recognition and correspondingly more reliable, which is why the project uses known transcripts for glyph extraction rather than the reader's own output.

Method | The extended label sequence

The first step is to write out the target as a sequence that allows blanks everywhere they are permitted: a blank, then each character followed by a blank.

Formula purpose and system use. Construct a target-state sequence with blanks so repeated letters can be aligned legally. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{ext} \;=\; (\varnothing,\ \ell_1,\ \varnothing,\ \ell_2,\ \varnothing,\ \ldots,\ \ell_n,\ \varnothing), \qquad S \;=\; 2n+1 \tag{6.24}$$

Source: [34]

where

ext	the extended sequence of states the alignment moves through
$\varnothing$	the blank label, slot 0
ℓ_i	the slot number of the i-th character of the text
n	the number of characters
S	the length of the extended sequence, hence the number of states

Every valid path through the columns can now be described as a non-decreasing walk through these states: at each column the path is in some state, and as columns advance the state index only ever stays put or increases. Characters sit at odd positions and the optional blanks at even ones. A path that skips a blank writes two characters back to back; a path that lingers in a blank puts a gap between them.

Method | The allowed moves

From state s at the previous column, three moves are considered: stay at s, advance to s from s−1, or advance to s from s−2. The third, the skip, is what allows the optional blanks to be passed over, and it carries one restriction.

- **Stay.** The same label continues across another column: a wide letter, or a long gap.
- **Advance by one.** Move to the next state, either from a character into its following blank or from a blank into the next character.
- **Skip by two.** Jump over a blank, placing two characters in consecutive columns. Permitted only when the target state is a character and that character differs from the one two states back.

The restriction on the skip is the whole reason the blanks were inserted. If the two characters either side of a blank are the same letter, skipping the blank would make a doubled letter collapse back into

one, by the merge rule. So for a doubled letter the blank between them is compulsory, and the skip is forbidden. A skip onto a blank is likewise meaningless, since it would be jumping over a character. Both cases are enforced by setting the candidate score to a very large negative number, which the maximisation then never selects, which is the same device as the infinities in LeastCostSeam.

Method | The recurrence

This is dynamic programming again, with the same structure as the seam, but maximising a log probability instead of minimising a cost. Working in log space means the multiplication of probabilities along a path becomes an addition, as established under LogSoftmax.

Formula purpose and system use. Accumulate the best legal transcript-constrained prefix in log space. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$\alpha[t,s] \;=\; \log p_t(\mathrm{ext}[s]) \;+\; \max\Big(\alpha[t-1,s],\; \alpha[t-1,s-1],\; \alpha[t-1,s-2]\Big) \tag{6.25}$$

Source: [34]

where

$\alpha[t,s]$
: the score of the best path that reaches state s at column t

$\log p_t(\mathrm{ext}[s])$
: the reader's log probability, at column t, of the label belonging to state s

$\alpha[t-1,\,\cdot\,]$
: the best scores at the previous column, for the three states a path may arrive from

max
: the best of those three arrivals; the third is excluded when the skip is not allowed

The initial column may only be in state 0, the leading blank, or state 1, the first character, since a path cannot have progressed further than one state in one column. At the final column the path must have reached either the last state, the trailing blank, or the state before it, the last character; whichever of those two scores higher is the end of the best path.

Justification | Why maximum and not sum

Training CTC sums over all paths, because the quantity being maximised is the total probability of the target string. Alignment takes the maximum instead, because the quantity wanted is one specific path: the single most probable way the text could be laid out across the columns. Taking the maximum of a sequence of choices like this is the **Viterbi** algorithm [54], long used for exactly this purpose of recovering the most likely state sequence behind an observed signal.

Method | From the path to column spans

Backtracking the recorded moves gives the state occupied at every column. The i-th character of the text lives in state 1+2i, so the columns whose state equals that number are the columns that character occupies, and the first and last of them are its span. A character without a usable recovered span is handled as an alignment/extraction failure. A legal CTC transition cannot simply skip an obligatory character or assign two different target states to one frame.

Method | The confidence

Formula purpose and system use. Translate aligned frame intervals into a summary confidence for the constrained path. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless

evidence is stated.

$$\text{conf} = \frac{1}{T}\sum_{t=1}^{T}\log p_t(\text{ext}[\text{path}[t]]) \tag{6.26}$$

where

conf the returned confidence: the mean log probability per column along the chosen path

T the number of columns

path$[t]$
: the state the best path occupies at column t

$\log p_t(\cdot)$
: the reader's log probability for that state's label at that column

Dividing by the number of columns is what makes the figure comparable between lines. A total would favour short lines simply because fewer negative terms are added, so the mean is the per-column average, with zero meaning perfect certainty and more negative meaning less. The glyph extractor uses it to reject lines whose alignment is not trustworthy enough to cut letters from, which matters because a confidently wrong alignment would poison the library with mislabelled glyphs.

> *Two guards return nothing rather than a bad answer: a text with no recognisable characters, and a line with fewer columns than characters. The second is a genuine impossibility rather than a difficulty, since every character needs at least one column of its own, so no alignment exists at all.*

6.33 ReadTextScored

```
def ReadTextScored(image, model)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 294. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.30.

Purpose | What this contributes

Calls model.Forward and returns the greedy transcript plus mean best-class log probability over every frame. Blank-dominated frames contribute. This score is used to choose page inversion; it is not transcript probability, edit accuracy or a calibrated confidence that the text is correct.

Method | Current implementation

Train from complete line transcripts and recover transcript-constrained character locations for extraction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ReadTextScored(image,model):
    t = TensorFromResized(ResizeLineImageFixed(image))
    logProbs = model.Forward(t)
    best = np.max(logProbs[:,0,:],axis=1)
    return DecodeCtc(logProbs)[0],float(np.mean(best))
```

Justification | Interpret the method rather than infer a benchmark

CTC marginalises valid blank/repeat paths; greedy decoding selects a path cheaply; forced alignment maximises over paths for a supplied known transcript. [34, 42, 53].

6.34 ReadText

```
def ReadText(image, model)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 301. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.30.

Purpose | What this contributes

Delegates to ReadTextScored and keeps only the string. The result depends on the already-loaded model and complete crop preparation; changing a checkpoint filename does not replace an existing model object.

Method | Current implementation

Train from complete line transcripts and recover transcript-constrained character locations for extraction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ReadText(image,model):
    return ReadTextScored(image,model)[0]
```

Justification | Interpret the method rather than infer a benchmark

CTC marginalises valid blank/repeat paths; greedy decoding selects a path cheaply; forced alignment maximises over paths for a supplied known transcript. [34, 42, 53].

Stage: Edit distance and evaluation metrics

Report equation 34 Pooled character recovery

Method | Formula

$A_c = 100[1 - \sum_i d_i / \sum_i |t_i|]$.

Purpose | Why the system needs it

Weight characters rather than equally weighting lines. This differs from the clamped per-candidate CharAcc at extreme insertion counts. Case, spaces, punctuation and empty-reference handling must remain explicit.

Evidence | Current location and source

Report evaluation and server scoring. [55, 10].

6.35 Method background: Edit distance and evaluation metrics

Purpose | The role in this system

Separate text recovery, writer decisions, geometric agreement and physical accuracy.

Justification | The reason for this construction

Each metric answers a different requirement. Pooled character edits weight characters; writer decisions are binary per line; geometric F measures thresholded point proximity.

Alternatives | What changes if another method is used

Exact-line match is stricter but less diagnostic. WER edits differ from word LCS recall. IoU, Pearson correlation and human judgements are distinct quantities and cannot be relabelled interchangeably.

Library | Implementation speed and accuracy

Edit-distance libraries can accelerate dynamic programming; SciPy spatial trees or distance transforms can compute nearest-point distances. Match normalisation, empty cases, tolerance and aggregation. Faster scoring does not improve handwriting. [55, 56, 57].

6.36 Levenshtein

```
def Levenshtein(a, b)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 305. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.35.

Purpose | What this contributes

Separate text recovery, writer decisions, geometric agreement and physical accuracy.

Inputs

`a` one string

`b` the other string

Output. the smallest number of single-character insertions, deletions and substitutions that turn one string into the other.

Purpose | What it is for

Scoring a reader requires a way to say how wrong an answer is. Counting matching positions does not work, because a single missing letter shifts everything after it and would be scored as if every later letter were also wrong. What is needed is a measure that understands insertion and deletion, and that is the **edit distance**, or **Levenshtein distance** [55]: the least number of single-character edits needed to turn one string into the other.

- **Insertion:** add a character. `cat` to `cart` is one edit.
- **Deletion:** remove a character. `cart` to `cat` is one edit.
- **Substitution:** replace a character. `cat` to `cot` is one edit.

Method | The recurrence

This is dynamic programming once more. Let the problem be solved for every pair of prefixes: how many edits turn the first i characters of one string into the first j of the other. The answer for a larger pair follows from three smaller ones, because the last edit must be one of the three kinds.

Formula purpose and system use. Find the minimum insertions, deletions and substitutions between transcript prefixes. **Source basis:** [55, 56, 57]. Exact project constants remain implementation choices unless evidence is stated.

$$D[i,j] \;=\; \min \begin{cases} D[i-1,\,j]+1 & \text{delete } a_i \\ D[i,\,j-1]+1 & \text{insert } b_j \\ D[i-1,\,j-1]+\left[\,a_i \neq b_j\,\right] & \text{match or substitute} \end{cases} \tag{6.27}$$

Source: [55]

where

$D[i,j]$
: the edit distance between the first i characters of a and the first j characters of b

a_i, b_j
: the i-th and j-th characters of the two strings

$[\,a_i \neq b_j\,]$
: 1 when the two characters differ and 0 when they match, so a match is free and a substitution costs one

min
: the cheapest of the three ways to arrive

The boundary cases are immediate: turning an empty string into a prefix of length j takes j insertions, and the reverse takes i deletions. From there the table fills row by row, and the bottom right corner is the answer for the whole strings.

Method | Two implementation economies

Only the previous row is kept, not the whole table. Each row depends only on the one before it, so storing all of them would use memory proportional to the product of the lengths for no purpose; this version uses memory proportional to the shorter string alone. That is possible because the distance itself is wanted and not the sequence of edits, which would require the full table to backtrack through, exactly as the seam does.

The function also swaps its arguments so the shorter string controls the row length. The distance is symmetric, since every edit has an inverse of the same cost, so swapping cannot change the answer, and it makes the retained row as short as possible.

6.37 CharAcc

```
def CharAcc(pred, truth)
```

Evidence | Source and method link

`Software/DemoCode/recognise.py` line 319. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.35.

Purpose | What this contributes

Separate text recovery, writer decisions, geometric agreement and physical accuracy.

Inputs

`pred`
: the predicted text

`truth`
: the correct text

Output. the fraction of characters correct, between 0 and 1.

Method | How it is defined

The edit distance counts errors; accuracy expresses them as a proportion of the length of the correct text.

Formula purpose and system use. Normalise edit count by nonempty reference length and clamp the candidate character score. **Source basis:** [55, 56, 57]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{CharAcc} \;=\; \max\!\left(0,\; 1 - \frac{D(\text{pred},\, \text{truth})}{|\text{truth}|}\right) \tag{6.28}$$

where

CharAcc
: the character accuracy

D
: the edit distance between the two strings

$|\text{truth}|$
: the number of characters in the correct text

$\max(0, \cdot)$
: the floor at zero, explained below

Justification | Why it is divided by the truth length

The denominator is the length of the correct text, not of the prediction and not the longer of the two. That choice makes the figure answer "what fraction of what should have been written came out right", which is the question of interest, and it makes the measure comparable across lines of different lengths. Dividing by the prediction length instead would let a system improve its score by predicting less.

Justification | Why the floor at zero

The edit distance can exceed the length of the correct text, when the prediction is much longer than the truth: turning a two-character truth into a fifty-character prediction takes around fifty edits, giving a raw score around -24. Negative accuracy is meaningless and, more practically, averaging it across a page would let one runaway line dominate the mean and produce a negative overall accuracy. The floor caps the damage at zero, which is the honest statement that nothing on that line was right.

> *The empty-truth case is handled separately: an empty prediction scores 1, anything else scores 0. Without it the division would be by zero. This is the right convention, since correctly reading a blank line as blank is a success and reading text off a blank line is a complete failure.*

7 Glyph extraction and writer profiles

Evidence | Current implementation boundary

`Software/DemoCode/style.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Thin and trace labelled line ink; align transcript frames and cut characters; normalise glyph coordinates; reject implausible shapes; select medoids and optional prototypes; assemble the writer profile consumed by synthesis.

7.1 Expanded reasoning: alignment to a defensible writer profile

7.1.1 Body-band estimation: what the horizontal profile guesses

Let M be a binary line mask and $p(y) = \sum_x M(y, x)$ its horizontal ink projection. The body-band heuristic takes the first and last rows above $0.45 \max_y p(y)$ as top and baseline b, with $h_x = b - y_{\text{top}}$. The baseline here is an image-derived estimate, not an anatomical fact about every glyph. Capitals, long descenders, isolated punctuation and uneven ink can bias the threshold. Bands narrower than three pixels are rejected because normalised measurements would become unstable. A 45% profile threshold is different from a threshold on an individual pixel's brightness.

Constructed profile $(0, 2, 8, 10, 9, 3, 0)$ has cutoff 4.5, so rows 2 through 4 define the dense band. Using half-open row edges would give a different height by one pixel; the convention must follow the implementation. Changing 0.45 changes estimated x-height, and therefore changes all later quantities expressed in x-heights. A percentile of ink rows, fitted baseline, or character-specific estimates are alternatives, with different assumptions. The exact 0.45 and three-pixel guard are project heuristics [3].

7.1.2 Refining shared character cuts: every term and its purpose

CTC alignment gives approximate character spans. Their adjacent centres provide initial shared cuts. For column profile p_x smoothed over a short window, the report uses

Formula purpose and system use. Purpose in this explanation: Refining shared character cuts: every term and its purpose **Source basis:** [33, 31, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$C_x = \frac{p_x}{q_{0.60}(p_j : p_j > 0)} + 1.15 \operatorname{clip}\left(\frac{b - y_{\text{top}}(x)}{h_x}, 0, 3\right).$$

The first term measures how much ink a cut would cross relative to a typical nonempty column. Excluding zero columns avoids a zero scale on sparse text; the numerical denominator floor handles degenerate cases. The second term penalises columns containing tall ink, favouring low connectors near the baseline over stems. Clip bounds limit the influence of extreme ascenders. The two terms are dimensionless; 1.15 sets their relative priority.

For smoothed ink count 2, positive-column percentile 8, height above baseline 4 and $h_x = 20$, the terms are 0.25 and $1.15(0.2) = 0.23$, total 0.48. A taller column with the same count and height 20 scores 1.40. A true blank column is explicitly assigned zero cost. This is a constructed arithmetic

example, not a measured cut. There is no published reason why 1.15 rather than 1.0 is optimal for these writers.

Read-only inspection also shows a smoothing width near $0.12h_x$, snapped to an odd integer of at least three; a search radius near $0.5h_x$ of at least three; and a proximity penalty $0.6|x - x_{\mathrm{CTC}}|/r$ inside that search. This last 0.6 is dimensionless and is unrelated to the 0.6 source-pixel polyline tolerance or the 0.6 mm hardware join. Ordered boundaries must leave $\max(1, 0.08h_x)$ pixels per cell. At $h_x = 25$ that minimum is 2 pixels. Without ordering, two neighbouring boundaries can snap to the same gap and create an empty character. Increasing minimum width may instead force a cut into a narrow genuine glyph. These are interacting heuristics; a local cost minimum alone cannot establish correct character ownership.

7.1.3 Binarisation, square dilation and Zhang–Suen thinning

Binarisation maps grey values to an ink/not-ink decision. It discards intensity information; later thinning cannot restore a faint stroke lost at this step. Dilation with square radius r sets a pixel if any mask pixel within $\max(|\Delta x|, |\Delta y|) \leq r$ is on. For radius one, an isolated pixel becomes a 3×3 block. Erosion requires all neighbourhood pixels to be on; closing combines the two in order. Boundary padding changes their behaviour at the edge.

For Zhang–Suen thinning, label the clockwise neighbours $p_2 = N, p_3 = NE, p_4 = E, p_5 = SE, p_6 = S, p_7 = SW, p_8 = W, p_9 = NW$. Let $B = \sum_{i=2}^{9} p_i$, and A be the number of 0-to-1 transitions in that circular list. A foreground pixel is eligible when $2 \leq B \leq 6$ and $A = 1$. In the first subiteration also require $p_2 p_4 p_6 = 0$ and $p_4 p_6 p_8 = 0$; in the second require $p_2 p_4 p_8 = 0$ and $p_2 p_6 p_8 = 0$. Delete marked pixels simultaneously within each subiteration and repeat until stable [31, 58]. The rules protect endpoints and local connectivity while eroding boundaries. Sequential in-place deletion would implement a different algorithm.

The skeleton approximates stroke geometry; it does not reconstruct pen pressure, velocity, original stroke order or an exactly equidistant medial axis. Thinning the complete line before splitting cells avoids artificial end conditions at every character boundary. Nearby strokes may still form ambiguous junctions. A distance-transform medial axis and vector contour fitting are alternatives, but they have different branch/noise behaviour.

7.1.4 Graph tracing, junction continuation and detached marks

Treat skeleton pixels as vertices and touching neighbours as edges. Degree one suggests an endpoint; degree two suggests continuation; larger degree indicates a junction. Short chains can be raster artefacts or real dots. Therefore a length filter must be combined with location/character context. At a junction the tracer chooses compatible continuations rather than claiming the original hand followed that order.

For outward endpoint directions u and v, straight continuation means they point oppositely. A continuation deflection can be written $\alpha = \arccos[(-u) \cdot v/(\|u\|\|v\|)]$, zero for straight passage. The report's limit is 75 degrees. If directions were instead both measured along travel, the minus sign would be omitted; mixing these conventions reverses the test. The threshold prevents a sharp turn from being accepted as a smooth continuation. It is permissive compared with 30 degrees and restrictive compared with 90, but no universal optimum follows from geometry. Detached dots are recovered as short strokes; crossbars require association rules so they are not mistaken for inter-character connectors. There is inherent ambiguity without online pen traces.

7.1.5 Douglas–Peucker: bounded simplification, units and example

Given endpoints A, B and intermediate point P, point-to-segment distance is

Formula purpose and system use. Purpose in this explanation: Douglas–Peucker: bounded simplification, units and example **Source basis:** [33, 31, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$t = \operatorname{clip}\left(\frac{(P-A)\cdot(B-A)}{\|B-A\|^2}, 0, 1\right), \qquad d(P, AB) = \|P - [A + t(B-A)]\|.$$

If $A = B$, use $\|P - A\|$. Find the farthest intermediate point. If its distance exceeds ϵ, retain it and recurse on the two subpaths; otherwise replace that subpath by its chord. This is the Douglas–Peucker principle [33, 59]. The inspected project uses the infinite supporting-line distance $|\Delta x(P_y - A_y) - \Delta y(P_x - A_x)|/(\|B - A\| + 10^{-9})$, rather than the clipped-segment formula above. The distinction matters near endpoints: distance to a supporting line can be small beyond a chord's endpoints. Consequently the project's tolerance is not an unrestricted symmetric Hausdorff bound between original and simplified curves.

For $(0,0), (1,0.2), (2,0)$, the intermediate distance is 0.2. At $\epsilon = 0.6$ it can be removed; at 0.1 it must be retained. At source x-height 30 pixels, 0.6 pixels is 0.02 x-heights; drawing at 4 mm per x-height gives a nominal source-to-output scale of 0.08 mm. This is a scale conversion of the simplification tolerance, not a bound on physical pen error. Resampling, skeleton error, smoothing, joining and machine error all add different distortions.

The report records 3738 vertices reduced to 974 at 0.6 source pixels, leaving about 26.1% and removing about 73.9%. That demonstrates compression for one line, not why 0.6 is optimal. Larger tolerances reduce G-code segments and can erase small loops/corners; smaller ones preserve raster jitter and increase work. Closed loops require care because identical endpoints make the initial chord degenerate; the inspected routine treats near-closed paths specially. Polygon topology and curvature are not generally guaranteed by ordinary Douglas–Peucker simplification.

7.1.6 Coordinate normalisation and shear: a worked transformation

For source pixel (x, y), left cut x_L, baseline b, estimated x-height $h_x > 0$, slant ϕ and trim δ,

Formula purpose and system use. Purpose in this explanation: Coordinate normalisation and shear: a worked transformation **Source basis:** [33, 31, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$v = \frac{b - y}{h_x}, \qquad u = \frac{x - x_L}{h_x} - v\tan\phi - \delta.$$

The first equation flips image-down to glyph-up. Division removes source scale. Subtracting $v\tan\phi$ removes slant; subtracting δ trims empty lead space, recorded separately so spacing can be restored. Shear is an affine transformation, not rotation: it shifts horizontal position in proportion to vertical height [25].

Take $x = 40, y = 30, x_L = 20, b = 50, h_x = 20, \phi = 10$ degrees and $\delta = 0.1$. Then $v = 1$ and $u = 1 - 0.17633 - 0.1 \simeq 0.72367$. Restoring lead 0.1 and the same shear at $H = 4$ mm gives horizontal offset $4(0.1 + 0.72367 + 0.17633) = 4$ mm and vertical offset 4 mm. If slant were applied both during extraction and placement without undoing it, the letter would be sheared twice. A negative or tiny estimated x-height invalidates all these units, hence explicit guards.

7.1.7 Size gates: 2.1 and 2.9 are widths, not heights

In the inspected source, ordinary glyph width is limited to 2.1 x-heights and designated wide glyphs to 2.9 [3]. The supplied list calls these height/width factors; the report and named constants identify both as *width* limits. At $h_x = 30$ pixels, they correspond to 63 and 87 pixels of normalised source

span. They target bad cuts containing neighbouring letters while allowing wider forms such as m and w. They do not derive from typography as universal ratios.

Other gates inspect height class, longest stroke, total length, stroke count, advance and within-character distributions. Ascenders and descenders should not be filtered by the same height expectation as a dot. Relative distributions allow writer-specific scale, but small samples can make them unreliable. Relaxing gates retains more personal variation and more fragments; tightening them reduces corruption and can eliminate rare valid forms. Missing characters then need explicit fallbacks whose provenance must remain visible.

7.1.8 The 12×12 descriptor and two different distances

The inspected shape-grid routine samples points along each segment, maps the glyph's bounding box to a 12×12 grid, accumulates counts, takes their square roots, applies a small blur, flattens to 144 values and normalises to unit Euclidean length. Sampling avoids representing a long straight segment by only its endpoints. Square roots reduce dominance by heavily sampled cells. The blur weights are centre 0.25 and eight neighbours 0.09375 each, summing to one. Flattening changes storage shape, not geometry.

Independent horizontal/vertical bounding-box scaling reduces translation and size sensitivity but also removes aspect-ratio information. Width/height gates therefore provide information the descriptor deliberately discards. Grid size 12 is a resolution/cost compromise: coarse grids alias details, fine grids become more sensitive to alignment and sampling. It is a project choice.

The pooled class reference is a componentwise median of descriptors followed by unit normalisation. For unit vectors v_g, r_c, prior distance is $d = 1 - v_g^T r_c$. Identical descriptors give zero. Because these descriptors are nonnegative, their dot products lie in $[0, 1]$, so this distance lies in $[0, 1]$ (up to rounding), rather than the $[0, 2]$ range of arbitrary signed unit vectors. A cosine distance need not obey the triangle inequality, so "distance" does not automatically mean a mathematical metric.

Within-writer consensus instead uses $D_{ij} = \|v_i - v_j\|_1$, the sum of absolute coordinate differences. The medoid is $i^* = \arg\min_i \sum_j D_{ij}$. With scalar examples $0, 0.1, 0.9$, the sums are $1.0, 0.9, 1.7$, so the second observed sample is selected. The mean is 0.333 and is not an observed sample. A medoid therefore is *not* a geometrical average shape. The denoised prototype is a separate constructed shape combining compatible strokes; incompatible stroke counts or order make naive vertex averaging invalid. The implementation's L1 consensus and cosine prior should not be silently substituted for one another.

7.1.9 Feature extraction, connectedness and complete profile algorithm

Slant comes from a shear search that concentrates ink into columns. Ascender and descender extents, advances, lead spacing and word gaps are measured in x-height units; robust summaries reduce isolated extraction errors. These features are explicit geometric measurements, unlike learned CNN features. Their meaning still depends on body-band and alignment quality.

Connectedness $c = N_{\text{cross}} / \max(1, N_{\text{adj}})$ counts eligible adjacent non-space boundaries crossed by a traced centreline. Nine crossed boundaries out of twelve give $c = 0.75$. It is not fraction of ink pixels touching, probability every new pair must join, or proof of original pen continuity. A bad cut can manufacture a crossing, and detached crossbars can produce false joins. If there are no eligible pairs the guarded formula returns zero; that represents no observed crossings, not strong evidence of printing.

The full profile process is: pair source lines with labels; prepare the reader input; force-align each label; estimate the body band/slant; reject unusable alignments; refine one ordered boundary array; thin and trace the full line; split chains at cuts; associate detached marks; normalise geometry; record spacing and boundary crossings; cache extraction; reject weak lines; filter each character's pool; compute prior and consensus; retain ranked variants and representative forms; store robust style

summaries and provenance. Caching changes cost, not correctness, and stale cached preprocessing can make a profile incompatible with current source. A usable glyph pool proves only that the filters accepted it. It does not establish that all letters or every writer's full style were captured.

Current function reference

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
MAX_VARIANTS_PER_CHAR = 12
ALIGN_CONF_PCT = 25
FRAGMENT_PEN_FRAC = 0.5
MIN_PEN_ABS = 0.3
MIN_LONGEST_STROKE = 0.55
DOT_MIN_AREA = 0.004
MAX_GLYPH_W = 2.1
MAX_GLYPH_W_WIDE = 2.9
_WIDE_CHARS = set('mwMW')
_BAR_CHARS = set('tfTF')
BAR_MIN_W = 0.25
BAR_MAX_W = 2.0
BAR_FLATNESS = 2.5
BAR_MIN_HEIGHT = 0.6
_OFFS = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]
_ASCEND = set('bdfhklt') | set('ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789')
_DESCEND = set('gjpqy')
_XONLY = set('acemnorsuvwxz')
```

Stage: Skeleton graph tracing and continuation

7.2 Method background: Skeleton graph tracing and continuation

Purpose | The role in this system

Turn centreline pixels into ordered polylines, then prune stubs and reconnect plausible continuations.

Justification | The reason for this construction

Endpoints, degrees and edge visitation provide explicit topology. Angle tests use opposing endpoint directions to distinguish continuation from a sharp turn.

Alternatives | What changes if another method is used

Contour tracing follows an outline, not this graph. Euler or route optimisation changes traversal, not the original unknown pen order. A learned tracer adds training and ambiguity.

Library | Implementation speed and accuracy

Graph packages could manage connectivity but per-pixel object graphs add overhead. Image libraries can supply skeletons/contours, not guaranteed original pen chronology. The project uses diagonal suppression and project thresholds. [31, 3].

7.3 TracePolylines

```
def TracePolylines(skel)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 56. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.2.

Purpose | What this contributes

The current implementation is the split-at-junction tracer directly; the older alternate through-junction tracer and feature switch are absent. It builds a pixel graph, suppresses redundant diagonal edges when an orthogonal connection exists, visits edges, prunes short chains and rejoins compatible endpoint directions. It estimates a useful drawing order, not the historical pen chronology.

Method | Current implementation

Turn centreline pixels into ordered polylines, then prune stubs and reconnect plausible continuations. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def TracePolylines(skel):
    pts = set((int(y),int(x)) for y,x in zip(*np.nonzero(skel)))
    if not pts:
        return []

    def Nbrs(p):
        y,x = p
        out = []
        for dy,dx in _OFFS:
            q = (y+dy,x+dx)
            if q not in pts:
                continue
            if dy and dx and ((y+dy,x) in pts or (y,x+dx) in pts):
                continue
            out.append(q)
        return out

    deg = {p: len(Nbrs(p)) for p in pts}
    usedEdges = set()
    polylines = []

    def Walk(start,first):
        chain = [start,first]
        usedEdges.add((start,first))
        usedEdges.add((first,start))
        cur,prev = first,start
        while deg[cur]==2:
            nxt = [n for n in Nbrs(cur) if n!=prev]
            if not nxt:
                break
            n = nxt[0]
            if (cur,n) in usedEdges:
                break
            usedEdges.add((cur,n))
            usedEdges.add((n,cur))
            chain.append(n)
            prev,cur = cur,n
        return chain

    seeds = sorted([p for p in pts if deg[p]!=2]) or sorted(pts)[:1]
    for p in seeds:
```

```
        for n in Nbrs(p):
            if (p,n) not in usedEdges:
                polylines.append(Walk(p,n))
    covered = {q for ch in polylines for q in ch}
    for p in sorted(pts-covered):
        if deg.get(p,0)==2:
            ns = Nbrs(p)
            if ns and (p,ns[0]) not in usedEdges:
                polylines.append(Walk(p,ns[0]))

    chains = [[(float(x),float(y)) for (y,x) in ch] for ch in polylines
              if len(ch)>=2]
    chains = _PruneSpurs(chains)
    return _JoinAtJunctions(chains)
```

Justification | Interpret the method rather than infer a benchmark

Endpoints, degrees and edge visitation provide explicit topology. Angle tests use opposing endpoint directions to distinguish continuation from a sharp turn. [31, 3].

7.4 _Dir

```
def _Dir(pts, fromEnd, n=4)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 115. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.2.

Purpose | What this contributes

Turn centreline pixels into ordered polylines, then prune stubs and reconnect plausible continuations.

Inputs

`pts` a polyline

`fromEnd`
: true for the direction at the last point, false for the first

`n` how many points back to look, default 4

Output. a unit vector giving the direction the path is heading at that end.

Purpose | What it is for

Deciding whether two traced paths were one pen stroke requires knowing which way each is heading where they meet. This measures that direction at either end of a path.

Justification | Why it looks several points back

The direction could be taken from the last two points, but those are adjacent pixels, so the step between them is one of only eight possible directions, each 45 degrees apart. That resolution is far too coarse to decide whether two strokes continue smoothly. Measuring from a point four steps back gives a chord over several pixels, whose angle is continuous and which averages out the staircase the pixel grid imposes.

Formula purpose and system use. Estimate a short endpoint direction for a junction-continuation decision. **Source basis:** [31, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathbf{d} = \frac{\mathbf{p}_{\text{end}} - \mathbf{p}_{\text{end}-n}}{\left\| \mathbf{p}_{\text{end}} - \mathbf{p}_{\text{end}-n} \right\|} \tag{7.1}$$

where

$\mathbf{d}$ the returned direction, a vector of length 1

$\mathbf{p}_{\text{end}}$ the point at the end being measured

$\mathbf{p}_{\text{end}-n}$
the point n steps in from that end

n how far back to look, 4 by default

$\| \cdot \|$ the length of the vector, by Pythagoras

Justification | Why it is normalised to unit length

Dividing by the length leaves only the direction, discarding how far apart the two points were. That is what makes two directions comparable by a dot product, as the next function does: for unit vectors the dot product is the cosine of the angle between them, with no dependence on the lengths. Without normalisation the same comparison would confound the angle with how long the paths happened to be.

> *A degenerate path whose endpoints coincide has no direction, and a default pointing along the positive x axis is returned. The value is arbitrary; what matters is that it is a unit vector, so the caller's angle arithmetic stays well defined instead of producing a division by zero.*

7.5 _PruneSpurs

```
def _PruneSpurs(chains, minLen=2.5)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 126. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.2.

Purpose | What this contributes

Turn centreline pixels into ordered polylines, then prune stubs and reconnect plausible continuations.

Inputs

`chains`
the traced polylines

`minLen`
the shortest a dangling path may be before it is removed, default 2.5 pixels

Output. the polylines with the short dangling ones removed.

Method | The artefact it removes

Thinning a shape where strokes cross leaves a characteristic defect. Where the ink is thick, as at the junction of a t or where a cursive join meets a letter body, the thinning does not produce a clean

crossing but a small cluster from which short stubs project, going nowhere. These are **spurs**, and they are an artefact of the thinning rather than anything the pen did.

They matter because they are traced as polylines like any other, so a letter that should have two strokes acquires five, three of which are two-pixel whiskers. That inflates the stroke count, which several filters test, it disturbs the stroke ordering, and it confuses the averaging that forms a prototype, which matches strokes between variants by position.

Method | How a spur is recognised

Two conditions together. It is short, under 2.5 pixels of path length. And it is **dangling**, meaning at least one of its two endpoints is not shared with any other path, so it leads nowhere. Requiring both is what makes the test safe: a short path joined at both ends is a genuine short connection between two strokes and must be kept, while a long dangling path is a real stroke with a free end, such as the stem of a t.

The sharing is determined by counting how many paths have an endpoint at each coordinate. A count of one means that endpoint belongs to this path alone.

Justification | Why the threshold is in pixels and not x-heights

Almost every threshold in this file is in x-heights, for the scale-independence reasons given in the introduction. This one is deliberately not, because the artefact it targets is not a property of the writing at all: it is a property of the thinning algorithm, whose spurs are a few pixels long whatever the resolution, because thinning works pixel by pixel. A threshold in x-heights would grow with resolution and start removing real strokes from high-resolution crops.

> *If the pruning would remove everything, the original paths are returned instead. A glyph consisting entirely of short dangling paths is most likely debris, but returning nothing would hide it from the filters that are designed to judge it; returning it unchanged lets them do so.*

7.6 _JoinAtJunctions

```
def _JoinAtJunctions(chains, maxAngleDeg=75.0)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 146. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.2.

Purpose | What this contributes

Merges two paths that meet at a junction and continue in roughly the same direction, which means they were one pen stroke before thinning split them.

Inputs

`chains`
: the traced polylines

`maxAngleDeg`
: how much the direction may change at a join and still count as one stroke, default 75 degrees

Output. the polylines with those that continue through a junction merged into one.

Purpose | The problem it solves

The tracer stops at every junction, because locally there is no way to know which way to continue. But a pen usually does continue. Where a cursive join meets the body of a letter, or where one stroke crosses another, the tracer produces several short paths where the pen drew one long one. Leaving them separate misrepresents the writing: the glyph appears to have more strokes than it does, and a plotter drawing it would lift the pen where the writer did not.

Method | The criterion: continuation of direction

What distinguishes a continuation from a genuine corner is direction. If one path arrives at a junction heading one way and another leaves heading the same way, they were one stroke. If the second leaves at a sharp angle, the junction is a real corner or a crossing of two different strokes.

Formula purpose and system use. Measure deflection between opposing endpoint directions and reconnect plausible continuations. **Source basis:** [31, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\cos\theta \;=\; -\,\mathbf{d}_i \cdot \mathbf{d}_j, \qquad \theta \;=\; \arccos\left(\mathrm{clip}(\cos\theta,\,-1,\,1)\right) \tag{7.2}$$

where

θ the turn angle: 0 for a path continuing perfectly straight through the junction

$\mathbf{d}_i$, $\mathbf{d}_j$
: the unit directions at the two ends being considered, each from _Dir

$\cdot$ the dot product, which for unit vectors equals the cosine of the angle between them

$-$ the negation, explained below

arccos
: the inverse cosine, converting a cosine back to an angle

$\mathrm{clip}(\cdot, -1, 1)$
: the cosine forced into its valid range before the inverse cosine is taken

Justification | Why the dot product is negated

This is the step most easily got wrong. Both directions are measured *outward* from their own path's end. Two paths that continue through a junction therefore point in opposite directions: one arrives pointing into the junction, the other leaves pointing away, and as outward measurements those are roughly antiparallel. So perfect continuation gives a dot product of -1, not $+1$, and negating it turns continuation into a cosine of $+1$, which is what the angle formula expects.

Justification | Why the cosine is clipped

The inverse cosine is defined only on the range -1 to 1. Two unit vectors cannot mathematically produce a dot product outside it, but floating-point rounding can: a vector whose length is stored as 1.0000000001 yields a dot product a fraction beyond 1, and the inverse cosine of that is not a number. The result would then propagate silently through every comparison, since comparisons involving it are all false. Clipping first costs nothing and removes the possibility.

Justification | Why 75 degrees is generous

A threshold of 75 degrees permits substantial direction change at a skeleton junction. The qualitative trade-off is between leaving a plausible continuation split and joining branches that were separate. Incorrect joining can alter stroke order, connections and shape, not merely the stroke count. The exact 75-degree value is a project heuristic; its optimality and asymmetric error costs are not established by the algorithm citation.

Justification | Why it repeats until nothing changes

One merge can create a new opportunity. Three paths meeting in a line are joined by first merging two, after which the merged path has a new free end that may continue into the third. So the search repeats until a pass finds nothing to merge. Each pass also takes the single best candidate, the pair with the smallest turn angle, rather than merging everything that qualifies. That matters at a junction where three or more paths meet and more than one pairing is permissible: taking the best first means the most convincing continuation is committed to before the others are considered, which is the greedy strategy used for the line centres in segment.py, but this greedy order does not prove globally correct reconstruction.

> *The search compares every pair of ends on every pass, which is quadratic in the number of paths and runs inside a loop. That would matter for hundreds of paths; here it runs on the handful belonging to one letter, where the cost is irrelevant and the clarity of an exhaustive search is worth having.*

Stage: Douglas Peucker simplification

Method rationale, alternatives and library comparison: Section 5.63.

7.7 SimplifyPolyline

```
def SimplifyPolyline(poly, eps=0.6)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 180. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.63.

Purpose | What this contributes

Reduce path vertices and command count while retaining meaningful geometric detail.

Inputs

`poly`
: a polyline

`eps` how far a point may sit from the simplified path before it must be kept, default 0.6 pixels

Output. the polyline with the points that add nothing removed.

Purpose | What it is for

A traced path has one point per pixel, so a straight stroke fifty pixels long is stored as fifty points when two would describe it exactly. Removing the redundant points shrinks the stored profile, speeds up everything that iterates over points, and removes the pixel staircase, leaving a path described by where it actually changes direction.

Method | The Douglas-Peucker method

The algorithm [33] is recursive and rests on one idea: a path can be replaced by the straight line between its endpoints if no point on it strays far from that line. So measure the furthest stray; if it is within tolerance, the whole path becomes two points, and if not, the path is split at the furthest point and each half is simplified the same way.

Formula purpose and system use. Measure deviation from the endpoint supporting line when deciding whether to retain a vertex. **Source basis:** [33, 20]. Exact project constants remain implementation choices unless evidence is stated.

$$d_k = \frac{\left|\,(x_b - x_a)(y_k - y_a) - (y_b - y_a)(x_k - x_a)\,\right|}{\sqrt{(x_b - x_a)^2 + (y_b - y_a)^2}} \tag{7.3}$$

Source: [33]

where

d_k the perpendicular distance from point k to the straight line joining the two endpoints

(x_a, y_a)
: the first point of the path

(x_b, y_b)
: the last point of the path

(x_k, y_k)
: the point whose distance is being measured

$|\cdot|$ the absolute value, since a distance has no sign

$\sqrt{\cdot}$ the length of the line segment, which turns the numerator into a true distance

The numerator is the **cross product** of the vector along the line and the vector to the point. Its magnitude equals the area of the parallelogram the two vectors span, and that area also equals the base times the height. The base is the length of the line segment, so dividing by it leaves the height, which is the perpendicular distance. This is why the formula computes a distance without any trigonometry.

Justification | Why the tolerance is 0.6 pixels

The 0.6-pixel tolerance is a project simplification setting, not a proven boundary between sampling noise and intentional writing. Nearest rounding can move a point by up to half a pixel on each axis, or about 0.707 pixels in Euclidean distance; error relative to a line reconstructed from rounded endpoints is a different quantity again. Raising epsilon generally removes more vertices and may erase real small features; lowering it retains more detail and more sampling noise. No literature result or reviewed sweep here establishes 0.6 as optimal.

Method | The closed-path special case

A loop, such as the body of an o, defeats the basic method. Its endpoints coincide or nearly so, which makes the reference line a point rather than a line, and every distance measured from it is meaningless. The function detects this by checking whether the far end of the path returns close to the start, and if so splits the path first at the point furthest from the start, simplifying the two halves separately. Each half is then an ordinary open path with distinct endpoints, and the method applies.

> *The two halves are rejoined by dropping the last point of the first before concatenating, since the split point belongs to both and would otherwise be stored twice. A duplicated point is a zero-length segment, which is harmless to draw but divides by zero in any calculation that normalises by segment length.*

Stage: Alignment guided glyph extraction

Report equation 25 Forced transcript alignment

Method | Formula

$V(t,s) = \log p_t(\ell'_s) + \max_{r \in \mathcal{P}(s)} V(t-1, r)$.

Purpose | Why the system needs it

Recover a high-scoring legal path consistent with the known label before extracting characters. Replacing path summation by maximisation gives a hard alignment, not proof of correct geometric boundaries.

Evidence | Current location and source

CtcForcedAlign and ExtractLineGlyphs. [34, 54].

Report equation 26 Character cut cost

Method | Formula

$C_x = p_x / q_{0.60}(p_j : p_j > 0) + 1.15\,\mathrm{clip}[(b - y_{\mathrm{top}}(x))/h_x, 0, 3]$.

Purpose | Why the system needs it

Prefer thin low connectors over stems near the recogniser's suggested cut. Blank columns have zero cost. Ordered boundaries and minimum cell width prevent neighbouring cuts collapsing. All exact weights/percentiles are project heuristics, not derived from CTC.

Evidence | Current location and source

ExtractLineGlyphs. [3].

Report equation 27 Stored glyph coordinates

Method | Formula

$v = (b - y)/h_x$, $u_0 = (x - x_L)/h_x - v \tan\phi$, and $u = u_0 - \delta$.

Purpose | Why the system needs it

Flip image-down to glyph-up, remove source scale and slant, and store trim as lead spacing. Shear is not rotation. The inverse placement must restore the same conventions.

Evidence | Current location and source

ExtractLineGlyphs and SynthesizeText. [25, 28, 3].

Report equation 28 Measured connectedness

Method | Formula

$c = N_{\mathrm{cross}} / \max(1, N_{\mathrm{adj}})$.

Purpose | Why the system needs it

Summarise eligible in-word boundaries crossed by a traced centreline. It is not measured original pen-lift probability: touching and bad cuts can inflate it. Current synthesis suppresses joins below 0.45 and chooses cursive fallback style at 0.5.

Evidence | Current location and source

BuildAuthorProfile, _CoreGlyph and SynthesizeText. [3, 60].

7.8 Method background: Alignment guided glyph extraction

Purpose | The role in this system

Recover labelled character geometry from offline lines without manually supplied character boxes.

Justification | The reason for this construction

The known transcript constrains CTC; shared ordered cuts, body-band estimates and shear normalisation separate shape from source size and spacing.

Alternatives | What changes if another method is used

Connected-component letters work mainly for separated print. Independent cuts can collapse into the same gap. Direct generative synthesis needs extra training and does not expose these stored shapes.

Library | Implementation speed and accuracy

A library OCR aligner can supply another alignment distribution, but changing it changes extraction quality and provenance. NumPy, OpenCV or SciPy supply parts; the complete cut/ownership/profile policy is project-specific. [34, 28, 3].

7.9 _TrimLigatureTails

```
def _TrimLigatureTails(strokes, connL=True, connR=True, lowY=0.42, minKeep=0.18, climbY
    =1.6, climbSlope=2.2)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 205. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Removes eligible low outward tails left at character cuts and records connection endpoints. Guards retain a minimum body and reject suspicious climbing tails. These local rules can confuse genuine flourishes with connectors; the thresholds are project heuristics.

Method | Current implementation

Recover labelled character geometry from offline lines without manually supplied character boxes. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _TrimLigatureTails(strokes,connL=True,connR=True,lowY=0.42,minKeep=0.18,
                       climbY=1.6,climbSlope=2.2):
    if not strokes:
        return strokes,(0.0,0.0),(0.0,0.0)
    allPts = [p for s in strokes for p in s]
    entry,exitPt = strokes[0][0],strokes[-1][-1]
    xs = [p[0] for p in allPts]
    span = max(xs)-min(xs)
    if span<=1e-6:
```

```
        return strokes,entry,exitPt

    def RunLen(s,connected,maxHeight,maxSlope):
        if not connected:
            return 0
        i = 0
        while i+1<len(s):
            dx = s[i+1][0]-s[i][0]
            dy = s[i+1][1]-s[i][1]
            if s[i][1]>=maxHeight or dx<0 or abs(dy)>maxSlope*max(dx,1e-6):
                break
            i+=1
        return i

    out = []
    for k,s in enumerate(strokes):
        t = list(s)
        if k==0:
            i = RunLen(t,connL,lowY,0.7) or RunLen(t,connL,climbY,climbSlope)
            if i and (t[-1][0]-t[i][0])>minKeep*span:
                t = t[i:]
        if k==len(strokes)-1 and len(t)>=2:
            rev = [(-x,y) for (x,y) in t[::-1]]
            r = RunLen(rev,connR,lowY,0.7) or RunLen(rev,connR,climbY,climbSlope)
            if r and (t[len(t)-1-r][0]-t[0][0])>minKeep*span:
                t = t[:len(t)-r]
        if len(t)>=2:
            out.append(t)
    if not out:
        return strokes,entry,exitPt
    return out,out[0][0],out[-1][-1]
```

Justification | Interpret the method rather than infer a benchmark

The known transcript constrains CTC; shared ordered cuts, body-band estimates and shear normalisation separate shape from source size and spacing. [34, 28, 3].

7.10 ExtractLineGlyphs

```
def ExtractLineGlyphs(gray, text, model)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 254. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Uses model.Forward, forced CTC alignment, an estimated body band and slant, then one shared strictly increasing cut array. It thins the full line before cell splitting, traces paths, handles dots/crossbars and stores upright x-height coordinates plus lead/advance metadata. A legal transcript alignment is not a verified character boundary.

Method | Current implementation

Recover labelled character geometry from offline lines without manually supplied character boxes. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ExtractLineGlyphs(gray,text,model):
```

```
    rawW = gray.shape[1]
    t = TensorFromResized(ResizeLineImageFixed(gray))
    lp = model.Forward(t)[:,0,:]
    res = CtcForcedAlign(lp,text)
    if res is None:
        return None, None
    align,conf = res
    T = lp.shape[0]

    ink = BinarizeLine(gray)
    band = CoreBand(ink)
    if band is None:
        return None, None
    top,base = band
    xh = float(base-top)
    slant = EstimateSlantDeg(ink)
    shear = np.tan(np.radians(slant))

    colProf = ink.sum(axis=0).astype(np.float64)
    k = max(3,int(round(xh*0.12)))|1
    colSm = np.convolve(colProf,np.ones(k)/k,mode='same')
    rowsIdx = np.arange(ink.shape[0])[:, None]
    hiY = np.where(ink,rowsIdx,ink.shape[0]).min(axis=0).astype(np.float64)
    topAbove = np.clip((base-hiY)/max(1.0,xh),0.0,3.0)   #highest ink per column, above the baseline
    thin = colSm/max(1e-6,float(np.percentile(colSm[colSm>0],60))
                      if (colSm>0).any() else 1.0)

    cutCost = thin+1.15*topAbove
    cutCost[colProf==0] = 0.0   #a true gap is always a free cut
    snapR = max(3,int(round(xh*0.5)))

    def _Snap(x,floor=None):
        a = int(np.clip(round(x)-snapR,0,rawW-1))
        b = int(np.clip(round(x)+snapR+1,1,rawW))
        if b-a<2:
            return float(x)
        win = cutCost[a:b].copy()
        off = np.abs(np.arange(a,b)-x)/max(1.0,snapR)
        return float(a+int(np.argmin(win+0.6*off)))

    origH = gray.shape[0]
    charXs = []
    for ch,sp in align:
        if sp is None:
            charXs.append((ch, None, None))
            continue
        x0 = FrameXToPixel(sp[0]/T*INPUT_WIDTH,rawW,origH)
        x1 = FrameXToPixel(sp[1]/T*INPUT_WIDTH,rawW,origH)
        charXs.append((ch,x0,x1))
    valid = [(i,c) for i,c in enumerate(charXs) if c[1] is not None]
    centres = [0.5*(c[1]+c[2]) for _,c in valid]

    nV = len(valid)
    minCell = max(1.0,0.08*xh)
    bnd = [0.0]*(nV+1)
    bnd[0] = max(0.0,centres[0]-(0.5*(centres[1]-centres[0])
                                  if nV>1 else xh))
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

The known transcript constrains CTC; shared ordered cuts, body-band estimates and shear normalisation separate shape from source size and spacing. [34, 28, 3].

7.11 _ClassOk

```
def _ClassOk(ch, top, bot)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 505. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Recover labelled character geometry from offline lines without manually supplied character boxes.

Inputs

`ch` the character this glyph is supposed to be

`top` its highest point, in x-heights above the baseline

`bot` its lowest point, negative below the baseline

Output. true if the glyph reaches where that character should reach.

Justification | Why this is the cheapest and most reliable gate

The height class of a letter is one of the few facts about a letterform that is true for every writer. Individual hands vary enormously in slant, width, roundness and how joined they are, but an l reaches above the x-height band in all of them, because otherwise it would not be an l. A test on reach is therefore almost free of false rejections while catching a large class of bad cuts, which is why it runs first.

Method | The three tests

Formula purpose and system use. Apply character-height expectations without forcing all letters to have the same extent. **Source basis:** [34, 28, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\begin{aligned} \text{x-only:} &\quad \text{top} \le 1.42 \ \text{ and } \ \text{bot} \ge -0.42 \\ \text{ascender:} &\quad \text{top} \ge 1.25 \\ \text{descender:} &\quad \text{bot} \le -0.20 \end{aligned} \tag{7.4}$$

where

top the glyph's highest point, in x-heights, where 1.0 is the top of the x-height band

bot its lowest point, negative below the baseline

1.42 how far above the band an x-only letter may stray before it is judged to have collected a neighbour's ascender

1.25 how far above the band an ascender letter must reach

−0.20 how far below the baseline a descender letter must drop

−0.42 how far below an x-only letter may stray

Justification | Why the thresholds are not at 1.0

The band is nominally 0 to 1, but its measured boundaries are uncertain. Margins can tolerate that uncertainty. The code uses different admissible extents for x-height, ascender and descender classes; these are heuristic gates rather than confidence intervals inferred from a measured error distribution.

The 1.42 and 1.25 upper-band tests serve different classes: rejecting excess height in an x-height letter versus demanding some extension for an ascender. Their overlap means a height between these values may be admissible under either class test. It does not prove a measured ambiguity interval or establish that contamination always exceeds a particular height. No reviewed source establishes these exact cutoffs as universal.

7.12 _PenLength

```
def _PenLength(g)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 515. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Recover labelled character geometry from offline lines without manually supplied character boxes.

Inputs

`g` a glyph dictionary

Output. the total length of all its strokes, in x-heights.

Method | What it measures

How far the pen travelled to draw this glyph, obtained by summing the lengths of every stroke, each of which is the sum of the straight-line distances between its consecutive points. It is the quantity the fragment test compares against a per-character floor, for the reasons set out under FRAGMENT_PEN_FRAC.

Justification | Why total rather than per stroke

A letter may legitimately be drawn in one stroke or several, depending on the hand and on whether the pen lifted. The same letter from the same writer can appear with two strokes on one line and one on the next, where a join happened to survive the thinning. The total is invariant to that, so it measures the letter rather than the accident of how it came out.

7.13 _LongestStroke

```
def _LongestStroke(g)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 524. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Recover labelled character geometry from offline lines without manually supplied character boxes.

Inputs

`g` a glyph dictionary

Output. the length of its single longest stroke, in x-heights.

Justification | What it measures, and why it is a separate test

The longest single continuous stroke. Where the total pen length asks how much writing is present, this asks whether any of it is a real stroke. The two are independent: a scatter of eight short scraps passes the total test and fails this one, while a single clean stroke that is slightly short passes this and fails the total. Keeping them separate means each can be set to catch its own defect, rather than one compromise threshold catching neither well.

> *Both measures ignore strokes of fewer than two points, which have no length to contribute. A single isolated point can survive into a glyph from the dot recovery, and including it as a zero-length stroke would be harmless here but would divide by zero in the resampling that forms prototypes.*

7.14 RenderRefStats

```
def RenderRefStats(gray)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 724. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Measures one real line as the networks see it, ink density, aspect, paper and ink levels, word gap, slant and reach.

Inputs

`gray`
: one real line crop from this writer

Output. a dictionary of measurements describing how that line looks, both to the networks and geometrically.

Purpose | What it is for, and the idea behind it

Reproducing a hand is not only a matter of letter shapes. The output must also *look* like a photograph of handwriting to the networks that judge it, and that involves properties no glyph library captures: how much of the image is ink, how dark the ink is, how bright the paper is, the proportions of the line. This function measures those on a real line, so that synthesis has a target to match rather than having to guess.

Method | Background: domain shift

Domain shift is the name for the situation where a model is used on data drawn from a different distribution than it was trained on. A network trained on photographs of handwriting has learned

the statistics of photographs: paper that is not pure white, ink that is not pure black, and soft grey edges where the two meet. Rendering a synthetic line as pure black on pure white produces an image that is correct as geometry but statistically unlike anything in training, and the network's response to it is unreliable, which would corrupt the judgement of the output.

This is why the paper and ink levels are measured as percentiles of the real crop rather than assumed: the median of the bright pixels is what this writer's paper actually is, and the median of the dark pixels is what their ink actually is. The fraction of intermediate greys is measured too, since that describes the softness of the stroke edges.

Justification | Why the word gap is measured from the ink, not the recogniser

The width of the space character as the aligner reports it is not a reliable measure of how widely this writer spaces their words. CTC concentrates its output and tends to assign a space a narrow territory, well inside the actual gap. So the gap is measured directly instead: the columns with no ink are found, the runs between inked columns are collected, those under half an x-height are discarded as being gaps within a word, and the median of the rest is taken.

The half-x-height threshold is what separates the two kinds of gap. Within a word the pen may lift, leaving a small gap, but a word boundary is a deliberate space and is substantially wider. The median of the wide gaps therefore describes word spacing, and the median again rather than the mean, so that one unusually wide gap at a paragraph end does not inflate it.

Justification | Why slant, ascender and descender are measured here too

These are also measured in the profile from the glyphs themselves, so measuring them again on the rendered line looks redundant. It is not, and the comment in the code gives the reason: these are measured *the same way* they will be measured on the synthesised output. A quantity measured by two different methods is not comparable with itself, so to check whether the synthesis reproduced the writer's slant, both sides must be measured by the same procedure. These values exist to be the reference half of that comparison.

> *Fields that cannot be measured are filled with a not-a-number value rather than a plausible default. That is deliberate: the caller takes a median across lines with those values excluded, so a line that could not be measured contributes nothing, whereas a default would contribute a fabricated measurement that is indistinguishable from a real one.*

7.15 _MedianWordGap

```
def _MedianWordGap(refs, spaceAdvs)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 763. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Recover labelled character geometry from offline lines without manually supplied character boxes.

Inputs

`refs`
: the per-line reference statistics

`spaceAdvs`
: the widths the aligner gave to space characters

Output. the typical gap this writer leaves between words, in x-heights.

Method | The preference order

Three sources are tried in order. The measured ink gaps from RenderRefStats are preferred, because they measure the real thing. If none is available, the aligner's space widths are used, which are biased narrow for the reason given above but are at least derived from this writer. If there are no spaces at all, 1.2 x-heights is returned as a generic value.

Justification | Why the fallback order is this way round

It descends from the most direct evidence to the least. The same cascade pattern appears in LinePitch and in the checkpoint selection: try the best-informed method, fall back only when it cannot apply. The consequence worth noting is that the fallback is known to be biased, so a profile built from a writer with no measurable gaps will have word spacing that is too narrow, and that is a better failure than having none at all.

> *Not-a-number values are dropped by testing whether each value equals itself, which is false only for not-a-number. That is the standard idiom and it works without importing anything, which is why it is used in a file that avoids dependencies.*

7.16 ExtractAuthorRaw

```
def ExtractAuthorRaw(authorId, lineItems, model)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 774. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Runs the cutting over every line of a writer, returning the uncut, unfiltered result per line.

Inputs

`authorId`
: which writer these lines belong to

`lineItems`
: pairs of a line crop and its known text

`model`
: a loaded reader for alignment

Output. a list of per-line results, each a pair of the glyphs cut from that line and its statistics.

Purpose | What it is for

Runs the cutter over every line of one writer and collects the results. It is the expensive half of profile building, and the half that gets cached.

Justification | Why the output is deliberately unfiltered

Nothing is rejected here. Every glyph the cutter produced is kept, including the fragments and the over-wide cuts. This is the design decision that makes the filters tunable: the filtering happens in BuildAuthorProfile, which runs on this output, so a threshold can be changed and the effect measured

in seconds against a cache that is still valid. Had filtering happened here, every change to a filter would require re-reading and re-cutting every page.

It is also what made the project's rejection statistics possible. Reporting what fraction of cuts were discarded, and why, requires having kept the discarded ones; a pipeline that filtered as it extracted could report only what survived.

> *A line whose extraction returned nothing is skipped rather than recorded as empty. Those are the lines where alignment failed outright or no x-height band could be found, which carry no glyphs and no usable statistics, so an entry for them would only have to be filtered out again by every consumer.*

7.17 BuildAuthorProfile

```
def BuildAuthorProfile(authorId, parsed, refs=None, prior=None, verbose=True)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 789. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.8.

Purpose | What this contributes

Rejects low-confidence source lines when enough are available, accumulates measured style, filters each character pool, computes prior and consensus, and stores variants, denoised prototypes and medoids. The profile stores ink-related reference statistics even though current RenderTrajectory uses uniform ink. Such stored metadata is not evidence that a live branch consumes it.

Method | Current implementation

Recover labelled character geometry from offline lines without manually supplied character boxes. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def BuildAuthorProfile(authorId,parsed,refs=None,prior=None,
                       verbose=True):
    store = {}
    slants,xhs,strokeWs = [],[],[]
    ascRatios,descRatios = [],[]
    advances = {}
    spaceAdvs = []
    connPairs,adjPairs = 0,0

    if not parsed:
        return None
    confs = [st['alignConf'] for _,st in parsed]
    confCut = (float(np.percentile(confs,ALIGN_CONF_PCT))
               if len(confs)>=8 else -1e9)

    for glyphs,stats in parsed:
        if stats['alignConf']<confCut:
            continue
        slants.append(stats['slant'])
        xhs.append(stats['xh'])
        strokeWs.append(stats['strokeW']/max(1.0,stats['xh']))
        for k,g in enumerate(glyphs):
            if g is None:
                continue
            ch = g['char']
            if ch==' ':
```

```
                spaceAdvs.append(g['width'])
                continue
            if ch.islower():
                if g['top']>1.35:
                    ascRatios.append(g['top'])
                if g['bot']<-0.25:
                    descRatios.append(g['bot'])
            advances.setdefault(ch,[]).append(g['advance'])
            store.setdefault(ch,[]).append(g)
            if k+1<len(glyphs) and glyphs[k+1] is not None and \
                    glyphs[k+1].get('char',' ')!=' ' and 'connR' in g:
                adjPairs+=1
                if g['connR']:
                    connPairs+=1

    if not xhs:
        return None

    storeKept = {}
    for ch,insts in store.items():
        wMax = MAX_GLYPH_W_WIDE if ch in _WIDE_CHARS else MAX_GLYPH_W
        penFloor = max(MIN_PEN_ABS,FRAGMENT_PEN_FRAC*
                       float(np.median([_PenLength(g) for g in insts])))
        good = [g for g in insts
                if 0.05<g['width']<wMax and (g['top']-g['bot'])>0.25
                and len(g['strokes'])<=6
                and _ClassOk(ch,g['top'],g['bot'])
                and g['lead']+g['width']<=1.45*g['advance']+0.25
                and _PenLength(g)>=penFloor
                and _LongestStroke(g)>=MIN_LONGEST_STROKE]
        if not good:
            continue
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

The known transcript constrains CTC; shared ordered cuts, body-band estimates and shear normalisation separate shape from source size and spacing. [34, 28, 3].

Stage: Occupancy descriptors and shape distances

Method | Writer texture allographs and ink width

Textural edge-direction distributions summarise local stroke orientations; allographs are alternative realisations of a character. The explicit glyph pool stores allographic variation, while the CNN learns discriminative maps. Ink width can carry writer evidence but changes when the robot uses one pen. That motivates shape normalisation, without proving all ink information is undesirable or every normalised prediction is more accurate [60, 61].

7.18 Method background: Occupancy descriptors and shape distances

Purpose | The role in this system

Reject malformed cuts and compare stored shapes without requiring matching vertex counts.

Justification | The reason for this construction

A sampled twelve-by-twelve occupancy grid, square-root compression, blur and unit norm provide a

compact shift-tolerant representation. Width gates separately retain aspect information lost by box normalisation.

Alternatives | What changes if another method is used

Raw pixel differences are sensitive to translation; point-set distances require registration; learned embeddings need training. Higher grid resolution keeps detail but is less tolerant of shifts.

Library | Implementation speed and accuracy

SciPy distances and image resizers offer primitives; a learned embedding is not an arithmetic replacement. Faster distance computation preserves the metric only if normalisation and sampling are identical. [3, 60].

7.19 _ShapeGrid

```
def _ShapeGrid(g, n=12)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 536. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.18.

Purpose | What this contributes

Describes a shape as a normalised 12 by 12 grid of ink counts, blurred a little so two writings of one letter need not line up exactly.

Inputs

`g` a glyph dictionary

`n` the grid resolution, default 12

Output. a vector of 144 numbers describing the glyph's shape, scaled to unit length; or nothing if the glyph has no points.

Purpose | What it is for

Several filters need to ask how similar two glyphs are, and one needs to ask how similar a glyph is to the general idea of a letter. Comparing two lists of strokes directly is not workable: they may have different numbers of strokes, different numbers of points, and the points need not correspond. What is needed is a **descriptor**, a fixed-length vector computed from a shape such that similar shapes give similar vectors. Then comparing shapes becomes comparing vectors, which is arithmetic.

Method | Step one: turn strokes into points

Each segment of each stroke is filled in with evenly spaced points along it, at a density of about 24 per unit length. This is necessary because the stored strokes have been simplified, so a long straight run is two points, and counting points directly would weight a simplified stroke far less than a wiggly one of the same length. Filling in restores the proportionality between a stroke's length and the number of points representing it.

Method | Step two: scale the shape into the grid

The points are rescaled so the shape exactly fills the grid in both directions, and each point is assigned to a cell. This discards absolute size and aspect ratio deliberately, so that the same letter written

large and small gives the same descriptor. Size is already tested separately by the width and height gates, so there is nothing lost by removing it here, and a great deal gained: the descriptor then responds to shape alone.

Method | Step three: count, then damp

Formula purpose and system use. Map sampled glyph coordinates into a compact occupancy grid. **Source basis:** [3, 60]. Exact project constants remain implementation choices unless evidence is stated.

$$G[i,j] \;=\; \sqrt{\left|\{\,\text{points falling in cell } (i,j)\,\}\right|} \tag{7.5}$$

where

$G[i,j]$
: the value stored for grid cell i, j

$|\{\cdot\}|$: the number of points that landed in that cell

The square root is a **damping** transformation. Without it, a cell where the pen dwelt, at a sharp corner or where two strokes overlap, accumulates many times the count of an ordinary cell and dominates the whole descriptor, so two glyphs would be judged similar or not chiefly by where their pen dwelt. Taking the square root compresses large counts toward small ones while keeping the ordering, so a cell with ink still counts for more than an empty one but not overwhelmingly more.

Method | Step four: blur

The grid is blurred with a small 3 by 3 kernel, weight 0.25 in the centre and 0.09375 at each of the eight neighbours. The problem it solves is that a grid is a hard partition: two writings of the same letter that differ by a hair can put corresponding ink in adjacent cells, which as vectors makes them share nothing at all in those positions. Blurring spreads each cell's value into its neighbours, so near-misses contribute to the similarity instead of being lost. This is the same reasoning that puts tolerance into any grid-based descriptor, and without it the measure would be far too brittle to be useful.

Method | Step five: normalise to unit length

Formula purpose and system use. Normalise the descriptor so comparisons focus on relative occupancy rather than total count. **Source basis:** [3, 60]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathbf{v} \;=\; \frac{\mathbf{g}}{\|\mathbf{g}\|}, \qquad \|\mathbf{g}\| \;=\; \sqrt{\textstyle\sum_k g_k^2} \tag{7.6}$$

where

$\mathbf{v}$
: the returned descriptor, of length 1

$\mathbf{g}$
: the blurred grid flattened into a 144-element vector

$\|\mathbf{g}\|$: its Euclidean length, the square root of the sum of the squares of its entries

Scaling to unit length removes the total amount of ink from the descriptor, leaving only how it is distributed. The decisive benefit is what it does to the comparison: for two unit vectors, the dot product is exactly the cosine of the angle between them, which is 1 for identical shapes and falls toward 0 as they diverge. So similarity becomes a single multiplication and sum, with a value on a fixed and interpretable scale. This is **cosine similarity**, and it is the standard way to compare descriptors precisely because normalisation makes magnitude irrelevant.

The resolution of 12 by 12 is a compromise. Coarser would stop distinguishing letters that differ in a small region, such as a and o, which agree everywhere but the stem. Finer would make the descriptor sensitive to exactly the variation between two writings of one letter that it is meant to tolerate, and the blur would have to widen to compensate, which gives back the resolution that was bought.

7.20 BuildLetterPrior

```
def BuildLetterPrior(storeByAuthor, minSamples=12)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 570. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.18.

Purpose | What this contributes

Pools descriptors across writers by character, requires a minimum sample count, takes a componentwise median and unit-normalises it. It supplies a rejection reference, not another writer glyph to be copied. Absent priors must not be interpreted as evidence of a perfect glyph.

Method | Current implementation

Reject malformed cuts and compare stored shapes without requiring matching vertex counts. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def BuildLetterPrior(storeByAuthor,minSamples=12):
    byChar = {}
    for store in storeByAuthor.values():
        for ch,vs in store.items():
            byChar.setdefault(ch,[]).extend(vs)
    prior = {}
    for ch,vs in byChar.items():
        grids = [g for g in (_ShapeGrid(v) for v in vs) if g is not None]
        if len(grids)<minSamples:
            continue
        G = np.vstack(grids)
        proto = np.median(G,axis=0)
        nrm = float(np.linalg.norm(proto))
        if nrm>1e-9:
            prior[ch] = proto/nrm
    return prior
```

Justification | Interpret the method rather than infer a benchmark

A sampled twelve-by-twelve occupancy grid, square-root compression, blur and unit norm provide a compact shift-tolerant representation. Width gates separately retain aspect information lost by box normalisation. [3, 60].

7.21 _PriorScore

```
def _PriorScore(g, ch, prior)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 588. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.18.

Purpose | What this contributes

Reject malformed cuts and compare stored shapes without requiring matching vertex counts.

Inputs

`g` a glyph dictionary

`ch` the character it claims to be

`prior`
the mapping of reference descriptors

Output. a distance from 0, identical to the reference, upward; 0 when no comparison can be made.

Method | What it measures

How far this glyph's shape sits from the cross-writer idea of its letter. It is the quantity stored on each glyph as `priorD`, and synthesis later uses it to judge how far a writer's own stored shape can be trusted.

Formula purpose and system use. Measure disagreement between a unit glyph descriptor and its pooled character reference. **Source basis:** [3, 60]. Exact project constants remain implementation choices unless evidence is stated.

$$D = 1 - \mathbf{v} \cdot \mathbf{p}_c \tag{7.7}$$

where

D the returned distance: 0 when the glyph matches the reference exactly, larger as it diverges

$\mathbf{v}$ the glyph's own descriptor, a unit vector

$\mathbf{p}_c$ the reference descriptor for that character, also a unit vector

$\cdot$ the dot product, which for unit vectors is the cosine of the angle between them

Subtracting the cosine similarity from 1 converts a similarity into a distance, so that small means good. This is the **cosine distance**. Since both descriptors have non-negative entries, their dot product cannot be negative, so the result lies between 0 and 1.

Justification | Why a missing reference scores zero rather than failing

When no reference exists for a character, or the glyph yields no descriptor, the score returned is 0, which is the value meaning a perfect match. That is deliberate: it makes the gate abstain. Every filter in this file can only reject, so the safe value in the absence of evidence is the one that rejects nothing. Returning a large value instead would quietly delete every glyph of every rare character, since those are exactly the characters without enough examples to have a reference.

7.22 PriorFilter

```
def PriorFilter(variants, ch, prior, keepFrac=0.7, minKeep=3)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 596. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.18.

Purpose | What this contributes

Reject malformed cuts and compare stored shapes without requiring matching vertex counts.

Inputs

`variants`
: this writer's glyphs for one character

`ch`
: that character

`prior`
: the mapping of reference descriptors

`keepFrac`
: the fraction to keep, default 0.7

`minKeep`
: the least number to keep, default 3

Output. the kept variants, sorted best first.

Method | What it does

Sorts this writer's variants of one character by their distance from the cross-writer reference and keeps the best 70 percent. The worst 30 percent are the ones least like the letter is supposed to look, which is where the mis-cuts that survived the geometric gates are concentrated.

Justification | Why a proportion rather than a threshold

A percentile gate ranks variants within a writer instead of imposing one absolute distance cutoff across all writers. This can tolerate a writer whose whole pool differs from the cross-writer reference, but also retains poor samples if every variant is poor. Tie handling, small-pool exceptions and earlier filters mean a stated retained fraction is not necessarily the fraction of all original glyphs kept.

Method | The floor, and the abstention

At least three variants are kept whatever the proportion says, and the filter does nothing at all when fewer than three exist or when no reference is available for the character. Three is the smallest number from which the consensus filter downstream can work and from which a prototype can be formed, so dropping below it costs more than the bad variant does. For a rare character, having three imperfect examples is better than having one perfect one, because synthesis needs something to choose among and a prototype needs several shapes to average.

> *The variants come back sorted best first, which is not incidental. Later stages take the first few, and the ranking established here is part of what determines which of a writer's shapes actually get drawn.*

Stage: Consensus medoids and prototypes

7.23 Method background: Consensus medoids and prototypes

Purpose | The role in this system

Choose representative writer glyphs while rejecting fragmented outliers.

Justification | The reason for this construction

A medoid minimises summed within-pool L1 descriptor distance and remains an actual sample. A denoised prototype combines compatible resampled strokes; it is a separate constructed object.

Alternatives | What changes if another method is used

A mean descriptor can be unrepresentative and supplies no pen path directly. Clustering handles multiple modes but needs a cluster rule. One arbitrarily chosen sample has no consensus guarantee.

Library | Implementation speed and accuracy

Pairwise-distance libraries can compute the same matrix; generic clustering is a different selection policy. Small pools limit the possible speed benefit. No library can fix a pool whose samples all contain bad cuts. [3, 62].

7.24 _ConsensusFilter

```
def _ConsensusFilter(variants, keepFrac=0.75)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 604. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.23.

Purpose | What this contributes

Choose representative writer glyphs while rejecting fragmented outliers.

Inputs

`variants`
: this writer's glyphs for one character

`keepFrac`
: the fraction to keep, default 0.75

Output. the kept variants, ordered by how typical they are.

Method | What it adds that the prior filter cannot

The prior filter compares a writer against everyone else, which catches a shape that is not really the letter. This one compares a writer against themselves, which catches a shape that is not really *their* letter. The two find different faults. A glyph carrying half of its neighbour may still resemble the general idea of the letter closely enough to pass the prior, while differing clearly from the writer's own twenty other examples, because those twenty agree with each other and it does not.

Method | Background: the medoid

The most typical member must be identified before the outliers can be found. A mean cannot serve, because the mean of several descriptors is not itself any real glyph and, being an average, is pulled by the outliers. Instead the **medoid** is used: the member of the set whose total distance to all the others is least. It is an actual glyph, and it is the multidimensional analogue of the median, with the same robustness.

Formula purpose and system use. Select the actual stored glyph minimising total within-pool L1 disagreement. **Source basis:** [3, 62]. Exact project constants remain implementation choices unless evidence is stated.

$$m \;=\; \arg\min_{i} \sum_{j} \left\| \mathbf{v}^{(i)} - \mathbf{v}^{(j)} \right\|_1, \qquad \left\| \mathbf{a} - \mathbf{b} \right\|_1 \;=\; \sum_{k} \left| a_k - b_k \right| \tag{7.8}$$

where

m the index of the medoid variant

$\mathbf{v}^{(i)}$ the descriptor of variant i

$\| \cdot \|_1$
: the **L1 distance**, or Manhattan distance: the sum of the absolute differences, as opposed to the square root of the sum of squares

k which descriptor position

$\arg\min_i$
: the variant for which that total is smallest

The L1 distance is used rather than the ordinary straight-line distance because squaring amplifies large differences, so a single badly disagreeing cell would dominate the comparison. Summing absolute differences treats every cell's disagreement in proportion to its size, which is the behaviour wanted when the question is overall typicality rather than worst-case mismatch. It is the same robustness argument that favours the median over the mean.

Method | The filter itself

With the medoid identified, the variants are ordered by distance from it and the furthest quarter is dropped. Ordering by distance from the most typical member means the kept variants are the ones that agree with the writer's own consensus, and the ordering also survives into the output, so later stages taking the first few get the most typical ones.

Justification | Why both filters are run, in this order

The prior filter runs first and the consensus filter second. The order matters because the consensus filter trusts the set it is given: it finds the most typical member of whatever it receives. If most of the variants were mis-cuts, the medoid would be a mis-cut and the filter would discard the correct letters as outliers. Running the prior filter first removes the glyphs that are not the letter at all, using an external reference that cannot be corrupted by this writer's cuts, so the consensus filter operates on a set in which the majority really is correct, which is the condition it needs.

> *The distance matrix is computed between every pair, which is quadratic in the number of variants. With at most a few dozen variants per character that is trivial, and the whole matrix is wanted anyway, since the medoid is defined by row sums of exactly that matrix.*

7.25 _ResampleN

```
def _ResampleN(poly, n)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 620. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.23.

Purpose | What this contributes

Choose representative writer glyphs while rejecting fragmented outliers.

Inputs

`poly`
: a polyline

`n`
: how many points to produce

Output. an array of n points evenly spaced along the path.

Purpose | What it is for

Averaging several writings of a letter requires adding corresponding points together, which requires that they correspond. Traced strokes have no correspondence at all: one may have 40 points and another 55, placed wherever the pixel grid and the simplification happened to put them. Resampling puts every stroke on the same footing, n points spaced equally along its length, so that the k-th point of one stroke and the k-th of another both sit at the same fraction of the way along.

Method | How it works: arc-length parameterisation

The cumulative distance along the path is computed, giving for each original point how far along the path it sits. That turns the path into a function of distance, and the new points are then read off at n equally spaced distances by linear interpolation between the original points.

Formula purpose and system use. Place a fixed number of samples along cumulative path length before comparing strokes. **Source basis:** [3, 62]. Exact project constants remain implementation choices unless evidence is stated.

$$s_i = \sum_{j<i} \|\mathbf{p}_{j+1} - \mathbf{p}_j\|, \qquad t_k = \frac{k}{n-1}\, s_{\text{last}}, \qquad k = 0, \ldots, n-1 \tag{7.9}$$

where

s_i
: the cumulative distance from the start of the path to its i-th point

$\mathbf{p}_j$
: the j-th original point

$\|\cdot\|$
: the straight-line distance between two consecutive points

t_k
: the distance at which the k-th new point is placed

s_{last}
: the total path length

n
: how many points are wanted

This is called **arc-length parameterisation**, and the alternative makes clear why it is necessary. Spacing the new points evenly by index rather than by distance would place them densely wherever the original happened to be dense, so two strokes resampled that way would still not correspond. Distance is the only parameterisation that both strokes share.

> *A path of zero length, which a dot recovery can produce, returns n copies of its single point. That is the correct degenerate answer and it keeps the array shape uniform, so the averaging that follows needs no special case.*

7.26 _AlignStroke

```
def _AlignStroke(s, N)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 632. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.23.

Purpose | What this contributes

Choose representative writer glyphs while rejecting fragmented outliers.

Inputs

`s` one stroke

`N` the number of points to resample to

Output. the stroke resampled to N points with a consistent starting point and direction, or nothing if it is too short.

Purpose | The problem beyond resampling

Resampling makes two strokes have the same number of points, but corresponding points still need to be at corresponding places, and two things can spoil that. A stroke may be traced in either direction, so one writing of a letter may run left to right and another right to left, in which case point 0 of one corresponds to point N−1 of the other. And a closed loop has no inherent starting point at all, so two tracings of the same o may begin anywhere on the circle.

Method | The two fixes

- **Open strokes are oriented left to right.** If the stroke ends left of where it began, it is reversed. Left to right is chosen because it is the direction of writing, so it is the orientation most strokes already have.
- **Closed strokes are rotated to begin at their leftmost point.** A stroke is recognised as closed when its two ends come within 0.3 x-heights of each other. The points are then rotated so the leftmost becomes first, and the original start is appended to close the loop. The leftmost point is used because it is well defined for any loop and is found identically in every tracing of it.

Both fixes work by choosing a canonical form: a rule that picks, from among the several equivalent ways of representing the same stroke, one particular way that depends only on the shape. Any such rule works provided every stroke obeys the same one, which is the real requirement.

> *Without these fixes the averaging would not merely be inaccurate; it would be nonsense. Averaging a stroke with a reversed copy of itself yields a shape collapsing toward its own midpoint, since the first point of one is averaged with the last of the other. The prototype would be a smear through the middle of the letter.*

7.27 _DenoisedPrototype

```
def _DenoisedPrototype(variants, minN=3, N=44)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 647. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.23.

Purpose | What this contributes

Averages several writings of a letter into one clean shape by taking the median point of the aligned, respaced variants.

Inputs

`variants`
: this writer's kept glyphs for one character

`minN`
: the least variants needed, default 3

`N`
: the number of points per stroke, default 44

Output. one clean letterform as a list of strokes, or nothing if no reliable prototype can be formed.

Purpose | What it is for

Every individual variant carries noise: the staircase left by the pixel grid, a boundary placed a little wrongly, a tremor in the hand. Those errors differ between variants while the letter does not, so averaging several writings cancels the noise and leaves the shape the writer meant. The result is the **prototype**, the cleanest available statement of how this writer forms this letter.

Method | Step one: group by stroke count

Only variants with the same number of strokes can be averaged, because the averaging matches strokes one to one. The variants are grouped by stroke count and the largest group is used. Taking the largest rather than any particular count lets the data decide how the writer usually forms the letter: if most of their a's are one stroke, the prototype is a one-stroke a.

A group with fewer than three members, or with more than five strokes, is refused outright. Three is the least from which an average means anything. The five-stroke limit guards against averaging glyphs whose stroke counts are high because the tracing fragmented them, where the stroke-to-stroke matching would be matching pieces that do not correspond.

Method | Step two: match strokes between variants

Within the group, each variant's strokes are sorted by their leftmost point and matched by position in that order, then aligned and resampled to a common 44 points. Sorting by position is a weak way to establish correspondence, but a sound one for this case: the strokes of one letter written by one person occur in a consistent left-to-right arrangement, so the leftmost stroke of one a is the same stroke as the leftmost of another.

Method | Step three: iterative alignment to the median

This is the step that distinguishes the method from a plain average. The variants are not yet aligned horizontally, because each was cut at a slightly different boundary, so one may sit a tenth of an x-height right of another. Averaging them in that state blurs the prototype horizontally. So each variant is shifted along x to whichever offset, among nine candidates spanning plus and minus 0.22 x-heights, brings it closest to the current median, and the median is then recomputed from the shifted variants. That is repeated twice.

Formula purpose and system use. Combine geometrically compatible resampled strokes into a separate prototype. **Source basis:** [3, 62]. Exact project constants remain implementation choices unless evidence is stated.

$$\delta_c^* = \arg\min_{\delta} \frac{1}{N} \sum_{k=1}^{N} \left\| (\mathbf{c}_k + \delta\,\hat{\mathbf{x}}) - \mathbf{m}_k \right\| \tag{7.10}$$

where

δ_c^* the best horizontal shift for variant c

$\mathbf{c}_k$ the k-th point of that variant's stroke

$\mathbf{m}_k$ the k-th point of the current median stroke

$\hat{\mathbf{x}}$ the unit vector along the horizontal axis, so the shift moves x only

δ a candidate shift, one of nine values from -0.22 to $+0.22$ x-heights

N the number of points per stroke, 44

Aligning variants to an estimate and then recomputing that estimate is iterative refinement. It can reduce misregistration, but this implementation does not establish a monotone decrease for every quality measure or a global optimum. Two rounds are the implementation budget, not a demonstrated sufficient number for every writer or variant pool.

Method | Step four: reject the worst fifth, then take the median

After alignment, each variant's mean deviation from the median is computed, and the worst 20 percent are dropped before the final median is taken. This is a second line of defence: a variant that survived every earlier gate but is still wrong will disagree with the aligned consensus, and here it is identified by that disagreement alone, with no reference to what kind of defect it has.

The median, not the mean, at every stage. Its robustness is exactly what is wanted: a prototype must not be bent toward a single bad variant, and with the median one outlier among several has no influence at all on the result rather than a proportional one.

> *Returning nothing is common and is not a failure. A character with few examples, or whose examples disagree about stroke count, has no reliable prototype, and inventing one from two variants would produce a shape less trustworthy than either. The caller falls back on the writer's actual variants, which is the right behaviour: a real if imperfect letterform beats a confidently wrong average.*

7.28 SmoothStroke

```
def SmoothStroke(pts, step=0.055, passes=2)
```

Evidence | Source and method link

`Software/DemoCode/style.py` line 703. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.23.

Purpose | What this contributes

Respaces a path evenly and runs a three-point average over it, taking the pixel staircase out.

Inputs

`pts` a stroke

`step`
: the spacing to resample to, in x-heights, default 0.055

`passes`
: how many smoothing passes, default 2

Output. the stroke resampled evenly and smoothed, as rounded coordinates.

Purpose | What it is for

A traced stroke still carries the pixel staircase: it was assembled from whole-pixel positions, so it zigzags at the scale of one pixel even where the writer's stroke was smooth. Drawn by a plotter that staircase is visible, and it also misrepresents the writing, since the writer drew a curve. This function removes it.

Method | Step one: resample evenly

The stroke is first resampled to points about 0.055 x-heights apart. Even spacing is required before smoothing, because the smoothing averages each point with its neighbours by index, and if the points are unevenly spaced then that averaging reaches different distances in different places, smoothing the sparse parts far more than the dense ones.

Method | Step two: the smoothing pass

Formula purpose and system use. Reduce local path noise through a weighted neighbouring-point average. **Source basis:** [3, 62]. Exact project constants remain implementation choices unless evidence is stated.

$$q'_k \;=\; 0.25\,q_{k-1} \;+\; 0.5\,q_k \;+\; 0.25\,q_{k+1} \tag{7.11}$$

where

q'_k the new position of the k-th point

q_k its current position

q_{k-1}, q_{k+1}
: its neighbours along the stroke

0.25, 0.5, 0.25
: the weights, which sum to 1 and preserve a constant coordinate signal; curved geometry can still shrink

This is convolution along the sampled path with a three-point kernel. Unit-sum weights preserve constant coordinate sequences and make the operation equivariant to translating the whole path. They do not preserve every curve's size: averaging can round corners and shrink curved structures even when the weights sum to one.

Two passes are applied. Away from endpoint handling, repeated convolution is equivalent to convolution with the combined kernel. Additional passes smooth more strongly and can also remove genuine small features. The exact pass count is not supported here by an independent optimum or a universal curvature-preservation bound.

Justification | Why the endpoints are held fixed

Only the interior points are smoothed; the first and last are kept exactly. The endpoints carry meaning that the interior does not: they are where the pen entered and left, they are what a join is drawn to, and they define the glyph's extent. Smoothing them would pull them inward, shortening every stroke slightly with each pass and, worse, moving the attachment points that synthesis relies on.

Coordinates are rounded to three decimal places on output. At a scale where 1 is the x-height, three decimals is a thousandth of a letter's height, far finer than any plotter can draw or any descriptor can distinguish. The rounding roughly halves the size of the stored profile, which runs to thousands of strokes, at no cost in fidelity.

8 Glyph curation

Evidence | Current implementation boundary

`Software/DemoCode/curate.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Inspect the stored profile and its variants, choose or remove entries, then persist the curated profile. This is profile preparation rather than inference-time model training.

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
HERE = Path(__file__).resolve().parent
OUT = HERE / 'results' / 'glyph_curation'
```

Stage: Profile curation

8.1 Method background: Profile curation

Purpose | The role in this system

Remove selected bad glyphs and rebuild representatives without training a new classifier.

Justification | The reason for this construction

Explicit variant indices and a saved curated profile make manual quality selection inspectable. A classifier label remains the base writer, not a new class for the suffix.

Alternatives | What changes if another method is used

Automatic prior/consensus filtering scales better but can retain systematic bad cuts. Human selection adds judgement and should not be hidden when interpreting results.

Library | Implementation speed and accuracy

Pillow draws inspection sheets; geometry and pool policy remain project-specific. A library UI or clustering tool changes workflow rather than proving better style. [3, 62, 63].

8.2 ParseSpec

```
def ParseSpec(spec)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 42. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.1.

Purpose | What this contributes

Remove selected bad glyphs and rebuild representatives without training a new classifier.

Method | Current implementation

Remove selected bad glyphs and rebuild representatives without training a new classifier. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ParseSpec(spec):
    out = {}
    for part in (spec or "").split():
        if ":" not in part:
            raise SystemExit("expected char:indices, got %r" % part)
        ch, idx = part.split(":", 1)
        if len(ch) != 1:
            raise SystemExit("one character per entry, got %r" % ch)
        out.setdefault(ch, set()).update(int(i) for i in idx.split(",") if i != "")
    return out
```

Justification | Interpret the method rather than infer a benchmark

Explicit variant indices and a saved curated profile make manual quality selection inspectable. A classifier label remains the base writer, not a new class for the suffix. [3, 62, 63].

8.3 LoadProfile

```
def LoadProfile(writer, curated=False)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 55. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.1.

Purpose | What this contributes

Remove selected bad glyphs and rebuild representatives without training a new classifier.

Method | Current implementation

Remove selected bad glyphs and rebuild representatives without training a new classifier. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def LoadProfile(writer, curated=False):
    name = "%s_curated.json" % writer if curated else "%s.json" % writer
    path = paths.Profiles/name
    if not path.exists():
        raise SystemExit("no profile at %s" % path)
    return json.load(open(path, encoding="utf-8")), path
```

Justification | Interpret the method rather than infer a benchmark

Explicit variant indices and a saved curated profile make manual quality selection inspectable. A classifier label remains the base writer, not a new class for the suffix. [3, 62, 63].

8.4 Listing

```
def Listing(writer, curated=False)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 132. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.1.

Purpose | What this contributes

Remove selected bad glyphs and rebuild representatives without training a new classifier.

Method | Current implementation

Remove selected bad glyphs and rebuild representatives without training a new classifier. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Listing(writer, curated=False):
    prof, path = LoadProfile(writer, curated)
    glyphs = prof.get("glyphs") or {}
    print("%s, from %s: %d characters, %d variants in total"
          % (writer, path.name, len(glyphs), sum(len(v) for v in glyphs.values())))
    for ch in sorted(glyphs):
        vs = glyphs[ch]
        pd = [float(g.get("priorD", 0.5)) for g in vs]
        proto = sum(1 for g in vs if g.get("denoised"))
        print("  %-4r %3d variants   prior distance min %.2f median %.2f max %.2f%s"
              % (ch, len(vs), min(pd), float(np.median(pd)), max(pd),
                 "   (%d prototype)" % proto if proto else ""))
```

Justification | Interpret the method rather than infer a benchmark

Explicit variant indices and a saved curated profile make manual quality selection inspectable. A classifier label remains the base writer, not a new class for the suffix. [3, 62, 63].

8.5 Variants

```
def Variants(writer, curated=False)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 198. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.1.

Purpose | What this contributes

The whole set in a form something else can draw the command line draws its own sheets, but the curation page in the web app needs the strokes themselves so it can draw them and be clicked on. the indices here are the ones Apply takes, so what is seen and what is removed cannot drift apart.

Method | Current implementation

Remove selected bad glyphs and rebuild representatives without training a new classifier. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Variants(writer, curated=False):
    prof, path = LoadProfile(writer, curated)
    glyphs = prof.get("glyphs") or {}
    chars = []
    for ch in sorted(glyphs):
        vs = []
        for i, g in enumerate(glyphs[ch]):
            vs.append(dict(index=i,
                           priorD=round(float(g.get("priorD", 0.5)), 4),
                           advance=round(float(g.get("advance", 0.9)), 3),
                           denoised=bool(g.get("denoised")),
                           strokes=[[[round(float(p[0]), 4), round(float(p[1]), 4)]
                                     for p in s] for s in (g.get("strokes") or [])]))
        chars.append(dict(ch=ch, variants=vs))
    return dict(writer=writer, source=path.name, characters=chars,
                nChars=len(chars), nVariants=sum(len(c["variants"]) for c in chars),
                curatedFrom=prof.get("curatedFrom"),
                hasCurated=(paths.Profiles/("%s_curated.json" % writer)).exists())
```

Justification | Interpret the method rather than infer a benchmark

Explicit variant indices and a saved curated profile make manual quality selection inspectable. A classifier label remains the base writer, not a new class for the suffix. [3, 62, 63].

8.6 Drop

```
def Drop(writer, dropSpec=None, keepSpec=None, curated=False, outName=None)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 219. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.1.

Purpose | What this contributes

Remove selected bad glyphs and rebuild representatives without training a new classifier.

Method | Current implementation

Remove selected bad glyphs and rebuild representatives without training a new classifier. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Drop(writer, dropSpec=None, keepSpec=None, curated=False, outName=None):
    return Apply(writer, ParseSpec(dropSpec), ParseSpec(keepSpec), curated, outName)
```

Justification | Interpret the method rather than infer a benchmark

Explicit variant indices and a saved curated profile make manual quality selection inspectable. A classifier label remains the base writer, not a new class for the suffix. [3, 62, 63].

8.7 Apply

```
def Apply(writer, drop=None, keep=None, curated=False, outName=None)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 224. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.1.

Purpose | What this contributes

Remove selected bad glyphs and rebuild representatives without training a new classifier.

Method | Current implementation

Remove selected bad glyphs and rebuild representatives without training a new classifier. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Apply(writer, drop=None, keep=None, curated=False, outName=None):
    prof, path = LoadProfile(writer, curated)
    glyphs = dict(prof.get("glyphs") or {})
    drop = {k: set(v) for k, v in (drop or {}).items() if v}
    keep = {k: set(v) for k, v in (keep or {}).items()}
    for ch in list(drop)+list(keep):
        if ch not in glyphs:
            raise SystemExit("%r has no variants in %s" % (ch, path.name))

    removed, emptied = 0, []
    for ch, vs in list(glyphs.items()):
        if ch in keep:
            chosen = [g for i, g in enumerate(vs) if i in keep[ch]]
        elif ch in drop:
            chosen = [g for i, g in enumerate(vs) if i not in drop[ch]]
        else:
            continue
        removed += len(vs)-len(chosen)
        if not chosen:
            emptied.append(ch)
            del glyphs[ch]
        else:
            glyphs[ch] = chosen

    prof["glyphs"] = glyphs
    ideal = dict(prof.get("idealGlyphs") or {})
    medoid = dict(prof.get("idealMedoid") or {})
    adv = dict(prof.get("letterAdvance") or {})
    touched = set(drop) | set(keep)
    for ch in touched:
        if ch not in glyphs:
            ideal.pop(ch, None)
            medoid.pop(ch, None)
            adv.pop(ch, None)
            continue
        vs = glyphs[ch]
        p = Prototype(ch, vs)
        m = Medoid(ch, vs)
        if p is None:
            ideal.pop(ch, None)
        else:
            ideal[ch] = p
        if m is None:
            medoid.pop(ch, None)
        else:
            medoid[ch] = m
        adv[ch] = round(float(np.median([g["advance"] for g in vs])), 3)
    prof["idealGlyphs"] = ideal
    prof["idealMedoid"] = medoid
    prof["letterAdvance"] = adv
```

```
    prof["curatedFrom"] = path.name
    prof["curatedRemoved"] = removed
    prof["curatedNote"] = ("glyphs, idealGlyphs, idealMedoid and letterAdvance "
                           "were rebuilt from the kept variants; connectedness "
                           "is carried over unchanged because the adjacency it "
                           "was counted from is not stored")

    out = paths.Profiles/(outName or ("%s_curated.json" % writer))
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Explicit variant indices and a saved curated profile make manual quality selection inspectable. A classifier label remains the base writer, not a new class for the suffix. [3, 62, 63].

8.8 Main

```
def Main()
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 303. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.1.

Purpose | What this contributes

Remove selected bad glyphs and rebuild representatives without training a new classifier.

Method | Current implementation

Remove selected bad glyphs and rebuild representatives without training a new classifier. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Main():
    ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    ap.add_argument("action", choices=["sheet", "list", "drop"])
    ap.add_argument("writer")
    ap.add_argument("--drop", default=None, help='e.g. "a:2,5 e:0"')
    ap.add_argument("--keep", default=None, help='e.g. "a:0,1" keeps only those')
    ap.add_argument("--curated", action="store_true",
                    help="work from <writer>_curated.json instead of the original")
    ap.add_argument("--out", default=None, help="name for the written profile")
    a = ap.parse_args()
    if a.action == "sheet":
        Sheet(a.writer, a.curated)
    elif a.action == "list":
        Listing(a.writer, a.curated)
    else:
        if not (a.drop or a.keep):
            raise SystemExit("give --drop or --keep")
        Drop(a.writer, a.drop, a.keep, a.curated, a.out)
```

Justification | Interpret the method rather than infer a benchmark

Explicit variant indices and a saved curated profile make manual quality selection inspectable. A classifier label remains the base writer, not a new class for the suffix. [3, 62, 63].

Stage: Consensus medoids and prototypes

Method rationale, alternatives and library comparison: Section 7.23.

8.9 Medoid

```
def Medoid(ch, vs)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 153. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.23.

Purpose | What this contributes

REBUILDING WHAT THE SET DECIDES the library is not the only thing that comes from the variants: the prototype letterform, the most typical real variant and the letter advances are all derived from them, so all three are recomputed rather than left describing glyphs that are no longer there.

Method | Current implementation

Choose representative writer glyphs while rejecting fragmented outliers. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Medoid(ch, vs):
    grids = [(ST._ShapeGrid(g), g) for g in vs]
    grids = [(v, g) for v, g in grids if v is not None]
    if len(grids) < 2:
        return None
    G = np.stack([v for v, _ in grids])
    D = np.abs(G[:, None, :]-G[None, :, :]).sum(-1)
    med = grids[int(np.argmin(D.sum(1)))][1]
    return dict(strokes=[ST.SmoothStroke(s) for s in med["strokes"]],
                priorD=round(float(med.get("priorD", 0.5)), 4),
                advance=round(float(med.get("advance", 0.9)), 3),
                lead=round(float(med.get("lead", 0.0)), 3),
                width=round(float(med.get("width", 0.9)), 3),
                top=round(float(med.get("top", 1.0)), 3),
                bot=round(float(med.get("bot", 0.0)), 3),
                entryY=round(float(med.get("entryY", 0.3)), 3),
                exitY=round(float(med.get("exitY", 0.3)), 3),
                nPool=len(vs))
```

Justification | Interpret the method rather than infer a benchmark

A medoid minimises summed within-pool L1 descriptor distance and remains an actual sample. A denoised prototype combines compatible resampled strokes; it is a separate constructed object. [3, 62].

8.10 Prototype

```
def Prototype(ch, vs)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 173. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 7.23.

Purpose | What this contributes

Choose representative writer glyphs while rejecting fragmented outliers.

Method | Current implementation

Choose representative writer glyphs while rejecting fragmented outliers. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Prototype(ch, vs):
    proto = ST._DenoisedPrototype(vs)
    if proto is None:
        return None
    pts = [p for s in proto for p in s]
    xs = [p[0] for p in pts]
    ys = [p[1] for p in pts]
    return dict(strokes=[ST.SmoothStroke(s) for s in proto],
                priorD=round(float(np.median([g.get("priorD", 0.5) for g in vs])), 4),
                advance=round(float(np.median([g["advance"] for g in vs])), 3),
                lead=round(float(np.median([g.get("lead", 0.0) for g in vs])), 3),
                width=round(max(xs)-min(xs), 3),
                top=round(max(ys), 3), bot=round(min(ys), 3),
                entryY=round(float(np.median([g.get("entryY", 0.3) for g in vs])), 3),
                exitY=round(float(np.median([g.get("exitY", 0.3) for g in vs])), 3),
                nPool=len(vs))
```

Justification | Interpret the method rather than infer a benchmark

A medoid minimises summed within-pool L1 descriptor distance and remains an actual sample. A denoised prototype combines compatible resampled strokes; it is a separate constructed object. [3, 62].

Stage: Trajectory rasterisation

8.11 Method background: Trajectory rasterisation

Purpose | The role in this system

Make an image of a generated path for preview and recognition-based scoring.

Justification | The reason for this construction

Sampling straight segments and applying a controlled width makes candidate rendering explicit. The uniform width models one drawing pen rather than source ink intensity.

Alternatives | What changes if another method is used

Vector display keeps exact paths for viewing but classifiers need rasters. Antialiased line drawing and supersampling change edge pixels and cost. Rendering width is not physical pressure.

Library | Implementation speed and accuracy

Pillow ImageDraw or OpenCV polylines can rasterise paths. Coverage rules, antialiasing, round caps and blur alter classifier inputs. Pixel similarity and performance should be compared separately. [63, 64, 3].

8.12 DrawGlyph

```
def DrawGlyph(g, cell=150, pad=14)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 67. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.11.

Purpose | What this contributes

Make an image of a generated path for preview and recognition-based scoring.

Method | Current implementation

Make an image of a generated path for preview and recognition-based scoring. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def DrawGlyph(g, cell=150, pad=14):
    from PIL import Image, ImageDraw
    im = Image.new("L", (cell, cell), 255)
    d = ImageDraw.Draw(im)
    strokes = g.get("strokes") or []
    pts = [p for s in strokes for p in s]
    if not pts:
        return im
    xs = [p[0] for p in pts]
    ys = [p[1] for p in pts]
    w = max(1e-6, max(xs)-min(xs))
    h = max(1e-6, max(ys)-min(ys))
    k = (cell-2*pad)/max(w, h)
    ox = (cell-w*k)/2.0
    oy = (cell-h*k)/2.0
    for s in strokes:
        q = [(ox+(p[0]-min(xs))*k, cell-(oy+(p[1]-min(ys))*k)) for p in s]
        if len(q) > 1:
            d.line(q, fill=0, width=2)
    return im
```

Justification | Interpret the method rather than infer a benchmark

Sampling straight segments and applying a controlled width makes candidate rendering explicit. The uniform width models one drawing pen rather than source ink intensity. [63, 64, 3].

8.13 Sheet

```
def Sheet(writer, curated=False, cell=150)
```

Evidence | Source and method link

`Software/DemoCode/curate.py` line 95. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.11.

Purpose | What this contributes

One image per character, every variant numbered the number under a variant is its index, which is what drop and keep take. a variant marked proto is the denoised prototype the builder prepended rather than one the writer actually wrote.

Method | Current implementation

Make an image of a generated path for preview and recognition-based scoring. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Sheet(writer, curated=False, cell=150):
    from PIL import Image, ImageDraw
    prof, path = LoadProfile(writer, curated)
    glyphs = prof.get("glyphs") or {}
    outDir = OUT/writer
    outDir.mkdir(parents=True, exist_ok=True)
    made = []
    for ch in sorted(glyphs):
        vs = glyphs[ch]
        cols = min(10, max(1, len(vs)))
        rows = (len(vs)+cols-1)//cols
        im = Image.new("L", (cols*cell, rows*(cell+22)+26), 255)
        d = ImageDraw.Draw(im)
        d.text((6, 6), "%s  %r  %d variants" % (writer, ch, len(vs)), fill=0)
        for i, g in enumerate(vs):
            cx, cy = (i % cols)*cell, 26+(i//cols)*(cell+22)
            im.paste(DrawGlyph(g, cell), (cx, cy))
            tag = "%d%s" % (i, "  proto" if g.get("denoised") else "")
            d.text((cx+4, cy+cell+4), tag, fill=0)
            d.text((cx+4, cy+cell+12), "pd %.2f" % float(g.get("priorD", 0.5)), fill=0)
        safe = "".join(c if c.isalnum() else "_%02x" % ord(c) for c in ch)
        f = outDir/("%s.png" % safe)
        im.save(f)
        made.append((ch, len(vs), f.name))
    index = outDir/"INDEX.txt"
    with open(index, "w", encoding="utf-8") as fh:
        fh.write("%s, from %s\n\n" % (writer, path.name))
        fh.write("the number under a variant is its index, for --drop and --keep\n")
        fh.write("pd is the prior distance: lower means closer to the usual shape\n\n")
        for ch, n, name in made:
            fh.write("%-4r %3d variants  %s\n" % (ch, n, name))
    print("wrote %d character sheets to %s" % (len(made), outDir))
    print("  index: %s" % index)
    return made
```

Justification | Interpret the method rather than infer a benchmark

Sampling straight segments and applying a controlled width makes candidate rendering explicit. The uniform width models one drawing pen rather than source ink intensity. [63, 64, 3].

9 Handwriting synthesis and candidate selection

Evidence | Current implementation boundary

`Software/DemoCode/synthesise.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Read a writer profile and text; choose eligible source glyphs and calibration; lay out words and joins; render candidates; use text/writer judges and accepted local repair to select a trajectory. The text and writer judges score a candidate rather than train during the request.

9.1 Expanded reasoning: placement, joins, anchors and candidate search

9.1.1 From stored glyph coordinates to new x, y positions

For glyph point (u, v), lead l, shear s, baseline drift d, origin (p_x, p_y) and physical x-height H in millimetres,

Formula purpose and system use. Purpose in this explanation: From stored glyph coordinates to new x,y positions **Source basis:** [3]. Exact project constants remain implementation choices unless evidence is stated.

$$x = p_x + H(l + u + sv), \qquad y = p_y + H(v + d).$$

Only p_x, p_y, H carry millimetres; other quantities are dimensionless. If $H = 4, p_x = 10, p_y = 20, l = 0.1, u = 0.5, v = 1, s = 0.2, d = 0.05$, the placed point is $(13.2, 24.2)$ mm. Setting shear to zero removes slant but does not remove writer-specific stroke shape. Setting drift to zero straightens the baseline but does not erase advance/word-gap variation.

Character advance moves the placement cursor; it is not necessarily the glyph's bounding width. Overhanging strokes may extend beyond the advance. Word gap is a separate measurement. Before placing a word, estimate its advance sum and compare with remaining line width. If it does not fit, wrap and start a new baseline. A word wider than a whole line requires a defined overflow policy; wrapping alone cannot make it fit. Image-down coordinates and physical-up glyph coordinates require an explicit vertical transformation later, rather than silently changing this equation.

The reported line step is $(a_{\text{asc}} - a_{\text{desc}} + 0.85)H$, where a descender extent is signed negative. For ascender 1.7, descender -0.6, $H = 4$, the pitch is $3.15(4) = 12.6$ mm. The added 0.85 is line clearance in x-height units, not the drift persistence that happens to share the same number. Source comments give later ruling-specific refinements; they should not be retroactively read into every reported physical run.

9.1.2 Random variation, slow drift and collision guards

Small random translation/rotation/scale perturbations reduce mechanical repetition. Their amplitude must remain small enough not to change character identity. A fixed pseudorandom seed reproduces the same choices under the same algorithm and iteration order; it does not make them naturally human.

The inspected drift update is conceptually $v_t = 0.85v_{t-1}+\eta_t$ followed by $d_t = \mathrm{clip}(d_{t-1}+v_t, -0.25, 0.25)$. Thus persistence acts on drift *velocity*, not simply on baseline position. If noise stopped, velocity retains 0.85^k after k glyphs; its half-life is $\log(0.5)/\log(0.85) \simeq 4.27$ glyphs. Position is an integrated, clipped process and does not inherit a simple stationary AR(1) variance formula. The ± 0.25 limit means ± 1 mm at $H = 4$ mm. Larger persistence produces longer correlated runs; larger bounds allow more baseline wander. The precise values are project heuristics.

A $0.05H$ clearance is 0.2 mm at that size. It limits unwanted collisions between separate glyph extents; legitimate cursive attachment is a different case. Bounds-based clearance can be conservative because two overlapping boxes need not contain overlapping strokes. Removing the guard may let letters collide; increasing it can create unnatural loose spacing. This is an explainable trade-off, not a demonstrated perceptual optimum.

9.1.3 Automatic anchoring: piecewise formula and provenance

Let d be median stored prior distance, where low means agreement with the character reference. The reported automatic anchoring strength is

Formula purpose and system use. Purpose in this explanation: Automatic anchoring: piecewise formula and provenance **Source basis:** [3]. Exact project constants remain implementation choices unless evidence is stated.

$$\lambda = \mathrm{clip}\left(\frac{d - 0.24}{0.32 - 0.24}, 0, 1\right).$$

It equals zero for $d \leq 0.24$, rises linearly through 0.5 at $d = 0.28$, and saturates at one for $d \geq 0.32$. The interior slope is 12.5, so a 0.01 distance change gives 0.125 in λ before clipping. These endpoints encode a sensitive quality transition; they are project settings, not values derived from the cosine-distance definition.

Anchoring strength controls fallback selection policy. It is not necessarily the interpolation weight between corresponding vertices of two strokes. A writer's medoid, a separately stored prototype, a case-related glyph and a generic anchor have different provenance. A generic anchor with the writer's slant applied does not become an observed sample of that writer. If the pooled prior is missing and the score routine returns a guarded zero, that is an absence of evidence, not proof of a perfect glyph. Alternative policies could use per-character confidence or a learned quality predictor, requiring labelled quality examples and validation.

9.1.4 Connectedness suppression, source compatibility and gap limits

Measured connectedness can include accidental touching or bad character cuts. The report suppresses automatic joins below 0.45. Unlike most exact constants, this one has a documented *intent* in source comments: example measured values include Dylan 0.195, Owen 0.319 and Yeukita 0.417, described as mostly print, versus writer 384 at 0.500 and 551 at 0.557, described as joined. The chosen 0.45 lies between the two groups [3]. Those comments supply a local rationale, not a newly verified dataset study or proof that the threshold transfers to new writers. Rebuilding profiles with different extraction can change the measurement even if the nominal formula is unchanged.

A gap at most 0.80 x-heights limits a connector to a local relation. With $H = 4$ mm the limit is 3.2 mm. Joining additionally checks compatible sources, eligible body strokes and endpoint geometry, so passing distance alone is insufficient. Dots and crossbars should not become entry/exit anchors. Height-compatible variants can be weighted more strongly without forcing every pair to join. A shallow connector preserves readable body connections; long straight spans can introduce false strokes or cross other glyphs. The 0.80 limit is a geometric heuristic, not a physiological maximum.

These synthesis joins differ from the G-code writer's 0.6 mm stroke join. The former encodes handwriting structure in x-height units; the latter avoids a physical pen toggle between sufficiently close path endpoints. The second can add a connector even after the first stage has finished. Confusing them hides where a visible extra mark originated.

9.1.5 Character score A, probability P and joint score J

For intended text t and recognised text $\hat{t}$, $A = \max(0, 1 - d_{\text{edit}}(\hat{t}, t)/|t|)$ for nonempty t. P is the requested writer's softmax probability from the chosen classifier. The reported combination is

Formula purpose and system use. Purpose in this explanation: Character score A, probability P and joint score J **Source basis:** [3]. Exact project constants remain implementation choices unless evidence is stated.

$$J = \begin{cases} 2AP/(A+P), & A+P > 0, \\ 0, & A+P = 0. \end{cases}$$

This is the harmonic mean. For positive A, P, $1/J = \frac{1}{2}(1/A + 1/P)$. The smaller quantity therefore has stronger influence than under an arithmetic mean. With $A = 1, P = 0.1$, $J \simeq 0.182$ while the arithmetic mean is 0.55. A zero in either input gives zero. This is a project objective joining two differently interpreted scores, not a likelihood or a percentage of physically correct handwriting.

For the report's example $A = 18/19 \simeq 0.9474, P = 0.58$, $J \simeq 0.7195$. The displayed 0.72 is rounding. Derivatives $\partial J/\partial A = 2P^2/(A+P)^2$ and $\partial J/\partial P = 2A^2/(A+P)^2$ show that when A is large and P small, improving P matters more. This algebra explains the preference; it does not establish that raw writer probabilities are calibrated or commensurate with edit accuracy.

Alternatives include a product, a weighted mean, minimum score, or maximising one subject to a threshold on the other. Each encodes different priorities. A product penalises two middling values more strongly; a weighted mean can hide failure in one term; a hard constraint has a discontinuity. The harmonic choice is defensible as a balanced preference, while its exact practical benefit remains an empirical question.

9.1.6 Full candidate and repair algorithm, with limitations

Generate at least one candidate, using seed $s+977k$ for trial k; render at controlled ink width; recognise its text; obtain writer probability; compute J; retain the strictly better complete candidate. Stop at the trial budget or the configured near-perfect score. The integer 977 spaces seeds deterministically. Being prime does not prove statistically independent draws. The direct joint function defaults use six tries, three repair rounds and two new weak positions per round, with early thresholds 0.99 and 0.995. These are search-cost limits and stopping preferences, not character-accuracy targets guaranteed on paper.

Repair aligns the known text to find weak positions, strengthens selected anchors, regenerates a complete trajectory and rescores it. Commit the new anchor assignments only if the complete joint score improves. The result's best score is then nondecreasing for deterministic scoring, but there is no global-optimum guarantee: regenerated geometry, wrapping and writer cues interact. A failed local change does not prove that character cannot be improved by another choice. The removed older legibility-only routine had different defaults and acceptance bookkeeping; it must not be described as the joint algorithm merely because names are similar.

The same reader/classifier selecting candidates cannot provide an independent post-selection validation. More trials can exploit model preferences and raise J without improving human legibility or physical reproduction. The reported diagnostic candidate plots should be read as optimisation examples. Photos, held-out read-back or separately specified geometric measurements answer different

questions. No candidate search, rendering inference or physical reproduction was run in preparing these explanations.

Current function reference

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
_FB_ADV = 1.0
_SIMILAR = {'I': 'l', 'O': '0', 'l': 'I', '0': 'O', 'o': '0', ';': ':', '"': "'", '!': 'l'}
UNIFORM_PEN_WIDTH_XH = 0.085
UNIFORM_SOFT_FRAC = 0.3
GAP_HI_PCT = 90
MAX_LETTER_GAP_XH = 0.3
MIN_LETTER_GAP_XH = 0.03
MAX_JOIN_GAP_XH = 0.8
JOIN_FLAT_MAX_ASPECT = 0.35
JOIN_VERT_MIN_XSPAN_XH = 0.25
CORE_SLANT_CAP_DEG = 22.0
CORE_JITTER_FRAC = 0.15
COLLISION_MIN_GAP_XH = 0.05
_ELIGIBLE_VARIANTS = 5
CORE_CURSIVE_CONN = 0.5
JOIN_PRINT_CONN_FLOOR = 0.45
IDEAL_ANCHOR_MAX_PRIORD = 0.3
_GLYPH_CLEAN = 0.12
_GLYPH_GARBAGE = 0.55
_COMMIT_LAM = 0.85
_AUTO_LAM_GOOD = 0.24
_AUTO_LAM_BAD = 0.32
_STYLE_CAL = {}
```

Stage: Glyph placement and candidate search

Report equation 29 New glyph placement

Method | Formula

$x = p_x + H(l + u + sv)$ and $y = p_y + H(v + d)$.

Purpose | Why the system needs it

Restore writer shear, physical size, lead and baseline drift at the new text location. H and origins have millimetre units; stored glyph coordinates are dimensionless. Additional jitter/collision handling changes placement.

Evidence | Current location and source

SynthesizeText. [3, 25].

Report equation 30 Automatic anchoring strength

Method | Formula

$\lambda = \text{clip}[(d - 0.24)/(0.32 - 0.24), 0, 1]$.

Purpose | Why the system needs it

Increase fallback preference when median stored prior distance is poor. The endpoints are project settings, not universal perceptual thresholds. A stored explicit legibilityLambda can override automatic calculation.

Evidence | Current location and source

AutoLegibility, _SwapProb and SynthesizeJointBestOf. [3].

Report equation 31 Candidate objective

Method | Formula

$A = \max(0, 1 - d_{\text{edit}}/|t|)$ and $J = 2AP/(A + P)$ for $A + P > 0$.

Purpose | Why the system needs it

Prefer candidates with both text recovery and requested-writer probability. The harmonic mean penalises a weak component, but P is a model posterior and A is an edit score, so J is not a joint likelihood or independent physical style judgement.

Evidence | Current location and source

SynthesizeJointBestOf and CharAcc. [3, 55].

Alternatives | Recurrent stroke generation GANwriting and One DM

Graves's recurrent synthesis predicts successive online movements; the project has offline images with no observed stroke order. GANwriting conditions raster generation on content and writer style; its image still needs a pen-path conversion for this robot. One-DM iteratively denoises an image and can use one style reference at generation, but it was trained on a larger corpus. A small reference requirement is not a small training-data requirement. The current glyph library avoids training a separate generator and exposes controllable geometry, while finite variants approximate contextual cursive poorly. These are report-grounded trade-offs, not new head-to-head measurements [65, 66, 67, 62].

9.2 Method background: Glyph placement and candidate search

Purpose | The role in this system

Generate new text in a stored writer style and balance readability with model-based writer identity.

Justification | The reason for this construction

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation.

Alternatives | What changes if another method is used

Recurrent stroke generators need online trajectories; GANwriting and One-DM produce raster images and add generative training plus path recovery. They may model context better but are not drop-in arithmetic replacements.

Library | Implementation speed and accuracy

PyTorch can implement neural generators or accelerate judging networks; it does not supply this project glyph library and joining policy as one operation. No measured synthesis quality or time advantage is claimed for an untested replacement. [62, 65, 66, 67].

9.3 _BaselineGlyph

```
def _BaselineGlyph(ch, profile, printOnly=False)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 182. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Fetches a fallback letter, stretched to this writer's own reach above and below the line.

Inputs

`ch` the character wanted

`profile`
: the writer's style profile

`printOnly`
: force the printed alphabet even for a cursive hand

Output. a pair of the strokes and the advance, scaled to this writer's reach; or nothing if the character is in neither alphabet.

Purpose | What it is for

Fetches a built-in letterform and adapts it to the writer. Without the adaptation a substituted letter would be visibly foreign: a generic l reaching 1.75 x-heights dropped into a hand whose own ascenders reach 2.4 looks short, and in a hand that reaches only 1.4 it towers over its neighbours. The letterform itself is generic; its proportions should not be.

Justification | Why only the zones outside the band are rescaled

Formula purpose and system use. Set a fallback advance from its geometric width plus designed clearance. **Source basis:** [62, 65, 66, 67]. Exact project constants remain implementation choices unless evidence is stated.

$$y' = \begin{cases} 1 + (y-1)\,\max\!\left(0.35,\ \frac{\text{asc}-1}{0.75}\right) & y > 1 \\ y \cdot \max\!\left(0.35,\ \frac{|\text{desc}|}{0.55}\right) & y < 0 \\ y & 0 \le y \le 1 \end{cases} \tag{9.1}$$

where

y the built-in letterform's height at this point, in x-heights

y' the adapted height

asc this writer's measured ascender reach, in x-heights above the baseline

desc their measured descender reach, negative

0.75 the ascender reach above the band that the built-in forms were drawn with, so dividing by it gives a ratio of 1 for a writer matching them

0.55 the same for the descender zone

$\max(0.35, \cdot)$
a floor, so a writer with very little measured reach does not collapse the zone to nothing

The x-height band itself, from 0 to 1, is left untouched, and that is the point of the piecewise form. The band is the reference against which everything is measured, so it is 1 by definition and rescaling it would be meaningless. What varies between writers is how far they reach *beyond* the band, and those are exactly the two zones the formula adjusts. A letter like an o, entirely inside the band, is therefore unchanged, while an l is stretched or compressed only above $y = 1$, leaving its body the same size as its neighbours'.

Justification | Why there is a floor of 0.35

A writer whose measured ascender is close to 1.0, which happens when the measurement failed or when most of their ascender letters were rejected, would otherwise give a ratio near zero and flatten every ascender to the height of the band. An l indistinguishable from an e is worse than a slightly wrong l, so the floor keeps at least a third of the designed reach whatever the measurement says.

Method | How the advance is derived

The advance is the shape's own horizontal extent plus 0.12 x-heights of spacing, computed from the points rather than stored. Deriving it means the two alphabets need no advance table to be maintained alongside them, and the margin is added so that consecutive fallback letters do not touch.

9.4 Trajectory

```
class Trajectory
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 255. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

The pen path in millimetres, a list of strokes, each a list of points drawn with the pen down, with y up.

Method | Methods exposed by this current class

```
def __init__(self, strokes, meta)
def Bounds(self)
def PenTravelMm(self)
```

Output. an object holding the pen path: a list of pen-down strokes in millimetres, plus metadata describing how it was produced.

Method | What it represents

The output of synthesis, and the thing the rest of the project consumes. It is a list of **pen-down paths**: each entry is a sequence of points the pen visits with its tip on the paper, and the gaps between

consecutive entries are where the pen lifts. That representation carries exactly the information a plotter needs and nothing more.

Justification | Why the pen-up moves are implicit

Nothing records the lifts explicitly. They are implied by the boundaries between strokes, because by definition the pen must rise at the end of one and fall at the start of the next. Storing them would duplicate information already present and create the possibility of the two disagreeing, which is a class of bug that cannot arise if the state is stored once.

Method | The coordinate convention, and the one place it changes

Coordinates are millimetres with y increasing upward, which is the natural orientation for describing writing and the convention the stored glyphs use. Images have y increasing downward, and some machines expect one or the other. Rather than taking a position on that, the trajectory states its own convention and each consumer converts: the rasteriser flips y as it draws, and the G-code writer flips it if the machine needs it. The flip therefore happens at the boundary, once per consumer, rather than being half-applied inside the trajectory.

Method | What the two methods report

- **Bounds** returns the smallest rectangle containing every point, which the rasteriser needs to size its canvas and the G-code writer needs to check the drawing fits the machine's travel.
- **PenTravelMm** returns the distance drawn with the pen down and the distance moved with it up. The first is how much ink the page needs; the second is wasted motion, and it is the quantity a stroke-ordering optimisation would try to reduce.

> *This is one of only two classes in the project, and it earns the form: it bundles data with two derived measurements of that same data, which is what a class is for. The rest of the pipeline is plain functions, because elsewhere there is no state that several operations need to share.*

9.5 _SwapProb

```
def _SwapProb(lam, priorD)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 290. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

The chance a stored shape is swapped for the fallback, rising with both the legibility setting and how rough the shape is.

Inputs

`lam` the legibility setting, from 0 to 1

`priorD`
: this shape's distance from the cross-writer reference

Output. the probability, from 0 to 1, that this shape is replaced by the anchor.

Purpose | What it is for

This small function is where the project's central trade-off is actually resolved. For one glyph, given how far it sits from what the letter should look like and how much the library as a whole is trusted, it returns the chance that the writer's own shape is set aside in favour of a clean one.

Method | The four cases, and the order they are tested in

Formula purpose and system use. Convert prior-distance quality into an anchor substitution preference. **Source basis:** [62, 65, 66, 67]. Exact project constants remain implementation choices unless evidence is stated.

$$P_{\text{swap}} = \begin{cases} 1 & d \geq 0.55 \\ 1 & \text{lam} \geq 0.85 \\ 0 & \text{lam} \leq 0.001 \\ 0.06\,\text{lam} & d \leq 0.12 \\ \text{clip}\big(\text{lam}\,(0.55 + 0.9\,b),\ 0,\ 1\big) & \text{otherwise} \end{cases} \tag{9.2}$$

where

P_{swap}
: the returned swap probability

d
: the shape's prior distance

lam
: the legibility setting

b
: how bad the shape is on a 0 to 1 scale, defined below

0.55
: the base swap rate for a shape of middling quality

0.9
: how much worse quality raises that rate

$\text{clip}(\cdot, 0, 1)$
: the result held to a valid probability

The order matters because the cases overlap. Garbage is tested first, so a hopeless shape is replaced even at lam 0. Full commitment is next, so above 0.85 everything goes regardless of quality. Only then is lam 0 tested, which keeps every shape that survived the first test: this is what makes lam 0 mean "faithful reproduction, defects and all" rather than "no filtering whatever".

Method | The interpolation in the middle range

Formula purpose and system use. Limit that substitution preference to a valid probability range. **Source basis:** [62, 65, 66, 67]. Exact project constants remain implementation choices unless evidence is stated.

$$b = \text{clip}\left(\frac{d - 0.12}{0.55 - 0.12},\ 0,\ 1\right) \tag{9.3}$$

where

b
: the badness of the shape: 0 at the clean threshold, 1 at the garbage threshold

d
: the shape's prior distance

0.12, 0.55
: the clean and garbage reference points

$\mathrm{clip}(\cdot, 0, 1)$
: held in range, although the earlier cases already guarantee d lies between the two

This maps the prior distance onto a 0 to 1 scale between the two reference points, which is what lets a single expression cover the whole middle range. The swap probability is then lam times a factor running from 0.55 at the clean end to 1.45 at the garbage end, clipped to 1. So at a given lam, a worse shape is swapped more often, and at a given quality, a less trusted library is swapped more often. The two inputs act multiplicatively, which is the behaviour wanted: neither alone should force a swap.

Justification | Why a probability and not a decision

Returning a probability rather than a yes or no is what produces a mixed page, and a mixed page is the point. A threshold rule would draw every instance of a given character from the same source, so a letter whose shape sits near the line would be either always the writer's or always the anchor. With a probability, some instances are theirs and some are not, which both preserves more of the hand and spreads the risk: one bad instance of a letter in a line is recoverable from context, where every instance being bad is not.

> *Because the draw is made per glyph from a seeded generator, the same text and seed give the same page. That is what makes the best-of-N search meaningful, since each attempt differs only by its seed, and what makes a result reproducible rather than something that happened once.*

9.6 AutoLegibility

```
def AutoLegibility(profile)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 314. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

How far to trust this writer's glyph storage, from the median distance from typical of all their stored shapes.

Inputs

`profile`
: the writer's style profile

Output. the legibility setting for that writer, from 0 to 1.

Purpose | What it is for

Derives the legibility setting from the profile instead of requiring it to be chosen. The question it answers is: how far can this writer's extracted library be trusted?

Formula purpose and system use. Map median glyph prior distance between project quality endpoints into anchoring strength. **Source basis:** [62, 65, 66, 67]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathrm{lam} \;=\; \mathrm{clip}\!\left(\frac{\mathrm{median}\{d_g\} - 0.24}{0.32 - 0.24},\; 0,\; 1\right) \tag{9.4}$$

where

lam the returned legibility setting

d_g the prior distance of stored variant g, over every variant of every character in the library

median
: the middle value of those distances

0.24, 0.32
: the good and bad reference points above

clip$(\cdot, 0, 1)$
: held within the valid range, which is what makes the mapping flat outside the two reference points

Justification | Why this works, and what it is really measuring

The chain of reasoning behind it is the substantive part. The prior distance measures how far a stored shape sits from what its letter should look like. A library whose shapes sit far from that is a library of bad cuts. Bad cuts come from boundaries placed where no honest boundary exists. And that happens exactly when the writing is joined up, because in cursive the pen does not lift between letters, so there is no place to cut that does not damage something.

So the median prior distance is, indirectly, a measure of how cursive the hand is, arrived at by measuring the consequence rather than the cause. That is why it works as a trust signal: it responds to the thing that actually makes an extraction unreliable. And it is why heavily cursive writers end up drawn from anchor forms, which is the mechanism behind this project's finding that such hands reproduce less faithfully.

Justification | Why an empty library returns 1

With no variants there is nothing to measure and nothing to draw from, so the anchor is the only available source and lam 1 states that. Returning 0 instead would claim a library worth trusting, and the caller would then try to draw from one that does not exist.

> *A profile may carry an explicit legibilityLambda, which callers prefer over this function. That is how a value found by experiment for a particular writer can override the derived one, and it is why the stored lam column appears in this project's profile tables even though it is usually empty.*

9.7 _GapRange

```
def _GapRange(profile)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 327. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Caches a gap interval in profile _gapRange from the ninetieth percentile of stored advance minus lead minus width. Bounds MIN_LETTER_GAP_XH and MAX_LETTER_GAP_XH constrain it. Later parameter or pool edits may need this cached interval recalculated.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _GapRange(profile):
    r = profile.get('_gapRange')
    if r is None:
        gaps = []
        for ch,vs in (profile.get('glyphs') or {}).items():
            if len(ch)!=1 or not ch.isalpha():
                continue
            for g in vs:
                gaps.append(float(g.get('advance',0.9))
                            -float(g.get('lead',0.0))
                            -float(g.get('width',0.9)))
        hi = float(np.percentile(gaps,GAP_HI_PCT)) if gaps else 0.25
        lo = MIN_LETTER_GAP_XH
        hi = min(MAX_LETTER_GAP_XH,max(lo+0.02,hi))
        r = (lo,hi)
        profile['_gapRange'] = r
    return r
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

9.8 _Advance

```
def _Advance(g, profile)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 346. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Uses the selected glyph own lead and width plus a bounded residual gap. Editing only the profile letterAdvance summary does not generally change normal stored glyph placement.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _Advance(g,profile):
    lead = float(g.get('lead',0.0))
    wid = float(g.get('width',0.9))
    lo,hi = _GapRange(profile)
    gap = float(g.get('advance',0.9))-lead-wid
    return lead+wid+min(max(gap,lo),hi)
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

9.9 _CoreGlyph

```
def _CoreGlyph(ch, profile)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 357. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

The current source always prefers profile idealMedoid if acceptable; the older IDEAL_SOURCE switch is absent. It falls back to the designed alphabet when the stored representative is absent or too rough. The profile connectedness threshold 0.5 chooses print versus cursive fallback style.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _CoreGlyph(ch,profile):
    cursive = float(profile.get('connectedness',0.0))>=CORE_CURSIVE_CONN
    src = profile.get('idealMedoid') or {}
    own = src.get(ch)
    if own and float(own.get('priorD',1.0))<=IDEAL_ANCHOR_MAX_PRIORD:
        strokes = [[tuple(p) for p in s] for s in own['strokes']]
        adv = _Advance(own,profile)
        return strokes,adv,float(own.get('lead',0.0)),cursive
    b = _BaselineGlyph(ch,profile,printOnly=not cursive)
    return None if b is None else (b[0],b[1],0.0,cursive)
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

9.10 _GlyphSource

```
def _GlyphSource(profile, ch, rng, prevExitY=None, lam=0.0, forceCore=False, forcePrint=
    False)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 374. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Sorts a character pool by prior distance and offers only the first _ELIGIBLE_VARIANTS, currently five. Rank weight is 1/(1+1.4*i), with an entry-height compatibility factor when joining. A probability policy may substitute the representative; case-related, similar, designed and blank fallbacks cover missing characters. It records own/core/case/similar/fallback provenance rather than silently treating all shapes as captured.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _GlyphSource(profile,ch,rng,prevExitY=None,lam=0.0,forceCore=False,
                 forcePrint=False):
    store = profile['glyphs']
    adv = profile.get('letterAdvance',{})
    if forcePrint:
        b = _BaselineGlyph(ch,profile,printOnly=True)
        if b is not None:
            return b[0],b[1],0.0,'core'

    def Pack(g,k=1.0):
        strokes = [[(x*k,y*k) for (x,y) in s] for s in g['strokes']]
        a = _Advance(g,profile)
        return strokes,a*k,float(g.get('lead',0.0))*k

    def Pick(cands):
        cands = cands[:_ELIGIBLE_VARIANTS]
        w = []
        for i,g in enumerate(cands):
            wi = 1.0/(1.0+1.4*i)
            if prevExitY is not None:
                dy = abs(float(g.get('entryY',0.0))-prevExitY)
                wi*=math.exp(-(dy/0.45)**2)
            w.append(wi)
        tot = sum(w)
        if tot<=1e-9:
            return cands[0]
        r = rng.random()*tot
        acc = 0.0
        for i,wi in enumerate(w):
            acc+=wi
            if r<=acc:
                return cands[i]
        return cands[-1]

    if ch in store and store[ch]:
        cands = store[ch]
        ranked = sorted(cands,key=lambda g: g.get('priorD',0.5))
        g = Pick(ranked)
        d = float(g.get('priorD',0.5))
        core = _CoreGlyph(ch,profile)
        if core is not None and (forceCore
                                 or rng.random()<_SwapProb(lam,d)):
            return core[0],core[1],core[2],'core'
        s,a,lead = Pack(g)
        return s,a,lead,'own'
    for alt in (ch.swapcase(),_SIMILAR.get(ch,'')):
        if alt and alt in store and store[alt]:
            core = _CoreGlyph(ch,profile)
            if core is not None and (forceCore
                                     or rng.random()<_SwapProb(lam,0.4)):
                return core[0],core[1],core[2],'core'
            g = Pick(sorted(store[alt],key=lambda x: x.get('priorD',0.5)))
            if alt==ch.swapcase() and ch.isupper():
                k = profile.get('ascender',1.7)/max(0.6,g['top'] or 1.0)
                k = float(np.clip(k,1.0,2.0))
                s,a,lead = Pack(g,k)
                return s,a,lead,'case'
            s,a,lead = Pack(g)
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

9.11 _Jitter

```
def _Jitter(strokes, rng, amp, rotAmp, scaleAmp)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 445. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Generate new text in a stored writer style and balance readability with model-based writer identity.

Inputs

`strokes`
: one glyph's strokes

`rng` the seeded random generator

`amp` the position shift amplitude, in x-heights

`rotAmp`
: the rotation amplitude, in degrees

`scaleAmp`
: the scale variation amplitude, as a fraction

Output. the strokes shifted, rotated and scaled by small random amounts.

Justification | Why randomness is added deliberately

Without it, every instance of a letter on the page is identical to the pixel, because the same stored variant is drawn the same way. Identical repetition is the visual signature of print, and a page of it reads as a font however handwritten the individual letterforms are. Real writing never repeats a letter exactly, so a small random variation per glyph is not a flaw introduced into the output; it is a property of handwriting that must be put back.

Method | The three transformations

Formula purpose and system use. Apply small coordinate, rotation and scale perturbations to vary repeated glyphs. **Source basis:** [62, 65, 66, 67]. Exact project constants remain implementation choices unless evidence is stated.

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} s_x x \\ s_y y \end{pmatrix} + \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} \tag{9.5}$$

where

x, y the glyph's original coordinates, in x-heights

x', y'
: the jittered coordinates

θ a small random rotation, drawn from a normal distribution

s_x, s_y
: independent random scale factors, each near 1

δ_x, δ_y
: a random translation

$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$
: the standard 2-D rotation matrix, which turns a point anticlockwise about the origin by theta

The order is scale, then rotate, then translate, and it is not interchangeable: rotating before scaling would make the independent x and y scalings act along the rotated axes, shearing the glyph rather than stretching it. Scaling x and y independently is itself deliberate, since a writer's letters vary slightly in proportion as well as in size.

Justification | Background: why a normal distribution

Each amount is drawn from a **normal distribution**, the bell-shaped curve, rather than uniformly from a range. The difference is that a normal distribution makes small deviations much commoner than large ones, with no hard limit, whereas a uniform draw makes a large deviation exactly as likely as a tiny one. Real variation behaves the first way: most letters are close to the writer's usual form and a few are noticeably off. A uniform draw produces output that wobbles unconvincingly, because every letter is about equally displaced.

Justification | Why the vertical shift is smaller

The vertical amplitude is 0.6 of the horizontal. Vertical position is read more sharply than horizontal: the eye uses the baseline as a reference, so a letter sitting high stands out in a way that one sitting slightly right does not. The recogniser is similarly sensitive, since its features were learned relative to the baseline. Reducing the vertical component keeps the variation from reading as a fault.

> *The amplitudes reaching this function are already scaled by the caller's jitter setting and, for an anchor glyph, by CORE_JITTER_FRAC. So the same function serves for a fully varied extracted glyph and a nearly rigid anchor, with the policy decision kept in the caller rather than here.*

9.12 _Resample

```
def _Resample(poly, step)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 462. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Generate new text in a stored writer style and balance readability with model-based writer identity.

Inputs

`poly`
: a path

`step`
: the desired spacing between points

Output. the path with points evenly spaced at about that interval.

Purpose | What it is for

Respaces a path so its points sit at roughly equal distances along it, by the arc-length method described under _ResampleN in style.py: cumulative distance along the path is computed and new points are read off at even distances by interpolation.

Justification | Why layout needs it

Paths reaching this point come from several sources with very different point densities. A stored glyph was simplified, so a straight run is two points. A ligature is generated from three control points. A built-in letterform was laid out by hand at whatever spacing was convenient. Concatenating them gives a path whose density varies several-fold along its length, and two later stages are sensitive to that.

- **The rasteriser** uses distances to complete line segments in DrawPolyline, so sparse vertices do not inherently create dotted straight segments; resampling changes the represented geometry and amount of segment work.
- **The G-code writer** emits moves between points; point density affects command count and curve approximation. Actual speed also depends on pulse timing and mechanics, so even spacing does not establish constant physical pen speed.

Justification | Why 0.35 millimetres

The 0.35 mm sampling step controls trajectory point density. Finer spacing increases point storage and later processing; coarser spacing gives fewer samples and can underrepresent curved geometry. Segment-based drawing and motion interpolation also matter, so uniform samples alone do not guarantee continuous raster coverage, smoother physical motion or constant actual pen speed. This exact step is a project setting, not a literature optimum.

> *This duplicates _ResampleN in style.py, differing in that it takes a spacing rather than a count and returns tuples rather than an array. Both forms are needed: averaging requires a fixed count so that points correspond between strokes, while drawing requires a fixed spacing so that density is uniform. The two interfaces serve different callers; count and spacing can be related through total path length, subject to endpoint and rounding conventions.*

9.13 StyleCalibration

```
def StyleCalibration(profile, mmPerXh=4.0, pxPerMm=18.0)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 484. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Always used unless a caller passes an explicit _cal into SynthesizeText. The retired USE_STYLE_CALIBRATION switch is absent. A cache keyed by authorId, physical x-height and render density stores corrections from a short fixed probe. Changes to other profile fields can reuse an old correction unless the cache is invalidated.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def StyleCalibration(profile,mmPerXh=4.0,pxPerMm=18.0):
    key = (profile.get('authorId'),round(mmPerXh,3),round(pxPerMm,3))
    if key in _STYLE_CAL:
        return _STYLE_CAL[key]
    ref = (profile.get('slantMeasRef'),profile.get('ascMeasRef'),
           profile.get('descMeasRef'))
    cal = dict(shearDelta=0.0,ascK=1.0,descK=1.0)
    if None in ref or any(r!=r for r in ref):
        _STYLE_CAL[key] = cal
        return cal
    slantRef,ascRef,descRef = ref
    sample = ('the quick brown fox jumps over lazy dogs and by half past '
              'eight it kept flying quietly')
    for _ in range(3):
        traj = SynthesizeText(sample,profile,mmPerXh=mmPerXh,seed=17,
                              lineWidthMm=10_000.0,jitter=0.0,_cal=cal)
        img = RenderTrajectory(traj,pxPerMm=pxPerMm)
        ink = BinarizeLine(np.array(img))
        band = CoreBand(ink)
        if band is None or not ink.any():
            break
        top,base = band
        xh = float(base-top)
        if xh<4:
            break
        ys = np.nonzero(ink.any(axis=1))[0]
        gotSlant = EstimateSlantDeg(ink)
        gotAsc = float(base-ys.min())/xh
        gotDesc = float(base-ys.max())/xh
        cal = dict(
            shearDelta=cal['shearDelta']+
            math.tan(math.radians(slantRef))-math.tan(math.radians(gotSlant)),
            ascK=float(np.clip(cal['ascK']*(ascRef-1.0)/
                               max(1e-3,gotAsc-1.0),0.5,1.6)),
            descK=float(np.clip(cal['descK']*descRef/min(-1e-3,gotDesc),
                                0.5,1.6)))
    _STYLE_CAL[key] = cal
    return cal
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

9.14 _ApplyCal

```
def _ApplyCal(strokes, cal)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 524. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Generate new text in a stored writer style and balance readability with model-based writer identity.

Inputs

`strokes`
: one glyph's strokes

`cal` the correction dictionary

Output. the strokes with the ascender and descender zones rescaled.

Method | What it does

Applies the two zone corrections, stretching or compressing only the parts of a glyph outside the x-height band. The shear correction is not applied here; it was folded into the slant before layout began, since shear depends on the final height and must be applied as the glyph is placed.

Formula purpose and system use. Adjust ascender and descender extents while preserving the body-band reference. **Source basis:** [62, 65, 66, 67]. Exact project constants remain implementation choices unless evidence is stated.

$$y' = \begin{cases} 1 + (y - 1)\, k_{\text{asc}} & y > 1 \\ y \cdot k_{\text{desc}} & y < 0 \\ y & 0 \le y \le 1 \end{cases} \tag{9.6}$$

where

y the glyph's height at this point, in x-heights

y' the corrected height

k_{asc} the ascender correction factor

k_{desc} the descender correction factor

This is the same piecewise form as _BaselineGlyph uses for adapting a built-in letterform, and for the same reason: the band from 0 to 1 is the reference and must not move, so only the reach beyond it is scaled. Writing the ascender case as 1 plus a scaled excess, rather than scaling y directly, is what keeps the band's top edge fixed at exactly 1.

Method | The early exit

When both factors are within a thousandth of 1, the strokes are returned untouched. That is the common case, since many writers need no zone correction, and the check avoids rebuilding every point of every glyph to multiply by one.

> *This is applied only to extracted glyphs, never to an anchor. The anchor's proportions were either designed to be legible or are the writer's own best letterform, and re-stretching them would reintroduce the distortion the anchor exists to avoid. The caller enforces that by testing the source label before calling.*

9.15 _Ligature

```
def _Ligature(p0, p1, xh, rng)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 544. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

The current connector uses the shallow project construction; the optional tangent-ligature implementation and switch are absent. A join adds new ink that was not observed in either stored character. It should not be represented as recovered original pen motion.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _Ligature(p0,p1,xh,rng):
    dx = p1[0]-p0[0]
    if abs(dx)<0.06*xh:
        return [p0,p1]
    sag = min(0.11*xh,0.16*abs(dx))+rng.gauss(0.0,0.015*xh)
    ctrlY = min(p0[1],p1[1])+0.35*abs(p1[1]-p0[1])-sag
    ctrl = ((p0[0]+p1[0])*0.5,ctrlY)
    out = []
    for i in range(9):
        t = i/8.0
        u = 1.0-t
        out.append((u*u*p0[0]+2*u*t*ctrl[0]+t*t*p1[0],
                    u*u*p0[1]+2*u*t*ctrl[1]+t*t*p1[1]))
    return _Resample(out,step=0.35)
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

9.16 _BodyStroke

```
def _BodyStroke(placed, xh)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 564. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Selects the body path and eligible entry/exit rather than joining via dots or crossbars. Flatness and small-horizontal-span guards address ambiguous skeleton ordering. The former JOIN_ANATOMY feature switch is absent; the actual helper determines the rule.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _BodyStroke(placed,xh):
    spans = [max(p[0] for p in s)-min(p[0] for p in s) for s in placed]
    highs = [max(p[1] for p in s)-min(p[1] for p in s) for s in placed]
    cand = [i for i in range(len(placed))
            if highs[i]>=JOIN_FLAT_MAX_ASPECT*max(spans[i],1e-9)]
    if not cand:
        cand = list(range(len(placed)))
```

```
    bi = max(cand,key=lambda i: spans[i])
    body = list(placed[bi])
    if spans[bi]<JOIN_VERT_MIN_XSPAN_XH*xh:
        if body[0][1]<body[-1][1]:
            body = body[::-1]   #y is up, so the lower end goes last
    return bi,body
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

9.17 _SplitWords

```
def _SplitWords(text)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 579. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Generate new text in a stored writer style and balance readability with model-based writer identity.

Inputs

`text`
: the text to be written

Output. a list of words with newline markers between the lines.

Purpose | What it is for

Breaks the text into the units layout works with. Words are the unit because wrapping happens at word boundaries: a word that will not fit on the current line moves to the next one whole, since handwriting is not hyphenated.

Justification | Why newlines survive as markers

The text is split on newlines first and a marker is inserted between the resulting chunks, so an explicit line break in the input is preserved rather than being treated as a space. The distinction matters because an explicit break is the writer's instruction and must be obeyed wherever the line happens to be, while a space is merely a place where a break is permitted. Keeping the two distinct lets layout handle them as what they are.

> *Empty strings between consecutive spaces are discarded, so repeated spaces collapse to one gap, and a trailing marker is removed so text ending in a newline does not produce an empty final line. Both are small conveniences that keep the layout loop from needing to test for them.*

9.18 _WordWidth

```
def _WordWidth(word, profile)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 587. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Generate new text in a stored writer style and balance readability with model-based writer identity.

Inputs

`word`
: one word

`profile`
: the writer's style profile

Output. approximately how wide the word will be, in x-heights.

Purpose | What it is for

An estimate used only to decide whether a word fits on the current line. It sums the writer's median advance for each character, falling back to 0.9 x-heights for a character with no recorded advance.

Justification | Why an estimate is acceptable here

The true width is not available in advance, because it depends on which variant of each letter is chosen, how the jitter scales them, and whether the collision guard pushes anything right. All of that happens while the word is being drawn, which is after the decision to put it on this line has been made. Determining the exact width would mean laying the word out, measuring it, and discarding the result if it did not fit, which is several times the work for a decision whose only consequence is where a line breaks.

The estimate is slightly low on average, since it ignores the collision guard, which only ever widens. So a word occasionally overruns the requested line width by a small margin. That is the right direction for the error to fall: a line a few percent too long is unremarkable, where a conservative estimate that broke lines early would leave visible ragged gaps.

9.19 SynthesizeText

```
def SynthesizeText(text, profile, mmPerXh=4.0, seed=None, lineWidthMm=180.0, jitter=0.15,
    legibility=None, perCharLam=None, _cal=None)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 596. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

Always computes or accepts style correction, clamps writer slant, suppresses joins below connectedness 0.45, and otherwise uses connEff equal to measured connectedness. There is no JOIN_PROB_CAP or JOIN_LEGIBILITY_COUPLING in this tree. Glyph variants, bounded spacing, jitter, integrated drift, collision clearance and compatible joins create millimetre paths; lineStepMm is (ascender - descender + 0.85) times x-height.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def SynthesizeText(text,profile,mmPerXh=4.0,seed=None,lineWidthMm=180.0,
                   jitter=0.15,legibility=None,perCharLam=None,_cal=None):
    if legibility is None:
        legibility = float(profile.get('legibilityLambda',
                                       AutoLegibility(profile)))
    rng = random.Random(seed)
    xh = float(mmPerXh)
    cal = _cal if _cal is not None else StyleCalibration(profile,mmPerXh)
    slantDeg = profile.get('slantDeg',0.0)
    authorSlantCap = 45.0-19.0*float(np.clip(legibility,0.0,1.0))
    slantDeg = float(np.clip(slantDeg,-authorSlantCap,authorSlantCap))
    slant = math.tan(math.radians(slantDeg))+cal['shearDelta']
    conn = float(profile.get('connectedness',0.0))
    connEff = 0.0 if conn<JOIN_PRINT_CONN_FLOOR else conn
    wordGap = float(profile.get('wordSpaceXh',1.2))
    ascender = float(profile.get('ascender',1.7))
    descender = float(profile.get('descender',-0.6))
    lineStep = (ascender-descender+0.85)*xh

    ampP = 0.022*jitter   #position jitter, in x-heights
    ampR = 2.2*jitter   #rotation jitter, in degrees
    ampS = 0.05*jitter   #scale jitter
    driftAmp = 0.05*jitter   #slow baseline drift, in x-heights

    strokes = []
    penX,penY = 0.0,0.0
    lineIdx = 0
    drift = 0.0
    driftV = 0.0
    usage = {}
    vpos = 0   #index into the text with newlines as spaces, for perCharLam

    def Flush(chain):
        if len(chain)>=2:
            strokes.append(_Resample(chain,step=0.35))
        elif len(chain)==1:
            strokes.append(chain+[(chain[0][0]+0.05,chain[0][1])])

    for word in _SplitWords(text):
        if word=='\n':
            lineIdx+=1
            penX = 0.0
            penY = -lineIdx*lineStep
            vpos+=1
            continue
        wWidth = _WordWidth(word,profile)*xh
        if penX>1e-6 and penX+wWidth>lineWidthMm:
            lineIdx+=1
            penX = 0.0
            penY = -lineIdx*lineStep
        chain = []   #the current pen-down polyline
        prevExit = None
        prevBodyMaxX = None
        prevSrc = None
        prevCoreCursive = False
        for ci,ch in enumerate(word):
            lam = legibility
            forceCore = forcePrint = False
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

9.20 SynthesizeJointBestOf

```
def SynthesizeJointBestOf(author, text, profile, nTries=6, mmPerXh=4.0, lineWidthMm=180.0,
    jitter=0.5, seed=0, reader=None, authorModel=None, authorMapping=None, pxPerMm=18.0,
    legibility=None, repair=True, repairRounds=3, repairWorstK=2)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 813. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 9.2.

Purpose | What this contributes

The direct function default is six tries, while server DEFAULT_TRIES is fourteen unless WEBAPP_TRIES overrides it. Each candidate is rendered and scored by text edit agreement and requested-writer probability using their harmonic mean. Up to three repair rounds target two new weak positions per round; only an improved complete score commits the repair. Search stopping thresholds and seed spacing 977 are heuristics, not published optima.

Method | Current implementation

Generate new text in a stored writer style and balance readability with model-based writer identity. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def SynthesizeJointBestOf(author,text,profile,nTries=6,mmPerXh=4.0,
                          lineWidthMm=180.0,jitter=0.5,seed=0,
                          reader=None,authorModel=None,authorMapping=None,
                          pxPerMm=18.0,legibility=None,
                          repair=True,repairRounds=3,repairWorstK=2):
    if legibility is None:
        legibility = float(profile.get('legibilityLambda',
                                       AutoLegibility(profile)))
    if reader is None:
        reader = PaperCRNNNumpy()
    if authorModel is None:
        authorModel = AuthorClassifierCNNNumpy()
    if authorMapping is None:
        authorMapping = authorModel.authorMapping
    visible = text.replace('\n',' ')
    authorIdx = authorMapping[author]

    def Score(img):
        textAcc = CharAcc(ReadText(img,reader),visible)
        _predIdx,probs = ClassifyImage(img,authorModel)
        widConf = float(probs[authorIdx])
        combined = (2*textAcc*widConf/(textAcc+widConf)
                    if (textAcc+widConf)>1e-9 else 0.0)
        return combined,textAcc,widConf

    base = 0 if seed is None else int(seed)
    best,bestTraj,bestImg = -1.0, None, None
    for k in range(max(1,nTries)):
        traj = SynthesizeText(text,profile,mmPerXh=mmPerXh,
                              seed=base+977*k,lineWidthMm=lineWidthMm,
                              jitter=jitter,legibility=legibility)
```

```
        img = RenderTrajectory(traj,pxPerMm=pxPerMm)
        combined,_textAcc,_widConf = Score(img)
        if combined>best:
            best,bestTraj,bestImg = combined,traj,img
        if best>=0.99:
            break

    perChar = {}
    if repair:
        for _round in range(repairRounds):
            if best>=0.995:
                break
            weak = _WeakChars(reader,bestImg,visible)
            fresh = [p for p in weak if p not in perChar][:repairWorstK]
            if not fresh:
                break
            trial = dict(perChar)
            for p in fresh:
                trial[p] = 1.0
            cand = SynthesizeText(text,profile,mmPerXh=mmPerXh,seed=base,
                                  lineWidthMm=lineWidthMm,jitter=jitter,
                                  legibility=legibility,perCharLam=trial)
            img = RenderTrajectory(cand,pxPerMm=pxPerMm)
            combined,_textAcc,_widConf = Score(img)
            if combined>best:
                best,bestTraj,bestImg = combined,cand,img
                perChar = trial
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Explicit glyphs suit a small offline collection and expose size, slant, spacing, joins and fallback provenance. Repeated candidates and accepted repair search this finite representation. [62, 65, 66, 67].

Stage: Trajectory rasterisation

Method rationale, alternatives and library comparison: Section 8.11.

9.21 _Draw

```
def _Draw(traj, pxPerMm, padMm, strokePx, paper=255, ink=0, softPx=0.0)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 752. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.11.

Purpose | What this contributes

Rasterises the pen path as black ink on white paper with the round pen, then softens the edge slightly.

Inputs

`traj`
: the trajectory to draw

`pxPerMm`
: the resolution in pixels per millimetre

`padMm`
: the margin in millimetres

`strokePx`
: the pen width in pixels

`paper`
: the background value, default 255

`ink` the stroke value, default 0

`softPx`
: the blur radius in pixels, 0 for none

Output. a grey image of the trajectory, ink on paper.

Justification | Why the pen path is turned into an image at all

The plotter draws from the trajectory and never needs an image. The image exists because the two judging networks score pictures, not paths. Every evaluation in this project, and the best-of-N search itself, works by rendering a candidate and asking the networks what they see. So the rasteriser is not a display convenience; it is part of the optimisation loop, and how faithfully it imitates a photograph determines whether the scores mean anything.

Method | The coordinate flip

Formula purpose and system use. Convert millimetre trajectory coordinates into image pixels with a vertical-axis flip. **Source basis:** [63, 64, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$x_{\text{px}} \;=\; (x - x_0 + \text{pad}) \cdot \text{ppm}, \qquad y_{\text{px}} \;=\; (y_1 - y + \text{pad}) \cdot \text{ppm} \tag{9.7}$$

where

x_{px}, y_{px}
: the pixel position

x, y the trajectory position in millimetres, y upward

x_0 the leftmost extent of the trajectory

y_1 the topmost extent

pad the margin in millimetres

ppm
: pixels per millimetre

The x expression merely shifts the origin and scales. The y expression also reverses direction, by subtracting from the top extent rather than adding to the bottom one, because the trajectory has y increasing upward and an image has row 0 at the top. This is the single place that conversion happens on the rendering path, which is the discipline described under Trajectory: each consumer converts at its own boundary.

Justification | Why the round pen needs no end caps

Each stroke is drawn by DrawPolyline, which sets a pixel according to its distance from the path rather than by stepping along and stamping a shape. A consequence worth drawing out is that round ends and round joins come out automatically: the set of points within a fixed distance of a line segment is a rectangle with a semicircle at each end, and the union of two such regions is already

correctly rounded where they meet. A stamping rasteriser would need explicit end caps and join handling, and would leave notches at sharp corners.

Justification | Why the blur comes last

All the strokes are drawn and then the whole image is blurred once. Blurring each stroke as it was drawn would be both slower and wrong: where two strokes overlap, each having been individually softened, the overlap would be darker than a single stroke's core, producing a visible dark patch at every crossing. Blurring the finished image treats a crossing as the single region of ink that it is.

> *The canvas is sized from the trajectory's own bounds plus the margin, so it is exactly as large as the writing requires. A fixed canvas size would either clip long text or surround short text with blank paper, and the second matters as much as the first, because the proportion of the crop that is ink is one of the statistics the networks respond to.*

9.22 RenderTrajectory

```
def RenderTrajectory(traj, pxPerMm=8.0, padMm=None, strokePx=None, size=None)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 771. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.11.

Purpose | What this contributes

Current accepted parameters are traj, pxPerMm, padMm, strokePx and size. There are no profile or uniformInk parameters. Rendering always uses the uniform-width path, default width fraction 0.085 and blur fraction 0.30. Optional size performs a subsequent raster resize and can change aspect ratio if both dimensions are specified independently.

Method | Current implementation

Make an image of a generated path for preview and recognition-based scoring. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def RenderTrajectory(traj,pxPerMm=8.0,padMm=None,strokePx=None,size=None):
    mm = traj.meta.get('mmPerXh',4.0)
    if padMm is None:
        padMm = 0.18*mm   #padding proportional to the writing
    sp = strokePx or max(1,int(round(UNIFORM_PEN_WIDTH_XH*mm*pxPerMm)))
    img = _Draw(traj,pxPerMm,padMm,sp,paper=255,ink=0,
                softPx=UNIFORM_SOFT_FRAC*max(1.0,sp))
    if size is not None:
        img = Resize(np.asarray(img,dtype=np.uint8),size[0],size[1],'linear')
    return img
```

Justification | Interpret the method rather than infer a benchmark

Sampling straight segments and applying a controlled width makes candidate rendering explicit. The uniform width models one drawing pen rather than source ink intensity. [63, 64, 3].

Stage: CTC decoding and forced alignment

Method rationale, alternatives and library comparison: Section 6.30.

9.23 _LineLogProbs

```
def _LineLogProbs(reader, img)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 783. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.30.

Purpose | What this contributes

Fits the rendered crop to the reader canvas and calls reader.Forward. It is a model-scoring input path, not a geometric measurement of the physical gantry output.

Method | Current implementation

Train from complete line transcripts and recover transcript-constrained character locations for extraction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _LineLogProbs(reader,img):
    t = TensorFromResized(ResizeLineImageFixed(img))
    return reader.Forward(t)[:,0,:]
```

Justification | Interpret the method rather than infer a benchmark

CTC marginalises valid blank/repeat paths; greedy decoding selects a path cheaply; forced alignment maximises over paths for a supplied known transcript. [34, 42, 53].

9.24 _WeakChars

```
def _WeakChars(reader, img, visible)
```

Evidence | Source and method link

`Software/DemoCode/synthesise.py` line 788. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.30.

Purpose | What this contributes

Train from complete line transcripts and recover transcript-constrained character locations for extraction.

Inputs

`reader`
: a loaded reader

`img` a rendered line image

`visible`
: the text that was supposed to be written

Output. the positions of the characters the reader scored worst, worst first.

Purpose | What it is for

Identifies which characters of the rendered line the reader is least sure about. The repair loop uses the answer to redraw only those characters, instead of redrawing the whole line and hoping for a better outcome.

Justification | Why forced alignment rather than decoding

The text is known, so there is no recognition to be done. What is wanted is per-character evidence, and forced alignment provides it: it finds where each known character sits in the rendered line, after which the reader's own score for that character over exactly those columns is a direct measure of how legible that character came out.

Formula purpose and system use. Rank transcript-aligned character positions by mean log probability for targeted repair. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$s_i \;=\; \frac{1}{t_1 - t_0} \sum_{t=t_0}^{t_1-1} \log p_t(c_i) \tag{9.8}$$

where

s_i the score for the i-th character: higher is more legible

c_i that character

t_0, t_1
: the first and last columns the alignment assigned to it

$\log p_t(c_i)$
: the reader's log probability for that character at column t

$\frac{1}{t_1 - t_0}$
: the mean over its columns, so a wide letter is not favoured over a narrow one merely for occupying more columns

Averaging rather than summing is the point of that factor. A summed score would rank an m below an i automatically, since it spans more columns and so accumulates more negative terms, and the repair loop would spend its attention on wide letters regardless of how legible they were.

Method | The unaligned case

A character the alignment could not place at all is given a very large negative score, so it sorts to the front. That is the correct ranking: a character with no columns of its own was so poorly drawn that the aligner had to pass through it in a single step shared with its neighbours, which is a more complete failure than a character that was placed but scored badly.

> *Whitespace is skipped, since a space is not a drawn shape and cannot be repaired by substituting a letterform. Positions are returned as indices into the original text, not into the filtered character list, because that is what the per-character override table is keyed by.*

10 Direct page retracing

Evidence | Current implementation boundary

`Software/DemoCode/retrace.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Take page pixels through line segmentation, skeleton tracing and pruning, then map recovered trajectories for reproduction. The current SegmentLines call mismatch is described in the pipeline audit; this is not a verified successful run.

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
HERE = Path(__file__).resolve().parent
MIN_STROKE_PTS = 4
MIN_STROKE_XH = 0.12
```

Stage: Direct ink retracing

10.1 Method background: Direct ink retracing

Purpose | The role in this system

Bypass recognition and glyph synthesis to compare a traced source path with physical output.

Justification | The reason for this construction

It removes two sources of variation but still includes binarisation, thinning, tracing, pruning, fitting and motion; it is not a pure mechanical ground truth.

Alternatives | What changes if another method is used

Direct known geometric paths isolate motion more tightly. OCR-and-synthesise asks a different end-to-end question. An online digitiser would supply a better observed trajectory.

Library | Implementation speed and accuracy

Image/vector tracing libraries could replace centreline extraction, with different topology and simplification. The current PageStrokes timer keyword is incompatible with current SegmentLines and is documented as a static integration discrepancy. [3, 31, 33].

10.2 PageStrokes

```
def PageStrokes(gray, timer=None)
```

Evidence | Source and method link

`Software/DemoCode/retrace.py` line 49. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 10.1.

Purpose | What this contributes

Calls SegmentLines(gray, timer=timer), although the captured core does not accept timer. If that mismatch were resolved elsewhere, the remaining path would still binarise, thin, trace and prune source ink; the code comment claiming only mechanical differences is stronger than this processing supports.

Method | Current implementation

Bypass recognition and glyph synthesis to compare a traced source path with physical output. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def PageStrokes(gray, timer=None):
    lines, _dbg, info = SEG.SegmentLines(gray, timer=timer)
    strokes = []
    perLine = []
    for ln in lines:
        crop = np.asarray(ln["image"], dtype=np.uint8)
        ink = SEG.BinarizeLine(crop)
        if not ink.any():
            perLine.append(0)
            continue
        polys = ST.TracePolylines(SEG.Skeletonize(ink))
        x0, y0 = (ln["bbox"][0], ln["bbox"][1]) if "bbox" in ln else (0, 0)
        kept = 0
        for p in polys:
            if len(p) < MIN_STROKE_PTS:
                continue
            strokes.append([(float(x)+x0, -(float(y)+y0)) for (x, y) in p])
            kept += 1
        perLine.append(kept)
    return strokes, dict(lines=len(lines), strokesPerLine=perLine, **info)
```

Justification | Interpret the method rather than infer a benchmark

It removes two sources of variation but still includes binarisation, thinning, tracing, pruning, fitting and motion; it is not a pure mechanical ground truth. [3, 31, 33].

10.3 LinePitchMm

```
def LinePitchMm(info, cfg, traj)
```

Evidence | Source and method link

`Software/DemoCode/retrace.py` line 77. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 10.1.

Purpose | What this contributes

The segmenter reports the pitch in half-resolution pixels; the trajectory is in full-resolution page pixels, so the pitch doubles, and ToMachine's own scale converts it to millimetres.

Method | Current implementation

Bypass recognition and glyph synthesis to compare a traced source path with physical output. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def LinePitchMm(info,cfg,traj):
    _p,scale = REP.ToMachine(traj,cfg,optimizeOrder=False)
    return max(1.0,float(info.get("pitch",40))*2.0*scale)
```

Justification | Interpret the method rather than infer a benchmark

It removes two sources of variation but still includes binarisation, thinning, tracing, pruning, fitting and motion; it is not a pure mechanical ground truth. [3, 31, 33].

10.4 PruneSpecks

```
def PruneSpecks(strokes, xhPx)
```

Evidence | Source and method link

`Software/DemoCode/retrace.py` line 83. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 10.1.

Purpose | What this contributes

Bypass recognition and glyph synthesis to compare a traced source path with physical output.

Method | Current implementation

Bypass recognition and glyph synthesis to compare a traced source path with physical output. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def PruneSpecks(strokes, xhPx):
    limit = MIN_STROKE_XH*max(1.0, xhPx)
    out = []
    for s in strokes:
        span = max(max(p[0] for p in s)-min(p[0] for p in s),
                   max(p[1] for p in s)-min(p[1] for p in s))
        if len(s) >= MIN_STROKE_PTS and span >= limit:
            out.append(s)
    return out
```

Justification | Interpret the method rather than infer a benchmark

It removes two sources of variation but still includes binarisation, thinning, tracing, pruning, fitting and motion; it is not a pure mechanical ground truth. [3, 31, 33].

10.5 RetracePage

```
def RetracePage(imagePath, outPath, cfg=None, previewPath=None)
```

Evidence | Source and method link

`Software/DemoCode/retrace.py` line 100. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 10.1.

Purpose | What this contributes

The whole job, photograph in and a g-code file out the trajectory is handed to the ordinary writer, so the page is placed, scaled to the work area and ordered by exactly the same code a synthesised line goes through. only the source of the strokes differs.

Method | Current implementation

Bypass recognition and glyph synthesis to compare a traced source path with physical output. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def RetracePage(imagePath, outPath, cfg=None, previewPath=None):
    cfg = cfg or REP.GantryConfig()
    run = TIMING.Run("retrace")

    with run.Step("loading"):
        gray = TR.ReadImageFile(imagePath)

    strokes, info = PageStrokes(gray, timer=run)
    with run.Step("tracing"):
        strokes = PruneSpecks(strokes, info.get("pitch", 40)*0.4)
    if not strokes:
        raise SystemExit("no ink found to trace in %s" % imagePath)

    traj = SYN.Trajectory(strokes, dict(author="retrace", source=str(imagePath),
                                        lines=info["lines"]))
    with run.Step("gcode"):
        written = REP.WriteGcode(traj, cfg, str(outPath),
                                 title="retrace of %s" % Path(imagePath).name,
                                 rowTolMm=LinePitchMm(info, cfg, traj))

    preview = None
    if previewPath:
        with run.Step("preview"):
            preview = SYN.RenderTrajectory(traj, pxPerMm=6.0)
            from PIL import Image
            Image.fromarray(np.asarray(preview, dtype=np.uint8)).save(previewPath)

    run.chars = 0
    run.lines = info["lines"]
    out = dict(written)
    out.update(gcode=str(outPath), tracedStrokes=len(strokes), timing=run.AsDict(),
               preview=str(previewPath) if previewPath else None)
    return out
```

Justification | Interpret the method rather than infer a benchmark

It removes two sources of variation but still includes binarisation, thinning, tracing, pruning, fitting and motion; it is not a pure mechanical ground truth. [3, 31, 33].

10.6 Main

```
def Main()
```

Evidence | Source and method link

`Software/DemoCode/retrace.py` line 141. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 10.1.

Purpose | What this contributes

Bypass recognition and glyph synthesis to compare a traced source path with physical output.

Method | Current implementation

Bypass recognition and glyph synthesis to compare a traced source path with physical output. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Main():
    ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    ap.add_argument("image")
    ap.add_argument("-o", "--out", default=None)
    ap.add_argument("--preview", default=None)
    args = ap.parse_args()
    out = args.out or str(Path(args.image).with_suffix(".gcode").name)
    r = RetracePage(args.image, out, previewPath=args.preview)
    print("traced %d lines into %d strokes" % (r["timing"]["lines"], r["tracedStrokes"]))
    print("  %s: %d g-code lines, %d pen pulses, scale %.3f"
          % (r["gcode"], r["lines"], r["penPulses"], r["scale"]))
    if r["clipped"]:
        print("  WARNING: %d point(s) clamped to the work area" % r["clipped"])
    if r["preview"]:
        print("  preview: %s" % r["preview"])
```

Justification | Interpret the method rather than infer a benchmark

It removes two sources of variation but still includes binarisation, thinning, tracing, pruning, fitting and motion; it is not a pure mechanical ground truth. [3, 31, 33].

11 Gantry mapping and control

Evidence | Current implementation boundary

`Software/DemoCode/reproduce.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Establish units, workspace and axis calibration; map and order trajectories; describe pen state and switches; generate coordinated integer pulses and G-code; interpret the job with stop and connection checks. Hardware sizing explains assumptions before the motion equations.

11.1 Expanded reasoning: mechanics, electronics and motion algorithms

11.1.1 Pulley travel, speed and torque: assumptions before arithmetic

For belt pitch $p = 2$ mm and $N_t = 20$ teeth, travel per revolution is $L_{\text{rev}} = N_t p = 40$ mm, and pitch radius $r = L_{\text{rev}}/(2\pi) = 6.3662$ mm. This is the belt pitch radius, not necessarily the measured outer pulley radius. At 4000 mm/min, motor speed is $4000/40 = 100$ rev/min. The report rounded a cited 3600 mm/min handwriting speed up to 4000, an increase of about 11.1%; this is a design allowance, not an experimentally recovered maximum gantry speed [10].

Torque is the moment of force; for tangential belt force F, $T = rF$. The general vector definition is $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$ and magnitude $rF \sin\theta$ [68]. Assuming steady horizontal motion and a Coulomb-style equivalent friction $F_f = \mu m g$, the report uses $\mu = 0.004, m = 0.950$ kg and $g = 9.81$ m/s^2:

Formula purpose and system use. Purpose in this explanation: Pulley travel, speed and torque: assumptions before arithmetic **Source basis:** [3, 69]. Exact project constants remain implementation choices unless evidence is stated.

$$F_f = 0.037278 \text{ N}, \qquad T_f = 0.0063662(0.037278) = 0.0002373 \text{ N m} = 0.02373 \text{ N cm}.$$

The conversion multiplies N m by 100 to obtain N cm. The small answer follows from a small assumed friction coefficient and short radius. It does not establish the actual rolling resistance of this assembly.

A fuller horizontal acceleration model is

Formula purpose and system use. Purpose in this explanation: Pulley travel, speed and torque: assumptions before arithmetic **Source basis:** [3, 69]. Exact project constants remain implementation choices unless evidence is stated.

$$T_{\text{req}} \simeq \frac{r}{\eta}(ma + F_f + F_{\text{pen}} + F_{\text{other}}) + J_{\text{rot}}\frac{a}{r},$$

where η is transmission efficiency, F_{pen} effective drag, F_{other} other opposing forces and J_{rot} rotary inertia. Terms must be referred to the same shaft. Constructed example: at $a = 0.4$ m/s^2, translational acceleration adds $rma = 0.002419$ N m $= 0.2419$ N cm before other terms, over ten times the report's steady-friction estimate. This illustrates an omitted load, not a newly measured torque or a motor redesign.

The report interpolates curve points $(60, 7)$ and $(120, 21)$ in RPM and N cm: $T(100) = 7 + (100 - 60)(21 - 7)/(120 - 60) = 16.33$ N cm. Linear interpolation assumes the correct curve and operating

conditions. The report literally leaves the curve citation as “SOURCE”. The unusual increasing pair cannot be independently authenticated here. Do not turn 16.33 into an established available torque or use its ratio to 0.02373 as a validated safety factor. Pull-out torque at speed, holding torque at rest and pull-in/start capability are different. Supply voltage, phase current, driver mode and acceleration affect the comparison.

11.1.2 Resolution, quantisation and why they are not accuracy

For $N_s = 200$ full steps/rev and microstep factor $m_s = 16$, $K = N_s m_s / L_{\text{rev}} = 80$ commanded steps/mm, spacing $\Delta = 1/K = 0.0125$ mm. Full-step angle is $360/200 = 1.8$ degrees. Nearest-grid rounding has per-axis error at most $\Delta/2 = 0.00625$ mm. With equal axes, endpoint Euclidean rounding error is at most $\sqrt{2}\Delta/2 = 0.00884$ mm. If two independently rounded endpoints define a length, their relative position can differ by up to twice the endpoint bound. Truncation instead of nearest rounding has a different bound.

These are ideal coordinate quantisation bounds. Microstep current ratios do not ensure exactly equal loaded rotor angle increments. Belt compliance, backlash, missed steps, pen flex and calibration error are separate. An open-loop counter records requested position; it does not observe actual position. Consequently the 0.5 mm physical length requirement requires measurement even though the command lattice is much finer.

11.1.3 A4988 current, reference voltage and every driver signal

The manufacturer gives $I_{\text{trip,max}} = V_{\text{REF}}/(8R_S)$. With confirmed $R_S = 0.1\,\Omega$ and desired peak limit 1.68 A, $V_{\text{REF}} = 8(0.1)(1.68) = 1.344$ V [69]. The factor 8 belongs to the driver’s internal current-sense scale. It is not the microstep factor, number of coils or supply voltage. For a different carrier with $R_S = 0.068\,\Omega$, the same limit would require 0.914 V. Pololu explicitly warns that carrier sense resistance varies [70]. Thus “R10 means 0.1 ohm” is a component-identification assumption requiring the actual board marking, not a universal carrier value.

This equation sets an electrical current limit; it does not by itself prove a thermally safe operating point. The final report’s main text claims an adjustment, whereas its connection table says actual V_{REF} remains unmeasured. This edition retains that as unresolved provenance. The external supply’s displayed current is not coil current: chopping and energy storage make input and winding currents different quantities. Full-step two-coil current also differs from the peak microstep limit.

STEP and DIR.
STEP’s active edge advances the translator; DIR chooses sequence direction. Logical inversion maps positive software travel to the installed motor orientation. The datasheet lists minimum STEP high and low times of 1 microsecond, and DIR setup/hold of 200 ns. These are electrical minimums, not guarantees of Linux scheduling precision.

MS1, MS2, MS3.
Select microstep mode; all high selects 1/16 in the manufacturer’s truth table. The code assumes this setting; it does not read back physical straps.

ENABLE, RESET, SLEEP.
Enable controls bridge output state, reset returns the translator to a defined home state, and sleep reduces activity. Their installed wiring is not fully documented in the final report. Do not infer GPIO assignments merely from the driver’s pin list.

VDD and GND.
Logic supply and common signal reference, distinct from motor supply. A shared reference is required for meaningful logic levels.

VMOT / VBB and winding outputs.
Motor supply feeds the bridge; 1A/1B and 2A/2B connect to winding pairs. They carry switched winding current, not GPIO STEP waveforms. Reversing a winding pair changes orientation.

VREF and SENSE.
Reference and current-sense network establish the regulated limit. They are analog/current-control nodes, not ordinary logic commands. Carrier capacitors and charge-pump components have support roles described in the datasheet; no undocumented installed values are assumed.

11.1.4 BC337 low-side switch, base resistor and switch pull-down

In a low-side NPN arrangement, motor positive goes to its supply, motor negative to collector, emitter to common ground, and GPIO through a base resistor. GPIO supplies base current, not motor current. The approximate base relation is $I_B = (V_{\text{GPIO}} - V_{BE})/R_B$ [71]. Assuming 3.3 V, $V_{BE} = 0.7$ V and the report's stated 4 mA limit, $R_{B,\min} = (3.3 - 0.7)/0.004 = 650\,\Omega$. The limit is a *minimum resistance*; lowering it raises GPIO current. At $R_B = 1$ kΩ, nominal $I_B = 2.6$ mA and resistor dissipation is $I_B^2 R_B = 6.76$ mW. Output-high droop and actual saturation V_{BE} change that estimate.

The transistor's active-region DC gain is not a guaranteed saturation gain. At a forced gain of ten, 2.6 mA base current corresponds to a 26 mA load check. The report lists motor rated current 30 mA and stall current 230 mA; these require forced gains 11.5 and 88.5 respectively at that nominal base current. This arithmetic alone therefore does not establish saturation at either load. The onsemi saturation-test conditions should be consulted directly; automated retrieval was blocked [72]. No circuit change or pressure fix is performed here.

An inductive motor stores $E = \frac{1}{2}LI^2$. Interrupting current produces $v = L\,di/dt$, explaining why turn-off behaviour differs from a resistor. The report records no fitted flyback diode; this manual does not invent one. The physical transient and transistor dissipation cannot be quantified without motor inductance, current and switching measurements.

For an active-high switch with 10 kΩ pull-down, an open input is held near ground; closure connects it to the high rail. At 3.3 V, closed pull-down current is 0.33 mA and resistor power 1.089 mW. For input leakage I_{leak}, open-node error is approximately $I_{\text{leak}}R$; cable/input capacitance gives time constant RC. A weaker pull-down saves current but increases susceptibility to leakage and coupled noise. Ten kilohms is a common engineering scale, not a value proved optimal by these equations. A base-emitter pull-down has a different purpose from a switch-input pull-down; the report does not fully establish the placement of its stated base pull-down.

11.1.5 All documented controller pins and supply connections

Header number means physical connector position; line offset means a GPIO line within the requested chip. They are not interchangeable [73]. The local source uses `/dev/gpiochip0`. The following mapping agrees with report Table 17 and inspected source, not a new electrical pin test. The manufacturer expansion reference was inaccessible to the automated reader [74].

Header	Line	Signal	Role
7	64	DIR1	X direction output
11	68	STEP1	X step output
12	81	DIR2	Y direction output
13	69	STEP2	Y step output
15	72	X minimum	Active-high switch input
16	65	X maximum	Active-high switch input
18	66	Y minimum	Active-high switch input
22	67	Y maximum	Active-high switch input
33	70	Pen switch	Raised-state feedback input
35	71	Pen motor	Transistor-base control output

The report separately lists USB camera data/power, network communication, ODROID DC input, external 24 V to driver motor supply, a dedicated 3.3 V rail for logic/pen supply, and common grounds. It does not record exact physical power/ground pins, complete return routing, actual adapter output or total 3.3 V rail load. USB and Ethernet are interfaces, not spare GPIO numbers. The quoted 4 mA GPIO capability is conditional because the report leaves its specific source unresolved; it must not be inferred from the stepper driver or BC337 datasheet.

11.1.6 Pen state machine, dwell, overrun and debounce

The pen motor turns one way and the cam alternates states. The conceptual state machine is: check interlocks; read current state; if already at target, do nothing; otherwise energise; poll until target or timeout; optionally continue the pen-up seating overrun; de-energise; report success/failure. The inspected `SetPen` compares feedback with the requested target, not always "up" as one report pseudocode line suggests. A switch reporting not-up does not measure nib contact force. The mechanism can satisfy a binary switch while pen pressure remains unsuitable.

The 40 ms up-overrun, 2 ms polling sleep, 3 s timeout and 350 ms G-code settling dwell serve different roles. The report attributes dwell to visual tuning; it supplies no formal optimum for each. At an illustrative unloaded 150 RPM, a quarter-turn takes 0.1 s, but gearing load, cam contact and switch position make this unsuitable as a measured timing prediction. The same 350 ms dwell need not represent the same physical settling process for raising and lowering.

Endstop confirmation requires three high reads, two gaps of 6 ms, hence 12 ms from first to last sample. Under an unrealistic independent spurious high probability q, three samples would give q^3. Real contact bounce is time-correlated, so this is not a reliability estimate. Debounce rejects short chatter at a cost of latency; it cannot ensure switches are "never" triggered by noise. The pen feedback routine is distinct from the debounced endstop routine. At nominal 7.8125 mm/s, 12 ms corresponds to 0.09375 mm of travel *if motion continued*; the inspected polling can pause pulses during confirmation, so that multiplication is not a measured stopping distance.

11.1.7 Calibration and G-code motion mapping

For each axis, move to a switch, reverse, count N pulses to the other switch, use measured travel L to obtain $K = N/L$ steps/mm, establish which end is minimum/maximum, then retract 5 mm. With $L = 200.6$ mm, the two 5 mm end clearances give 190.6 mm usable span; the recorded Y travel 261.3 gives 251.3 mm. The 5 mm is a clearance choice, not an inferred switch precision. Homing at 1.6 ms per nominal step gives $1/(0.0016 \cdot 80) = 7.8125$ mm/s before overhead. The 200000-step bound is a failure guard, not measured travel.

Calibration estimates scale from an external length and reference contacts. It does not calibrate squareness, backlash or flex. In $K = N/L$, errors in L propagate approximately as $\delta K/K \simeq -\delta L/L$ for fixed N. Calibrating against the same length later used for acceptance can conceal systematic error, so provenance of independent checks matters.

G21 selects millimetres; G90 absolute positions; G0/G1 specify linear targets with feed handling; F is mm/min, so 900 becomes 15 mm/s and 2400 becomes 40 mm/s. G4 P is a dwell with this project's P convention in seconds. M3/M5 are repurposed here to lower/raise the pen; in other controllers they often refer to spindle control. M2 terminates. Modal settings persist until changed. Parsing a command and updating a commanded counter does not verify that the carriage reached its target.

11.1.8 Bresenham movement and how to read its staircase

For integer pulse displacements (d_x, d_y), set $a_x = |d_x|, a_y = |d_y|$ and direction signs. If $a_x \geq a_y$, initialise $e = \lfloor a_x/2 \rfloor$. At each X pulse subtract a_y; if $e < 0$, also pulse Y and add a_x. Swap axes for a steep line. This distributes minor pulses along major pulses with integer arithmetic [75].

For $(5, 2)$, errors after each complete iteration are $0, 3, 1, 4, 2$ and the endpoint sequence is

Formula purpose and system use. Purpose in this explanation: Bresenham movement and how to read its staircase **Source basis:** [3, 69]. Exact project constants remain implementation choices unless evidence is stated.

$$(1, 0),\ (2, 1),\ (3, 1),\ (4, 2),\ (5, 2).$$

The ideal Y coordinates are $0.4, 0.8, 1.2, 1.6, 2.0$; discrepancies are $-0.4, 0.2, -0.2, 0.4, 0$ steps. Signs reflect axes; the absolute deviations are at most half a minor step in this example. The plot's staircase shows the command lattice, not measured carriage vibration. An equal-axis move pulses both axes per iteration. A zero-length move needs no pulses.

Geometry and timing are distinct. The inspected routine pulses major and minor axes sequentially, sleeping for each high pulse. Approximately $a_x + a_y$ pulse sleeps are therefore incurred. For nominal path length L and feed v, a naive N_{major}-based delay can make diagonal motion slower unless this sequential cost is accounted for. GPIO calls, debounce and OS scheduling add overhead. Bresenham alone supplies neither acceleration control nor hard-real-time timing.

11.1.9 Bounds, joins and nominal timing budget

For polyline vertices $p_i = (x_i, y_i)$, bounds are the coordinate minima and maxima. Straight segments lie within their endpoint box, so all vertices inside a rectangular workspace imply the interpolated straight segments do too. This does not cover pen radius, calibration uncertainty, cam geometry or curved interpolation extrema. The empty-trajectory zero box is a convention, not a drawn dot at the origin.

Pen-down length is $L_d = \sum \|p_{i+1} - p_i\|$ within strokes; pen-up travel is the distance between successive stroke endpoints. At feeds F_d, F_u in mm/min, a nominal time is $t_0 = 60L_d/F_d + 60L_u/F_u + N_{\text{toggle}}\tau$ plus any explicitly counted dwells. For $L_d = 300$ mm, $L_u = 120$ mm, feeds 900/2400 and twenty 0.35 s dwells, $t_0 = 20 + 3 + 7 = 30$ s. This is constructed ideal accounting, not an ODROID prediction. Avoid double-counting a dwell already included in a toggle term. Acceleration, sequential pulses, software overhead and generation time explain why measured full-request timing differs.

The 0.6 mm endpoint-join threshold avoids a lift/drop pair by drawing a short connector. It trades fewer toggles for a possible extra mark; at $H = 4$ mm it is 0.15 x-heights. It is not a Douglas–Peucker error bound. Ring closure, decimation and stroke-order optimisation likewise change different aspects of a trajectory. Their combined output must be considered when interpreting physical lengths or geometric fidelity; none is tuned or tested by this document-only expansion.

Current function reference

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
_EXTENT_HI_PCT = 98.0
_EXTENT_LO_PCT = 2.0
CHIP_PATH = '/dev/gpiochip0'
DIR1_PIN, STEP1_PIN = (64, 68)
DIR2_PIN, STEP2_PIN = (81, 69)
X_MIN_PIN, X_MAX_PIN = (72, 65)
Y_MIN_PIN, Y_MAX_PIN = (66, 67)
PEN_SWITCH_PIN = 70
```

```
PEN_MOTOR_PIN = 71
ENDSTOP_PINS = {'X_MIN': X_MIN_PIN, 'X_MAX': X_MAX_PIN, 'Y_MIN': Y_MIN_PIN, 'Y_MAX': Y_MAX_PIN}
INVERT_X = True
INVERT_Y = True
MICROSTEPS = 16
STEPS_PER_REV = 200 * MICROSTEPS
STEPS_PER_MM = 80.0
MIN_STEP_DELAY_S = 2e-06
X_SWITCH_TRAVEL_MM = 200.6
Y_SWITCH_TRAVEL_MM = 261.3
EDGE_TOLERANCE_MM = 5.0
HOMING_STEP_DELAY_S = 0.0008
HOMING_MAX_STEPS = 200000
ENDSTOP_DEBOUNCE_S = 0.006
ENDSTOP_CONFIRM_SAMPLES = 3
RETRACT_MM = 5.0
PEN_UP_OVERRUN_S = 0.04
GCODE_MIN_FEED_MM_S = 0.1
_GCODE_WORD_RE = re.compile('([A-Za-z])\\s*(-?[0-9]*\\.?[0-9]+)')
_RETREAT_SIGN = {'X_MIN': +1, 'X_MAX': -1, 'Y_MIN': +1, 'Y_MAX': -1}
```

Stage: Coordinate mapping and path planning

Report equation 1 Belt travel and pitch radius

Method | Formula

$\ell_{\mathrm{rev}} = N_t p = 40\,\mathrm{mm/rev}$ and $r = \ell_{\mathrm{rev}}/(2\pi) = 6.366\,\mathrm{mm}$.

Purpose | Why the system needs it

Translate motor rotation to belt motion and establish the moment arm for torque. The current configuration uses 20 teeth, 2 mm pitch and 40 mm per revolution. The pitch radius is not necessarily the pulley outer radius.

Evidence | Current location and source

GantryConfig and its steps-per-mm properties. Geometry derived from the stated transmission; motor/driver relations are conditional on actual hardware. [69, 3].

Report equation 2 Linear speed to motor speed

Method | Formula

$n = v/\ell_{\mathrm{rev}}$, so $4000/40 = 100$ revolutions per minute.

Purpose | Why the system needs it

Compare a proposed traverse rate with a speed-dependent motor curve. Millimetres per minute divided by millimetres per revolution yields revolutions per minute. It is a design calculation, not a measured maximum feed.

Evidence | Current location and source

Mechanical sizing and _GcodeLinearMove feed interpretation. [3, 68].

Report equation 3 Torque curve interpolation

Method | Formula

$T(n) = T_1 + (n - n_1)(T_2 - T_1)/(n_2 - n_1)$.

Purpose | Why the system needs it

Estimate between two verified curve samples. The report's $(60, 7)$ and $(120, 21)$ RPM/N cm points give 16.33 N cm at 100 RPM, but its curve citation is literally left as SOURCE. The arithmetic is reproducible; the points and their operating conditions remain unauthenticated.

Evidence | Current location and source

Report hardware justification, not a torque lookup in the current controller. [10, 68].

Report equation 4 Steady opposing load

Method | Formula

$T = r\mu mg = 0.0002373\,\mathrm{N\,m} = 0.02373\,\mathrm{N\,cm}$ under the report inputs.

Purpose | Why the system needs it

Estimate one steady-friction term. The assumed $\mu = 0.004$ is not an observed assembly coefficient. Acceleration, pen drag, belt forces and rotary inertia are omitted, so this cannot establish the available dynamic margin.

Evidence | Current location and source

Report mechanical sizing; the fuller load discussion is in the motion chapter. [68, 10].

Report equation 5 Command lattice and rounding

Method | Formula

$K = 200(16)/40 = 80\,\mathrm{steps/mm}$ and $\Delta = 1/K = 0.0125$ mm.

Purpose | Why the system needs it

Relate integer pulse targets to millimetres. Nearest rounding gives at most $\Delta/2$ per axis and $\sqrt{2}\Delta/2$ endpoint distance for equal axes. These are quantisation bounds, not loaded microstep accuracy or measured reproduction precision.

Evidence | Current location and source

GantryConfig, Calibrate and _GcodeLinearMove. [69, 3].

Report equation 6 Driver current limit

Method | Formula

$I_{\mathrm{trip,max}} = V_{\mathrm{REF}}/(8R_S)$; with $R_S = 0.1\,\Omega$ and 1.68 A, $V_{\mathrm{REF}} = 1.344$ V.

Purpose | Why the system needs it

Set a conditional electrical current limit. Eight is the driver's internal sense scaling, not the microstep setting. Installed sense resistance, actual reference voltage and thermal conditions must be established separately.

Evidence | Current location and source

Hardware setting, not a software-assigned phase current. [69, 70].

Report equation 32 Machine placement

Method | Formula

$x_m = o_x + m_x + a(x - x_0)$, $y_m = o_y - m_y - d_y - a(y_1 - y) + \Delta y$.

Purpose | Why the system needs it

Place the trajectory within margins and fit to bounds. The current ToMachine retains scaled synthesis pitch and removes measured row-centre wander; it has no ignoreRuling choice. Both frames here are y-up.

Evidence | Current location and source

ToMachine and _Clamp. [3, 25].

Alternatives | Mechanisms lifts and control

A Cartesian gantry maps orthogonal axes directly; serial arms require coupled kinematics and can vary stiffness over the workspace; parallel mechanisms trade workspace and coupling differently. The report's one-way geared cam replaces a position-commanded servo or third stepper with a switched motor and binary raised-state feedback. A4988 current chopping controls winding current; it is not the same as the controller's STEP sequence. Open-loop interior motion and switch-referenced homing are not continuous position feedback. These mechanical trade-offs are discussed here because they determine the meaning of the software calculations, not because another mechanism was built or tested.

Evidence | System decisions mentioned outside the algorithms

The camera, frame/clamps, Cartesian mechanism, geared lift, endstops, transistor and power/driver connections influence assumptions in the software and are covered in the motion/hardware explanation. Ergonomics, cost, maintainability, privacy and environmental discussions are report design context rather than additional image-processing functions. They do not create undocumented sensor feedback, trained classes or software safety guarantees.

11.2 Method background: Coordinate mapping and path planning

Purpose | The role in this system

Place and fit the generated trajectory into the measured machine work area.

Justification | The reason for this construction

Uniform scaling preserves shape while margins/origin define placement. Row grouping and nearest-end stroke order reduce travel; clamping makes out-of-bounds points explicit.

Alternatives | What changes if another method is used

Anisotropic fitting distorts glyphs. Optimal travelling-salesperson routing is more expensive and still must respect stroke/row semantics. A fixed sequence preserves order but may travel farther.

Library | Implementation speed and accuracy

Geometry libraries provide affine transforms and route solvers, but hardware bounds and pen semantics remain application-specific. Algorithmic path length is different from measured motion time. [75, 3].

11.3 GantryConfig

```
class GantryConfig
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 18. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Current fields include gearing, bounds, origin, margins, ruling fractions, feeds, decimation, pen dwell and endpoint/ring join distances. They do not include ignoreRuling, fitToRuling or accelMmS2. Properties derive commanded steps/mm from 200 full steps, sixteen microsteps and forty millimetres per revolution; controller calibration later uses measured per-axis factors.

Method | Methods exposed by this current class

```
def __post_init__(self)
def usableWidthMm(self)
def microstepsPerRev(self)
def stepsPerMmX(self)
def stepsPerMmY(self)
```

Method | Current implementation

Place and fit the generated trajectory into the measured machine work area. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class GantryConfig:
    fullStepsPerRev: int = 200   #NEMA 17
    microstepping: int = 16   #A4988 MS1/MS2/MS3 set to 1/16
    mmPerRevX: float = 40.0   #GT2 2 mm pitch, 20 teeth
    mmPerRevY: float = 40.0
    boundsMinXmm: float = 0.0
    boundsMinYmm: float = 0.0
    boundsMaxXmm: float = 0.0
    boundsMaxYmm: float = 0.0
    originXmm: float = 0.0
    originYmm: float = 0.0   #zero means the top of the calibrated area
    pageMarginXmm: float = 15.0
    pageMarginYmm: float = 10.0
    lineHeightMm: float = 8.4
    lineOffsetMm: float = 0.0
    rulingXhFrac: float = 0.46
    rulingMaxExtent: float = 1.25
    drawFeedMmMin: float = 900.0
    travelFeedMmMin: float = 2400.0
    minSegmentMm: float = 0.12   #shorter moves are collapsed
```

```
    penUpDwellMs: float = 350.0
    penDownDwellMs: float = 350.0   #longer, to let the nib seat before drawing
    penUpCode: str = 'M5'
    penDownCode: str = 'M3'
    penStartsUp: bool = True
    joinStrokesMm: float = 0.6
    closeRingMm: float = 0.6

    def __post_init__(self):
        usableW = X_SWITCH_TRAVEL_MM-2*EDGE_TOLERANCE_MM
        usableH = Y_SWITCH_TRAVEL_MM-2*EDGE_TOLERANCE_MM
        self.boundsMaxXmm = (usableW if self.boundsMaxXmm <= 0
                             else min(self.boundsMaxXmm, usableW))
        self.boundsMaxYmm = (usableH if self.boundsMaxYmm <= 0
                             else min(self.boundsMaxYmm, usableH))
        if self.originYmm <= 0:
            self.originYmm = self.boundsMaxYmm

    @property
    def usableWidthMm(self):
        return self.boundsMaxXmm-self.originXmm-2*self.pageMarginXmm

    @property
    def microstepsPerRev(self):
        return self.fullStepsPerRev*self.microstepping

    @property
    def stepsPerMmX(self):
        return self.microstepsPerRev/self.mmPerRevX

    @property
    def stepsPerMmY(self):
        return self.microstepsPerRev/self.mmPerRevY
```

Justification | Interpret the method rather than infer a benchmark

Uniform scaling preserves shape while margins/origin define placement. Row grouping and nearest-end stroke order reduce travel; clamping makes out-of-bounds points explicit. [75, 3].

11.4 XhForRuling

```
def XhForRuling(profile, cfg, text=None, default=4.0, probe=True)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 91. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Chooses x-height before line wrapping from profile xhOverPitch if between 0.15 and 0.90, otherwise rulingXhFrac, multiplied by lineHeightMm. An optional plain synthesis probe reduces size if the tallest line exceeds rulingMaxExtent. It supplies a size constraint, not guaranteed actual line spacing.

Method | Current implementation

Place and fit the generated trajectory into the measured machine work area. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def XhForRuling(profile,cfg,text=None,default=4.0,probe=True):
    ruling = float(getattr(cfg,'lineHeightMm',8.4))
    frac = float(profile.get('xhOverPitch',0.0) or 0.0)
    if not 0.15<=frac<=0.90:
        frac = float(getattr(cfg,'rulingXhFrac',0.46))
    xh = ruling*frac
    if not probe or not text:
        return xh
    try:
        import synthesise as SYN
        t = SYN.SynthesizeText(text,profile,mmPerXh=xh,seed=0,
                               lineWidthMm=getattr(cfg,'usableWidthMm',180.0))
        got = _TallestLine(t)
        cap = ruling*float(getattr(cfg,'rulingMaxExtent',1.60))
        if got>cap>0:
            xh *= cap/got
    except Exception:
        pass   #a probe that will not run must not stop a job being written
    return xh
```

Justification | Interpret the method rather than infer a benchmark

Uniform scaling preserves shape while margins/origin define placement. Row grouping and nearest-end stroke order reduce travel; clamping makes out-of-bounds points explicit. [75, 3].

11.5 _TallestLine

```
def _TallestLine(traj)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 120. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Groups trajectory strokes around the stored synthesis pitch and measures each row extent for the size cap. Grouping can be imperfect when stroke geometry spans neighbouring rows; the returned extent is a geometric proxy.

Method | Current implementation

Place and fit the generated trajectory into the measured machine work area. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _TallestLine(traj):
    pitch = float(traj.meta.get('lineStepMm',0.0) or 0.0)
    rows = {}
    for s in traj.strokes:
        if not s:
            continue
        ys = [p[1] for p in s]
        if pitch>1e-6:
            k = round(-(sum(ys)/len(ys))/pitch)
            base = -k*pitch
        else:
            k,base = 0,0.0
        rows.setdefault(k,[]).append((max(ys)-base,min(ys)-base))
    if not rows:
```

```
        return 0.0
    worst = 0.0
    for v in rows.values():
        tops = np.array([t for t,_b in v],dtype=np.float64)
        bots = np.array([b for _t,b in v],dtype=np.float64)
        if tops.size<8:   #too few strokes for a percentile to mean anything
            worst = max(worst,float(tops.max()-bots.min()))
        else:
            worst = max(worst,float(np.percentile(tops,_EXTENT_HI_PCT)
                                    -np.percentile(bots,_EXTENT_LO_PCT)))
    return worst
```

Justification | Interpret the method rather than infer a benchmark

Uniform scaling preserves shape while margins/origin define placement. Row grouping and nearest-end stroke order reduce travel; clamping makes out-of-bounds points explicit. [75, 3].

11.6 ToMachine

```
def ToMachine(traj, cfg, fitToBounds=True, optimizeOrder=True, rowTolMm=None)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 151. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Uniformly shrinks only when work-area width/height requires it, then uses each detected line centre to remove baseline wander while retaining synthesis pitch times scale. There is no selectable ignoreRuling branch. Coordinates remain y-up; top alignment and margin/offset subtraction do not introduce a second image-axis flip.

Method | Current implementation

Place and fit the generated trajectory into the measured machine work area. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ToMachine(traj,cfg,fitToBounds=True,optimizeOrder=True,rowTolMm=None):
    x0,y0,x1,y1 = traj.Bounds()
    w,h = max(1e-6,x1-x0),max(1e-6,y1-y0)
    scale = 1.0
    mx,my = cfg.pageMarginXmm,cfg.pageMarginYmm
    off = max(0.0,float(getattr(cfg,'lineOffsetMm',0.0)))
    if fitToBounds:
        availW = cfg.boundsMaxXmm-cfg.originXmm-2*mx
        availH = (cfg.originYmm-my-off)-cfg.boundsMinYmm
        scale = min(1.0,availW/w,availH/h)
    pitch = float(traj.meta.get('lineStepMm',0.0) or 0.0)
    ruledPitch = pitch*scale
    respace = pitch>1e-6 and int(traj.meta.get('lines',1) or 1)>1
    centres = {}
    if respace:
        rows = {}
        for s in traj.strokes:
            if not s:
                continue
            k = round(-(sum(p[1] for p in s)/len(s))/pitch)
            ys = [p[1] for p in s]
```

```
            lo,hi = rows.get(k,(1e18,-1e18))
            rows[k] = (min(lo,min(ys)),max(hi,max(ys)))
        if rows:
            k0 = min(rows)   #the topmost line stays where it is
            c0 = (rows[k0][0]+rows[k0][1])*0.5
            for k,(lo,hi) in rows.items():
                ck = (lo+hi)*0.5
                centres[k] = (c0-ck)*scale-(k-k0)*ruledPitch
    out = []
    for s in traj.strokes:
        shift = 0.0
        if respace and s:
            k = round(-(sum(p[1] for p in s)/len(s))/pitch)
            shift = centres.get(k,0.0)
        q = [(cfg.originXmm+mx+(x-x0)*scale,
              cfg.originYmm-my-off-(y1-y)*scale+shift)
             for (x,y) in s]
        out.append(_Decimate(q,cfg.minSegmentMm))
    if optimizeOrder:
        out = OptimizeStrokeOrder(out,rowTolMm=rowTolMm)
    return out,scale
```

Justification | Interpret the method rather than infer a benchmark

Uniform scaling preserves shape while margins/origin define placement. Row grouping and nearest-end stroke order reduce travel; clamping makes out-of-bounds points explicit. [75, 3].

11.7 _Decimate

```
def _Decimate(poly, minSeg)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 198. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Place and fit the generated trajectory into the measured machine work area.

Inputs

`poly`
: a path in millimetres

`minSeg`
: the shortest segment to keep, in millimetres

Output. the path with points closer together than that merged away.

Purpose | What it is for

A path may carry far more points than the machine can act on. A traced stroke has a point per pixel, and resampling during layout placed points every 0.35 mm. This removes the points that are closer together than the machine can resolve.

Justification | The method, and why it is not Douglas-Peucker

Each point is kept only if it is at least the minimum distance from the last point kept. This is a different rule from the path simplification in style.py, which keeps points according to how far they

stray from a straight line, and the difference is appropriate to the different purpose. Here the question is not whether a point carries shape information but whether the machine can act on it at all: a commanded move shorter than the minimum segment is below the resolution of the motion, so it costs a line in the file and produces nothing.

Justification | Why the comparison is against the last kept point

Comparing against the last kept point rather than the previous input point is what makes the rule accumulate correctly. Against the previous input point, a long run of points each slightly under the threshold would all be dropped, however far the path travelled in total. Against the last kept point, the distance accumulates until the threshold is reached, so a point is emitted every minimum segment along the path regardless of how finely the input was sampled.

> *The first and last points are always kept, whatever the spacing. The endpoints define where a stroke begins and ends, so dropping one would shorten the stroke, and the final point in particular is where the pen lifts and where the next stroke's travel is measured from.*

11.8 OptimizeStrokeOrder

```
def OptimizeStrokeOrder(strokes, rowTolMm=None)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 212. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Orders the strokes in bands down the page and, inside a band, takes the nearest unvisited stroke next, which cuts the pen-up travel.

Inputs

`strokes`
: the strokes in bed coordinates

`rowTolMm`
: how far apart two strokes may sit vertically and still count as the same row; derived if not given

Output. the same strokes in an order that reduces pen-up travel.

Purpose | What it is for

The strokes may be drawn in any order, since each is independent. The order determines how far the pen travels raised between them, and that travel is pure waste: it takes time and draws nothing. For a page of handwriting with hundreds of strokes the difference between a good order and a poor one is substantial.

Purpose | Background: this is a travelling salesman problem

Finding the order that minimises total travel is the **travelling salesman problem**: given a set of places, find the shortest route visiting all of them. It is one of the classic hard problems in computing, with no known method that finds the exact best route in reasonable time as the number of places grows. With several hundred strokes an exact answer is out of reach, so a **heuristic** is used: a rule that gives a good answer quickly with no guarantee of being best.

Method | The heuristic: bands, then nearest neighbour

Two ideas are combined. The strokes are first grouped into horizontal bands by their vertical centres, and the bands are processed from top to bottom. Within a band, the nearest unvisited stroke is taken next.

- **Banding** exploits the structure of the problem. Text is arranged in lines, and almost all the strokes of one line of writing are near each other, so a route that finishes one line before starting the next is close to optimal. A pure nearest-neighbour search without banding can wander between lines, since a stroke on the line below may happen to be nearer than the next stroke along.
- **Nearest neighbour** within a band is the standard simple heuristic: from where the pen is, go to the closest place not yet visited. It is greedy, so it can paint itself into a corner and leave a long move to the last stroke, but within one line of text the strokes are nearly collinear and the greedy route is close to the best.

Justification | Why the band tolerance is derived from the strokes

Formula purpose and system use. Place and fit the generated trajectory into the measured machine work area. **Source basis:** [75, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{rowTol} \;=\; \max\left(1,\; 1.2 \cdot \text{median}\{\, h_s \,\}\right) \tag{11.1}$$

where

rowTol
: how far apart two strokes' vertical centres may be and still share a band, in millimetres

h_s : the height of stroke s, from its lowest point to its highest

1.2 : a margin above the typical stroke height

1 : a floor in millimetres, for a drawing of very small strokes

A fixed tolerance would be wrong at every size but one. Deriving it from the median stroke height ties it to the writing: a band should be about as tall as a line of text, and a typical stroke is about the height of a letter. The median again rather than the mean, so that one tall stroke spanning several lines, which a long descender or an underline can produce, does not widen every band.

Method | Reversing a stroke to enter from the nearer end

A stroke can be drawn from either end, and the result is identical ink. So when the far end is closer to the pen than the near end, the stroke is reversed. The code comment notes that this is free for a plotter, and it is worth saying why: unlike a milling cut or an extrusion, a pen stroke has no direction-dependent behaviour, so nothing is lost. It can nearly halve the travel to a long stroke.

> *Fewer than three strokes are returned unchanged. With two strokes there is only one meaningful order, and the banding statistics would be computed from a sample too small to describe anything.*

11.9 _Clamp

```
def _Clamp(strokes, cfg)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 252. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Place and fit the generated trajectory into the measured machine work area.

Inputs

`strokes`
: the strokes in bed coordinates

`cfg` the machine configuration

Output. a pair: the strokes with every point forced inside the work area, and how many points were moved.

Purpose | What it is for

A final safety net. Every point is forced inside the machine's declared bounds, and the number of points that had to be moved is counted and reported.

Justification | Why it exists when ToMachine already fits the drawing

The fitting scales the drawing to the available area, so in principle nothing can lie outside it. This clamp exists for the cases where that reasoning does not hold: the fitting can be switched off by the caller, the origin can be set such that the available area is computed wrongly, and a stroke can be reversed or reordered by code that has no knowledge of the bounds. A point outside the work area is not a cosmetic problem; it commands the machine to drive into its own frame.

Justification | Why the count is returned

Clamping silently would hide a real fault. A drawing that needed clamping was mispositioned, and the clamped version is not what was asked for: the affected strokes are flattened against the boundary, which is visible as a straight edge where writing should be. Returning the count lets the writer emit a warning into the file, so the condition is recorded where someone will see it rather than inferred from the drawing.

> *Clamping each coordinate independently is not geometrically principled: a stroke crossing the boundary diagonally is bent rather than cut, so its shape changes. Properly clipping the path to the rectangle would preserve the shape of the part inside. That is not done because the condition is a fault to be reported rather than a case to be handled well, and the simple form cannot itself fail.*

11.10 _ClampToUsableArea

```
def _ClampToUsableArea(xMm, yMm)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 819. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Place and fit the generated trajectory into the measured machine work area.

Inputs

`xMm` the target X coordinate

`yMm` the target Y coordinate

Output. the coordinates forced inside the homed usable area.

Justification | Why the machine clamps as well as the writer

The G-code writer already clamps to the configured bounds. This clamps again, against the *measured* usable area, and the two are not the same check. The writer's bounds are configuration, stated in advance; this one is what calibration actually found, which may be smaller, and it applies to any file whatever its origin, including a hand-written one or one from another program.

The principle is that the machine must protect itself rather than trusting its input. A clamp at the point of motion is the last line of defence and the only one that cannot be bypassed by using a different generator.

Justification | Why it prints when it acts

A clamp that fires means the file asked for something outside the machine, so the drawing will be wrong: strokes are flattened against the boundary. Printing the original and clamped coordinates makes that visible while the job runs, so the operator can stop it rather than discovering afterwards that part of the drawing is a straight line along one edge.

Stage: Homing and axis calibration

Report equation 10 Measured axis scale

Method | Formula

$K_{\text{axis}} = N/L$ and $x = n/K - 5$ mm in the clearance-offset frame.

Purpose | Why the system needs it

Use counted travel between contacts and an external length reference to establish position. The 5 mm offset reserves switch clearance; it is not a measurement of switch repeatability.

Evidence | Current location and source

Calibrate and _GcodeLinearMove use per-axis calibrated factors. [3, 75].

11.11 Method background: Homing and axis calibration

Purpose | The role in this system

Establish a reference and convert commanded pulse counts into machine coordinates.

Justification | The reason for this construction

Counting pulses between switches against measured travel estimates steps/mm; clearance retracts keep drawing away from the contacts.

Alternatives | What changes if another method is used

Nominal belt geometry avoids a measurement but misses actual scale errors. Encoders observe motion continuously at extra hardware cost. Multi-point calibration can assess squareness and backlash beyond one scale factor.

Library | Implementation speed and accuracy

Motion-control firmware can home and scale axes but still needs correct wiring, travel and configuration. A library cannot supply missing reference measurements. [69, 75, 3].

11.12 AxisDirValue

```
def AxisDirValue(invert, positive)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 564. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.11.

Purpose | What this contributes

Establish a reference and convert commanded pulse counts into machine coordinates.

Inputs

`invert`
: whether this axis's direction signal is reversed

`positive`
: whether the wanted motion is in the positive direction

Output. the level to set on the direction pin.

Method | What it resolves

Turns a wanted direction into a pin level, accounting for the axis's wiring inversion. It is the single place the inversion is applied, so every motion routine inherits it by asking for a direction by meaning rather than by level.

Formula purpose and system use. Establish a reference and convert commanded pulse counts into machine coordinates. **Source basis:** [69, 75, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{forward} = \text{positive} \oplus \text{invert} \tag{11.2}$$

where

forward
: whether the direction pin should be driven active

positive
: whether the wanted motion is toward increasing coordinates

invert
: this axis's wiring inversion flag

$\oplus$ **exclusive or**: true when exactly one of the two is true

Exclusive or is exactly the right operation, and it is worth checking the four cases. Wanting positive motion on an uninverted axis drives the pin active; wanting negative on the same axis drives it inactive. On an inverted axis both answers swap. So the output is active when the two inputs disagree, which is what exclusive or computes. Written as nested conditions it would be four branches that must each be right; as an exclusive or it is one expression that cannot be half correct.

11.13 _RetractOffSwitch

```
def _RetractOffSwitch(hit, ignore=frozenset())
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 569. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.11.

Purpose | What this contributes

Establish a reference and convert commanded pulse counts into machine coordinates.

Inputs

`hit` the name of the pressed switch

`ignore`
: other switches to disregard while retracting

Output. nothing; it steps the axis clear and updates the position count.

Method | What it does

Steps the affected axis away from the switch by the retract distance, at homing speed, updating the step count as it goes. The direction comes from the retreat table and the axis and pins from the switch's name.

Justification | Why it watches the other switches

Each step checks for any switch other than the one being escaped and the ones explicitly ignored, and stops if one trips. The code comment calls that a real fault, and it is: moving away from one limit should not approach another, so if it does, the machine's geometry is not what is believed, perhaps because a switch is stuck closed or an axis is shorter than its recorded travel. Continuing to drive would push against a second limit.

Justification | Why the position count is updated

The retract moves the carriage, so the recorded position must follow, or every subsequent absolute move is out by the retract distance. The count is accumulated during the loop and applied to the right axis afterwards, which also means a retract cut short by another switch updates by the distance actually travelled rather than the distance intended.

> *Retracting at homing speed rather than drawing speed is deliberate. The machine is at a limit, in a state it was not supposed to reach, and the geometry is not trusted. Moving slowly means the other-switch check has time to act before much further travel occurs.*

11.14 _StepAxis

```
def _StepAxis(stepPin, dirPin, dirValue, stopWhen, ignore=frozenset())
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 738. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.11.

Purpose | What this contributes

Establish a reference and convert commanded pulse counts into machine coordinates.

Inputs

`stepPin`
: the step pin for this axis

`dirPin`
: its direction pin

`dirValue`
: the level to drive the direction pin at

`stopWhen`
: a test saying which switch is the expected one

`ignore`
: switches to disregard

Output. the switch that stopped the motion and how many steps were taken.

Purpose | What it is for

Drives one axis in one direction until a switch stops it, counting the steps. It is the primitive the homing and calibration are built from, and the step count it returns is the measurement that calibration converts into steps per millimetre.

Justification | Why the stopping condition is a test rather than a name

The caller supplies a function saying which switch is the expected one, rather than naming it. That is necessary because homing begins without knowing which end of the axis it will reach: the carriage may be anywhere, so driving in one direction may bring it to either switch. The first call accepts either, and the second accepts only the opposite one.

Justification | Why an unexpected switch is an error rather than a stop

A switch that trips which is not the expected one raises an error naming the wiring, rather than being treated as a normal stop. The distinction matters because the step count is about to be used as a measurement: stopping on the wrong switch means the count does not measure what it is believed to, and a calibration built on it would scale every subsequent move wrongly. A measurement known to be invalid should fail rather than be used.

Justification | Why the step limit is also an error

Exceeding the step bound raises an error too. A correctly functioning machine reaches a switch in a fraction of that distance, so the bound being reached means no switch is going to trip, which is the disconnected-wiring case. Again the appropriate response is to stop and say so, since there is no sensible way to proceed without a reference position.

11.15 _CalibrateAxis

```
def _CalibrateAxis(stepPin, dirPin, minName, maxName, OtherAxisNames=frozenset())
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 756. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.11.

Purpose | What this contributes

Establish a reference and convert commanded pulse counts into machine coordinates.

Inputs

`stepPin`
: the step pin

`dirPin`
: the direction pin

`minName`
: the minimum switch's name

`maxName`
: the maximum switch's name

`OtherAxisNames`
: the other axis's switches, to disregard

Output. the first switch reached, the second, and the steps measured between them.

Method | The measurement

The axis is driven in one direction until either switch trips, then reversed until the opposite one does, counting the steps of the second move. That count is the full travel of the axis in steps, which divided by the known travel in millimetres gives the steps per millimetre.

Justification | Why the count comes from the second move

The first move starts from wherever the carriage happened to be and ends at a switch, so its length is arbitrary and carries no information. The second starts at a known position, one switch, and ends at another known position, the other switch, so its length is the distance between two physical references. Only a move between two known points measures anything.

Justification | Why the first switch is ignored during the reverse

The reverse move begins with the first switch still pressed, since the carriage is resting against it, and it stays pressed for the first few steps until the lever releases. The code comment calls this release lag and notes it is not a fault. Without adding that switch to the ignore set, the reverse move would immediately detect it, decide it was unexpected, and raise an error on every calibration.

Justification | Why the other axis is ignored throughout

The other axis may be resting against one of its own limits while this one is being calibrated, which is a perfectly normal state and nothing to do with this axis's motion. Ignoring its switches lets each axis be calibrated regardless of where the other happens to be parked.

11.16 Calibrate

```
def Calibrate()
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 775. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.11.

Purpose | What this contributes

Homes both axes, sets steps per millimetre from the travel it counts, and stores the usable area, no absolute move is accepted before this runs.

Output. a copy of the calibration record: the measured scaling for both axes and the usable area.

Method | What it establishes

Two things the machine cannot otherwise know: where it is, and how far a step moves it. Both come from driving to physical references, since the motors report neither.

Method | The sequence

- **Refuse if a switch is already pressed.** Calibration drives until a switch trips, so starting against one makes the first move meaningless.
- **Measure the X axis** between its two switches, and set its steps per millimetre from the count.
- **Retract off the switch** it finished against, so it is clear of its limit.
- **Measure the Y axis** the same way, ignoring the X switches.
- **Retract** again, record the usable area, and park at the safe corner.

Method | Where the position count comes from

After each axis, the step count is set to the full travel if the axis finished at its maximum switch, and to zero if at its minimum. That is the whole point of homing: the carriage is at a switch, the switch is at a known place, so the count can be set rather than guessed. Every subsequent absolute move is measured from it.

Method | The usable area and the switch-travel allowance

Formula purpose and system use. Establish a reference and convert commanded pulse counts into machine coordinates. **Source basis:** [69, 75, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{usable} = \text{switchTravel} - 2 \cdot \text{edgeTolerance} \tag{11.3}$$

where

usable
: the area the machine will address, per axis

switchTravel
: the hand-measured distance between the two switches

edgeTolerance
: the margin kept clear at each end

The usable area is deliberately smaller than the measured travel, for the reasons given under EDGE_TOLERANCE_MM. The coordinate origin is placed at the margin's distance in from the minimum switches, so a commanded zero is a position the machine is allowed to occupy rather than the switch itself.

Justification | Why it parks at the end

Calibration finishes with a move to the origin of the usable area. That leaves the machine at a known, legal position, clear of every switch, which is what a drawing job expects to start from. Finishing while resting against a limit would mean the first motion check of the next job found a tripped switch.

Stage: Pen state and switch confirmation

Report equation 7 Minimum base resistance

Method | Formula

$R_{B,\min} = (V_{\mathrm{GPIO}} - V_{BE})/I_{\mathrm{GPIO,max}} = 650\,\Omega$ for the stated assumptions.

Purpose | Why the system needs it

Limit GPIO base current. This is a minimum resistance, not a maximum. The report's 4 mA GPIO capability lacks a resolved specific source, and 0.7 V is an approximation; neither is certified by the arithmetic.

Evidence | Current location and source

Pen transistor circuit and PEN_MOTOR_PIN output. [71, 72, 10].

Report equation 8 Base current and saturation check

Method | Formula

$I_B = (3.3 - 0.7)/1000 = 2.6$ mA.

Purpose | Why the system needs it

Check the selected resistor against the stated GPIO budget and transistor drive. A forced gain of ten corresponds to 26 mA, below the listed 30 mA motor rated current and far below its 230 mA stall current. Active-region gain does not prove saturation.

Evidence | Current location and source

Pen switching, not a pressure controller. [71, 72, 10].

Report equation 9 Switch pull down current

Method | Formula

$I = V/R = 3.3/10000 = 0.33$ mA.

Purpose | Why the system needs it

Explain the closed-switch load and why an open logic node needs a defined level. Leakage and noise sensitivity constrain resistance too; the result does not prove that 10 kΩ is optimal.

Evidence | Current location and source

Endstop and pen feedback wiring. [73, 3].

11.17 Method background: Pen state and switch confirmation

Purpose | The role in this system

Avoid unnecessary one-way cam toggles and wait for the requested feedback state.

Justification | The reason for this construction

An idempotent state request does nothing when already at target; finite confirmation, timeout and motor-off paths bound actuation. Switch state does not measure contact pressure.

Alternatives | What changes if another method is used

A servo gives positional commands; a third stepper adds another axis; hardware debounce filters electrical transitions. These are hardware/design alternatives rather than interchangeable constants.

Library | Implementation speed and accuracy

GPIO libraries handle line access; they do not define cam geometry, debounce policy or pressure feedback. Hardware timing can be steadier than userspace sleeps. No new reliability or force measurement is inferred. [73, 71, 3].

11.18 NewPenState

```
def NewPenState(cfg)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 269. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.17.

Purpose | What this contributes

Avoid unnecessary one-way cam toggles and wait for the requested feedback state.

Inputs

`cfg` the machine configuration

Output. a small mutable record holding whether the pen is up and how many pulses have been emitted.

Justification | Why the pen's state must be tracked at all

This is the central peculiarity of the machine and it shapes the whole of the G-code writing. The pen is raised and lowered by a geared motor that turns 90 degrees per pulse. It is a **one-way actuator**: it accepts "move" and nothing else. There is no command meaning "be up", and nothing in the G-code can ask where the pen currently is.

The consequence is that the writer must know the pen's state by remembering it. The state begins at a known value, assumed at power-on, and every pulse emitted flips it. Emitting a toggle when the pen is already in the wanted state would move it to the wrong one, and from then on every pen command in the file would be inverted: the machine would travel with the pen down and draw with it up.

Justification | Why it is a list rather than a plain value

The record is a two-element list so that the function updating it can modify it in place and the caller sees the change. A tuple or an integer would be copied, and the caller would keep the old state. This is the smallest possible piece of shared mutable state and it exists because the pen genuinely has a state that persists across many lines of output.

> *The starting state is an assumption, not a measurement. If the machine is powered on with the pen down, every pen command in the file is inverted. The hardware half resolves this properly, with a switch that reports the pen's actual position, but a written file cannot: it has no way to ask.*

11.19 SetPenLines

```
def SetPenLines(state, cfg, wantUp)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 277. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.17.

Purpose | What this contributes

Maintains the generated-job pen state and emits M3/M5 plus direction-specific dwell only when the requested state differs. It models commanded state; physical switch confirmation belongs to SetPen, not this G-code construction helper.

Method | Current implementation

Avoid unnecessary one-way cam toggles and wait for the requested feedback state. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def SetPenLines(state,cfg,wantUp):
    if wantUp==state[0]:
        return []
    state[0] = wantUp
    state[1]+= 1
    code = cfg.penUpCode if wantUp else cfg.penDownCode
    dwell = (cfg.penUpDwellMs if wantUp else cfg.penDownDwellMs)/1000.0
    out = [f"{code}   ; pen {'UP' if wantUp else 'DOWN'} "
           f"(one 90deg pulse of the one-way gear motor)"]
    if dwell>0:
        out.append(f"G4 P{dwell:.3f}   ; wait for the 90deg rotation to finish")
    return out
```

Justification | Interpret the method rather than infer a benchmark

An idempotent state request does nothing when already at target; finite confirmation, timeout and motor-off paths bound actuation. Switch state does not measure contact pressure. [73, 71, 3].

11.20 TriggeredEndstop

```
def TriggeredEndstop(ignore=frozenset())
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 545. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.17.

Purpose | What this contributes

The name of a limit switch reading triggered, confirmed over several samples because a long wire picks up noise.

Inputs

`ignore`
: switch names to disregard

Output. the name of a switch confirmed to be pressed, or nothing.

Purpose | What it is for

The safety check, called before essentially every step of every motion. It reads each switch and returns the name of the first confirmed to be pressed.

Method | How the debounce is arranged

A switch reading open is dismissed immediately. One reading closed is then sampled twice more, spaced by the debounce delay, and is believed only if all three agree. The reasoning behind the spacing and the count is given under ENDSTOP_DEBOUNCE_S and ENDSTOP_CONFIRM_SAMPLES.

Justification | Why the asymmetry is what makes this affordable

This function runs before every step pulse, so its cost is multiplied by the tens of thousands of steps in a drawing. The early dismissal is what keeps that cost down: in the normal case every switch reads open and the function performs four pin reads and returns, with no delays at all. The twelve milliseconds of confirmation are spent only when a switch actually reads closed, which happens once per job at most.

Justification | Why some switches can be ignored

The ignore set exists because there are legitimate situations in which a switch is pressed and should not stop the machine. Retracting off a switch necessarily begins with that switch pressed, and homing one axis may hold a switch while the other axis is moved. Without the ability to disregard a named switch, the only remedy would be to omit the safety check entirely during those operations, which would remove it exactly where the machine is deliberately being driven against its limits.

> *The check returns at the first switch it confirms rather than examining all four. For stopping that is sufficient, since one tripped switch is reason enough. The separate _AllTriggeredEndstops exists for the emergency stop, which does need the full list so that it can retract off every one of them.*

11.21 _AllTriggeredEndstops

```
def _AllTriggeredEndstops()
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 560. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.17.

Purpose | What this contributes

Avoid unnecessary one-way cam toggles and wait for the requested feedback state.

Output. a list of every switch currently confirmed as pressed.

Justification | Why a complete list is needed

An emergency stop must retract off every pressed switch, not just the one that was noticed first. Two can be pressed at once: a carriage driven into a corner reaches an X and a Y limit together, and retracting off only one leaves the machine still held against the other, so the next motion check still finds a tripped switch and nothing can move.

Method | How it is built from the single check

Each switch is tested by calling the single check with every *other* switch in the ignore set, so the answer concerns that switch alone. It is a roundabout construction, but it reuses the debounce logic rather than duplicating it, which matters for a safety check: two implementations of a debounce could diverge, and the one in the rarely exercised path is the one that would not be noticed.

11.22 PenIsUp

```
def PenIsUp()
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 703. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.17.

Purpose | What this contributes

Avoid unnecessary one-way cam toggles and wait for the requested feedback state.

Output. whether the pen switch currently reads as up.

Purpose | What it is for

Reads the pen position switch. This is the capability the written G-code file lacks: the machine can ask where the pen is, so it never has to assume.

> *The read is not debounced, unlike the endstop check. That is acceptable because of how it is used: SetPen polls it repeatedly in a loop while the motor runs, so a single spurious reading advances the loop by one iteration and the next reading corrects it. A spurious endstop reading, by contrast, aborts the whole job, which is why that one is confirmed and this one is not.*

11.23 SetPen

```
def SetPen(targetUp, timeoutS=3.0, force=False)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 710. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.17.

Purpose | What this contributes

First respects endstop interlocks unless forced, returns immediately if already at the requested feedback state, then runs the one-way motor until target/timeout and adds up-overrun only after successful pen-up. Returns success/failure. It cannot measure force, and the current G-code runner logs pen-confirmation failure but continues.

Method | Current implementation

Avoid unnecessary one-way cam toggles and wait for the requested feedback state. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def SetPen(targetUp,timeoutS=3.0,force=False):
    hit = TriggeredEndstop()
    if hit is not None and not force:
        print(f"Refusing pen move -- end-stop {hit} still tripped.")
        return False
    if PenIsUp()==targetUp:
        return True

    req.set_value(PEN_MOTOR_PIN,Value.ACTIVE)
    deadline = time.time()+timeoutS
    reached = False
    while time.time()<deadline:
        if not force and TriggeredEndstop() is not None:
            break
        if PenIsUp()==targetUp:
            reached = True
            break
        time.sleep(0.002)
    if reached and targetUp:
        time.sleep(PEN_UP_OVERRUN_S)
    req.set_value(PEN_MOTOR_PIN,Value.INACTIVE)

    if not reached:
        print("Pen move did NOT confirm via switch -- check motor/wiring.")
    return reached
```

Justification | Interpret the method rather than infer a benchmark

An idempotent state request does nothing when already at target; finite confirmation, timeout and motor-off paths bound actuation. Switch state does not measure contact pressure. [73, 71, 3].

Stage: Bresenham stepping and feed timing

Report equation 11 Bresenham minor step count

Method | Formula

$m_k = \lceil [ka_y - \lfloor a_x/2 \rfloor]/a_x \rceil$ for the stated major-axis accumulator convention.

Purpose | Why the system needs it

Explain how integer error updates distribute minor-axis pulses along a line. Initial error and the strict negative test determine ties. This is spatial interpolation; sequential pulses and sleeps affect timing separately.

Evidence | Current location and source

BresenhamMove. [75, 3].

11.24 Method background: Bresenham stepping and feed timing

Purpose | The role in this system

Coordinate two integer step counts along one line using a bounded error accumulator.

Justification | The reason for this construction

Major-axis stepping distributes minor-axis pulses without a division per step. Feed conversion turns desired length and rate into a nominal pulse delay.

Alternatives | What changes if another method is used

Independent axis moves draw an L shape. DDA uses fractional accumulation. Dedicated motion firmware can add acceleration and more deterministic pulse timing.

Library | Implementation speed and accuracy

An image-library line drawer generates pixels, not GPIO pulses. A motion controller or kernel/hardware pulse engine is the relevant timing alternative. Geometry equivalence cannot establish equal scheduling, missed steps or physical accuracy. [75, 69, 73].

11.25 BresenhamMove

```
def BresenhamMove(dx, dy, stepDelayS=MIN_STEP_DELAY_S)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 663. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.24.

Purpose | What this contributes

The current default delay is MIN_STEP_DELAY_S, not the old circle-demo constant. It updates commanded counters after sequential GPIO pulses, checks abort/endstops per major iteration and distributes minor pulses through an integer error. A diagonal iteration can incur two pulse sleeps; the geometry algorithm alone does not enforce constant physical feed.

Method | Current implementation

Coordinate two integer step counts along one line using a bounded error accumulator. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def BresenhamMove(dx,dy,stepDelayS=MIN_STEP_DELAY_S):
    global CurrentXSteps,CurrentYSteps
    ax,ay = abs(dx),abs(dy)
    sx = 1 if dx>=0 else -1
    sy = 1 if dy>=0 else -1
    req.set_value(DIR1_PIN,AxisDirValue(INVERT_X,sx>0))
    req.set_value(DIR2_PIN,AxisDirValue(INVERT_Y,sy>0))

    def Pulse(stepPin):
        req.set_value(stepPin,Value.ACTIVE)
        time.sleep(stepDelayS)
        req.set_value(stepPin,Value.INACTIVE)

    if ax>=ay:   #x is the major axis
        err = ax//2
        for _ in range(ax):
            if abortEvent.is_set() or TriggeredEndstop() is not None:
                return
            Pulse(STEP1_PIN)
            CurrentXSteps+=sx
            err-=ay
            if err<0:   #the minor axis catches up by one step
                Pulse(STEP2_PIN)
                CurrentYSteps+=sy
                err+=ax
    else:   #y is the major axis
        err = ay//2
```

```
        for _ in range(ay):
            if abortEvent.is_set() or TriggeredEndstop() is not None:
                return
            Pulse(STEP2_PIN)
            CurrentYSteps+=sy
            err-=ax
            if err<0:
                Pulse(STEP1_PIN)
                CurrentXSteps+=sx
                err+=ay
```

Justification | Interpret the method rather than infer a benchmark

Major-axis stepping distributes minor-axis pulses without a division per step. Feed conversion turns desired length and rate into a nominal pulse delay. [75, 69, 73].

11.26 _GcodeLinearMove

```
def _GcodeLinearMove(targetXMm, targetYMm, feedMmMin)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 827. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.24.

Purpose | What this contributes

Clamps the requested machine target, uses calibrated X/Y steps/mm and a switch-clearance offset, rounds to integer target steps, and computes nominal major-iteration delay from Euclidean length and F/60. It then calls sequential-pulse Bresenham. This explains direction-dependent timing overhead and why the feed is not a verified speed.

Method | Current implementation

Coordinate two integer step counts along one line using a bounded error accumulator. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _GcodeLinearMove(targetXMm,targetYMm,feedMmMin):
    targetXMm,targetYMm = _ClampToUsableArea(targetXMm,targetYMm)
    spmmX,spmmY = Calibration["steps_per_mm_x"],Calibration["steps_per_mm_y"]
    x0Mm = CurrentXSteps/spmmX-EDGE_TOLERANCE_MM
    y0Mm = CurrentYSteps/spmmY-EDGE_TOLERANCE_MM
    distMm = math.hypot(targetXMm-x0Mm,targetYMm-y0Mm)
    TargetXSteps = round((targetXMm+EDGE_TOLERANCE_MM)*spmmX)
    TargetYSteps = round((targetYMm+EDGE_TOLERANCE_MM)*spmmY)
    dxSteps = TargetXSteps-CurrentXSteps
    dySteps = TargetYSteps-CurrentYSteps
    if dxSteps==0 and dySteps==0:
        return
    majorSteps = max(abs(dxSteps),abs(dySteps),1)
    feedMmS = max(feedMmMin/60.0,GCODE_MIN_FEED_MM_S)
    stepDelayS = max((distMm/feedMmS)/majorSteps,MIN_STEP_DELAY_S)
    BresenhamMove(dxSteps,dySteps,stepDelayS=stepDelayS)
```

Justification | Interpret the method rather than infer a benchmark

Major-axis stepping distributes minor-axis pulses without a division per step. Feed conversion turns desired length and rate into a nominal pulse delay. [75, 69, 73].

11.27 _GcodeArcMove

```
def _GcodeArcMove(x0Mm, y0Mm, x1Mm, y1Mm, iMm, jMm, clockwise, feedMmMin)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 849. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.24.

Purpose | What this contributes

Converts an I/J-centred G2/G3 arc to short straight chords and executes them through the linear mover. Approximation depends on chord segmentation, and the current parser supports only its implemented dialect rather than every CNC command.

Method | Current implementation

Coordinate two integer step counts along one line using a bounded error accumulator. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _GcodeArcMove(x0Mm,y0Mm,x1Mm,y1Mm,iMm,jMm,clockwise,feedMmMin):
    cx,cy = x0Mm+iMm,y0Mm+jMm
    radius = math.hypot(iMm,jMm)
    if radius<1e-6:
        _GcodeLinearMove(x1Mm,y1Mm,feedMmMin)
        return
    startAngle = math.atan2(y0Mm-cy,x0Mm-cx)
    endAngle = math.atan2(y1Mm-cy,x1Mm-cx)
    if clockwise:
        while endAngle>=startAngle:   #force a strictly decreasing sweep
            endAngle-=2.0*math.pi
    else:
        while endAngle<=startAngle:
            endAngle+=2.0*math.pi
    angleSpan = endAngle-startAngle
    nSegments = max(4,int(abs(angleSpan)*radius/0.4))   #about 0.4 mm per chord
    for k in range(1,nSegments+1):
        if abortEvent.is_set() or TriggeredEndstop() is not None:
            return
        a = startAngle+angleSpan*(k/nSegments)
        _GcodeLinearMove(cx+radius*math.cos(a),cy+radius*math.sin(a),feedMmMin)
```

Justification | Interpret the method rather than infer a benchmark

Major-axis stepping distributes minor-axis pulses without a division per step. Feed conversion turns desired length and rate into a nominal pulse delay. [75, 69, 73].

Stage: G code parsing and job generation

Report equation 33 Nominal timing budget

Method | Formula

$t_{\mathrm{nom}} = L_d/v_d + L_u/v_u + N_{\mathrm{toggle}}\tau.$

Purpose | Why the system needs it

Explain drawing, travel and dwell contributions. Current WriteGcode uses feeds in mm/min, multiplies length/feed by 60, and approximates dwell using the mean of up/down dwell. Initial/final travel, actual toggles and software overhead prevent treating it as a rigorous measured lower bound.

Evidence | Current location and source

WriteGcode and timing.Run; keep estimates distinct from completed-job logs. [3, 76].

11.28 Method background: G code parsing and job generation

Purpose | The role in this system

Connect abstract pen paths to modal movement and pen-state commands.

Justification | The reason for this construction

Explicit units, absolute positioning, feeds and dwell make a job inspectable; calibrated execution maintains a commanded position and checks interruptions.

Alternatives | What changes if another method is used

Direct streaming avoids a stored file but loses a convenient review/replay artefact. Standard firmware has a richer parser and motion planner, requiring agreed dialect and pen-code mapping.

Library | Implementation speed and accuracy

A G-code parser can recognise syntax but cannot assume this project M3/M5 pen semantics or P dwell units. LinuxCNC/GRBL-style conventions must be checked rather than treated as identical. [3, 77].

11.29 WriteGcode

```
def WriteGcode(traj, cfg, path, title='', rowTolMm=None)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 294. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Maps, decimates, orders and clamps paths, optionally closes near-rings and draws short endpoint bridges. It explicitly restates modal feed on travel and drawing transitions, counts lengths and writes an inspectable file. Its timing summary uses toggle count times the mean of up/down dwells and excludes some initial/final travel and execution overhead; it is nominal accounting, not a rigorous measured lower bound.

Method | Current implementation

Connect abstract pen paths to modal movement and pen-state commands. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def WriteGcode(traj,cfg,path,title='',rowTolMm=None):
    strokes,scale = ToMachine(traj,cfg,rowTolMm=rowTolMm)
    strokes,clipped = _Clamp(strokes,cfg)
    pen = NewPenState(cfg)
    L = []
    L.append(f"; Handwriting-Robot G-code{(' -- '+title) if title else ''}")
    L.append(f"; author={traj.meta.get('author')} strokes={len(strokes)} "
             f"scale={scale:.3f}")
    L.append(f"; steps/mm  X={cfg.stepsPerMmX:.3f}  Y={cfg.stepsPerMmY:.3f} "
             f"({cfg.microstepsPerRev} microsteps/rev)")
    L.append(f"; pen: one-way gear motor, {cfg.penUpCode}=up "
             f"{cfg.penDownCode}=down, each = one 90deg pulse")
    if clipped:
        L.append(f"; WARNING: {clipped} point(s) clamped to work area")
    L.append('G21   ; millimetres')
    L.append('G90   ; absolute')
    L+=SetPenLines(pen,cfg,True)
    L.append(f"G0 F{cfg.travelFeedMmMin:.0f}")
    here = None        #where the pen is, in millimetres
    joined = rings = 0
    joinedMm = 0.0
    drawMm = travelMm = 0.0
    for s in strokes:
        if len(s)<2:
            continue
        if (cfg.closeRingMm>0 and len(s)>3
                and math.dist(s[0],s[-1])<=cfg.closeRingMm
                and s[0]!=s[-1]):
            s = list(s)+[s[0]]
            rings += 1
        if here is not None and math.dist(here,s[0])<=cfg.joinStrokesMm:
            L.append(f"G1 F{cfg.drawFeedMmMin:.0f}")
            joined += 1
            joinedMm += math.dist(here,s[0])
            drawMm += math.dist(here,s[0])
        else:
            L+=SetPenLines(pen,cfg,True)
            L.append(f"G0 F{cfg.travelFeedMmMin:.0f} "
                     f"X{s[0][0]:.3f} Y{s[0][1]:.3f}")
            if here is not None:
                travelMm += math.dist(here,s[0])
            L+=SetPenLines(pen,cfg,False)
            L.append(f"G1 F{cfg.drawFeedMmMin:.0f}")
        prev = s[0]
        for (x,y) in s[1:]:
            L.append(f"G1 X{x:.3f} Y{y:.3f}")
            drawMm += math.dist(prev,(x,y))
            prev = (x,y)
        here = s[-1]
    L+=SetPenLines(pen,cfg,True)
    L.append(f"G0 F{cfg.travelFeedMmMin:.0f} "
             f"X{cfg.originXmm:.3f} Y{cfg.originYmm:.3f}   ; park")
    if here is not None:
        travelMm += math.dist(here,(cfg.originXmm,cfg.originYmm))
    L.append('M2')
    Path(path).parent.mkdir(parents=True,exist_ok=True)
    with open(path,'w',encoding='utf-8') as f:
        f.write('\n'.join(L)+'\n')
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Explicit units, absolute positioning, feeds and dwell make a job inspectable; calibrated execution maintains a commanded position and checks interruptions. [3, 77].

11.30 GantryNotConnectedError

```
class GantryNotConnectedError
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 406. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Connect abstract pen paths to modal movement and pen-state commands.

Output. an error type raised when something needs the hardware and the hardware is not available.

Purpose | What it is for

Raised when the GPIO library is missing or the chip device cannot be opened, which is the ordinary situation on any computer that is not the machine's own controller.

Justification | Why a named error rather than letting the import fail

An unguarded import of the hardware library would make this module unimportable off the machine, and since the test and evaluation scripts all import it, none of them could run anywhere else. A distinct error type lets the hardware functions fail cleanly when called, while the arithmetic half of the file remains usable.

Being a distinct type rather than a generic error is what lets a caller distinguish "there is no machine here" from "the machine is there and something went wrong". The first is expected and should be reported as a plain message; the second is a fault.

11.31 NotCalibratedError

```
class NotCalibratedError
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 410. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Connect abstract pen paths to modal movement and pen-state commands.

Output. an error type raised when an absolute move is attempted before homing.

Justification | Why calibration is a precondition and not a convenience

A stepper motor has no idea where it is. It moves a commanded number of steps from wherever it happens to be, and nothing in the motor or the driver reports a position. So a command to move to a particular place on the bed is meaningless until the machine has established where that place is relative to its current position, which is what homing does.

Executing such a command anyway is not a silent error. The machine drives toward a position computed from an unknown origin, which is as likely as not to be outside the bed, and it will drive into its own frame until the limit switches catch it. A distinct error type refusing the command is the appropriate response to a request that cannot be carried out safely.

This is one of the few places in the project where a function refuses rather than falling back. The usual pattern is to return a degraded result, but there is no degraded version of a physical move: it either goes to the right place or it drives the machine somewhere arbitrary.

11.32 Connect

```
def Connect()
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 501. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Claims the configured chip lines with gpiod v2, sets initial direction/step/motor states, and records the live handle. Failure must be interpreted as no connected controller; reading this function does not verify actual header wiring or GPIO current capability.

Method | Current implementation

Connect abstract pen paths to modal movement and pen-state commands. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Connect():
    global req
    if req is not None:
        return
    if gpiod is None:
        raise GantryNotConnectedError(f"gpiod not available: {_GPIOD_IMPORT_ERROR}")
    try:
        req = gpiod.request_lines(
            CHIP_PATH,
            consumer="stepper-control",
            config={
                DIR1_PIN: gpiod.LineSettings(direction=Direction.OUTPUT,output_value=Value.ACTIVE),
                STEP1_PIN: gpiod.LineSettings(direction=Direction.OUTPUT,output_value=Value.INACTIVE)
    ,
                DIR2_PIN: gpiod.LineSettings(direction=Direction.OUTPUT,output_value=Value.ACTIVE),
                STEP2_PIN: gpiod.LineSettings(direction=Direction.OUTPUT,output_value=Value.INACTIVE)
    ,
                X_MIN_PIN: gpiod.LineSettings(direction=Direction.INPUT),
                X_MAX_PIN: gpiod.LineSettings(direction=Direction.INPUT),
                Y_MIN_PIN: gpiod.LineSettings(direction=Direction.INPUT),
                Y_MAX_PIN: gpiod.LineSettings(direction=Direction.INPUT),
                PEN_SWITCH_PIN: gpiod.LineSettings(direction=Direction.INPUT),
                PEN_MOTOR_PIN: gpiod.LineSettings(direction=Direction.OUTPUT,output_value=Value.
    INACTIVE),
            },
        )
    except Exception as e:
        req = None
        raise GantryNotConnectedError(f"Could not open {CHIP_PATH}: {e}") from e
```

Justification | Interpret the method rather than infer a benchmark

Explicit units, absolute positioning, feeds and dwell make a job inspectable; calibrated execution maintains a commanded position and checks interruptions. [3, 77].

11.33 Disconnect

```
def Disconnect()
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 529. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Connect abstract pen paths to modal movement and pen-state commands.

Output. nothing; it sets the outputs safe and releases the lines.

Method | The order of operations

The step pins and the pen motor are set inactive first, and only then are the lines released. The order is necessary: once a line is released, its level is no longer controlled, and a line released while active may stay active. Releasing a step pin while high would leave the driver holding a coil energised, which heats the motor with nothing running to notice.

Justification | Why the release is in a finally clause

The setting of safe levels is wrapped so that the release happens whether or not it succeeded. If a pin cannot be set, because the chip has gone away or the device was unplugged, the lines must still be released, or they stay claimed by a process that has exited and the next run cannot acquire them. Releasing a resource must not depend on the success of the tidying done before it.

11.34 _TriggerEmergencyStop

```
def _TriggerEmergencyStop(hit)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 598. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Cuts the drive, retracts off every tripped switch and marks the machine stopped until it is cleared.

Inputs

`hit` the switch that caused the stop

Output. nothing; it stops the drive, retracts and latches the fault.

Justification | The sequence, and why it is in this order

- **Latch the fault first,** before anything else. Setting the flag and recording the switch immediately means that any other code checking it from this point on sees the stop, including the retract that follows.
- **Set the step pins inactive.** This ends any pulse in progress so no coil is left energised mid-step.

- **Find every pressed switch,** since more than one may be.
- **Retract off each of them,** so the machine is clear of its limits and able to move again once the fault is cleared.
- **Report,** naming the switches and the retract distance.

Justification | Why re-entry is refused

The function returns immediately if the fault is already latched. Without that, a second switch tripping during the retract would trigger another stop, which would retract again, which could trip again, and the recursion would continue until the program ran out of stack. The guard makes the stop happen once and the retract complete, which is the behaviour wanted: the first stop has already established that the machine is to halt, and a second detection adds nothing.

Justification | Why the pen is raised

Raising the pen is part of stopping, not a courtesy. A pen left down during an abort rests on the paper at the point of failure, where the ink bleeds into a visible blot, and if the machine is subsequently moved by hand it drags a line across the drawing. The drawing is already spoiled by the abort; the object is to avoid spoiling it further.

11.35 RequestAbort

```
def RequestAbort()
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 613. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Sets an event consumed inside motion/dwell loops. Requesting interruption and observing that the active job has stopped are separate events.

Method | Current implementation

Connect abstract pen paths to modal movement and pen-state commands. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def RequestAbort():
    abortEvent.set()
```

Justification | Interpret the method rather than infer a benchmark

Explicit units, absolute positioning, feeds and dwell make a job inspectable; calibrated execution maintains a commanded position and checks interruptions. [3, 77].

11.36 EmergencyStop

```
def EmergencyStop(waitS=3.0, park=True)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 620. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Requests abort, waits a bounded interval for the active-job event, turns outputs inactive and attempts pen-up/parking according to state. It returns whether execution remains active or latched. It does not supply a hard real-time maximum stop latency or guarantee the pen successfully raised.

Method | Current implementation

Connect abstract pen paths to modal movement and pen-state commands. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def EmergencyStop(waitS=3.0,park=True):
    Connect()
    abortEvent.set()
    deadline = time.time()+waitS
    while _jobActive.is_set() and time.time()<deadline:
        time.sleep(0.005)
    stillRunning = _jobActive.is_set()

    req.set_value(STEP1_PIN,Value.INACTIVE)
    req.set_value(STEP2_PIN,Value.INACTIVE)

    penUp = SetPen(True,force=True)
    parked,detail = False,""
    if stillRunning:
        detail = ("the drawing thread did not let go within %.1f s, so the "
                  "stop is still latched and the corner move was skipped"%waitS)
    else:
        abortEvent.clear()   #cleared before the park move
        if not park:
            detail = "parking not asked for"
        elif not Calibration["done"]:
            detail = ("not calibrated, so there is no known corner to drive "
                      "to -- the pen is up and the motors are idle")
        elif TriggeredEndstop() is not None:
            detail = ("end-stop %s is tripped, so the corner move was skipped "
                      "-- clear it, then re-home"%TriggeredEndstop())
        else:
            cfg = GantryConfig()
            _GcodeLinearMove(cfg.originXmm,cfg.originYmm,cfg.travelFeedMmMin)
            parked = True
    print("[stop] pen %s, %s%s"%("up" if penUp else "UP FAILED",
                                 "parked at the corner" if parked else "not parked",
                                 " -- "+detail if detail else ""))
    return {"stopped": True,"pen_up": bool(penUp),"parked": parked,
            "still_running": stillRunning,"latched": stillRunning,
            "detail": detail,"estopped": estopped,
            "tripped_switch": TrippedSwitch}
```

Justification | Interpret the method rather than infer a benchmark

Explicit units, absolute positioning, feeds and dwell make a job inspectable; calibrated execution maintains a commanded position and checks interruptions. [3, 77].

11.37 ParseGcodeLine

```
def ParseGcodeLine(raw)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 803. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Connect abstract pen paths to modal movement and pen-state commands.

Inputs

`raw` one line of a G-code file

Output. the command and its parameters, or nothing for a line with no command.

Method | What it does

Strips comments, splits the line into words, takes the first G or M word as the command, and collects the rest as parameters keyed by their letters.

Method | The two comment conventions

G-code has two: a semicolon makes the rest of the line a comment, and text in parentheses is a comment wherever it appears, including in the middle of a line. Both are handled, the first by truncating at the semicolon and the second by removing every parenthesised span. Handling only the first would leave comment text to be parsed as words, and a comment containing a letter followed by a digit would be read as a parameter.

Justification | Why the first G or M word becomes the command

A line has one command and any number of parameters, and the command is identified by being a G or M word. Taking the first such word rather than the first word generally is what lets a line with a leading line number be parsed, and the test that the command slot is still empty is what stops a later G word being mistaken for a second command.

> *The command's number is converted to an integer while parameters stay as floating point. That is correct for each: G1 and G01 denote the same command, so an integer normalises them, while a coordinate of 12.5 must keep its fraction.*

11.38 RunGcodeFile

```
def RunGcodeFile(path)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 875. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Requires calibration, opens the job, initialises modal state and encloses execution in a finally block clearing _jobActive. Initial parser coordinates use nominal STEPS_PER_MM, whereas _GcodeLinearMove obtains physical coordinates from calibrated factors. Do not assume these are identical in every modal/no-coordinate case.

Method | Current implementation

Connect abstract pen paths to modal movement and pen-state commands. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def RunGcodeFile(path):
    global lastJobAborted
    lastJobAborted = False
    if not Calibration["done"]:
        raise NotCalibratedError("Run calibration before running a G-code file.")
    try:
        with open(path,'r') as f:
            lines = f.readlines()
    except OSError as e:
        print(f"Couldn't open '{path}': {e}")
        return

    absMode = True
    feedMmMin = 900.0
    curXMm = CurrentXSteps/STEPS_PER_MM
    curYMm = CurrentYSteps/STEPS_PER_MM
    print(f"Running G-code: {path} ({len(lines)} lines)")

    _jobActive.set()
    try:
        _RunGcodeLines(path,lines,absMode,feedMmMin,curXMm,curYMm)
    finally:
        _jobActive.clear()
```

Justification | Interpret the method rather than infer a benchmark

Explicit units, absolute positioning, feeds and dwell make a job inspectable; calibrated execution maintains a commanded position and checks interruptions. [3, 77].

11.39 _RunGcodeLines

```
def _RunGcodeLines(path, lines, absMode, feedMmMin, curXMm, curYMm)
```

Evidence | Source and method link

`Software/DemoCode/reproduce.py` line 900. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.28.

Purpose | What this contributes

Processes commands with abort and endstop checks, modal feed and absolute/relative positioning. M3/M5 call SetPen; a false return is printed and execution continues. Unsupported commands are logged/skipped. Commanded parser targets and clamped/actually reached coordinates need not agree after interruptions; successful parsing is not independent position feedback.

Method | Current implementation

Connect abstract pen paths to modal movement and pen-state commands. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _RunGcodeLines(path,lines,absMode,feedMmMin,curXMm,curYMm):
    global lastJobAborted
    for lineno,raw in enumerate(lines,1):
```

```
        if abortEvent.is_set():
            lastJobAborted = True
            print(f"G-code job STOPPED at line {lineno} -- stop requested.")
            return
        hit = TriggeredEndstop()
        if hit is not None:
            _TriggerEmergencyStop(hit)
            print(f"G-code job ABORTED at line {lineno}.")
            return
        if estopped:
            print(f"G-code job ABORTED at line {lineno} -- already emergency-stopped.")
            return

        parsed = ParseGcodeLine(raw)
        if parsed is None:
            continue
        cmd,p = parsed

        if cmd in ("G0","G1"):
            if "F" in p:
                feedMmMin = p["F"]
            targetX = p.get("X",curXMm) if absMode else curXMm+p.get("X",0.0)
            targetY = p.get("Y",curYMm) if absMode else curYMm+p.get("Y",0.0)
            _GcodeLinearMove(targetX,targetY,feedMmMin)
            curXMm,curYMm = targetX,targetY

        elif cmd in ("G2","G3"):
            if "F" in p:
                feedMmMin = p["F"]
            targetX = p.get("X",curXMm) if absMode else curXMm+p.get("X",0.0)
            targetY = p.get("Y",curYMm) if absMode else curYMm+p.get("Y",0.0)
            _GcodeArcMove(curXMm,curYMm,targetX,targetY,
                          p.get("I",0.0),p.get("J",0.0),cmd=="G2",feedMmMin)
            curXMm,curYMm = targetX,targetY

        elif cmd=="G4":
            deadline = time.time()+p.get("P",0.0)
            while time.time()<deadline:
                if abortEvent.is_set() or TriggeredEndstop() is not None:
                    break
                time.sleep(0.005)

        elif cmd=="G90":
            absMode = True
        elif cmd=="G91":
            absMode = False
        elif cmd=="G21":
            pass
        elif cmd=="M3":
            if not SetPen(False):
                print(f"[Line {lineno}] Pen-down did not confirm -- continuing anyway.")
        elif cmd=="M5":
            if not SetPen(True):
                print(f"[Line {lineno}] Pen-up did not confirm -- continuing anyway.")
        elif cmd in ("M2","M30"):
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Explicit units, absolute positioning, feeds and dwell make a job inspectable; calibrated execution maintains a commanded position and checks interruptions. [3, 77].

12 Web orchestration and evaluation

Evidence | Current implementation boundary

`Software/DemoCode/web/server.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Load the networks and profiles; ingest and orient an image; segment/read/name its lines; orchestrate classification, generation or retracing; expose reports, G-code, calibration and writing routes. API endpoints are alternative request paths, not a single sequential call chain.

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
HERE = Path(__file__).resolve().parent
CP3 = HERE.parent
FRONTEND = HERE / 'frontend'
UPLOADS = paths.Cache / 'WebUploads'
JOBS = paths.Cache / 'WebJobs'
REPORTS = paths.Cache / 'WebReports'
CAPTURE_ROTATE_DEG = 90
NAME_FROM_LINES = 0
ORIENT_PROBE_LINES = 3
API_KEY = os.environ.get('WEBAPP_API_KEY', '')
PORT = int(os.environ.get('PORT', '5000'))
DEFAULT_TRIES = int(os.environ.get('WEBAPP_TRIES', '14'))
READER_PATH = paths.Weights / _reader if _reader else REC.DEFAULT_TEXT_WEIGHTS
TEXT_MODEL = REC.PaperCRNNNumpy(checkpointPath=READER_PATH)
INK_MODEL = REC.AuthorClassifierCNNNumpy(checkpointPath=REC.DEFAULT_AUTHOR_WEIGHTS)
INK_MAP = INK_MODEL.authorMapping
INK_BY_INDEX = {v: k for k, v in INK_MAP.items()}
SHAPE_MODEL = INK_MODEL
SHAPE_MAP = SHAPE_MODEL.authorMapping
PAGE_NAMER = SHAPE_MODEL
PAGE_NAMER_STROKE_NORM = False
PAGE_BY_INDEX = {v: k for k, v in PAGE_NAMER.authorMapping.items()}
PROFILES = LoadProfiles()
TIMINGS = []
TIMINGS_LOCK = threading.Lock()
```

Stage: Orientation and skew estimation

Method rationale, alternatives and library comparison: Section 5.40.

12.1 OpenUpright

```
def OpenUpright(src)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 125. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

Align the page line axis while preserving the writer slant needed for reproduction.

Method | Current implementation

Align the page line axis while preserving the writer slant needed for reproduction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def OpenUpright(src):
    """See source snapshot for the original docstring."""
    return ImageOps.exif_transpose(Image.open(src))
```

Justification | Interpret the method rather than infer a benchmark

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up. [11, 25, 24, 6].

12.2 IngestImage

```
def IngestImage(src)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 149. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

Align the page line axis while preserving the writer slant needed for reproduction.

Method | Current implementation

Align the page line axis while preserving the writer slant needed for reproduction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def IngestImage(src):
    """See source snapshot for the original docstring."""
    img = OpenUpright(src)
    deg = int(CAPTURE_ROTATE_DEG) % 360
    if deg:
        img = img.rotate(deg, expand=True)
    return img
```

Justification | Interpret the method rather than infer a benchmark

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up. [11, 25, 24, 6].

12.3 StandUp

```
def StandUp(gray)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 388. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

Urned by a fixed CAPTURE_ROTATE_DEG while an upload was not turned at all, on the assumption that an uploaded file is a phone photograph and so already upright. the webcam on the gantry writes no EXIF orientation tag, so the SAME photograph read correctly through the camera button and as nonsense when its saved file was uploaded: 91.1 per cent of characters against 19.8. both paths go through this now, so they cannot disagree. it is also idempotent, which is why the camera path can go on standing its capture up as it saves it: a page that is already upright is left alone.

Method | Current implementation

Align the page line axis while preserving the writer slant needed for reproduction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def StandUp(gray):
    """See source snapshot for the original docstring."""
    up, deg, s0, s90 = SEG.UprightPage(gray)
    return up, deg, {"axis_0": round(s0, 3), "axis_90": round(s90, 3)}
```

Justification | Interpret the method rather than infer a benchmark

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up. [11, 25, 24, 6].

12.4 ReadAllLines

```
def ReadAllLines(crops)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 394. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 5.40.

Purpose | What this contributes

Reads up to three sampled crops upright and inverted using mean best-frame log score. It reuses the chosen probe reads and maps remaining jobs over LW.Map. If inversion wins it reverses line order and rotates each crop to keep text and metadata consistent.

Method | Current implementation

Align the page line axis while preserving the writer slant needed for reproduction. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ReadAllLines(crops):
    """See source snapshot for the original docstring."""
    def Read(c, flip):
        img = np.rot90(c, 2) if flip else c
        return REC.ReadTextScored(Image.fromarray(img).convert("L"), TEXT_MODEL)

    n = len(crops)
    probe = list(np.unique(np.linspace(0, n-1, min(ORIENT_PROBE_LINES, n))
                           .round().astype(int)))
    fwd = {i: Read(crops[i], False) for i in probe}
    rev = {i: Read(crops[i], True) for i in probe}
    sF = float(np.mean([fwd[i][1] for i in probe]))
    sR = float(np.mean([rev[i][1] for i in probe]))

    flip = sR > sF
    cache = rev if flip else fwd
    todo = [(i, crops[i], flip) for i in range(n) if i not in cache]
    done = {i: (t, sc) for i, t, sc in LW.Map(LW.ReadOne, todo)}
    out = [cache[i] if i in cache else done[i] for i in range(n)]
    if flip:
        idx = list(range(n))[::-1]
        return ([np.rot90(crops[i], 2) for i in idx], [out[i][0] for i in idx],
                [out[i][1] for i in idx], idx, 180, sF, sR)
    idx = list(range(n))
    return (list(crops), [t for t, _s in out], [s for _t, s in out], idx, 0, sF, sR)
```

Justification | Interpret the method rather than infer a benchmark

The row-profile concentration objective is inexpensive and distinct from deslanting. Quarter turns resolve axis; reader-based inversion resolves which end is up. [11, 25, 24, 6].

Stage: Edit distance and evaluation metrics

Method rationale, alternatives and library comparison: Section 6.35.

12.5 NameWriter

```
def NameWriter(crops)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 439. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.35.

Purpose | What this contributes

Maps independent crops through the configured namer, then averages full class-probability vectors weighted by counts of pixels below 128. The current LW configuration inherits the same model and stroke-normalisation flag. A page probability is a weighted arithmetic pool, not a product of independent likelihoods.

Method | Current implementation

Separate text recovery, writer decisions, geometric agreement and physical accuracy. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def NameWriter(crops):
    """See source snapshot for the original docstring."""
    got = LW.Map(LW.NameOne, [(i, c) for i, c in enumerate(crops)])
    per = [p for _i, p in sorted(got, key=lambda r: r[0])]
    w = np.array([float((c < 128).sum()) for c in crops], dtype=np.float64)
    if w.sum() <= 0:
        w = np.ones(len(crops), dtype=np.float64)
    avg = (np.vstack(per)*w[:, None]).sum(axis=0)/w.sum()
    return per, avg, w
```

Justification | Interpret the method rather than infer a benchmark

Each metric answers a different requirement. Pooled character edits weight characters; writer decisions are binary per line; geometric F measures thresholded point proximity. [55, 56, 57].

12.6 SplitExpected

```
def SplitExpected(expectedText, n)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 468. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.35.

Purpose | What this contributes

Separate text recovery, writer decisions, geometric agreement and physical accuracy.

Method | Current implementation

Separate text recovery, writer decisions, geometric agreement and physical accuracy. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def SplitExpected(expectedText, n):
    """See source snapshot for the original docstring."""
    got = [l.strip() for l in (expectedText or "").splitlines() if l.strip()]
    return got if len(got) == n and n > 0 else None
```

Justification | Interpret the method rather than infer a benchmark

Each metric answers a different requirement. Pooled character edits weight characters; writer decisions are binary per line; geometric F measures thresholded point proximity. [55, 56, 57].

12.7 WordAcc

```
def WordAcc(pred, truth)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 517. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.35.

Purpose | What this contributes

Computes case-insensitive word-level longest-common-subsequence recall divided by reference word count. It is not standard word error rate and is not strict positional exact-word accuracy. The function now resides in the server rather than recognise.py.

Method | Current implementation

Separate text recovery, writer decisions, geometric agreement and physical accuracy. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def WordAcc(pred,truth):
    tw,pw = truth.split(),pred.split()
    if not tw:
        return 1.0
    m,n = len(tw),len(pw)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m):
        for j in range(n):
            dp[i+1][j+1] = (dp[i][j]+1 if tw[i].lower()==pw[j].lower()
                            else max(dp[i][j+1],dp[i+1][j]))
    return dp[m][n]/m
```

Justification | Interpret the method rather than infer a benchmark

Each metric answers a different requirement. Pooled character edits weight characters; writer decisions are binary per line; geometric F measures thresholded point proximity. [55, 56, 57].

Stage: Coordinate mapping and path planning

Method rationale, alternatives and library comparison: Section 11.2.

12.8 _Clamp

```
def _Clamp(value, default, lo, hi)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 715. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 11.2.

Purpose | What this contributes

Place and fit the generated trajectory into the measured machine work area.

Method | Current implementation

Place and fit the generated trajectory into the measured machine work area. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _Clamp(value, default, lo, hi):
    """See source snapshot for the original docstring."""
    try:
        v = float(value)
    except (TypeError, ValueError):
        return float(default)
    return float(min(max(v, lo), hi))
```

Justification | Interpret the method rather than infer a benchmark

Uniform scaling preserves shape while margins/origin define placement. Row grouping and nearest-end stroke order reduce travel; clamping makes out-of-bounds points explicit. [75, 3].

Stage: Web orchestration and job state

12.9 Method background: Web orchestration and job state

Purpose | The role in this system

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Justification | The reason for this construction

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract.

Alternatives | What changes if another method is used

A monolithic request is simpler but couples generation and hardware execution. A queue/worker service scales differently and needs explicit state/failure handling.

Library | Implementation speed and accuracy

Flask handles routing and responses, Pillow handles images, and NumPy performs arithmetic. These are actual dependencies; the complete demo cannot truthfully be called library-free. File sanitisation/API keys do not constitute full external-service security. [78, 6, 3].

12.10 BaseAuthor

```
def BaseAuthor(name)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 242. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Curate.py writes <writer>_curated.json beside the original, and the server lists every profile it finds, so these names reach the endpoints. the writer models were trained on the plain names and know nothing of the suffix, so anything asking them has to ask about the writer.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def BaseAuthor(name):
    return name[:-8] if name.endswith("_curated") else name
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.11 LoadProfiles

```
def LoadProfiles()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 247. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def LoadProfiles():
    out = {}
    for f in sorted(paths.Profiles.glob("*.json")):
        try:
            out[f.stem] = json.load(open(f, encoding="utf-8"))
        except Exception as e:
            print("[server] skipping profile %s: %s" % (f.name, e))
    return out
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.12 Remember2

```
def Remember2(d)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 265. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Remember2(d):
    """See source snapshot for the original docstring."""
    with TIMINGS_LOCK:
        TIMINGS.append(d)
        del TIMINGS[:-200]
    return d
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.13 Remember

```
def Remember(run)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 273. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Remember(run):
    d = run.AsDict()
    with TIMINGS_LOCK:
        TIMINGS.append(d)
        del TIMINGS[:-200]        #keep the last two hundred, no more
    print(run.Report(), flush=True)
    return d
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.14 CheckApiKey

```
def CheckApiKey()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 283. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def CheckApiKey():
    if API_KEY and request.headers.get("X-API-Key") != API_KEY:
        abort(401)
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.15 Index

```
def Index()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 292. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Index():
    return send_from_directory(FRONTEND, "index.html")
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.16 Static

```
def Static(name)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 297. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Static(name):
    if (FRONTEND/name).exists():
        return send_from_directory(FRONTEND, name)
    abort(404)
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.17 ApiAuthors

```
def ApiAuthors()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 307. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiAuthors():
    out = []
    for a in sorted(PROFILES):
        p = PROFILES[a]
        out.append(dict(id=a,
                        glyphs=len(p.get("glyphs") or {}),
                        legibility=p.get("legibilityLambda"),
                        inGantryModels=a in SHAPE_MAP))
    return jsonify(out)
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.18 ApiGetImage

```
def ApiGetImage(imageId)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 319. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiGetImage(imageId):
    safe = "".join(c for c in imageId if c.isalnum() or c == "_")
    if safe.startswith("job_"):
        p = JOBS/safe[4:]/"preview.png"
    else:
        p = UPLOADS/("%s.jpg" % safe)
    if not p.exists():
        abort(404)
    return send_file(str(p))
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.19 ApiCameraCapture

```
def ApiCameraCapture()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 331. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiCameraCapture():
    try:
        captured = CAM.CaptureImage()
    except Exception as e:
        return jsonify({"error": "Camera failed: %s" % e}), 200
    imageId = uuid.uuid4().hex
    IngestImage(captured).convert("RGB").save(UPLOADS/("%s.jpg" % imageId))
    return jsonify({"image_id": imageId, "image_url": "/api/images/%s" % imageId})
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.20 ApiPreviewStart

```
def ApiPreviewStart()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 342. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiPreviewStart():
    try:
        CAM.PreviewStart()
    except Exception as e:
        return jsonify({"active": False, "error": str(e)}), 200
    return jsonify({"active": True})
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.21 ApiPreviewStop

```
def ApiPreviewStop()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 351. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiPreviewStop():
    CAM.PreviewStop()
    return jsonify({"active": False})
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.22 ApiPreviewStream

```
def ApiPreviewStream()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 357. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiPreviewStream():
    if not CAM.PreviewActive():
        abort(409)
    def Frames():
        while CAM.PreviewActive():
            try:
                jpeg = CAM.PreviewFrameJpeg()
            except RuntimeError:
                break
            yield (b"--frame\r\nContent-Type: image/jpeg\r\n\r\n"+jpeg+b"\r\n")
    return Response(Frames(),
                    mimetype="multipart/x-mixed-replace; boundary=frame")
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.23 WriteReport

```
def WriteReport(reportId, payload)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 479. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def WriteReport(reportId, payload):
    """See source snapshot for the original docstring."""
    (REPORTS/("%s.json" % reportId)).write_text(json.dumps(payload, indent=2),
                                               encoding="utf-8")
    cols = ["line", "author_named", "author_confidence_pct",
```

```
            "page_author_probability_pct", "char_accuracy_pct",
            "word_accuracy_pct", "read_confidence", "ink_pct",
            "text_read", "text_expected"]
    with (REPORTS/("%s.csv" % reportId)).open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        for k in ("image_id", "predicted_author", "author_confidence_pct",
                  "author_line_agreement_pct", "author_line_correct_pct",
                  "text_accuracy_pct", "word_accuracy_pct",
                  "mean_line_char_accuracy_pct", "lines_detected",
                  "lines_dropped_as_blots", "orientation_applied_deg"):
            w.writerow(["# %s" % k, payload.get(k)])
        w.writerow([])
        w.writerow(cols)
        for r in payload["lines"]:
            w.writerow([r.get(c) for c in cols])
    return "/api/reports/%s" % reportId
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.24 ApiReport

```
def ApiReport(reportId, ext='csv')
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 504. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiReport(reportId, ext="csv"):
    safe = "".join(c for c in reportId if c.isalnum())
    if ext not in ("csv", "json"):
        abort(404)
    p = REPORTS/("%s.%s" % (safe, ext))
    if not p.exists():
        abort(404)
    return send_file(str(p), as_attachment=True,
                     download_name="per_line_report_%s.%s" % (safe, ext))
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.25 ApiClassify

```
def ApiClassify()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 535. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Applies upload/capture orientation handling, times SegmentLines as one combined stage, resolves inversion, reads and names lines, and emits paired metrics only where supplied truth is compatible. It does not report the earlier three separate conditioning/segmenting/cropping timings from the core API.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiClassify():
    expectedText = (request.form.get("expected_text") or "").strip()
    expectedAuthor = request.form.get("expected_author") or ""

    run = TIMING.Run("classify")
    with run.Step("loading", chars=0):
        if "image" in request.files and request.files["image"].filename:
            page = OpenUpright(request.files["image"].stream).convert("L")
            imageId = uuid.uuid4().hex
        elif request.form.get("image_id"):
            imageId = "".join(c for c in request.form["image_id"] if c.isalnum())
            p = UPLOADS/("%s.jpg" % imageId)
            if not p.exists():
                return jsonify({"error": "Unknown image_id"}), 400
            page = Image.open(p).convert("L")
        else:
            return jsonify({"error": "Provide either 'image' or 'image_id'"}), 400
        gray, axisDeg, axisScores = StandUp(np.asarray(page, dtype=np.uint8))
        Image.fromarray(gray).convert("RGB").save(UPLOADS/("%s.jpg" % imageId))

    with run.Step("segmenting", chars=0):
        lines, _dbg, info = SEG.SegmentLines(gray)
    crops = [np.asarray(ln["image"]) for ln in lines]
    if not crops:
        return jsonify({"error": "No handwriting lines were detected.",
                        "timing": Remember(run)}), 422

    with run.Step("reading", chars=0) as step:
        crops, readLines, readScores, order, flipDeg, sF, sR = ReadAllLines(crops)
        lines = [lines[i] for i in order]   #keep the crop metadata in step
        step.chars = sum(len(t) for t in readLines)
    predictedText = " ".join(readLines)

    with run.Step("naming", chars=len(predictedText)):
        perLineProbs, avg, inkWeights = NameWriter(crops)
        bestIdx = int(np.argmax(avg))
        predictedAuthor = PAGE_BY_INDEX[bestIdx]

    truthLines = SplitExpected(expectedText, len(crops))
    named = [PAGE_BY_INDEX[int(np.argmax(p))] for p in perLineProbs]
    agree = sum(1 for n in named if n == predictedAuthor)
    correct = (sum(1 for n in named if n == expectedAuthor)
               if expectedAuthor else None)
```

```
    lineRows = []
    for i, p in enumerate(perLineProbs):
        t = truthLines[i] if truthLines else None
        lineRows.append({
            "line": i+1,
            "author_named": named[i],
            "author_confidence_pct": round(100*float(p[int(np.argmax(p))]), 1),
            "page_author_probability_pct": round(100*float(p[bestIdx]), 1),
            "author_matches_page": named[i] == predictedAuthor,
            "author_correct": (named[i] == expectedAuthor) if expectedAuthor else None,
            "char_accuracy_pct": (round(100*REC.CharAcc(readLines[i], t), 1)
                                  if t is not None else None),
            "word_accuracy_pct": (round(100*WordAcc(readLines[i], t), 1)
                                  if t is not None else None),
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.26 ReadBackText

```
def ReadBackText(previewImg, pxPerMm)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 656. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

WRITING TEXT IN A WRITER'S HAND the joint search draws nTries candidates and keeps whichever scores best on the harmonic mean of how readable it is and how much it looks like the requested writer, which is the selection that lifted both figures.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ReadBackText(previewImg, pxPerMm):
    """See source snapshot for the original docstring."""
    arr = np.asarray(previewImg, dtype=np.uint8)
    pad = max(8, int(8.0*float(pxPerMm or 8.0)))   #8 mm of paper, as the writers' own sheets have
    padded = np.full((arr.shape[0]+2*pad, arr.shape[1]+2*pad), 255, np.uint8)
    padded[pad:pad+arr.shape[0], pad:pad+arr.shape[1]] = arr
    try:
        lines, _dbg, _info = SEG.SegmentLines(padded, mergeValley=0.72)
    except Exception:
        return REC.ReadText(previewImg, TEXT_MODEL)
    if len(lines) <= 1:
        return REC.ReadText(previewImg, TEXT_MODEL)
    return " ".join(filter(None, (
        REC.ReadText(Image.fromarray(np.asarray(ln["image"])).convert("L"), TEXT_MODEL)
        for ln in lines)))
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.27 JobResponse

```
def JobResponse(jobId, written, previewImg, run, expectedText=None, pxPerMm=8.0, **extra)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 696. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def JobResponse(jobId, written, previewImg, run, expectedText=None,
                pxPerMm=8.0, **extra):
    with run.Step("reading back", chars=len(expectedText or "")):
        readBack = ReadBackText(previewImg, pxPerMm)
    body = {
        "job_id": jobId,
        "gcode_url": "/api/gcode/%s" % jobId,
        "preview_url": "/api/images/job_%s" % jobId,
        "gcode_line_count": written.get("lines"),
        "pen_pulses": written.get("penPulses"),
        "read_back_text": readBack,
        "text_accuracy_pct": (round(100*REC.CharAcc(readBack, expectedText), 1)
                              if expectedText else None),
    }
    body.update(extra)
    body["timing"] = Remember(run)
    return jsonify(body)
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.28 ApiGenerate

```
def ApiGenerate()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 727. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

This endpoint expresses the intended full synthesis-to-job flow but has static interface conflicts in the captured tree: GantryConfig has no ignoreRuling field and RenderTrajectory accepts neither profile nor uniformInk. Explain the intended stages without claiming this snapshot successfully completes generation. No fix is made for this manual.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiGenerate():
    data = request.get_json(force=True) or {}
    text = (data.get("text") or "").strip()
    author = data.get("author")
    nTries = int(data.get("nTries", DEFAULT_TRIES))
    if not text:
        return jsonify({"error": "text is required"}), 400
    if author not in PROFILES:
        return jsonify({"error": "Unknown author %r" % author}), 400
    judged = BaseAuthor(author)
    if judged not in SHAPE_MAP:
        return jsonify({"error": "No writer model for %r, so its output cannot be "
                                 "judged during the search" % author}), 400

    profile = PROFILES[author]
    cfg = REP.GantryConfig()
    lineH = float(data.get("lineHeightMm") or cfg.lineHeightMm)
    lineIdx = max(0, int(data.get("line", 0)))
    drawFeed = _Clamp(data.get("drawFeedMmMin"), cfg.drawFeedMmMin, 60.0, 3000.0)
    travelFeed = _Clamp(data.get("travelFeedMmMin"), cfg.travelFeedMmMin, 60.0, 6000.0)
    penUpDwell = _Clamp(data.get("penUpDwellMs"), cfg.penUpDwellMs, 0.0, 1000.0)
    penDownDwell = _Clamp(data.get("penDownDwellMs"), cfg.penDownDwellMs, 0.0, 1000.0)
    ignore = data.get("ignoreRuling")
    ignore = cfg.ignoreRuling if ignore is None else bool(ignore)
    cfg = dataclasses.replace(cfg, lineHeightMm=lineH, lineOffsetMm=lineIdx*lineH,
                              ignoreRuling=ignore,
                              drawFeedMmMin=drawFeed, travelFeedMmMin=travelFeed,
                              penUpDwellMs=penUpDwell, penDownDwellMs=penDownDwell)
    run = TIMING.Run("reproduce", chars=len(text), lines=1)

    with run.Step("synthesis"):
        traj = SYN.SynthesizeJointBestOf(
            judged, text, profile, nTries=nTries,
            mmPerXh=REP.XhForRuling(profile, cfg, text),
            lineWidthMm=cfg.usableWidthMm,
            reader=TEXT_MODEL, authorModel=SHAPE_MODEL, authorMapping=SHAPE_MAP)

    jobId = uuid.uuid4().hex
    jobDir = JOBS/jobId
    jobDir.mkdir(parents=True, exist_ok=True)

    with run.Step("rendering"):
        preview = SYN.RenderTrajectory(traj, pxPerMm=18.0, profile=profile, uniformInk=True)
        Image.fromarray(np.asarray(preview, dtype=np.uint8)).save(jobDir/"preview.png")

    with run.Step("gcode"):
        written = REP.WriteGcode(traj, cfg, str(jobDir/"job.gcode"),
                                 title="web job %s" % author)

    (jobDir/"meta.json").write_text(json.dumps({"text": text, "author": author,
                                                "nTries": nTries, "line": lineIdx,
                                                "lineHeightMm": lineH,
```

```
                                                "ignoreRuling": ignore,
                                                "drawFeedMmMin": drawFeed,
                                                "travelFeedMmMin": travelFeed,
                                                "penUpDwellMs": penUpDwell,
                                                "penDownDwellMs": penDownDwell,
                                                "generateStages": run.stages}),
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.29 ApiRetrace

```
def ApiRetrace()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 839. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiRetrace():
    if "image" in request.files and request.files["image"].filename:
        imageId = uuid.uuid4().hex
        srcPath = UPLOADS/("%s.jpg" % imageId)
        OpenUpright(request.files["image"].stream).convert("RGB").save(srcPath)
    elif request.form.get("image_id"):
        imageId = "".join(c for c in request.form["image_id"] if c.isalnum())
        srcPath = UPLOADS/("%s.jpg" % imageId)
        if not srcPath.exists():
            return jsonify({"error": "Unknown image_id"}), 400
    else:
        return jsonify({"error": "Provide either 'image' or 'image_id'"}), 400

    jobId = uuid.uuid4().hex
    jobDir = JOBS/jobId
    jobDir.mkdir(parents=True, exist_ok=True)
    try:
        result = RETRACE.RetracePage(srcPath, jobDir/"job.gcode",
                                     cfg=REP.GantryConfig(),
                                     previewPath=jobDir/"preview.png")
    except SystemExit as e:
        return jsonify({"error": str(e)}), 422

    (jobDir/"meta.json").write_text(json.dumps(
        {"text": "", "author": "retrace", "source": str(srcPath),
         "generateStages": result["timing"]["stages"]}), encoding="utf-8")
    Remember2(result["timing"])
    return jsonify({
        "job_id": jobId,
```

```
        "gcode_url": "/api/gcode/%s" % jobId,
        "preview_url": "/api/images/job_%s" % jobId,
        "source_url": "/api/images/%s" % imageId,
        "lines_traced": result["timing"]["lines"],
        "strokes_traced": result["tracedStrokes"],
        "gcode_line_count": result["lines"],
        "pen_pulses": result["penPulses"],
        "scale": round(result["scale"], 4),
        "clamped_points": result["clipped"],
        "timing": result["timing"],
    })
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.30 IsConnected

```
def IsConnected()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 889. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def IsConnected():
    return getattr(REP,"req",None) is not None
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.31 WriteGcodeFile

```
def WriteGcodeFile(path)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 896. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

The three flags are read back off the module after the run, because RunGcodeFile sets them as it goes and returns nothing itself.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def WriteGcodeFile(path):
    REP.Connect()
    REP.RunGcodeFile(path)
    return {"estopped": REP.estopped,
            "tripped_switch": REP.TrippedSwitch,
            "aborted": REP.lastJobAborted}
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.32 ParseGcode

```
def ParseGcode(path)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 910. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Read a g-code file back into strokes written as its own reader rather than sharing code with the writer, so that comparing the two actually tests the file instead of testing one shared assumption twice over.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ParseGcode(path):
    strokes = []
    cur = []
    penDown = False
    x = y = 0.0
    with open(path,encoding="utf-8") as f:
        for raw in f:
            line = raw.split(";")[0].strip()
            if not line:
                continue
            up = line.upper()
            code = up.split()[0]
            if code=="M5":
                if penDown and len(cur)>=2:
                    strokes.append(cur)
                cur,penDown = [],False
                continue
            if code=="M3":
                penDown = True
                cur = [(x,y)]
                continue
            if code in ("G0","G1"):
                nx = re.search(r"X(-?\d+\.?\d*)",up)
```

```
                ny = re.search(r"Y(-?\d+\.?\d*)",up)
                if nx:
                    x = float(nx.group(1))
                if ny:
                    y = float(ny.group(1))
                if (nx or ny) and penDown:
                    cur.append((x,y))
    if penDown and len(cur)>=2:
        strokes.append(cur)
    return strokes
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.33 ApiGcodeUpload

```
def ApiGcodeUpload()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 946. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiGcodeUpload():
    if "gcode" not in request.files or not request.files["gcode"].filename:
        return jsonify({"error": "Provide a 'gcode' file"}), 400
    expectedText = (request.form.get("expected_text") or "").strip()

    jobId = uuid.uuid4().hex
    jobDir = JOBS/jobId
    jobDir.mkdir(parents=True, exist_ok=True)
    gcodePath = jobDir/"job.gcode"
    request.files["gcode"].save(str(gcodePath))

    run = TIMING.Run("reproduce", chars=len(expectedText), lines=1)
    with run.Step("parsing gcode"):
        strokes = ParseGcode(str(gcodePath))
        traj = SYN.Trajectory(strokes, dict(author="uploaded",
                                            source=request.files["gcode"].filename))
    with run.Step("rendering"):
        preview = SYN.RenderTrajectory(traj, pxPerMm=8.0)
        Image.fromarray(np.asarray(preview, dtype=np.uint8)).save(jobDir/"preview.png")
    written = dict(lines=sum(1 for _ in open(gcodePath, encoding="utf-8", errors="replace")),
                   penPulses=None)
    return JobResponse(jobId, written, preview, run, pxPerMm=8.0,
                       expectedText=expectedText or None)
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.34 ApiGetGcode

```
def ApiGetGcode(jobId)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 976. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiGetGcode(jobId):
    safe = "".join(c for c in jobId if c.isalnum())
    p = JOBS/safe/"job.gcode"
    if not p.exists():
        abort(404)
    return send_file(str(p), mimetype="text/plain",
                     as_attachment=True, download_name="%s.gcode" % safe)
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.35 ApiGantryStatus

```
def ApiGantryStatus()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 992. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiGantryStatus():
    if not IsConnected():
        try:
            REP.Connect()
        except REP.GantryNotConnectedError as e:
            return jsonify({"connected": False, "calibrated": False, "detail": str(e)})
    return jsonify({"connected": True,
                    "calibrated": REP.Calibration["done"],
                    "calibration": REP.Calibration if REP.Calibration["done"] else None})
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.36 ApiGantryCalibrate

```
def ApiGantryCalibrate()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1004. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiGantryCalibrate():
    run = TIMING.Run("calibrate")
    try:
        REP.Connect()
        with run.Step("homing"):
            result = REP.Calibrate()
    except REP.GantryNotConnectedError as e:
        return jsonify({"calibrated": False, "message": "Gantry not connected: %s" % e})
    except RuntimeError as e:
        return jsonify({"calibrated": False, "message": "Calibration failed: %s" % e})
    return jsonify({"calibrated": True, "calibration": result, "timing": Remember(run)})
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.37 ApiGantryStop

```
def ApiGantryStop()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1025. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiGantryStop():
    try:
        result = REP.EmergencyStop()
    except REP.GantryNotConnectedError as e:
        return jsonify({"stopped": False, "gantry_connected": False,
                        "message": "Gantry not connected: %s" % e})
    except RuntimeError as e:
        return jsonify({"stopped": False, "gantry_connected": True,
                        "message": "Stop failed: %s" % e})
    bits = ["pen up" if result["pen_up"] else "PEN DID NOT LIFT"]
    bits.append("parked at the corner" if result["parked"] else "not parked")
    msg = "Stopped -- " + ", ".join(bits) + "."
    if result.get("detail"):
        msg += " " + result["detail"][0].upper() + result["detail"][1:] + "."
    result.update({"gantry_connected": True, "message": msg})
    return jsonify(result)
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.38 ApiWriteGcode

```
def ApiWriteGcode(jobId)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1044. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiWriteGcode(jobId):
    safe = "".join(c for c in jobId if c.isalnum())
    jobDir = JOBS/safe
    gcodePath = jobDir/"job.gcode"
```

```
    if not gcodePath.exists():
        return jsonify({"error": "Unknown job_id"}), 404

    meta = {}
    if (jobDir/"meta.json").exists():
        meta = json.loads((jobDir/"meta.json").read_text(encoding="utf-8"))
    chars = len(meta.get("text", ""))

    priorStages = meta.get("generateStages") or []
    run = TIMING.Run("reproduce", chars=chars, lines=1)
    for s in priorStages:
        run.Carry(s.get("stage", "earlier"), s.get("seconds", 0.0), s.get("characters", 0) or 0)
    try:
        with run.Step("drawing", chars=chars):
            result = WriteGcodeFile(str(gcodePath))
    except REP.GantryNotConnectedError as e:
        return jsonify({"sent": False, "gantry_connected": False,
                        "message": "Gantry not connected: %s" % e})
    except REP.NotCalibratedError as e:
        return jsonify({"sent": False, "gantry_connected": True, "calibrated": False,
                        "message": "Calibration required: %s" % e})

    if result.get("aborted"):
        return jsonify({"sent": False, "gantry_connected": True, "aborted": True,
                        "message": "Job stopped before it finished. The page is "
                                   "incomplete -- do not measure it.",
                        "timing": Remember(run)})
    if result.get("estopped"):
        return jsonify({"sent": False, "gantry_connected": True,
                        "message": "Job aborted by end-stop %r mid-run. Clear it on the "
                                   "gantry before sending another job."
                                   % result.get("tripped_switch"),
                        "timing": Remember(run)})
    return jsonify({"sent": True, "gantry_connected": True, "message": "Job completed.",
                    "timing": Remember(run)})
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.39 ApiModelInfo

```
def ApiModelInfo()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1096. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiModelInfo():
    def Params(model):
        return int(sum(arr.size for arr in model.sd.values()))

    def Mb(path):
        return round(path.stat().st_size/(1024*1024), 2) if path and path.exists() else None

    return jsonify({
        "text_recognizer": {
            "architecture": "CNN-BiLSTM-CTC, 12 convolutions then 2 bidirectional LSTM layers",
            "weights_file": TEXT_MODEL.checkpointPath.name,
            "parameters": Params(TEXT_MODEL),
            "file_size_mb": Mb(TEXT_MODEL.checkpointPath),
            "measured_accuracy": {
                "real_ink_characters_pct": 98.61,
                "real_ink_words_pct": 94.17,
                "note": "669 real line crops from five photographed writers. The "
                        "same measurement on the from-scratch checkpoint gives 24.1%, "
                        "which is why the joint checkpoint is the one loaded.",
            },
        },
        "writer_id_ink_based": {
            "architecture": "CNN classifier on raw grey crops",
            "weights_file": INK_MODEL.checkpointPath.name,
            "parameters": Params(INK_MODEL),
            "file_size_mb": Mb(INK_MODEL.checkpointPath),
            "measured_accuracy": {
                "real_photographed_pages_pct": 99.70,
                "note": "Use only on real photographed pages. A drawn line has one "
                        "pen width everywhere, so there is no ink evidence to judge.",
            },
        },
        "writer_id_stroke_normalised": {
            "architecture": "CNN classifier on stroke-normalised crops, ink-density invariant",
            "weights_file": SHAPE_MODEL.checkpointPath.name,
            "parameters": Params(SHAPE_MODEL),
            "file_size_mb": Mb(SHAPE_MODEL.checkpointPath),
            "measured_accuracy": {
                "real_ink_pct": 98.20,
                "note": "This is the judge used for drawn and synthesised output.",
            },
        },
        "reproduction_pipeline": {
            "authors": sorted(PROFILES),
            "n_authors": len(PROFILES),
            "measured_on_novel_sentences": {
                "writer_id_direct_pct": 77.86,
                "writer_id_gcode_pct": 77.86,
                "text_char_direct_pct": 80.07,
                "text_char_gcode_pct": 80.07,
                "text_word_pct": None,
                "writers_identified": "12 of 13",
                "seconds_per_line": 10.9,
                "method": "SynthesizeJointBestOf: %d candidates per line, scored by the "
                          "harmonic mean of text accuracy and writer-ID confidence "
                          "together, with the C1 join" % DEFAULT_TRIES,
                "measured_via": "testing/results/RESULTS.md, 13 writers on novel sentences",
            },
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.40 ApiTimings

```
def ApiTimings()
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1167. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiTimings():
    with TIMINGS_LOCK:
        rows = list(TIMINGS)
    summary = {}
    for r in rows:
        s = summary.setdefault(r["kind"], dict(runs=0, totalSeconds=0.0, characters=0))
        s["runs"] += 1
        s["totalSeconds"] += r["totalSeconds"]
        s["characters"] += r["characters"] or 0
    for kind, s in summary.items():
        s["meanSeconds"] = round(s["totalSeconds"]/s["runs"], 3)
        s["meanSecondsPerChar"] = (round(s["totalSeconds"]/s["characters"], 4)
                                   if s["characters"] else None)
        formula = TIMING.ALLOWANCE.get(kind)
        meanChars = s["characters"]/s["runs"] if s["runs"] else 0
        s["meanAllowanceSeconds"] = round(formula(meanChars), 2) if formula and meanChars else None
        s["allowanceFormula"] = TIMING.ALLOWANCE_TEXT.get(kind)
        s["meanWithinAllowance"] = (bool(s["meanSeconds"] <= s["meanAllowanceSeconds"])
                                    if s["meanAllowanceSeconds"] else None)
    return jsonify({"summary": summary, "runs": rows})
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.41 ApiSamples

```
def ApiSamples(author)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1192. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiSamples(author):
    if author not in PROFILES:
        abort(404)
    try:
        pairs = RENDER.SampleCrops(author, n=2)
    except Exception as e:
        return jsonify({"error": str(e)}), 200
    return jsonify([{"image_url": "/api/samples/%s/%s" % (author, p.name), "text": t}
                    for p, t in pairs])
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.42 ApiSampleFile

```
def ApiSampleFile(author, filename)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1205. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiSampleFile(author, filename):
    safe = "".join(c for c in filename if c.isalnum() or c in "._-")
    p = RENDER.CACHE_DIR/safe
    if not safe.startswith("sample_%s_" % author) or not p.exists():
        abort(404)
    return send_file(str(p))
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.43 ApiGlyphs

```
def ApiGlyphs(author)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1214. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiGlyphs(author):
    if author not in PROFILES:
        abort(404)
    try:
        return send_file(str(RENDER.GlyphGridPath(author, PROFILES[author])))
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.44 ApiCurateGet

```
def ApiCurateGet(author)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1233. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiCurateGet(author):
    base = author[:-8] if author.endswith("_curated") else author
    if base not in PROFILES and author not in PROFILES:
        abort(404)
    curated = request.args.get("curated") in ("1", "true", "yes")
    try:
        return jsonify(CUR.Variants(base, curated))
    except SystemExit as e:
        return jsonify({"error": str(e)}), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.45 ApiCuratePost

```
def ApiCuratePost(author)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1247. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiCuratePost(author):
    if author not in PROFILES:
        abort(404)
    body = request.get_json(silent=True) or {}
    drop = {k: [int(i) for i in v] for k, v in (body.get("drop") or {}).items()}
    keep = {k: [int(i) for i in v] for k, v in (body.get("keep") or {}).items()}
    if not drop and not keep:
        return jsonify({"error": "nothing was marked for removal"}), 200
    try:
        summary = CUR.Apply(author, drop, keep,
                            curated=bool(body.get("curated")),
                            outName=body.get("out") or None)
    except SystemExit as e:
        return jsonify({"error": str(e)}), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
    PROFILES.clear()
    PROFILES.update(LoadProfiles())
    summary["authorsNow"] = sorted(PROFILES)
    return jsonify(summary)
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

12.46 ApiCurateDelete

```
def ApiCurateDelete(author)
```

Evidence | Source and method link

`Software/DemoCode/web/server.py` line 1272. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 12.9.

Purpose | What this contributes

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests.

Method | Current implementation

Connect capture, inference, profile selection, files and calibrated drawing through explicit requests. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ApiCurateDelete(author):
    p = paths.Profiles/("%s_curated.json" % author)
    existed = p.exists()
    if existed:
        p.unlink()
    PROFILES.clear()
    PROFILES.update(LoadProfiles())
    return jsonify({"deleted": existed, "file": p.name, "authorsNow": sorted(PROFILES)})
```

Justification | Interpret the method rather than infer a benchmark

Separate stages and saved jobs make the flow reviewable. Shared models/profiles avoid reloading on every request; current imports determine the real API contract. [78, 6, 3].

13 Independent line workers

Evidence | Current implementation boundary

`Software/DemoCode/web/lineworkers.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Configure inherited readers/classifiers; decide whether pooling is eligible; initialise per-process thread limits; map independent line jobs and preserve order, with serial fallback. The server remains responsible for page-level orchestration.

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
LINE_WORKERS = int(os.environ.get('LINE_WORKERS', '0'))
MIN_LINES_FOR_POOL = 4
_CFG = {}
```

Stage: Independent line workers

13.1 Method background: Independent line workers

Purpose | The role in this system

Run independent text or writer passes on multiple processes while restoring page order.

Justification | The reason for this construction

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time.

Alternatives | What changes if another method is used

Serial processing avoids pool overhead for short pages. Threads share memory but interact with Python and BLAS scheduling. Spawn workers need explicit model initialisation rather than this fork-inherited configuration.

Library | Implementation speed and accuracy

The code already uses Python multiprocessing.Pool and NumPy/BLAS. Fork is copy-on-write, not a guarantee of no allocation; passing crops/results still needs communication. Forking a threaded server with native numerical state is a deployment consideration. No new speedup or identical-output test was run. [1, 2].

13.2 Configure

```
def Configure(reader, namer, strokeNormalise)
```

Evidence | Source and method link

`Software/DemoCode/web/lineworkers.py` line 30. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 13.1.

Purpose | What this contributes

Run independent text or writer passes on multiple processes while restoring page order.

Method | Current implementation

Run independent text or writer passes on multiple processes while restoring page order. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Configure(reader, namer, strokeNormalise):
    _CFG["reader"] = reader
    _CFG["namer"] = namer
    _CFG["norm"] = bool(strokeNormalise)
```

Justification | Interpret the method rather than infer a benchmark

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time. [1, 2].

13.3 CanPool

```
def CanPool(n)
```

Evidence | Source and method link

`Software/DemoCode/web/lineworkers.py` line 37. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 13.1.

Purpose | What this contributes

Allows a pool only when LINE_WORKERS is not one, at least MIN_LINES_FOR_POOL jobs are present, os.fork exists and configuration is nonempty. Eligibility uses the number of jobs passed to Map: after orientation probes this can be smaller than page line count.

Method | Current implementation

Run independent text or writer passes on multiple processes while restoring page order. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def CanPool(n):
    return (LINE_WORKERS != 1 and n >= MIN_LINES_FOR_POOL
            and hasattr(os, "fork") and bool(_CFG))
```

Justification | Interpret the method rather than infer a benchmark

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time. [1, 2].

13.4 _Workers

```
def _Workers(n)
```

Evidence | Source and method link

`Software/DemoCode/web/lineworkers.py` line 43. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 13.1.

Purpose | What this contributes

Run independent text or writer passes on multiple processes while restoring page order.

Method | Current implementation

Run independent text or writer passes on multiple processes while restoring page order. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _Workers(n):
    if LINE_WORKERS > 0:
        return min(LINE_WORKERS, n)
    return max(1, min(os.cpu_count() or 1, n))
```

Justification | Interpret the method rather than infer a benchmark

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time. [1, 2].

13.5 _CapThreads

```
def _CapThreads()
```

Evidence | Source and method link

`Software/DemoCode/web/lineworkers.py` line 58. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 13.1.

Purpose | What this contributes

Sets thread-count environment variables and tries seven OpenBLAS entry-point spellings (including SciPy-prefixed wheel symbols) found in NumPy shared libraries. It is a best-effort cap, not a portable guarantee for every BLAS/backend. Environment settings after initialisation alone may not reconfigure an already-loaded backend.

Method | Current implementation

Run independent text or writer passes on multiple processes while restoring page order. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _CapThreads():
    for v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
        os.environ[v] = "1"
    try:
        import ctypes
        import glob
        import numpy
```

```
        root = os.path.dirname(os.path.dirname(os.path.abspath(numpy.__file__)))
        pats = [os.path.join(root, "numpy.libs", "*openblas*.so*"),
                os.path.join(root, "numpy", ".libs", "*openblas*.so*")]
        names = ("scipy_openblas_set_num_threads64_",
                 "scipy_openblas_set_num_threads_64_",
                 "scipy_openblas_set_num_threads",
                 "openblas_set_num_threads64_",
                 "openblas_set_num_threads",
                 "scipy_goto_set_num_threads64_",
                 "goto_set_num_threads")
        for pat in pats:
            for so in glob.glob(pat):
                lib = ctypes.CDLL(so)
                for fn in names:
                    if hasattr(lib, fn):
                        getattr(lib, fn)(1)
                        return True
    except Exception:
        pass
    return False
```

Justification | Interpret the method rather than infer a benchmark

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time. [1, 2].

13.6 _Init

```
def _Init()
```

Evidence | Source and method link

`Software/DemoCode/web/lineworkers.py` line 91. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 13.1.

Purpose | What this contributes

Run independent text or writer passes on multiple processes while restoring page order.

Method | Current implementation

Run independent text or writer passes on multiple processes while restoring page order. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _Init():
    _CapThreads()
```

Justification | Interpret the method rather than infer a benchmark

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time. [1, 2].

13.7 ReadOne

```
def ReadOne(job)
```

Evidence | Source and method link

`Software/DemoCode/web/lineworkers.py` line 96. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 13.1.

Purpose | What this contributes

Run independent text or writer passes on multiple processes while restoring page order.

Method | Current implementation

Run independent text or writer passes on multiple processes while restoring page order. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ReadOne(job):
    import numpy as np
    from PIL import Image
    import recognise as REC
    i, crop, flip = job
    img = np.rot90(crop, 2) if flip else crop
    text, score = REC.ReadTextScored(Image.fromarray(img).convert("L"),
                                     _CFG["reader"])
    return i, text, score
```

Justification | Interpret the method rather than infer a benchmark

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time. [1, 2].

13.8 NameOne

```
def NameOne(job)
```

Evidence | Source and method link

`Software/DemoCode/web/lineworkers.py` line 108. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 13.1.

Purpose | What this contributes

Run independent text or writer passes on multiple processes while restoring page order.

Method | Current implementation

Run independent text or writer passes on multiple processes while restoring page order. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def NameOne(job):
    import numpy as np
    from PIL import Image
    import recognise as REC
    i, crop = job
    _idx, p = REC.ClassifyImage(Image.fromarray(crop).convert("L"),
                                _CFG["namer"], strokeNormalise=_CFG["norm"])
    return i, np.asarray(p, dtype=np.float64)
```

Justification | Interpret the method rather than infer a benchmark

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time. [1, 2].

13.9 Map

```
def Map(fn, jobs)
```

Evidence | Source and method link

`Software/DemoCode/web/lineworkers.py` line 124. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 13.1.

Purpose | What this contributes

Uses an explicit fork multiprocessing context, creates a pool for the current map and returns results in supplied job order. Failures fall back to serial calls of the same function. Successful model inheritance does not remove crop/result IPC or pool-startup cost, and failure during a serial retry can still propagate.

Method | Current implementation

Run independent text or writer passes on multiple processes while restoring page order. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Map(fn, jobs):
    jobs = list(jobs)
    if not jobs:
        return []
    if not CanPool(len(jobs)):
        return [fn(j) for j in jobs]
    try:
        import multiprocessing as mp
        ctx = mp.get_context("fork")
        with ctx.Pool(processes=_Workers(len(jobs)), initializer=_Init) as pool:
            out = pool.map(fn, jobs, chunksize=1)
        return out
    except Exception as e:
        print("[lineworkers] pool unavailable (%s), running in one process"
              % type(e).__name__, file=sys.stderr, flush=True)
        return [fn(j) for j in jobs]
```

Justification | Interpret the method rather than infer a benchmark

Explicit fork workers inherit configured models and cap numerical threads to reduce oversubscription. The serial path remains available; startup and IPC still cost time. [1, 2].

14 Inspection images and glyph sheets

Evidence | Current implementation boundary

`Software/DemoCode/web/render.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Render stored glyphs and sample crops into inspection sheets after profiles or datasets exist. These outputs support inspection rather than replace recognition or physical validation.

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
SCRIPT_DIR = Path(__file__).resolve().parent
CACHE_DIR = paths.Cache / 'WebCache'
_FONT_CACHE = {}
```

Stage: Trajectory rasterisation

Method rationale, alternatives and library comparison: Section 8.11.

14.1 _font

```
def _font(size, bold=False)
```

Evidence | Source and method link

`Software/DemoCode/web/render.py` line 31. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.11.

Purpose | What this contributes

Make an image of a generated path for preview and recognition-based scoring.

Method | Current implementation

Make an image of a generated path for preview and recognition-based scoring. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _font(size,bold=False):
    key = (size,bold)
    if key not in _FONT_CACHE:
        name = "consolab.ttf" if bold else "consola.ttf"
        try:
            _FONT_CACHE[key] = ImageFont.truetype(f"C:/Windows/Fonts/{name}",size)
        except Exception:
            _FONT_CACHE[key] = ImageFont.load_default()
    return _FONT_CACHE[key]
```

Justification | Interpret the method rather than infer a benchmark

Sampling straight segments and applying a controlled width makes candidate rendering explicit. The uniform width models one drawing pen rather than source ink intensity. [63, 64, 3].

14.2 _RenderOneGlyph

```
def _RenderOneGlyph(g, cell=90, pad=8)
```

Evidence | Source and method link

`Software/DemoCode/web/render.py` line 44. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.11.

Purpose | What this contributes

Renders one stored glyph (list of strokes, each a list of (x,y) points in the normalized glyph frame) into a small square cell image, scaled to fit.

Method | Current implementation

Make an image of a generated path for preview and recognition-based scoring. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def _RenderOneGlyph(g,cell=90,pad=8):
    strokes = g.get("strokes",[])
    im = Image.new("L",(cell,cell),255)
    draw = ImageDraw.Draw(im)
    allpts = [p for s in strokes for p in s]
    if not allpts:
        return im
    xs = [p[0] for p in allpts]
    ys = [p[1] for p in allpts]
    x0,x1,y0,y1 = min(xs),max(xs),min(ys),max(ys)
    w = max(x1-x0,0.01)
    h = max(y1-y0,0.01)
    scale = (cell-2*pad)/max(w,h)

    def tx(p):
        return (pad+(p[0]-x0)*scale,cell-pad-(p[1]-y0)*scale)

    for s in strokes:
        pts = [tx(p) for p in s]
        if len(pts)>=2:
            draw.line(pts,fill=0,width=2)
    return im
```

Justification | Interpret the method rather than infer a benchmark

Sampling straight segments and applying a controlled width makes candidate rendering explicit. The uniform width models one drawing pen rather than source ink intensity. [63, 64, 3].

14.3 GlyphGridPath

```
def GlyphGridPath(author, profile)
```

Evidence | Source and method link

`Software/DemoCode/web/render.py` line 72. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.11.

Purpose | What this contributes

Returns the path to a cached glyph-grid PNG for ‘author‘, rendering it first if it doesn’t exist yet. One row per character, one column per stored variant, matching the manual-inspection sheets used during development.

Method | Current implementation

Make an image of a generated path for preview and recognition-based scoring. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def GlyphGridPath(author,profile):
    outPath = CACHE_DIR/f"glyphs_{author}.png"
    if outPath.exists():
        return outPath

    glyphs = profile.get("glyphs") or {}
    if not glyphs:
        raise KeyError(f"No glyphs in the profile for {author!r}")
    chars = sorted(glyphs.keys())
    maxVariants = max(len(v) for v in glyphs.values())
    cell = 90
    labelW = 28
    W = labelW+maxVariants*cell
    H = len(chars)*cell
    sheet = Image.new("RGB",(W,H),"white")
    draw = ImageDraw.Draw(sheet)
    font = _font(13,bold=True)
    for ri,c in enumerate(chars):
        draw.text((4,ri*cell+cell//2-8),repr(c)[1:-1],fill=(0,0,150),font=font)
        for ci,g in enumerate(glyphs[c]):
            im = _RenderOneGlyph(g,cell=cell)
            sheet.paste(im.convert("RGB"),(labelW+ci*cell,ri*cell))
    sheet.save(outPath)
    return outPath
```

Justification | Interpret the method rather than infer a benchmark

Sampling straight segments and applying a controlled width makes candidate rendering explicit. The uniform width models one drawing pen rather than source ink intensity. [63, 64, 3].

14.4 SampleCrops

```
def SampleCrops(author, n=2)
```

Evidence | Source and method link

`Software/DemoCode/web/render.py` line 105. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 8.11.

Purpose | What this contributes

Up to n real line crops for one writer, with transcripts the stats page shows these beside the glyph grid, so a viewer can compare the writer’s actual hand against what the machine draws. holdout lines

are skipped, since those are the measurement set, and only mid length lines are picked because a two word line shows nothing.

Method | Current implementation

Make an image of a generated path for preview and recognition-based scoring. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def SampleCrops(author,n=2):
    metaPath = CACHE_DIR/f"sample_{author}_meta.txt"
    cached = sorted(CACHE_DIR.glob(f"sample_{author}_*.png"))
    if len(cached)>=n and metaPath.exists():
        texts = metaPath.read_text(encoding="utf-8").splitlines()
        return list(zip(cached[:n],texts[:n]))

    import numpy as np
    from PIL import Image
    import train as TR

    base = TR.IAMLineDatasetRaw(rootDir=str(TR.DEFAULT_AUTHOR_DATA_DIR),
                                cacheDir=str(TR.DEFAULT_AUTHOR_CACHE_DIR))
    if author in paths.DatabaseWriters:
        base.samples = [s for s in base.samples
                        if s["page_key"].split("/")[0]==author]
    else:
        base.samples = []
        TR.AddPersonalSamples(base)
        base.samples = [s for s in base.samples
                        if s["page_key"].split("/")[0]==author]

    picked = []
    for s in base.samples:
        if s.get("is_holdout"):
            continue
        if 20<=len(s["text"])<=70:
            picked.append(s)
        if len(picked)>=n:
            break

    outPaths,texts = [],[]
    for i,s in enumerate(picked):
        arr = TR.DecodeCrop(s["imageBytes"])
        path = CACHE_DIR/f"sample_{author}_{i}.png"
        Image.fromarray(np.asarray(arr,dtype=np.uint8)).convert("L").save(path)
        outPaths.append(path)
        texts.append(s["text"])
    metaPath.write_text("\n".join(texts),encoding="utf-8")
    return list(zip(outPaths,texts))
```

Justification | Interpret the method rather than infer a benchmark

Sampling straight segments and applying a controlled width makes candidate rendering explicit. The uniform width models one drawing pen rather than source ink intensity. [63, 64, 3].

15 Training and numerical differentiation

Evidence | Current implementation boundary

`Software/DemoCode/train.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Decode source images and labels; construct page-separated data views and batches; augment and initialise; run train-mode forward passes; calculate losses and reverse derivatives; clip/update parameters; validate and save checkpoints. This chapter explains the report training methods and is outside live demo inference.

15.1 Expanded reasoning: data partitions, loss derivatives and training choices

15.1.1 The 15% holdout, seed and 20% personal representation

The inspected paths specify validation fraction 0.15 and split seed zero. A seeded shuffle fixes which examples are withheld, provided the example list and ordering are unchanged. The fraction controls data allocation; the seed controls reproducibility, not statistical quality. For a constructed 100-line population a 15% split with exact rounding gives 15 validation and 85 training lines. Real per-writer rounding and minimum-set guards can change totals. Never infer actual held-out counts from the fraction alone.

A line-level split can put neighbouring lines from one photographed page in both sets. This tests new lines under partly shared capture conditions, not new pages, pens or writers. A page-level split reduces that dependency; a writer-disjoint split answers a different question from closed-set writer identification. Fifteen percent is a sample-efficiency/precision compromise, not a statistically universal optimum. After tuning on validation, it no longer serves as a fully untouched final test.

The text trainer's personal fraction 0.20 controls representation during mixed sampling [3]. It does not make public and personal data equal: the intended shares are 80/20. In an ideal twenty-example batch, that would mean sixteen public and four personal examples, subject to the routine's rounding/replacement behaviour. Oversampling a small personal set can repeat examples; an epoch need not then mean one visit per unique image. This coefficient is separate from 0.2 dropout and from the 15% holdout. Its rationale is to prevent abundant public data overwhelming the personal domain; it does not guarantee absence of bias or optimum transfer.

15.1.2 Training settings: what each number controls

The report's first-principles defaults use RMSprop for text, AdamW for writers, initial learning rate 10^{-3}, weight penalties 10^{-5} and 10^{-4} respectively; text plateau reduction factor 0.5, patience 5 and minimum 10^{-5}; writer cosine annealing and gradient norm limit 5. These describe code defaults, not a recovered configuration for every saved checkpoint. Architecture sizes and preparation must also match the checkpoint.

RMSprop keeps $v_t = \alpha v_{t-1} + (1-\alpha)g_t^2$ and updates $\theta \leftarrow \theta - \eta g_t/(\sqrt{v_t}+\epsilon)$. This scales steps by recent squared-gradient magnitude [79]. The inspected defaults are $\alpha = 0.99, \epsilon = 10^{-8}$. Larger α

gives a longer memory; the numerical floor prevents unstable small denominators. The inspected RMSprop adds $\lambda\theta$ to the gradient, so its weight penalty is coupled L2-style regularisation.

Adam forms $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$ and $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$, corrects their initial bias using $1-\beta_1^t$ and $1-\beta_2^t$, and scales the first moment by the square root of the second [80]. Defaults 0.9 and 0.999 express different smoothing times. AdamW additionally subtracts $\eta\lambda\theta$ separately from the adaptive gradient update. Decoupling matters because adding L2 to an adaptively scaled gradient is not equivalent [81]. Neither optimiser proves a global optimum for these nonconvex networks.

Halving learning rate after a plateau reduces update size when the monitored validation quantity stops improving. "Patience 5" must be read with the code's comparison: the inspected counter uses `badEpochs > patience`, so the reduction follows more than five counted bad epochs. Cosine decay typically follows $\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})[1 + \cos(\pi t/T_{\max})]$ across the configured run. These are schedules, not measured physical learning rates. Changing loss reduction changes gradient scale and can invalidate a learning rate comparison even if the displayed numerical rate is unchanged.

15.1.3 Cross-entropy and its logit derivative from first principles

For logits z_k, stable softmax is $p_k = e^{z_k - m}/\sum_j e^{z_j - m}$ with $m = \max_j z_j$. Subtracting the common m leaves ratios unchanged. With true class y, $L = -\log p_y = -z_y + \log\sum_j e^{z_j}$. Direct differentiation gives $\partial L/\partial z_k = p_k - \mathbf{1}[k=y]$. For probabilities $(0.7, 0.2, 0.1)$ and true class one, loss is $-\log 0.7 \simeq 0.3567$ and gradient $(-0.3, 0.2, 0.1)$, summing to zero. Gradient descent raises the true logit relative to the others. Class indices are discrete labels; the derivative is with respect to the real logits.

A batch average $L_B = B^{-1}\sum_i L_i$ contributes a factor $1/B$ to every per-example gradient. Apply that factor once, not at every backward layer. An average makes nominal scale less sensitive to batch size, but the examples' gradient directions still differ. Class imbalance and repeated samples can still dominate the mean.

15.1.4 CTC target occupancy, backward convention and batch scaling

Use $Z = P(\ell \mid X)$ for total valid-path likelihood. Let $\alpha_{t,s}$ include the emission at (t,s) and let $\beta_{t,s}$ include only later emissions. Then conditional probability of state s at frame t is $\exp(\alpha_{t,s} + \beta_{t,s} - \log Z)$ in log notation. Summing over states carrying class k gives

Formula purpose and system use. Purpose in this explanation: CTC target occupancy, backward convention and batch scaling **Source basis:** [48, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$q_{t,k} = \sum_{s:\ell'_s = k} e^{\alpha_{t,s} + \beta_{t,s} - \log Z}, \qquad \frac{\partial L}{\partial z_{t,k}} = p_t(k) - q_{t,k}.$$

The condition $\ell'_s = k$ matters: state index and class index differ. Different states can carry the same letter or blank. If backward scores also include the current emission, divide it out once; otherwise it is counted twice. This convention issue is a frequent source of incorrect CTC implementations. The occupancy definition follows the valid-path posterior, not the greedy path [48].

For the constructed two-frame a example above, at frame one valid-path mass emitting a is $0.42+0.18 = 0.60$ and blank mass is 0.28. Thus $q_1(a) = 0.60/0.88 \simeq 0.6818$, and the a logit derivative is $0.6 - 0.6818 = -0.0818$. Occupancy sums to one at each valid frame; the logit gradients sum to zero. These identities provide hand-checkable invariants. An impossible target has $Z = 0$ and requires explicit failure handling.

The project averages sequence losses across a batch without target-length division. Equal coefficients $1/B$ do not mean equal influence per character: long sequences can contribute larger loss/gradient. PyTorch's documented `CTCLoss(reduction='mean')` divides by target length before averaging;

matching the project's batch mean requires summing per-sequence losses and dividing by batch size [53]. One must match blank index, valid input/target lengths, tensor order and impossible-path behaviour before calling a loss discrepancy a numerical bug.

15.1.5 Backward pass: how every major layer receives a derivative

For affine $Y = XW^T + b$ and upstream $G = \partial L/\partial Y$, $dW = G^T X$, $dX = GW$, $db = \sum_{\text{batch}} G$. A convolution can be written as this matrix multiplication after stacking patches (im2col). Its input gradient folds overlapping patch contributions back by *adding* them. Overwriting overlaps loses gradients. Bias sums over batch and spatial positions. The padding added for forward computation must be removed from the input gradient.

ReLU masks upstream values at nonpositive inputs. Max pooling scatters to saved argmax locations, accumulating if needed. Average pooling distributes equally. For batch normalisation, write $g_i = \partial L/\partial u_i$, $r = (v + \epsilon)^{-1/2}$ and averages over the same m values:

Formula purpose and system use. Purpose in this explanation: Backward pass: how every major layer receives a derivative **Source basis:** [48, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$d\beta = \sum_i g_i, \quad d\gamma = \sum_i g_i \hat{z}_i, \qquad dz_i = \gamma r \left[g_i - \overline{g} - \hat{z}_i \overline{g\hat{z}}\right].$$

This includes dependence of mean and variance on all batch values. Treating them as constants gives the inference derivative, not the training derivative. The distinction is mathematical; tolerances cannot repair a wrong formula.

For one LSTM step, combine upstream cell derivative $\bar{c}_t$ with the hidden contribution $\bar{h}_t \odot o_t \odot (1 - \tanh^2 c_t)$. Then $\bar{f}_t = \bar{c}_t \odot c_{t-1}$, $\bar{i}_t = \bar{c}_t \odot g_t$, $\bar{g}_t = \bar{c}_t \odot i_t$, $\bar{o}_t = \bar{h}_t \odot \tanh c_t$ and $\bar{c}_{t-1} = \bar{c}_t \odot f_t$ along the direct cell path. Multiply gate derivatives by sigmoid/tanh derivatives, concatenate in the same gate order as forward, form weight outer products, and propagate to x_t, h_{t-1}. Reverse temporal order during backpropagation through time. For the reverse-direction LSTM, undo the reversal when merging input derivatives. For stacked layers, output derivatives split across directions and then pass through the lower recurrent layer. These are explicit chain-rule calculations, not an invocation of automatic differentiation.

15.1.6 Clipping, initialisation, augmentation and their alternatives

Global norm clipping uses $g' = g \min(1, \tau/\|g\|_2)$ with $\tau = 5$ for the inspected writer trainer. A norm of 20 is scaled by 0.25; a norm of 2 is unchanged. It preserves direction, unlike elementwise clipping. It limits a gradient norm, not necessarily an adaptive optimiser's parameter-update norm. The method addresses exploding gradients [82]; threshold 5 remains a project setting. The reported text route does not apply the same clipping.

Variance-scaled initialisation attempts to keep activations/gradients from shrinking or growing through depth. For rectifiers, variance around $2/n_{\text{in}}$ motivates He-style scaling [83]; Xavier-style scaling uses fan-in and fan-out for different activation assumptions [84]. This does not make a freshly initialised model accurate. Learned batchnorm scales starting at one and offsets at zero initially avoid an extra arbitrary rescaling.

Augmentation perturbs only training examples with plausible shear, rotation, perspective or elastic deformation [22]. It assumes the label is unchanged. An excessive warp can violate that assumption or create nonphysical strokes. Validation preparation remains fixed, with dropout disabled and saved normalisation statistics, so its score is not an average over random training perturbations. Stronger augmentation is not automatically better, especially for writer identity where style cues themselves are labels.

15.1.7 Extended training algorithm and historical time evidence

Conceptually: choose fixed partitions; sample a batch and any domain mixture; augment only training images; apply the checkpoint's preparation; run the train-mode backbone and task head while saving forward caches; compute CTC or cross-entropy; apply the correct batch reduction; backpropagate through the head, pooling and convolution stack; clip if configured; update optimiser state/weights; at epoch end validate deterministically; update schedule; save a progress checkpoint and replace best only under the selected criterion. Text selection uses validation loss; writer selection uses validation accuracy. Resume requires optimiser moments, epoch/scheduler state and preferably random state, not just weights, to reproduce a continuous run.

The final report's Appendix C.6 retains writer epoch times around 19.5–20.9 minutes and selected reader epochs 7, 8, 11 and 16 at 122.3, 120.1, 111.6 and 112.3 minutes. These are reported log rows, not timings measured here. Multiplying one selected epoch by a desired epoch count estimates neither the exact complete run time nor the time on another host. Caches, validation, batch sizes, implementation and interruptions matter. "Best saved epoch" is not necessarily final epoch, and one loss value is not an accuracy. The logs and checkpoint metadata are needed to tie any quoted result to its host, precision, training route and selected model. No new training, inference or numerical comparison was run for this edition.

Current function reference

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
IDCT_M = IdctMatrix()
AUGMENTATIONS = [AugmentShear, AugmentRotate, AugmentElastic, AugmentPerspective]
MAX_SAFE_LABEL_CHARS = int(INPUT_WIDTH // 4 * 0.75)
HIDDEN_SIZE = 256
WEIGHTS_DIR = paths.Weights
DEFAULT_TEXT_CACHE_DIR = paths.Cache / 'line_image_cache'
DEFAULT_AUTHOR_CACHE_DIR = paths.Cache / 'line_cache_authors10'
DEFAULT_AUTHOR_DATA_DIR = paths.Data.parents[1] / 'Data' / 'Datasets' / 'IAMpages10'
DEFAULT_TEXT_DATA_DIR = DEFAULT_AUTHOR_DATA_DIR
```

Stage: Image codecs and cached crop bytes

15.2 Method background: Image codecs and cached crop bytes

Purpose | The role in this system

Convert stored image bytes into arrays and retain compact intermediate examples.

Justification | The reason for this construction

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion.

Alternatives | What changes if another method is used

Pillow/OpenCV decoders support many formats and compiled paths. Writing a decoder trades coverage and maintenance effort for explicit understanding. JPEG is lossy; PNG is lossless relative to encoded pixels.

Library | Implementation speed and accuracy

Pillow/libjpeg or OpenCV imread is the practical library alternative. Exact JPEG rounding and colour conversion can differ. Do not claim file-byte equality or recognition equivalence from a decoder name alone. [85, 86, 26].

15.3 IdctMatrix

```
def IdctMatrix()
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 536. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

Convert stored image bytes into arrays and retain compact intermediate examples.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def IdctMatrix():
    u = np.arange(8).reshape(8,1)
    x = np.arange(8).reshape(1,8)
    m = np.cos((2*x+1)*u*np.pi/16)*0.5
    m[0,:] *= 1.0/np.sqrt(2.0)   #the u=0 basis carries half the weight
    return m
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.4 UnfilterPng

```
def UnfilterPng(raw, height, stride, bpp)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 550. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

Undoes the per-row prediction a PNG encoder applied, the first three filters act on a whole row at once, average and Paeth depend on the pixel just reconstructed, so those two walk the row.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def UnfilterPng(raw,height,stride,bpp):
    out = np.zeros((height,stride),dtype=np.uint8)
    prev = np.zeros(stride,dtype=np.uint8)
    pos = 0
    for r in range(height):
        ft = raw[pos]
        line = np.frombuffer(raw,dtype=np.uint8,count=stride,offset=pos+1)
        pos += 1+stride
        if ft==0:
            cur = line.copy()
        elif ft==1:                                 #sub: a running total along the row
            if bpp==1:
                cur = (np.cumsum(line,dtype=np.int64)&255).astype(np.uint8)
            else:
                pad = (-stride)%bpp
                wide = np.concatenate([line.astype(np.int64),np.zeros(pad,dtype=np.int64)])
                cur = (np.cumsum(wide.reshape(-1,bpp),axis=0).reshape(-1)[:stride]&255
                       ).astype(np.uint8)
        elif ft==2:                                 #up: add the row above, wrapping at 256
            cur = line+prev
        else:
            cur = bytearray(line)
            pv = prev.tobytes()
            if ft==3:                               #average of left and above
                for i in range(min(bpp,stride)):
                    cur[i] = (cur[i]+(pv[i]>>1))&255
                for i in range(bpp,stride):
                    cur[i] = (cur[i]+((cur[i-bpp]+pv[i])>>1))&255
            else:                                   #paeth: whichever neighbour predicts best
                for i in range(min(bpp,stride)):
                    cur[i] = (cur[i]+pv[i])&255
                for i in range(bpp,stride):
                    a,b,c = cur[i-bpp],pv[i],pv[i-bpp]
                    est = a+b-c
                    da,db,dc = abs(est-a),abs(est-b),abs(est-c)
                    cur[i] = (cur[i]+(a if (da<=db and da<=dc) else (b if db<=dc else c)))&255
            cur = np.frombuffer(bytes(cur),dtype=np.uint8)
        out[r] = cur
        prev = cur
    return out
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.5 DecodePng

```
def DecodePng(data)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 592. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

Convert stored image bytes into arrays and retain compact intermediate examples.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def DecodePng(data):
    if data[:8]!=b"\x89PNG\r\n\x1a\n":
        raise ValueError("not a png")
    width = height = None
    bitDepth = colourType = interlace = 0
    palette = None
    idat = []
    i = 8
    while i<len(data):
        length = int.from_bytes(data[i:i+4],"big")
        kind = data[i+4:i+8]
        body = data[i+8:i+8+length]
        if kind==b"IHDR":
            width = int.from_bytes(body[0:4],"big")
            height = int.from_bytes(body[4:8],"big")
            bitDepth,colourType,interlace = body[8],body[9],body[12]
        elif kind==b"PLTE":
            palette = np.frombuffer(body,dtype=np.uint8).reshape(-1,3)
        elif kind==b"IDAT":
            idat.append(body)
        elif kind==b"IEND":
            break
        i += 12+length
    if bitDepth!=8 or interlace!=0:
        raise ValueError("only 8 bit non interlaced png is handled, got depth %d interlace %d"
                         % (bitDepth,interlace))
    channels = {0:1,2:3,3:1,4:2,6:4}[colourType]
    raw = zlib.decompress(b"".join(idat))
    pixels = UnfilterPng(raw,height,width*channels,channels)
    pixels = pixels.reshape(height,width,channels)
    if colourType==3:
        return palette[pixels[:,:,0]]
    if colourType in (0,4):
        return pixels[:,:,0]          #the second channel of type 4 is alpha
    return pixels[:,:,:3]
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.6 BuildHuffTable

```
def BuildHuffTable(counts, values)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 632. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

Rebuilds a JPEG huffman table from the per-length code counts and the values, with codes of eight bits or fewer also in a 256-entry lookup.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def BuildHuffTable(counts,values):
    minCode = [0]*17
    maxCode = [-1]*17
    valPtr = [0]*17
    code = k = 0
    lut = np.full(256,-1,dtype=np.int32)
    lutLen = np.zeros(256,dtype=np.uint8)
    for length in range(1,17):
        valPtr[length] = k
        minCode[length] = code
        for _ in range(counts[length-1]):
            if length<=8:
                base = code<<(8-length)
                for fill in range(1<<(8-length)):
                    lut[base+fill] = values[k]
                    lutLen[base+fill] = length
            code += 1
            k += 1
        maxCode[length] = code-1
        code <<= 1
    return dict(minCode=minCode,maxCode=maxCode,valPtr=valPtr,values=values,
                lut=lut,lutLen=lutLen)
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.7 JpegBitReader

```
class JpegBitReader
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 659. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

The entropy-coded bytes as a stream of bits, dropping the zero stuffed after every 0xFF and stepping over restart markers on request.

Method | Methods exposed by this current class

```
def __init__(self, data, pos)
def Fill(self)
def Peek8(self)
def Skip(self, n)
```

```
def Receive(self, n)
def Restart(self)
```

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class JpegBitReader:
    def __init__(self,data,pos):
        self.data = data
        self.pos = pos
        self.bits = 0
        self.nBits = 0

    def Fill(self):
        while self.nBits<=24:
            if self.pos>=len(self.data):
                self.bits = (self.bits<<8)|0
                self.nBits += 8
                continue
            b = self.data[self.pos]
            if b==0xFF:
                nxt = self.data[self.pos+1] if self.pos+1<len(self.data) else 0xD9
                if nxt==0x00:
                    self.pos += 2
                else:
                    self.bits = (self.bits<<8)|0   #a marker: feed zeros
                    self.nBits += 8
                    continue
            else:
                self.pos += 1
            self.bits = (self.bits<<8)|b
            self.nBits += 8

    def Peek8(self):
        if self.nBits<8:
            self.Fill()
        return (self.bits>>(self.nBits-8))&0xFF

    def Skip(self,n):
        self.nBits -= n
        self.bits &= (1<<self.nBits)-1

    def Receive(self,n):
        if n==0:
            return 0
        if self.nBits<n:
            self.Fill()
        v = (self.bits>>(self.nBits-n))&((1<<n)-1)
        self.Skip(n)
        return v

    def Restart(self):
        self.bits = 0
        self.nBits = 0
        while self.pos+1<len(self.data):
            if self.data[self.pos]==0xFF and 0xD0<=self.data[self.pos+1]<=0xD7:
                self.pos += 2
                return
            self.pos += 1
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.8 DecodeHuffValue

```
def DecodeHuffValue(reader, table)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 715. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

Convert stored image bytes into arrays and retain compact intermediate examples.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def DecodeHuffValue(reader,table):
    peek = reader.Peek8()
    length = int(table["lutLen"][peek])
    if length:
        reader.Skip(length)
        return int(table["lut"][peek])
    code = 0
    for length in range(1,17):
        code = (code<<1)|reader.Receive(1)
        if code<=table["maxCode"][length]:
            return table["values"][table["valPtr"][length]+code-table["minCode"][length]]
    raise ValueError("bad huffman code")
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.9 Extend

```
def Extend(value, nBits)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 729. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

Convert stored image bytes into arrays and retain compact intermediate examples.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Extend(value,nBits):
    if nBits==0:
        return 0
    return value if value>=(1<<(nBits-1)) else value-(1<<nBits)+1
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.10 DecodeJpeg

```
def DecodeJpeg(data)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 740. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

A baseline JPEG file to a grey array, only the luminance component is reconstructed, because in a JPEG that channel already is the grey image, the colour blocks are still huffman decoded, since skipping them would lose the place in the bit stream, but are then discarded.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def DecodeJpeg(data):
    if data[:2]!=b"\xff\xd8":
        raise ValueError("not a jpeg")
    quant = {}
    huff = {}
    comps = []
    height = width = 0
    restartInterval = 0
    i = 2
    scanPos = None
    scanTables = {}
    while i<len(data)-1:
        if data[i]!=0xFF:
            i += 1
            continue
        marker = data[i+1]
        if marker in (0xD8,0x01) or 0xD0<=marker<=0xD7:
            i += 2
            continue
        if marker==0xD9:
            break
        length = int.from_bytes(data[i+2:i+4],"big")
```

```
        body = data[i+4:i+2+length]
        if marker==0xDB:                                    #quantisation tables
            p = 0
            while p<len(body):
                precision,tableId = body[p]>>4,body[p]&15
                step = 64*(2 if precision else 1)
                raw = np.frombuffer(body[p+1:p+1+step],
                                    dtype=">u2" if precision else np.uint8).astype(np.int32)
                quant[tableId] = raw
                p += 1+step
        elif marker in (0xC0,0xC1):                         #baseline frame
            height = int.from_bytes(body[1:3],"big")
            width = int.from_bytes(body[3:5],"big")
            for c in range(body[5]):
                cid,samp,qid = body[6+c*3],body[7+c*3],body[8+c*3]
                comps.append(dict(id=cid,h=samp>>4,v=samp&15,q=qid))
        elif marker in (0xC2,0xC3,0xC5,0xC6,0xC7,0xC9,0xCA,0xCB,0xCD,0xCE,0xCF):
            raise ValueError("only baseline jpeg is handled, this file is SOF%d" % (marker-0xC0))
        elif marker==0xC4:                                  #huffman tables
            p = 0
            while p<len(body):
                cls,tableId = body[p]>>4,body[p]&15
                counts = list(body[p+1:p+17])
                total = sum(counts)
                values = list(body[p+17:p+17+total])
                huff[(cls,tableId)] = BuildHuffTable(counts,values)
                p += 17+total
        elif marker==0xDD:                                  #restart interval
            restartInterval = int.from_bytes(body[0:2],"big")
        elif marker==0xDA:                                  #start of scan
            for c in range(body[0]):
                cid,tables = body[1+c*2],body[2+c*2]
                scanTables[cid] = (tables>>4,tables&15)
            scanPos = i+2+length
            break
        i += 2+length
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.11 ReadImageFile

```
def ReadImageFile(path)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 865. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

A page on disk as a grey array, with the decoder picked from the file's own signature and colour flattened by the luminance weights.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below

is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ReadImageFile(path):
    data = Path(path).read_bytes()
    if data[:8]==b"\x89PNG\r\n\x1a\n":
        arr = DecodePng(data)
    elif data[:2]==b"\xff\xd8":
        arr = DecodeJpeg(data)
    else:
        raise ValueError("unrecognised image format: %s" % path)
    if arr.ndim==3:
        arr = np.rint(arr[...,:3]@np.array([0.299,0.587,0.114])).astype(np.uint8)
    return arr
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

15.12 EncodeCrop

```
def EncodeCrop(arr)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 878. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

Convert stored image bytes into arrays and retain compact intermediate examples.

Inputs

`arr` a line crop as an array

Output. the crop packed into bytes for storage.

Justification | Why crops are cached at all

Extracting line crops from a page is slow: the page must be read, binarised, analysed for line positions and cut. Training visits every line once per epoch for dozens of epochs, so repeating that work would dominate the run. The crops are therefore extracted once and stored.

Method | What encoding means here

The array is flattened into a byte sequence with its shape recorded alongside, so it can be written into a single file and read back by offset. Storing each crop as a separate image file would mean tens of thousands of small files, which is slow to open and wasteful of disk, since a filesystem allocates a whole block to a file however small.

15.13 DecodeCrop

```
def DecodeCrop(raw)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 887. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.2.

Purpose | What this contributes

Dispatches cached crop decoding using current byte representation and returns pixels for the ordinary preparation path. Cached and fresh files must not be conflated; cache format is an implementation contract.

Method | Current implementation

Convert stored image bytes into arrays and retain compact intermediate examples. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def DecodeCrop(raw):
    try:
        return np.load(io.BytesIO(zlib.decompress(raw)),allow_pickle=False)
    except zlib.error:
        return np.asarray(DecodePng(raw),dtype=np.uint8)
```

Justification | Interpret the method rather than infer a benchmark

PNG reconstruction uses prediction filters and lossless decompression; JPEG uses entropy decoding and inverse DCT. This exposes training-file arithmetic, while the demo server uses Pillow for image ingestion. [85, 86, 26].

Stage: Datasets partitions and sampling

15.14 Method background: Datasets partitions and sampling

Purpose | The role in this system

Keep image and label identities aligned and separate held-out evaluation from learning.

Justification | The reason for this construction

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference.

Alternatives | What changes if another method is used

Page-disjoint partitions reduce shared capture leakage; writer-disjoint testing asks a different question from closed-set writer naming. Oversampling changes representation, not the number of independent examples.

Library | Implementation speed and accuracy

Framework Dataset/DataLoader objects organise batches and workers, not the validity of a partition. More worker processes can duplicate cache/model memory. Actual split manifests and run metadata outrank defaults. [87, 3, 88].

15.15 ReadLabelLines

```
def ReadLabelLines(imgPath, labelPath=None)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 894. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`imgPath`
: the page image's path

`labelPath`
: the transcript's path, derived from the image's if not given

Output. the transcript as a list of lines.

Purpose | What it is for

Reads the text that was written on a page. Supervised training needs the correct answer for every input, and for a page of handwriting that is the transcript, typed when the page was collected.

Justification | Why the transcript is per line rather than per page

The reader works on one line at a time, so each crop needs its own label. Keeping the transcript as one line of text per line of writing means the correspondence is positional: the n-th transcript line labels the n-th crop. That is what makes it possible to check the extraction, since a page whose crop count does not match its transcript line count has been segmented wrongly and can be rejected rather than silently mislabelled.

> *Deriving the transcript's path from the image's by convention, rather than being told it, is what lets a directory of pages be processed by listing the images alone. The cost is that the convention is implicit: a transcript named differently is simply not found, and the page is skipped.*

15.16 ClusterRows

```
def ClusterRows(rows, maxGap=5)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 902. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`rows`
: row indices

`maxGap`
: the largest gap that still counts as the same cluster, default 5

Output. the indices grouped into runs of nearby rows.

Purpose | What it is for

Given a set of interesting rows, such as those containing a printed rule, groups the ones that are close together. A single printed line a few pixels thick registers as several adjacent rows, and what is wanted is one entry per rule rather than one per row.

Method | The rule

The indices are taken in order and a new cluster is started whenever the gap to the previous one exceeds the limit. This is single-link clustering in one dimension, and it is the natural formulation when the data is ordered: two rows belong together if they are close, and closeness chains, so a run of rows each near the last forms one cluster however long it is.

> *A gap of five pixels suits the printed rules on ruled paper, which are a pixel or two thick with occasional breaks where writing crosses them. The chaining property is what bridges those breaks, and it is also the method's weakness: a dense enough sequence of separate features merges into one cluster.*

15.17 FindRuleLineClusters

```
def FindRuleLineClusters(binaryImg, rawGrayscaleArray=None)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 910. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`binaryImg`
: the binarised page

`rawGrayscaleArray`
: the original grey page, if available

Output. the row clusters that look like printed rules.

Purpose | What it is for

Finds the printed ruling on lined paper, so that it can be removed before the handwriting is analysed and so that its positions can hint at where the lines of writing sit.

Method | How a printed rule is recognised

By the same property segment.py uses: a printed rule crosses the whole page in one straight unbroken run, which handwriting never does. Rows containing an unusually long horizontal run of ink are therefore candidates, and clustering them groups each rule into one entry.

Justification | Why the rules are useful as well as unwanted

They are removed because they corrupt the analysis, joining every letter on a line into one blob. But their positions are also the best possible evidence about where lines of writing are, because the writer

wrote on them. So they are returned rather than merely deleted, and the line splitter takes them as hints.

> *This is a separate implementation from the equivalent in segment.py, because this file processes database pages and that one processes photographs, which differ enough that one routine serving both would be a compromise for each. The duplication is real but the inputs genuinely differ: a database scan has even lighting and no perspective.*

15.18 RemoveStraightFormLines

```
def RemoveStraightFormLines(hwRegion)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 919. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`hwRegion`
: the region of the page containing handwriting

Output. the region with the printed ruling deleted.

Method | What it does

Deletes the long straight runs identified as printing, leaving the handwriting. The reasoning is set out at length under RemovePrintedStructure in segment.py: length, not darkness or thickness, distinguishes printing from writing, and the threshold sits in a wide gap between the two.

Justification | Why the removal must be of whole runs

Removing the rule at the places where writing crosses it would cut the writing too, since at a crossing the same pixels belong to both. The remedy used here and in segment.py is to restrict the deletion to pixels that form part of a long run, which a letter's stroke does not, so a crossing loses the rule and keeps the letter.

15.19 DetectPageBounds

```
def DetectPageBounds(binaryImg, rawGrayscaleArray=None)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 930. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`binaryImg`
: the binarised page

`rawGrayscaleArray`
: the grey page, if available

Output. the rectangle of the page that holds handwriting.

Purpose | What it is for

A database page is a scanned form, with a printed header, a typed prompt and sometimes a footer, of which only the handwritten region is wanted. This finds that region so the rest can be ignored.

Justification | Why the printed parts must be excluded rather than tolerated

Printed text is text, so the reader would read it, and it has no corresponding transcript line, so every crop after it would be labelled with the wrong line. A single extra crop at the top of a page misaligns the whole page's labels, which is far worse than losing the page: a mislabelled example teaches the network something false, where a missing one teaches nothing.

> *This is why the crop count is checked against the transcript line count and the page rejected if they disagree. The check cannot tell which crop is wrong, only that something is, and rejecting the page is the right response: the database has thousands of pages and losing a few is cheaper than training on bad labels.*

15.20 MergeUndersizedFragments

```
def MergeUndersizedFragments(lines, minRelativeHeight=0.4)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 942. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`lines`
: the candidate line regions

`minRelativeHeight`
: the least height relative to the typical line, default 0.4

Output. the regions with the too-short ones merged into their neighbours.

Purpose | The problem it solves

Line splitting can divide one line of writing into two, typically when a row of low ink inside the line is mistaken for the gap between lines. The symptom is a region much shorter than the others, since one real line has been halved.

Justification | Why the test is relative

The height is compared with the typical line height on the same page rather than with a fixed figure, because the right height depends on the writing and the scan resolution. A region under 40 percent

of the typical height cannot be a line, since no writer varies their line height that much within a page, so it must be a fragment of one.

Justification | Why merging rather than discarding

A fragment contains real writing, which the transcript expects to find. Discarding it would lose that text and, worse, would change the crop count and so misalign every label after it. Merging it back into its neighbour restores both the writing and the count.

15.21 MergeOrSplitToExpectedLineCount

```
def MergeOrSplitToExpectedLineCount(lines, rowInkCount, expectedLineCount)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 975. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`lines`
: the candidate line regions

`rowInkCount`
: the ink per row profile

`expectedLineCount`
: how many lines the transcript has

Output. the regions adjusted to match that count.

Method | The idea: use the answer to fix the method

The transcript says how many lines the page has. That is information the splitter did not use, and it is reliable, because it was typed by a person reading the page. So if the splitter produced the wrong number, the discrepancy is known exactly, and the regions can be adjusted until the count agrees: merging the most doubtful boundary when there are too many, splitting the most promising region when there are too few.

Justification | Why this is legitimate and where its limit is

Using the label to guide the extraction would be improper if the label were being predicted, and it is not: the task here is to prepare training data, and the transcript is given for exactly that purpose. It is the same reasoning that justifies forced alignment in style.py, where the known text is used to locate characters rather than to recognise them.

The limit is that it cannot verify correctness, only count. A page split in the wrong places but into the right number of pieces passes, and its labels are then confidently wrong. The count is a necessary condition for a correct extraction, not a sufficient one.

15.22 EstimateLinePitch

```
def EstimateLinePitch(smoothedRowInk, minPitch=40, maxPitch=400)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1010. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`smoothedRowInk`
: the smoothed ink per row profile

`minPitch`
: the smallest pitch considered, default 40

`maxPitch`
: the largest, default 400

Output. the estimated spacing between lines of writing, in pixels.

Purpose | What it is for

The same quantity LinePitch finds in segment.py, estimated from a smoothed row profile, and used as the scale for the line splitting below. The reasoning for using a projection profile, and for smoothing it first, is given in that entry.

Justification | Why the search is bounded

The bounds keep the estimate within the range a line of writing on a scanned page can plausibly occupy. Without a lower bound the method can lock onto the spacing between the strokes within a word, which is a genuine periodicity in the profile and a much shorter one. Without an upper bound it can lock onto the spacing between paragraphs. Both are real patterns; the bounds say which scale is being asked about.

15.23 SplitLinesByPeriodicity

```
def SplitLinesByPeriodicity(binaryRegion, ruleLineHints=None, smoothWindow=9, minLineHeight
    =30)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1023. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Splits a block that is really several evenly spaced lines, at the valleys of its ink-per-row profile.

Inputs

`binaryRegion`
: the handwriting region

`ruleLineHints`
: the printed rule positions, if found

`smoothWindow`
: the profile smoothing width, default 9

`minLineHeight`
: the shortest acceptable line, default 30

Output. the row ranges of the individual lines of writing.

Justification | The approach, and why periodicity is the right tool

Lines of writing on a page are close to evenly spaced, because the writer followed the ruling or kept a steady hand. That regularity is strong evidence and it is what the splitter uses: rather than looking for each gap independently, it establishes the pitch and then expects boundaries at roughly that spacing.

The advantage over independent gap-finding is robustness to a single bad gap. A line whose descenders reach into the next line leaves almost no gap at all, and a method looking only for gaps misses that boundary entirely, merging two lines. A method that knows the expected spacing looks for a boundary where one ought to be and places it at the best available row, even where that row is not a clear gap.

Justification | Why the printed rules are used as hints

Where the ruling was found, it says where the writer intended each line to sit, which is better evidence than the ink profile: it is unaffected by descenders, by a line of unusual height, or by a blank line. The hints are used to guide the boundaries rather than to set them, because writers do not follow ruling exactly.

> *This whole routine is a separate, earlier solution to the problem segment.py solves with seam carving. The seam approach is better, since it cuts around descenders instead of choosing a row to cut through. This one remains because it is what the cached training data was extracted with, and re-extracting it would change the data the reported checkpoints were trained on.*

15.24 ExtractLinePatches

```
def ExtractLinePatches(page, targetHeight=32, maxWidth=1024, expectedLineCount=None,
    labelLines=None, isDataset=True)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1051. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`page`
: the page image

`targetHeight`
: the height to scale crops to, default 32

`maxWidth`
: the greatest width kept, default 1024

`expectedLineCount`
: how many lines the transcript has

`labelLines`
: the transcript lines

Output. the line crops, with their labels where the count matched.

Purpose | What it is for

The whole page-to-crops pipeline for database pages, in one call: find the page bounds, remove the printed ruling, split into lines, merge the fragments, reconcile against the expected count, and cut and scale each crop.

Justification | Why the crops are scaled here

Each crop is scaled to a fixed height, preserving its aspect ratio. Doing it at extraction rather than at training time means the cache holds crops of a known height, so loading a batch needs no resizing, and the cache is smaller, since a scanned line is several times taller than the network needs.

Justification | Why width is capped rather than scaled

An unusually wide crop is truncated rather than squeezed. Squeezing would change the aspect ratio and so the letter proportions, for the reasons given under ResizeLineImageFixed. An over-wide crop is also usually a fault rather than a long line, two lines having been merged, so truncating loses little that was right.

> *Where the crop count does not match the transcript, the crops are returned without labels and the page cannot be used for training the reader, though it remains usable for the writer namer, which needs only to know who wrote the page. That is why the two networks are trained from different views of the same cache.*

15.25 EncodeText

```
def EncodeText(text)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1169. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`text`
: a line of text

Output. the sequence of class indices for its characters.

Method | What it does

Converts a label into the numbers CTC works with, by looking each character up in the charset mapping from recognise.py and dropping anything not in it.

Justification | Why unknown characters are dropped rather than substituted

A character outside the charset has no output slot, so the network cannot be asked to produce it. Substituting a placeholder would teach it to emit that placeholder whenever something unfamiliar appears, which is worse than omitting it: the network would learn a response to a situation it can never handle correctly.

Dropping the character does introduce a small inconsistency, since the image contains a mark the label does not mention. CTC tolerates that, because it allows the network to emit blank there, and the alternative would be discarding every line containing any unusual character, which over a large database is a substantial loss.

15.26 FindPeaks

```
def FindPeaks(sm, floorFrac=0.0, minGap=0, prominence=None, distance=None)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1177. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Peaks in a profile, where a peak counts only if the profile falls by the prominence before it rises higher, and no two kept peaks are closer than the distance.

Inputs

`sm` a smoothed profile

`floorFrac`
: the least height as a fraction of the maximum

`minGap`
: the least separation between peaks

`prominence`
: the least prominence required

`distance`
: an alternative name for the separation

Output. the indices of the accepted peaks, in order.

Purpose | What it is for

A general peak finder, used by the line splitting. It applies three filters in turn, each addressing a different way a spurious peak arises.

Method | Finding candidates

A candidate is at least as large as its left neighbour and strictly larger than its right. The asymmetry is deliberate, as the code comment notes: if both comparisons were inclusive, a flat-topped peak several samples wide would report every one of those samples. With one strict comparison it reports exactly one.

Method | The height floor, and a trap in it

A candidate must exceed a fraction of the profile's maximum. The comment records a real pitfall: the floor must not default to zero, because the profile can be negative. A profile with its mean removed

has negative valleys, and a peak in a negative region is still a peak. Defaulting the floor to negative infinity rather than zero is what makes the filter optional rather than silently active.

Method | Prominence, which is the interesting filter

Prominence measures how much a peak stands out from its surroundings, as opposed to how tall it is. It is defined as the peak's height above the higher of the two lowest points reached before the profile rises above the peak again on either side.

Formula purpose and system use. Reject row-profile maxima whose prominence or separation is inadequate. **Source basis:** [87, 3, 88]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{prom}(i) \;=\; \text{sm}[i] \;-\; \max\Big(\min_{j\in\text{left}} \text{sm}[j],\; \min_{j\in\text{right}} \text{sm}[j]\Big) \tag{15.1}$$

where

prom(i)
: the prominence of the peak at index i

sm[i]
: the profile value at the peak

left, right
: the stretches on each side, walked outward until the profile exceeds the peak

min
: the lowest value reached in each stretch, called the key saddle

max
: the higher of the two, which is what the peak must rise above to be independent

The code comment names what this rejects: a ripple riding on a big peak's flank. Such a ripple can be tall in absolute terms, taller than a genuine small peak elsewhere, so a height filter keeps it. But it does not descend far on either side before the profile rises again, so its prominence is small and this filter removes it. Height asks how large a peak is; prominence asks whether it is a peak at all rather than a bump on something larger. The term is borrowed from topography, where it distinguishes an independent summit from a shoulder of a bigger mountain.

Method | The separation filter

Finally, peaks too close together are thinned by the same greedy strongest-first suppression used for the line centres in segment.py: take the tallest, exclude everything within the minimum separation, repeat. Processing tallest first is what makes it correct rather than arbitrary, for the reason given there.

> *The three filters are each optional and disabled by default, so the function is a plain local-maximum finder unless asked for more. That keeps one implementation usable for both the careful line-splitting case and the simple ones.*

15.27 LineImageCache

```
class LineImageCache
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1294. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keeps extracted line crops on disk, in shards with a manifest, so a rerun does not cut the pages up again.

Method | Methods exposed by this current class

```
def __init__(self, rootDir, cacheDir='line_image_cache', ForceRebuild=False, SaveEveryNPages=10,
    maxPages=None)
def Get(self, pageKey)
```

Output. an object holding extracted crops on disk and serving them by index.

Purpose | What it is for

Stores every extracted crop in a single file with an index of where each one starts, so a crop can be read by number without reading the rest. It is what makes the extraction cost payable once rather than once per epoch.

Justification | Why one file with an index

Tens of thousands of separate image files would be slow, because each open is a separate filesystem operation, and wasteful, because a filesystem allocates a whole block per file. One file read at an offset is a single operation, and the index is small enough to hold in memory.

Justification | Why the whole cache is not simply loaded into memory

The extracted crops for the full database run to several gigabytes, more than is comfortable to hold while also holding the activations a training step needs. Reading each crop as the batch needs it keeps the memory use to the batch, and the operating system's own file cache keeps the recently used parts in memory anyway, which recovers most of the speed without the program managing it.

15.28 IAMLineDatasetRaw

```
class IAMLineDatasetRaw
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1413. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

The database pages line by line, each paired with its transcript and marked held out or not, held-out lines never reach training.

Method | Methods exposed by this current class

```
def __init__(self, rootDir, MinPagesForHoldout=3, cacheDir='line_image_cache', ForceRebuild=False,
    SaveEveryNPages=10, maxPages=None, isTrain=True)
def ResolveDatasetRoot(rootDir)
def __len__(self)
def __getitem__(self, idx)
def HoldoutSplitIndices(self)
```

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class IAMLineDatasetRaw:
    def __init__(
        self,
        rootDir,
        MinPagesForHoldout=3,
        cacheDir="line_image_cache",
        ForceRebuild=False,
        SaveEveryNPages=10,
        maxPages=None,
        isTrain=True,
    ):
        self.samples = []
        self.isTrain = isTrain

        rootDir = self.ResolveDatasetRoot(Path(rootDir))

        ImageCache = LineImageCache(
            rootDir,
            cacheDir=cacheDir,
            ForceRebuild=ForceRebuild,
            SaveEveryNPages=SaveEveryNPages,
            maxPages=maxPages,
        )

        self.authorFolders = sorted(p.name for p in rootDir.iterdir() if p.is_dir())
        self.authorToIdx = {author: idx for idx,author in enumerate(self.authorFolders)}

        MismatchedPages = 0
        OversizedLabels = 0
        for authorId in self.authorFolders:
            authorDir = rootDir/authorId
            AuthorPages = sorted(authorDir.glob("*.png"))
            if not AuthorPages:
                continue
            LabeledPages = [p for p in AuthorPages if ReadLabelLines(str(p))]
            HoldoutPageName = LabeledPages[-1].name if len(LabeledPages)>=MinPagesForHoldout else
    None

            for imgPath in AuthorPages:
                pageKey = f"{authorId}/{imgPath.name}"
                LabelLines = ReadLabelLines(str(imgPath))
                if not LabelLines:
                    continue

                pngList = ImageCache.Get(pageKey)
                if pngList is None:
                    continue

                if len(pngList)!=len(LabelLines):
                    MismatchedPages+=1
                    print(f"[Dataset] Skipping {pageKey}: cached image has {len(pngList)} line(s) but
     "
                          f"the current label has {len(LabelLines)} line(s) -- segmentation and label
     "
                          f"disagree on line count for this page. This page's images need re-cropping
     "
                          f"(delete/rebuild line_image_cache/ for it) before it can be used.")
                    continue

                isHoldout = imgPath.name==HoldoutPageName
                for lineIdx in range(len(LabelLines)):
                    text = LabelLines[lineIdx]
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.29 TrainEvalView

```
class TrainEvalView
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1518. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Method | Methods exposed by this current class

```
def __init__(self, BaseDataset, indices, isTrain)
def __len__(self)
def __getitem__(self, i)
```

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class TrainEvalView:
    def __init__(self,BaseDataset,indices,isTrain):
        self.baseDataset = BaseDataset
        self.indices = indices
        self.isTrain = isTrain

    def __len__(self):
        return len(self.indices)

    def __getitem__(self,i):
        item = self.baseDataset.samples[self.indices[i]]
        img = ResizeLineImageFixed(DecodeCrop(item["imageBytes"]))
        if self.isTrain:
            img = AugmentLineImage(img)
        imgTensor = TensorFromResized(img)
        return imgTensor,item["target"],item["text"]
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.30 TextCollateFn

```
def TextCollateFn(batch)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1536. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`batch`
: a list of crop and label pairs

Output. the batch assembled into arrays.

Method | What collation is

A dataset serves one item at a time; a network wants a batch as one array. **Collation** is the step between: taking a list of items and stacking them, which for images means padding them to a common size.

Justification | Why the labels stay as a list

The crops are stacked into one array, but the labels cannot be: they are sequences of different lengths, and CTC requires each one's true length, since padding a label would add characters the image does not contain. So the labels are kept as a list of varying-length sequences, which is exactly what CtcLoss expects and why that function loops over the batch.

15.31 AuthorLabeledView

```
class AuthorLabeledView
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1547. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

The same crops labelled by writer instead of by text, each one redrawn at the one pen width ClassifyImage uses at inference.

Method | Methods exposed by this current class

```
def __init__(self, base, indices, isTrain)
def __len__(self)
def __getitem__(self, i)
```

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class AuthorLabeledView:
    def __init__(self,base,indices,isTrain):
        self.base = base
```

```
        self.indices = indices
        self.isTrain = isTrain

    def __len__(self):
        return len(self.indices)

    def __getitem__(self,i):
        item = self.base.samples[self.indices[i]]
        gray = DecodeCrop(item["imageBytes"])
        normImg = StrokeNormalize(np.array(gray))
        if normImg is not None:
            gray = normImg   #the raw crop is kept only if no usable ink was found
        img = ResizeLineImageFixed(gray)
        if self.isTrain:
            img = AugmentLineImage(img)
        imgTensor = TensorFromResized(img)
        authorId = item["page_key"].split("/")[0]
        authorIdx = self.base.authorToIdx[authorId]
        return imgTensor,authorIdx,item["page_key"]
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.32 AuthorCollateFn

```
def AuthorCollateFn(batch)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1571. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`batch`
: a list of crop and writer-index pairs

Output. the batch assembled into arrays.

Method | How it differs from the text version

Simpler, because the labels are single integers rather than sequences, so they stack into one array like the crops. There is no variable length to preserve and no padding question for the labels.

> *That simplicity is a direct consequence of the task: classification has one answer per example, where transcription has a sequence whose length varies. It is the same distinction that makes the namer need no CTC and no recurrence.*

15.33 AddPersonalSamples

```
def AddPersonalSamples(base)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1580. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Cuts the photographed writers' pages into lines and adds them to a dataset, keyed on file name, size and a digest of each transcript so the result can be cached across machines.

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def AddPersonalSamples(base):
    cachePath = paths.Cache/"personal_lines_cache.pkl"
    stamp = []
    for author in paths.PersonalWriters:
        for imgPath,labelPath in paths.PersonalWriterPages(author):
            imgSize = imgPath.stat().st_size if imgPath.exists() else 0
            digest = (hashlib.sha1(labelPath.read_bytes()).hexdigest()
                      if labelPath.exists() else "")
            stamp.append((imgPath.name,imgSize,digest))
    key = tuple(sorted(stamp))

    if cachePath.exists():
        try:
            with open(cachePath,"rb") as f:
                cached = pickle.load(f)
            if cached.get("key")==key:
                base.samples.extend(cached["samples"])
                print(f"[Personal] {len(cached['samples'])} line samples read from the cache",
                      flush=True)
                return
            print("[Personal] pages changed since the cache was written, cutting again",flush=True)
        except Exception as e:
            print(f"[Personal] cache unreadable ({e}), cutting again",flush=True)

    added = []
    rng = random.Random(paths.SplitSeed)
    nAdded = 0
    print(f"[Personal] cutting pages for {paths.PersonalWriters} into lines",flush=True)
    for author in paths.PersonalWriters:
        for imgPath,labelPath in paths.PersonalWriterPages(author):
            print(f"[Personal] {imgPath.name} ...",end="",flush=True)
            leanLines,_ov,_info = SL.SegmentLines(ReadImageFile(imgPath))
            crops = [np.asarray(ln["image"]) for ln in leanLines]
            gt = ReadLabelLines(str(imgPath),str(labelPath))
            gt = [g for g in gt if g.strip()!="MESS"]
            n = min(len(crops),len(gt))
            rows = list(zip(range(n),crops[:n],gt[:n]))
            rng.shuffle(rows)
            nVal = max(1,round(len(rows)*paths.ValFraction))
            for rank,(origIdx,crop,text) in enumerate(rows):
                target = EncodeText(text)
                if len(target)==0:
                    continue
                isHoldout = rank<nVal
                pageKey = f"{author}/{imgPath.stem}_{origIdx:02d}.png"
                sample = {
```

```
                    "page_key": pageKey,
                    "line_idx": origIdx,
                    "imageBytes": EncodeCrop(crop),
                    "target": target,
                    "text": text,
                    "is_holdout": isHoldout,
                }
                base.samples.append(sample)
                added.append(sample)
                nAdded+=1
    print(f"[Personal] added {nAdded} line samples from {len(paths.PersonalWriters)} writers")

# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.34 TekliaLineDataset

```
class TekliaLineDataset
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1646. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Method | Methods exposed by this current class

```
def __init__(self, rootDir, isTrain)
def __len__(self)
def __getitem__(self, i)
```

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class TekliaLineDataset:
    def __init__(self,rootDir,isTrain):
        rootDir = Path(rootDir)
        self.isTrain = isTrain
        self.items = []
        print(f"[Teklia] indexing {rootDir.name} ...",end="",flush=True)
        for imgPath in sorted(rootDir.glob("*.png")):
            labelPath = imgPath.with_name(imgPath.stem+"_labels.txt")
            if labelPath.exists():
                text = labelPath.read_text(encoding="utf-8").strip()
                if text:
                    self.items.append((imgPath,text))
        print(f" {len(self.items)} lines",flush=True)
        if not self.items:
            raise RuntimeError(f"No line images found under {rootDir}")
```

```
    def __len__(self):
        return len(self.items)

    def __getitem__(self,i):
        imgPath,text = self.items[i]
        img = ResizeLineImageFixed(ReadImageFile(imgPath))
        if self.isTrain:
            img = AugmentLineImage(img)
        imgTensor = TensorFromResized(img)
        return imgTensor,EncodeText(text),text
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.35 IterateBatches

```
def IterateBatches(dataset, batchSize, rng, shuffle)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1808. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`dataset`
: the dataset to iterate

`batchSize`
: how many items per batch

`rng` the random generator

`shuffle`
: whether to shuffle the order

Output. batches of items, one at a time.

Justification | Why the order is shuffled for training

The examples arrive grouped by page and therefore by writer. Without shuffling, consecutive batches would each contain one writer's hand, so the network would be pulled toward that hand and then away from it, which both slows convergence and makes the loss oscillate with the data order rather than with learning. Shuffling makes each batch an approximately representative sample, which is the assumption the gradient estimate rests on.

Justification | Why validation is not shuffled

Validation computes a mean over the whole set, and a mean does not depend on the order. Keeping the order fixed makes the measurement exactly reproducible between runs, which matters when comparing two checkpoints: any difference is then attributable to the weights rather than to the batching.

Batches are produced one at a time rather than built as a list, so only the current batch is in memory. For a dataset of tens of thousands of crops that is the difference between a few hundred megabytes and several gigabytes.

15.36 TextNumpyBatch

```
def TextNumpyBatch(batch)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1905. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`batch`
: a collated batch

Output. the images, the encoded labels and the original texts.

Method | What it does

Prepares a batch for the network: the images converted to the input convention and the labels encoded into class indices. The original texts are carried through as well so that validation can compare the decoded output against them.

The image conversion must match TensorFromResized in recognise.py exactly, including the inversion and the standardisation. A difference between the two would produce a network that trains successfully and reads nothing at inference, which is the hardest class of fault to find because both halves appear to work.

15.37 ValidateTextModel

```
def ValidateTextModel(valSet, params, batchSize, rng)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1918. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Decodes every held-out crop and compares it with its transcript, giving the mean loss and the share of characters right on lines training never saw.

Inputs

`valSet`
: the held-out set

`params`
: the parameters

`batchSize`
: the batch size

`rng` the random generator

Output. a pair: the mean validation loss and the character accuracy.

Justification | Why a held-out set is necessary

Training loss falls whether or not the network is learning anything general, because a network with enough capacity can memorise its training examples. The only way to tell memorisation from learning is to measure on data the network has never been trained on. That is the **held-out** or validation set, and the gap between training and validation loss is the measure of overfitting.

Justification | Why two numbers are reported

The loss and the character accuracy answer different questions, and both are needed. The loss is what training minimises, so it is the quantity the scheduler and the best-checkpoint logic must use, and it is sensitive to small changes in confidence. Character accuracy is what anyone actually cares about, but it is coarse: a network can improve its loss substantially with no change in accuracy, because it has become more confident about answers it already had right.

> *Augmentation is off here, through the evaluation view, and batch normalisation uses its stored statistics, through the evaluation forward pass. Both are necessary for the measurement to describe the checkpoint that will be saved rather than some variant of it.*

15.38 BuildPagesSplit

```
def BuildPagesSplit(maxPages, dataDir)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1939. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`maxPages`
: how many pages to use

`dataDir`
: where the pages are

Output. a pair of training and validation datasets.

Justification | Why the split is by page and not by line

This is the important decision. Lines from one page are written by one person, on one sheet, with one pen, in one session. Splitting by line would put some of a page's lines in training and others in validation, and the network could then score well on validation by recognising that particular page's characteristics rather than by reading. The validation figure would be optimistic, and the amount by which it was optimistic would be unknowable.

Splitting by page makes the validation lines come from sheets the network has never seen, which is the situation it will actually face. This is **leakage** avoidance, and it is the commonest way a held-out measurement is quietly invalidated: the split has to be made at the level at which examples are genuinely correlated, which here is the page.

> *The page limit exists to allow short runs during development. It takes the first n pages rather than a random sample, which means a short run always uses the same pages, so two such runs are comparable with each other even though neither represents the whole database.*

15.39 BuildTekliaSplit

```
def BuildTekliaSplit(dataDir)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1949. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Inputs

`dataDir`
: where the corpus is

Output. a pair of training and validation datasets from the alternative corpus.

Purpose | What it is for

The same split for the second corpus, which arrives already cut into lines. Having both behind the same interface is what lets the training function take a source argument and otherwise be identical, so a change to the training procedure applies to both without being made twice.

15.40 PersonalLineView

```
class PersonalLineView
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1955. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Method | Methods exposed by this current class

```
def __init__(self, samples, isTrain)
def __len__(self)
def __getitem__(self, i)
```

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class PersonalLineView:
    def __init__(self,samples,isTrain):
        self.samples = samples
        self.isTrain = isTrain

    def __len__(self):
        return len(self.samples)

    def __getitem__(self,i):
        item = self.samples[i]
        img = ResizeLineImageFixed(DecodeCrop(item["imageBytes"]))
        if self.isTrain:
            img = AugmentLineImage(img)
        return TensorFromResized(img),item["target"],item["text"]
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.41 CollectPersonalLines

```
def CollectPersonalLines()
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1971. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def CollectPersonalLines():
    holder = type("Holder",(),{})()
    holder.samples = []
    AddPersonalSamples(holder)
    train = [s for s in holder.samples if not s.get("is_holdout")]
    held = [s for s in holder.samples if s.get("is_holdout")]
    return train,held
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.42 BuildJointSplit

```
def BuildJointSplit(dataDir)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1983. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

The public set plus the photographed writers, which is the mix the deployed reader was trained on.

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def BuildJointSplit(dataDir):
    tekTrain,tekVal = BuildTekliaSplit(dataDir)
    perTrain,perHeld = CollectPersonalLines()
    print(f"[Mix] {len(tekTrain)} Teklia + {len(perTrain)} personal train lines, "
          f"{len(tekVal)} Teklia + {len(perHeld)} personal held out")
    return tekTrain,PersonalLineView(perTrain,isTrain=True),tekVal,PersonalLineView(perHeld,isTrain=
    False)
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.43 IterateMixedBatches

```
def IterateMixedBatches(mainSet, extraSet, batchSize, fraction, rng, shuffle=True)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1995. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Batches drawn from two sets at a fixed ratio, so every batch holds the same proportion of photographed lines, the smaller set is reused as often as needed.

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def IterateMixedBatches(mainSet,extraSet,batchSize,fraction,rng,shuffle=True):
    nExtra = min(len(extraSet),max(1,int(round(batchSize*fraction)))) if len(extraSet) else 0
    nMain = max(1,batchSize-nExtra)
    mainOrder = rng.permutation(len(mainSet)) if shuffle else np.arange(len(mainSet))
    extraOrder = rng.permutation(len(extraSet)) if (shuffle and nExtra) else np.arange(len(extraSet))
    extraAt = 0
```

```
    for start in range(0,len(mainSet),nMain):
        batch = [mainSet[int(i)] for i in mainOrder[start:start+nMain]]
        for _ in range(nExtra):
            if extraAt>=len(extraOrder):
                extraOrder = rng.permutation(len(extraSet))
                extraAt = 0
            batch.append(extraSet[int(extraOrder[extraAt])])
            extraAt += 1
        yield batch
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.44 TrainTextModel

```
def TrainTextModel(numEpochs, batchSize, lr, seed, maxPages, dataDir, source='pages',
    resume=False, warmStart=None, personalFraction=0.2, tag=None)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 2015. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

The reader's training run, batch, forward, CTC loss, backward, step, validate, and save whenever the held-out loss improves.

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def TrainTextModel(numEpochs,batchSize,lr,seed,maxPages,dataDir,source="pages",resume=False,
                   warmStart=None,personalFraction=0.20,tag=None):
    personalSet = personalVal = None
    if source=="joint":
        trainSet,personalSet,valSet,personalVal = BuildJointSplit(dataDir)
    elif source=="teklia":
        trainSet,valSet = BuildTekliaSplit(dataDir)
    else:
        trainSet,valSet = BuildPagesSplit(maxPages,dataDir)
    print(f"[Split] Train lines: {len(trainSet)} | Held-out val lines: {len(valSet)}")

    numClasses = len(CHARSET)+1
    params = InitTextModelParams(seed,numClasses)
    trainable = TrainableParams(params)
    optimizer = RMSprop(trainable,lr=lr,weightDecay=1e-5)
    scheduler = ReduceLrOnPlateau(optimizer,factor=0.5,patience=5,minLr=1e-5)

    rng = np.random.default_rng(seed)
    bestValLoss = np.inf
    bestPath = WEIGHTS_DIR/"paper_cnn_bilstm_ctc_scratch_best.npz"
    ckptPath = WEIGHTS_DIR/"paper_cnn_bilstm_ctc_scratch_checkpoint.npz"
    logPath = WEIGHTS_DIR/"paper_cnn_bilstm_ctc_scratch_log.txt"

    startEpoch = 1
    if warmStart:
```

```
        loaded = LoadParamsInto(params,warmStart)
        print(f"[WarmStart] loaded {loaded}/{len(params)} tensors from {warmStart}")
    if resume and ckptPath.exists():
        loaded = LoadParamsInto(params,ckptPath)
        prevEpoch,prevBest = LoadCheckpointMeta(ckptPath)
        startEpoch = prevEpoch+1
        if prevBest is not None:
            bestValLoss = prevBest
        print(f"[Resume] loaded {loaded}/{len(params)} tensors from {ckptPath}, "
              f"continuing at epoch {startEpoch} (best val loss so far {bestValLoss:.4f})")

    for epoch in range(startEpoch,startEpoch+numEpochs):
        epochStart = time.time()
        epochLoss,nBatches = 0.0,0
        batches = (IterateMixedBatches(trainSet,personalSet,batchSize,personalFraction,rng)
                   if personalSet is not None
                   else IterateBatches(trainSet,batchSize,rng,shuffle=True))
        for batch in batches:
            images,targets,_texts = TextNumpyBatch(batch)
            rawLogits,cache = TextModelForward(images,params)
            loss,dRawLogits = CtcLoss(rawLogits,targets)
            _,grads = TextModelBackward(dRawLogits,cache)
            trainableGrads = {k:grads[k] for k in trainable}
            optimizer.Step(trainableGrads)
            epochLoss+=loss
            nBatches+=1
            if nBatches==1 or nBatches%5==0:
                print(f"\r[TrainText] Epoch {epoch:03d} | Batch {nBatches:04d} | "
                      f"Running loss {epochLoss/nBatches:.3f}",end="",flush=True)
        print()
        trainLoss = epochLoss/max(1,nBatches)

        valLoss,valCharAcc = ValidateTextModel(valSet,params,batchSize,rng)
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.45 ValidateAuthorModel

```
def ValidateAuthorModel(valSet, params, batchSize, rng)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 2162. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

Keep image and label identities aligned and separate held-out evaluation from learning.

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def ValidateAuthorModel(valSet,params,batchSize,rng):
    if len(valSet)==0:
```

```
        return 0.0
    correct,total = 0,0
    for batch in IterateBatches(valSet,batchSize,rng,shuffle=False):
        images,authorIdxs,_ = AuthorCollateFn(batch)
        logits = EvalAuthorForward(params,images)
        preds = np.argmax(logits,axis=1)
        correct+=int((preds==authorIdxs).sum())
        total+=len(authorIdxs)
    return correct/max(1,total)
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.46 TrainAuthorModel

```
def TrainAuthorModel(numEpochs, batchSize, lr, seed, resume=False, warmStart=None, writers=
    None)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 2179. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

The namer's training run, the same shape of loop with cross entropy instead of CTC. every crop is stroke-normalised first, the same preparation ClassifyImage applies at inference.

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def TrainAuthorModel(numEpochs,batchSize,lr,seed,resume=False,warmStart=None,
                     writers=None):
    wanted = list(writers) if writers else list(paths.DatabaseWriters+paths.PersonalWriters)
    unknown = [w for w in wanted if w not in paths.DatabaseWriters+paths.PersonalWriters]
    if unknown:
        raise SystemExit("unknown writer(s): %s\nknown: %s"
                         % (", ".join(unknown),
                            ", ".join(paths.DatabaseWriters+paths.PersonalWriters)))
    wantedDb = [w for w in paths.DatabaseWriters if w in wanted]

    base = IAMLineDatasetRaw(rootDir=str(DEFAULT_AUTHOR_DATA_DIR),cacheDir=str(DEFAULT_AUTHOR_CACHE_
    DIR))
    before = len(base.samples)
    base.samples = [s for s in base.samples if s["page_key"].split("/")[0] in wantedDb]
    print(f"[Filter] kept {len(base.samples)}/{before} database samples for {wantedDb}")

    AddPersonalSamples(base)
    base.samples = [s for s in base.samples if s["page_key"].split("/")[0] in wanted]
    base.authorFolders = sorted(wanted)
    base.authorToIdx = {a:i for i,a in enumerate(base.authorFolders)}
    print(f"[Authors] {len(base.authorFolders)}: {base.authorFolders}")
    print(f"[Crops] stroke-normalised at {CLS.PenWidthXh:.2f} x-heights")

    trainIdx,valIdx = base.HoldoutSplitIndices()
    if not valIdx:
```

```
        raise RuntimeError("No holdout lines found.")
    trainSet = AuthorLabeledView(base,trainIdx,isTrain=True)
    valSet = AuthorLabeledView(base,valIdx,isTrain=False)
    print(f"[Split] Train lines: {len(trainIdx)} | Held-out val lines: {len(valIdx)}")

    numAuthors = len(base.authorFolders)
    params = InitAuthorModelParams(seed,numAuthors)
    trainable = TrainableParams(params)
    optimizer = AdamW(trainable,lr=lr,weightDecay=1e-4)
    scheduler = CosineAnnealingLr(optimizer,tMax=numEpochs)

    rngBatch = np.random.default_rng(seed)
    rngDropout = np.random.default_rng(seed+1)
    bestAcc = 0.0
    stem = "author_shape_11w_pw170_scratch"
    outPath = WEIGHTS_DIR/(stem+"_weights.npz")
    ckptPath = WEIGHTS_DIR/(stem+"_checkpoint.npz")
    logPath = WEIGHTS_DIR/(stem+"_log.txt")
    print(f"[Output] {outPath.name}")

    startEpoch = 1
    if warmStart:
        loaded = LoadParamsInto(params,warmStart)
        print(f"[WarmStart] loaded {loaded}/{len(params)} tensors from {warmStart}")
    if resume and ckptPath.exists():
        loaded = LoadParamsInto(params,ckptPath)
        prevEpoch,prevBest = LoadCheckpointMeta(ckptPath)
        startEpoch = prevEpoch+1
        if prevBest is not None:
            bestAcc = prevBest
        print(f"[Resume] loaded {loaded}/{len(params)} tensors from {ckptPath}, "
              f"continuing at epoch {startEpoch} (best val acc so far {bestAcc:.3f})")

    for epoch in range(startEpoch,startEpoch+numEpochs):
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

15.47 Main

```
def Main()
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 2280. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.14.

Purpose | What this contributes

The command line, –resume continues from this module's own checkpoint and –warm-start begins from any checkpoint on disk, neither restores the optimiser state, only the weights.

Method | Current implementation

Keep image and label identities aligned and separate held-out evaluation from learning. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def Main():
    import argparse
    ap = argparse.ArgumentParser(description="train the reader or the namer from first principles")
    ap.add_argument("task",choices=["text","author"])
    ap.add_argument("--epochs",type=int,default=40)
    ap.add_argument("--batch",type=int,default=8)
    ap.add_argument("--lr",type=float,default=1e-3)
    ap.add_argument("--seed",type=int,default=7)
    ap.add_argument("--source",choices=["joint","teklia","pages"],default="joint",
                    help="joint: the pre-cut public lines plus the photographed "
                         "writers. teklia: public lines only. pages: segment the "
                         "database pages here")
    ap.add_argument("--personal-fraction",dest="personalFraction",type=float,default=0.20,
                    help="joint only: share of every batch drawn from photographed lines")
    ap.add_argument("--pages",type=int,default=None,help="limit how many pages are cut up")
    ap.add_argument("--data",default=None,help="override where the pages or crops live")
    ap.add_argument("--writers",nargs="+",default=None,
                    help="author task only: train on this subset of writers")
    ap.add_argument("--resume",action="store_true",
                    help="continue from this module's own checkpoint")
    ap.add_argument("--warm-start",dest="warmStart",default=None,
                    help="start from a named checkpoint")
    args = ap.parse_args()

    warm = Path(args.warmStart) if args.warmStart else None
    if warm and not warm.exists():
        candidate = WEIGHTS_DIR/warm.name
        if candidate.exists():
            warm = candidate
        else:
            raise SystemExit("no checkpoint at %s" % args.warmStart)

    if args.task=="text":
        if args.data:
            dataDir = args.data
        elif args.source in ("teklia","joint"):
            dataDir = str(paths.Data.parents[1]/"Data"/"Datasets"/"TekliaIamLines")
        else:
            dataDir = str(DEFAULT_TEXT_DATA_DIR)
        TrainTextModel(args.epochs,args.batch,args.lr,args.seed,args.pages,dataDir,
                       source=args.source,resume=args.resume,warmStart=warm,
                       personalFraction=args.personalFraction)
    else:
        TrainAuthorModel(args.epochs,args.batch,args.lr,args.seed,
                         resume=args.resume,warmStart=warm,
                         writers=args.writers)
```

Justification | Interpret the method rather than infer a benchmark

Seeded splits and personal/public sampling expose exactly which population drives learning; caches avoid repeated decoding. Labels are not used as classifier answers during ordinary inference. [87, 3, 88].

Stage: Training data augmentation

15.48 Method background: Training data augmentation

Purpose | The role in this system

Expose the fixed-label learner to plausible capture and writing variation.

Justification | The reason for this construction

Small affine, perspective and elastic changes broaden the training distribution without collecting a new label for each transformed image. Validation remains deterministic.

Alternatives | What changes if another method is used

Collecting real varied captures avoids a synthetic-noise assumption but costs data effort. Large distortions can change a character or writer cue. Test-time augmentation is a different inference procedure.

Library | Implementation speed and accuracy

Torchvision/OpenCV/Pillow transforms can implement similar warps. Interpolation, coordinate convention and random distributions differ; stronger or faster augmentation is not automatically more accurate. [22, 25, 24].

15.49 MyAffine

```
def MyAffine(img, angleDeg=0.0, shearDeg=0.0, fillValue=255)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1207. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.48.

Purpose | What this contributes

Expose the fixed-label learner to plausible capture and writing variation.

Inputs

`img` the image

`angleDeg`
: the rotation in degrees

`shearDeg`
: the shear in degrees

`fillValue`
: what to fill uncovered pixels with, default 255

Output. the image rotated and sheared about its centre.

Method | Background: what an affine transformation is

An **affine** transformation is any combination of rotation, scaling, shearing and translation: it maps straight lines to straight lines and keeps parallel lines parallel. Every such transformation can be written as a matrix multiplication once the coordinates are expressed in **homogeneous** form, which means appending a 1 to each point so that a translation becomes part of the matrix rather than a separate addition.

Justification | Why the transformations are composed as matrices

Formula purpose and system use. Build a coordinate transform for a labelled training image. **Source basis:** [22, 25, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$M \;=\; T_{+c}\, R\, S\, T_{-c} \tag{15.2}$$

where

M the combined transformation

T_{-c} a translation moving the image's centre to the origin

S the shear

R the rotation

T_{+c} a translation moving the centre back

Rotation and shear are both defined about the origin, which for an image is a corner. Applying them directly would swing the image about that corner and out of frame. Translating the centre to the origin first, transforming, and translating back makes both act about the centre instead. Matrix composition is what allows the four steps to be combined into one, so the image is resampled once rather than four times, which matters because each resampling blurs it slightly.

The order reads right to left, since the rightmost matrix acts on the point first. That is the usual convention for column-vector coordinates and it is the opposite of the order the operations are listed in prose.

Justification | Why the matrix is inverted

The combined matrix is inverted before being used. The reason is the direction resampling works in: for each *output* pixel, the warp asks which input position it came from, which is the inverse map. Working forward instead, pushing each input pixel to its output position, leaves gaps where no input pixel happens to land and collisions where two do. Working backwards fills every output pixel exactly once, which is why it is universal in image warping. The same argument appears under Warp in segment.py.

15.50 AugmentShear

```
def AugmentShear(img)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1220. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.48.

Purpose | What this contributes

Expose the fixed-label learner to plausible capture and writing variation.

Inputs

`img` a line crop

Output. the crop with a random shear applied.

Method | Background: what augmentation is for

Data augmentation creates extra training examples by altering existing ones in ways that do not change their label. A sheared line of handwriting still says the same words, so it is a valid extra example. The purpose is to teach the network which variations are irrelevant: having seen the same line at many slants, it learns to read the letters rather than to recognise one particular slant.

Justification | Why shear in particular

Slant is one source of writer and page variation. Shear augmentation exposes training to changed stroke inclination, but it also changes horizontal geometry and interpolation. It is not established here as the largest source of variation.

Justification | The range, and why it is expressed as a tangent

The shear factor is drawn uniformly from -0.6 to 0.6 and then converted to an angle. Working in the factor rather than the angle is deliberate: the factor is what the shear matrix contains, so a uniform draw on it gives a uniform transformation strength, where a uniform draw on the angle would concentrate the effect near the extremes. A factor of 0.6 is about 31 degrees, which specifies the sampled augmentation range; it is not a measured bound on all real writers.

15.51 AugmentRotate

```
def AugmentRotate(img)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1226. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.48.

Purpose | What this contributes

Expose the fixed-label learner to plausible capture and writing variation.

Inputs

`img` a line crop

Output. the crop rotated by a small random angle.

Method | What it simulates

A line that is not quite level, which happens when a page is photographed slightly askew or when a writer's line drifts. The range is only 2.5 degrees either way, far smaller than the shear range.

Justification | Why the range is small

Because the real variation is small. The segmenter deskews every page before cutting crops, so by the time a crop reaches the network its residual tilt is a fraction of a degree. Augmenting much further than the real variation teaches the network to handle a situation that never occurs, at the cost of capacity it could spend on ones that do. A rotation also changes the measured slant and the apparent x-height, so a large one would conflict with the shear augmentation rather than complementing it.

15.52 FindPerspectiveCoeffs

```
def FindPerspectiveCoeffs(startpoints, endpoints)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1231. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.48.

Purpose | What this contributes

Expose the fixed-label learner to plausible capture and writing variation.

Inputs

`startpoints`
: the four source corners

`endpoints`
: where they should land

Output. the eight coefficients of the perspective transformation.

Method | Background: what a perspective transformation is

An affine transformation keeps parallel lines parallel, which a photograph taken at an angle does not: the far edge of a page appears shorter than the near one, so its sides converge. A **perspective** or **projective** transformation is the more general map that allows that convergence, and it is what actually happens when a flat page is photographed from an angle.

Formula purpose and system use. Solve for the projective mapping coefficients from paired points. **Source basis:** [22, 25, 24]. Exact project constants remain implementation choices unless evidence is stated.

$$x' = \frac{ax + by + c}{gx + hy + 1}, \qquad y' = \frac{dx + ey + f}{gx + hy + 1} \tag{15.3}$$

where

x, y
: a point in the source image

x', y'
: where it lands

$a \dots f$
: the six coefficients an affine map would also have

g, h
: the two extra coefficients that produce the perspective effect

1
: the fixed ninth coefficient; the map is unchanged by scaling all nine, so one can be fixed

The division by a function of position is what distinguishes it from an affine map and what makes parallel lines converge: points further away in the implied depth direction are divided by a larger number and so are compressed toward one another.

Method | How the eight coefficients are found

Four corner correspondences are given, each fixing two equations, giving eight equations for the eight unknowns. The equations above are not linear in the coefficients because of the division, but multiplying each through by its denominator makes them linear, and that rearrangement is what the rows of the matrix in the code are: each row is one such multiplied-out equation, with the cross terms carrying the negative signs the rearrangement produces.

The resulting system is then solved by standard linear algebra. The reason four corners are exactly right is that eight unknowns need eight equations: three corners would leave the system underdetermined and five would overdetermine it, requiring a least-squares fit rather than a solution.

15.53 AugmentPerspective

```
def AugmentPerspective(img)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1241. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.48.

Purpose | What this contributes

Expose the fixed-label learner to plausible capture and writing variation.

Inputs

`img` a line crop

Output. the crop with a random perspective distortion applied.

Purpose | What it simulates, and why it matters for this project

A page photographed from an angle rather than square on. This augmentation matters more here than it would for a database-only system, because this project's own pages are phone photographs: a page on a desk photographed by hand always carries some perspective. A network trained only on flat scans would be seeing a distortion it had never met.

Justification | Why it is applied to the line rather than the page

Strictly the perspective applies to the whole page, and a line crop is a narrow strip of it, so the distortion within one strip is a small part of the page's. Applying a crop-sized perspective is therefore an approximation, and a slightly stronger one than reality. That is acceptable for augmentation, whose purpose is to cover the variation rather than to model it exactly, and erring toward stronger distortion is the safer direction.

15.54 PerspectiveJitterPoints

```
def PerspectiveJitterPoints(width, height, DistortionScale)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1250. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.48.

Purpose | What this contributes

Expose the fixed-label learner to plausible capture and writing variation.

Inputs

`width`
: the crop width

`height`
: its height

`DistortionScale`
: how far the corners may move, as a fraction

Output. a pair of corner lists: the originals and the jittered destinations.

Method | What it does

Moves each of the four corners by a random amount within a fraction of the crop's size, producing the correspondences the coefficient solver needs.

Justification | Why the corners are moved independently

Independent corner displacements generate a general perspective distortion rather than a parameterised one. The alternative, choosing a viewing angle and deriving the corners from it, would be more physically faithful and would require a camera model, a focal length and an assumed distance, none of which are known. Moving the corners directly covers the same space of distortions with no model at all, which is what augmentation needs.

> *The displacement is a fraction of the crop's dimensions rather than a number of pixels, so the strength of the distortion is the same for a crop of any size. This is the same scale-relative reasoning that governs the thresholds throughout the project.*

15.55 AugmentElastic

```
def AugmentElastic(img)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1275. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.48.

Purpose | What this contributes

Expose the fixed-label learner to plausible capture and writing variation.

Inputs

`img` a line crop

Output. the crop with a smooth random warp applied.

Method | What it simulates

Elastic distortion [22] displaces every pixel by a small smoothly varying random amount, so the image is stretched here and compressed there. For handwriting it imitates the natural variation of a hand: the same person writing the same word twice produces shapes that differ in just this way, locally stretched and squeezed rather than globally transformed.

Justification | How the smoothness is obtained, and why it is essential

Two random displacement fields are generated, one per axis, and each is blurred with a Gaussian before being applied. The blurring is what makes the method work. An unblurred random displacement moves each pixel independently, which is simply noise: it destroys the strokes rather than deforming them. After blurring, nearby pixels move together, so a stroke stays a connected stroke and is merely bent.

The blur width therefore controls the character of the distortion. A wide blur gives a nearly uniform displacement, which is just a translation; a narrow one approaches noise. The useful range is a width comparable to the features being deformed, which for handwriting is around a letter's size. The amplitude scales the displacement after blurring, since blurring reduces the magnitude as well as smoothing it.

The implementation calls the Elastic function in segment.py, which is the same operation the glyph pipeline uses. Sharing it means the augmentation applied during training and any distortion applied elsewhere are identical, which matters if the two are ever compared.

15.56 AugmentLineImage

```
def AugmentLineImage(img)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1282. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.48.

Purpose | What this contributes

Expose the fixed-label learner to plausible capture and writing variation.

Method | Current implementation

Expose the fixed-label learner to plausible capture and writing variation. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def AugmentLineImage(img):
    candidates = [fn for fn in AUGMENTATIONS if random.random()<0.5]
    if not candidates:
        return img
    return random.choice(candidates)(img)
```

Justification | Interpret the method rather than infer a benchmark

Small affine, perspective and elastic changes broaden the training distribution without collecting a new label for each transformed image. Validation remains deterministic. [22, 25, 24].

Stage: Initialisation and learned parameter shapes

15.57 Method background: Initialisation and learned parameter shapes

Purpose | The role in this system

Start training with a controlled signal scale and checkpoint-compatible array layout.

Justification | The reason for this construction

Fan-in scaled convolution and explicit tensor shapes expose variance assumptions. Current linear and LSTM initialisers use uniform bounds; they are not the old Gaussian/Xavier descriptions.

Alternatives | What changes if another method is used

Zero weights preserve unwanted symmetry. Xavier and He gains use different activation assumptions. Orthogonal recurrent matrices are another training choice. None yields a useful reader before learning.

Library | Implementation speed and accuracy

PyTorch initialisers provide equivalent distributions if bounds, dtype, bias and random generator are matched. The same seed across different libraries need not draw the same arrays. [83, 84, 41, 49].

15.58 InitConv

```
def InitConv(rng, outCh, inCh, kh, kw)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1689. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.57.

Purpose | What this contributes

Random starting weights for a convolution, scaled by how many inputs feed each output.

Inputs

`rng` the seeded random generator

`outCh`
: the number of filters

`inCh`
: the input channel count

`kh` the filter height

`kw` its width

Output. a pair: the randomly initialised filters and a zero bias.

Justification | Why initialisation matters at all

The starting weights are random, but not arbitrarily so. If they are too small, the signal shrinks as it passes through each layer and arrives at the output as nearly nothing, so there is no gradient to learn from. If too large, it grows and saturates. With twelve convolutional layers either effect compounds twelve times, so the scale of the initial weights determines whether the network trains at all.

Method | He initialisation

Formula purpose and system use. Choose variance-scaled starting convolution weights for a rectifier backbone. **Source basis:** [83, 84, 41, 49]. Exact project constants remain implementation choices unless evidence is stated.

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{\text{fanIn}}}\right), \qquad \text{fanIn} = \text{inCh} \times k_h \times k_w \tag{15.4}$$

Source: [83]

where

w each weight, drawn independently

$\mathcal{N}(0, s)$
: a normal distribution with mean 0 and standard deviation s

fanIn
: how many input values each output is computed from

2 the factor that accounts for ReLU, explained below

The reasoning runs as follows. An output is a sum of fanIn products of a weight and an input. For independent terms, variances add, so the output's variance is fanIn times the product of the weight and input variances. Keeping the output's variance equal to the input's therefore requires the weight variance to be one over fanIn. That is the Glorot argument [84].

The factor of 2 is the correction for ReLU [83]. ReLU sets every negative value to zero, discarding about half the signal's variance, so without compensation the variance would halve at every layer and after twelve layers be reduced by a factor of four thousand. Doubling the weight variance exactly offsets it. The difference between the two initialisations is only that factor, and for a network this deep it is the difference between training and not.

Justification | Why the bias starts at zero

There is nothing to break symmetry in a bias, because the weights are already random, so each unit already computes something different. Zero is the neutral starting point, and it is also harmless here because the batch normalisation that follows removes any constant offset anyway, as noted under ConvBlockForward.

15.59 InitBatchNorm

```
def InitBatchNorm(ch)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1697. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.57.

Purpose | What this contributes

Start training with a controlled signal scale and checkpoint-compatible array layout.

Inputs

`ch` the channel count

Output. four arrays: the scale at one, the shift at zero, and the running statistics at zero and one.

Justification | Why the scale starts at one and the shift at zero

Those values make the learned part of the layer the identity: the output is the standardised input unchanged. So the layer begins by doing exactly the normalisation it exists for and nothing else, and any departure from that is something training chose. Starting the scale at a random value would add a per-channel distortion the network would have to learn to undo before making progress.

Justification | Why the running statistics start at zero and one

A mean of zero and a variance of one describe already-standardised data, so a network evaluated before any training behaves as though normalisation had been applied correctly. They are also the values the exponential average converges away from fastest: being the neutral assumption, any real data moves them in a meaningful direction immediately.

> *These two pairs are treated differently everywhere else in the file. The scale and shift are parameters with gradients and are updated by the optimiser; the running statistics have no gradients and are updated by observation. TrainableParams separates them by name, which is why the naming convention for them is load-bearing.*

15.60 InitLinear

```
def InitLinear(rng, outF, inF)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1702. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.57.

Purpose | What this contributes

Uses uniform weights and biases between plus/minus one over the square root of input width, not the Gaussian initialiser described in the old function note. The distribution matches a common framework linear-layer convention; it is a starting distribution, not learned evidence.

Method | Current implementation

Start training with a controlled signal scale and checkpoint-compatible array layout. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def InitLinear(rng,outF,inF):
    bound = 1.0/np.sqrt(inF)
    w = rng.uniform(-bound,bound,size=(outF,inF)).astype(np.float32)
    b = rng.uniform(-bound,bound,size=outF).astype(np.float32)
    return w,b
```

Justification | Interpret the method rather than infer a benchmark

Fan-in scaled convolution and explicit tensor shapes expose variance assumptions. Current linear and LSTM initialisers use uniform bounds; they are not the old Gaussian/Xavier descriptions. [83, 84, 41, 49].

15.61 InitLstmDirection

```
def InitLstmDirection(rng, inputSize, hiddenSize)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1709. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.57.

Purpose | What this contributes

Uses independent uniform gate matrices and both bias arrays between plus/minus one over square root of hidden size. The gate block layout must match inference. It does not explicitly set a positive forget-gate bias.

Method | Current implementation

Start training with a controlled signal scale and checkpoint-compatible array layout. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def InitLstmDirection(rng,inputSize,hiddenSize):
    bound = 1.0/np.sqrt(hiddenSize)
    wIh = rng.uniform(-bound,bound,size=(4*hiddenSize,inputSize)).astype(np.float32)
```

```
    wHh = rng.uniform(-bound,bound,size=(4*hiddenSize,hiddenSize)).astype(np.float32)
    bIh = rng.uniform(-bound,bound,size=4*hiddenSize).astype(np.float32)
    bHh = rng.uniform(-bound,bound,size=4*hiddenSize).astype(np.float32)
    return wIh,wHh,bIh,bHh
```

Justification | Interpret the method rather than infer a benchmark

Fan-in scaled convolution and explicit tensor shapes expose variance assumptions. Current linear and LSTM initialisers use uniform bounds; they are not the old Gaussian/Xavier descriptions. [83, 84, 41, 49].

15.62 InitConvBlock

```
def InitConvBlock(rng, params, prefix, inCh, outCh, nLayers)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1718. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.57.

Purpose | What this contributes

Start training with a controlled signal scale and checkpoint-compatible array layout.

Method | Current implementation

Start training with a controlled signal scale and checkpoint-compatible array layout. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def InitConvBlock(rng,params,prefix,inCh,outCh,nLayers):
    c = inCh
    for i in range(nLayers):
        idx = i*3
        w,b = InitConv(rng,outCh,c,3,3)
        params[f"{prefix}.{idx}.weight"] = w
        params[f"{prefix}.{idx}.bias"] = b
        g,be,rm,rv = InitBatchNorm(outCh)
        params[f"{prefix}.{idx+1}.weight"] = g
        params[f"{prefix}.{idx+1}.bias"] = be
        params[f"{prefix}.{idx+1}.running_mean"] = rm
        params[f"{prefix}.{idx+1}.running_var"] = rv
        c = outCh
```

Justification | Interpret the method rather than infer a benchmark

Fan-in scaled convolution and explicit tensor shapes expose variance assumptions. Current linear and LSTM initialisers use uniform bounds; they are not the old Gaussian/Xavier descriptions. [83, 84, 41, 49].

15.63 InitTextModelParams

```
def InitTextModelParams(seed, numClasses)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1819. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.57.

Purpose | What this contributes

Builds a fresh reader, the backbone, two bidirectional LSTM layers and a score per character.

Inputs

`seed`
: the random seed

`numClasses`
: the output size, the charset plus the blank

Output. the reader's complete parameter dictionary.

Method | What it builds

The shared backbone, two bidirectional LSTM layers, and a linear text head producing one score per class.

Justification | Why the seed is explicit

Every random draw comes from a generator created from this seed, so the same seed gives the same initial weights. That is what makes a training run reproducible, which matters for comparing two configurations: without it, a difference in the result could be a difference in the starting point rather than in the change being tested.

Justification | Why the class count is one more than the charset

The extra class is the CTC blank, which is not a character but needs an output slot of its own, as described under DecodeCtc. Deriving the count from the charset rather than stating it means the two cannot disagree, and a disagreement would be severe: the network would be trained with every character's index shifted by one relative to what inference assumes.

15.64 BuildLstmLayerParams

```
def BuildLstmLayerParams(sd)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1843. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.57.

Purpose | What this contributes

Start training with a controlled signal scale and checkpoint-compatible array layout.

Inputs

`sd` the parameter dictionary

Output. the LSTM parameters gathered into the nested structure the forward pass expects.

Purpose | What it is for

The parameters are stored flat, keyed by name, because that is what a checkpoint is and what the optimiser iterates over. The recurrent forward pass wants them nested: a list of layers, each with a forward and a backward direction, each with its four arrays. This converts one view to the other.

Justification | Why the flat storage is the primary form

Because three separate things need it flat: the optimiser, which iterates over every parameter without knowing the structure; the checkpoint, which is a flat collection of named arrays; and the gradient dictionary, which must be keyed the same way. The nested form is needed by one function, so it is the derived one.

> *The nested structure holds references to the same arrays, not copies, so an update applied by the optimiser to the flat dictionary is visible through the nested view. That is essential and would break silently if any step copied.*

15.65 InitAuthorModelParams

```
def InitAuthorModelParams(seed, numAuthors)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 2098. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.57.

Purpose | What this contributes

Start training with a controlled signal scale and checkpoint-compatible array layout.

Inputs

`seed`
: the random seed

`numAuthors`
: how many writers to distinguish

Output. the namer's complete parameter dictionary.

Method | What it builds

The same shared backbone, then a two-layer head: a linear layer, a ReLU, a dropout, and a second linear layer producing one score per writer. No recurrent layers, for the reasons given under AuthorClassifierCNNNumpy.

Justification | Why the writer count is a parameter

It sets the final layer's output size, so a checkpoint is tied to the number of writers it was trained on. Adding a writer means a new final layer and retraining it, and a checkpoint from a different count cannot be loaded. That is also why the writer names are stored inside the checkpoint, as described under LoadStateDictAny: the count alone is not enough, since the same ten writers in a different order would load without complaint and attribute every page to the wrong person.

Stage: Convolution and feature sequences

Method rationale, alternatives and library comparison: Section 6.7.

15.66 Conv2dForward

```
def Conv2dForward(x, weight, bias, padding=1)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 39. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`x` the input, shaped (batch, channels, height, width)

`weight`
: the filters

`bias`
: one number per output channel, or nothing

`padding`
: zeros added round the input, default 1

Output. a pair: the filtered result, and the cache the backward pass needs.

Method | What it adds to the inference version

The arithmetic is identical to Conv2d in recognise.py, including the im2col rearrangement and the strided view, both explained in that entry. The only difference is that this one keeps a cache.

Justification | What is cached, and why each item is needed

- **The input's shape,** not the input itself. The backward pass builds a gradient array of that shape and never needs the values.
- **The weights,** because the input gradient is formed by multiplying the output gradient by them.
- **The flattened columns,** which is the significant one. The weight gradient is the output gradient multiplied by exactly these patches, so keeping them avoids rebuilding them, which would mean redoing the strided view and the reshape.
- **The padding and the filter and output sizes,** so the backward pass can fold the column gradient back into image shape without recomputing the geometry.

Justification | The cost of caching, and why it is accepted

The columns are the memory cost of training. A 3 by 3 filter over three channels makes each patch 27 numbers, and there is one patch per output position, so the cached columns are roughly nine times the size of the input feature map. Every convolution in the network holds its own for the whole forward pass, since none can be released until the backward pass has used it. That, rather than the weights, is what sets the largest batch size that fits in memory, and it is why training needs far more memory than inference.

> *No bias-related value is cached. The bias gradient is just the output gradient summed over every position, which needs nothing from the forward pass, because adding a constant has a derivative of one.*

15.67 ConvBlockForwardTrain

```
def ConvBlockForwardTrain(x, params, prefix, nLayers)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 217. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`x`
: the input feature maps

`params`
: the parameter dictionary

`prefix`
: the block's name prefix

`nLayers`
: how many groups the block contains

Output. a pair: the output, and a list of per-group caches.

Method | What it adds to the inference version

The same convolution, normalise, activate repetition, with the batch-statistics version of the normalisation and with every group's three caches collected into a list.

Justification | Why the caches are kept in order

The list is built front to back as the groups are applied, and the backward pass walks it in reverse. That correspondence is the whole structure of backpropagation through a sequence of layers: the gradient must pass through them in the opposite order to the data, so the last cache created is the first one needed.

> *The running statistics are updated in place inside the parameter dictionary rather than returned. They are not parameters in the usual sense, since no gradient is computed for them and no optimiser touches them, but they must be saved in the checkpoint, so they live alongside the parameters and are excluded from the trainable set by name.*

15.68 InitBackboneParams

```
def InitBackboneParams(rng, params)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1733. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`rng` the random generator

`params`
: the parameter dictionary, filled in place

Output. nothing; the backbone's parameters are added.

Method | The channel progression

One channel in, then 32, then 64, then 128, with two, four and six groups in the three stages. The channel count doubles at each stage while the spatial size halves, which is the standard arrangement in a convolutional backbone.

Justification | Why that arrangement

Two reasons, one about capacity and one about cost. Deeper layers represent larger and more varied structures, strokes giving way to junctions and then to letter parts, and there are more distinguishable large structures than small ones, so more channels are needed to hold them. And since each pooling quarters the spatial size, doubling the channels leaves the work per layer roughly halved rather than growing, so the deeper stages are affordable despite being wider and having more groups.

Justification | Why the backbone is shared between the two networks

Both the reader and the namer use exactly this backbone, initialised by this one function. The features useful for recognising a letter and for recognising a hand are largely the same features, as argued under AuthorClassifierCNNNumpy, so there is no reason for the two to differ, and sharing the initialiser means a change to the architecture applies to both rather than to one.

15.69 BackboneForwardTrain

```
def BackboneForwardTrain(x, params)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1777. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`x` the input crops

`params`
: the parameters

Output. a pair: the collapsed feature sequence, and the cache.

Purpose | What it is for

The shared convolutional backbone with caching: three blocks, two poolings, and the collapse of the height. Both networks call it, so the part they have in common is written once.

Justification | Why sharing the implementation matters more than sharing the weights

The two networks do not share weights; each trains its own copy. What they share is the code, and that is what guarantees the architectures are identical, which in turn is what makes a warm start

from one to the other possible: the parameter names and shapes agree because they were produced by the same function.

15.70 EvalBackboneForward

```
def EvalBackboneForward(params, x)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1798. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`params`
: the parameters

`x`
: the input crops

Output. the backbone's output, with no cache.

Justification | Why a third version of the backbone exists

Validation needs a forward pass with the inference behaviour, using the stored batch statistics rather than the batch's own, and with no caching, since no backward pass follows. Running the training version and discarding the caches would use the wrong statistics and would allocate the caches anyway.

Justification | Why using the stored statistics for validation is the point

Validation estimates how the saved checkpoint will perform, and the saved checkpoint will be used with the stored statistics. Validating with batch statistics would measure something slightly different and, worse, something that depends on how the validation set happened to be divided into batches, so the same weights would score differently with a different batch size.

15.71 TextModelForward

```
def TextModelForward(x, params)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1854. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Method | Current implementation

Extract local stroke features shared across locations before sequence recognition or writer pooling. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def TextModelForward(x,params):
    backboneOut,backboneCache = BackboneForwardTrain(x,params)
    pooled,hpCache = AdaptiveMaxPoolHeightForward(backboneOut)
    B,C,_,W = pooled.shape
    squeezed = pooled[:,:,0,:]
    seqIn = np.transpose(squeezed,(2,0,1))   #width becomes the CTC time axis

    seqOut,lstmCaches = MultilayerBidirectionalLstmForwardTrain(
        seqIn,BuildLstmLayerParams(params),HIDDEN_SIZE)

    rawLogits,linCache = LinearForward(seqOut,params["text_head.weight"],params["text_head.bias"])
    cache = dict(backboneCache=backboneCache,hpCache=hpCache,lstmCaches=lstmCaches,linCache=linCache,
    bchw=(B,C,W))
    return rawLogits,cache
```

Justification | Interpret the method rather than infer a benchmark

Weight sharing reduces parameters compared with a fully connected image mapping. Patch-to-matrix multiplication reaches compiled NumPy linear algebra while exposing the computation. [35, 36, 46, 2].

15.72 EvalTextForward

```
def EvalTextForward(params, x)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1895. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`params`
: the parameters

`x`
: the input crops

Output. the log probabilities per column, with no cache.

Purpose | What it is for

The reader's forward pass in inference mode, for validation: stored batch statistics, no caching, and the log softmax applied, since validation decodes the output rather than passing it to the loss.

15.73 AuthorModelForward

```
def AuthorModelForward(x, params, training, rng)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 2114. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

The namer forward, backbone, pooling over height and width, one score per writer.

Inputs

`x` the input crops

`params`
: the parameters

`training`
: whether dropout should be active

`rng` the random generator for dropout

Output. a pair: the raw per-writer scores, and the cache.

Method | What it does

The backbone, the average pooling over position, and the two-layer head with dropout between the layers.

Justification | Why the training flag is an argument

Because dropout must be active during training and inactive during validation, and the same function serves both. Passing it explicitly rather than holding it as state makes it impossible to validate with dropout accidentally left on, which would add noise to the measurement, or to train with it off, which would silently remove the regularisation.

> *The random generator is passed in rather than created here, so that the dropout masks come from the run's own seeded generator and a training run is reproducible. A generator created inside the function would reseed itself on every call and the masks would be unrepeatable.*

15.74 EvalAuthorForward

```
def EvalAuthorForward(params, x)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 2152. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.7.

Purpose | What this contributes

Extract local stroke features shared across locations before sequence recognition or writer pooling.

Inputs

`params`
: the parameters

`x` the input crops

Output. the raw per-writer scores, with no cache and no dropout.

Purpose | What it is for

The namer's inference-mode forward pass for validation: stored batch statistics, dropout off, no caching. The same three differences as the reader's evaluation pass, and for the same reasons.

Stage: Batch normalisation

Method rationale, alternatives and library comparison: Section 6.13.

15.75 BatchNorm2dForwardTrain

```
def BatchNorm2dForwardTrain(x, gamma, beta, runningMean, runningVar, momentum=0.1, eps=1e
    -05)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 90. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.13.

Purpose | What this contributes

Normalises using this batch's own mean and variance, and updates the running values inference will use.

Inputs

`x` the input

`gamma`
: the learned scale per channel

`beta`
: the learned shift per channel

`runningMean`
: the stored mean, updated in place

`runningVar`
: the stored variance, updated in place

`momentum`
: how fast the stored statistics follow the batches, default 0.1

`eps` the guard on the division, default 1e-5

Output. a pair: the normalised and rescaled output, and the cache.

Method | How this differs from the inference version

The inference version, in recognise.py, standardises using statistics stored in the checkpoint. This one standardises using the current batch's own mean and variance, and updates the stored ones as a side effect.

Formula purpose and system use. Estimate the mean over the training normalisation population. **Source basis:** [37, 38]. Exact project constants remain implementation choices unless evidence is stated.

$$\mu_c = \frac{1}{N}\sum_{b,p,q} x[b,c,p,q], \qquad \sigma_c^2 = \frac{1}{N}\sum_{b,p,q}\left(x[b,c,p,q]-\mu_c\right)^2 \tag{15.5}$$

where

μ_c the mean of channel c over this batch

σ_c^2 its variance

N how many values that is: the batch size times the height times the width

$\sum_{b,p,q}$
summed over the batch and both spatial axes, but not over channels

The statistics are per channel and pooled over everything else, for the reason given in the inference entry: a channel is the map of where one filter responded, and that filter's typical response size is one property however many positions and examples it is measured over.

Justification | Why using the batch is what makes it work

Standardising by the batch is not a convenience; it is the mechanism. The purpose of the layer is to fix the scale a layer presents to the next one, and during training that scale is changing constantly because the layers below are being updated. Measuring it fresh from the current batch tracks those changes immediately, where stored statistics would always describe an earlier state of the network.

Justification | The running statistics, and why they are an exponential average

Formula purpose and system use. Estimate variance and produce normalised activations. **Source basis:** [37, 38]. Exact project constants remain implementation choices unless evidence is stated.

$$\hat{\mu} \leftarrow (1-m)\,\hat{\mu} + m\,\mu_c \tag{15.6}$$

where

$\hat{\mu}$ the stored mean, kept for inference

m the momentum, 0.1

μ_c this batch's mean

This is an **exponential moving average**: each update keeps 90 percent of the old value and takes 10 percent from the new observation, so the stored figure is a weighted average over recent batches with older ones fading away. It is used rather than a plain average over all batches for two reasons. It needs no count and no accumulator, only the current value. And the fading is correct here: early batches were produced by a network whose weights were nearly random, so their statistics do not describe the trained network and should be forgotten.

Justification | Why the stored variance is the unbiased estimate

Formula purpose and system use. Update running estimates used later during inference. **Source basis:** [37, 38]. Exact project constants remain implementation choices unless evidence is stated.

$$\sigma^2_{\text{unbiased}} = \sigma^2_{\text{biased}} \cdot \frac{N}{N-1} \tag{15.7}$$

where

σ^2_{biased}
the variance computed with the mean measured from the same data

$\sigma^2_{\text{unbiased}}$
the corrected estimate of the population variance

N the number of values

Measuring the variance about a mean estimated from the same sample systematically underestimates it, because the sample mean is by construction the point about which the squared deviations are smallest. **Bessel's correction**, dividing by N−1 rather than N, compensates. The biased form is used to normalise this batch, since that is the quantity that actually standardises these values, while the corrected form is stored, since that is an estimate of the variance of all data. The code comment notes this matches the reference framework's behaviour, which matters because checkpoints must be interchangeable.

Method | What is cached

The standardised values, the standard deviation, the scale and the element count. Notably the input itself is not cached: the backward derivation below is written entirely in terms of the standardised values, which is both less memory and fewer operations than recovering the input.

Stage: Nonlinear activation and affine transforms

Method rationale, alternatives and library comparison: Section 6.15.

15.76 ReluForward

```
def ReluForward(x)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 123. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.15.

Purpose | What this contributes

Let stacked learned layers express nonlinear image and sequence relationships, or convert features into class scores.

Inputs

`x` any array

Output. a pair: the rectified array, and a mask of which entries were positive.

Justification | What is cached and why it is cheap

The forward arithmetic is the same clamping as in recognise.py. What it adds is a boolean mask recording which inputs were positive, which is all the backward pass needs. A boolean array is one byte per element against eight for a double, so this is the cheapest cache of any layer, which matters because ReLU appears after every convolution.

> *The mask could be derived from the output instead, since the output is positive exactly where the input was. Caching it explicitly costs a little memory and keeps the backward pass independent of what the caller did with the output, which would otherwise be a hidden requirement that the output not be modified in place.*

15.77 LinearForward

```
def LinearForward(x, weight, bias)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 184. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.15.

Purpose | What this contributes

Let stacked learned layers express nonlinear image and sequence relationships, or convert features into class scores.

Inputs

`x` the input, with features along the last axis

`weight`
: the weight matrix

`bias`
: the bias, or nothing

Output. a pair: the result, and the cache holding the input and the weights.

Justification | What is cached and why both items are needed

Both the input and the weights, because the backward pass needs each for a different gradient: the weight gradient is formed from the input and the input gradient is formed from the weights. This is the only layer here that caches its full input values, which is affordable because a linear layer sits at the end of the network where the data is small, a vector of a few hundred numbers rather than a feature map.

Stage: Pooling and sequence formation

Method rationale, alternatives and library comparison: Section 6.19.

15.78 MaxPool2dForward

```
def MaxPool2dForward(x, kernel=2, stride=2)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 133. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.19.

Purpose | What this contributes

Reduce spatial resolution or summarise a feature map into sequence columns and writer features.

Inputs

`x` the input

`kernel`
: the window size, default 2

`stride`
: the step between windows, default 2

Output. a pair: the pooled output, and the cache recording which element won each window.

Justification | What must be cached, and why

The forward arithmetic is as in recognise.py. What training requires in addition is the **position** of the winner in each window, not merely its value. The backward pass must send the gradient to the element that was passed through, and only the position identifies it.

The position is stored as an index into the flattened window, a number from 0 to 3 for a 2 by 2 window, rather than as a pair of coordinates. A single index is one array instead of two and can be used directly by the indexed-write operation the backward pass uses to place the gradient.

> *This is why the forward pass flattens the window axes and takes the maximum by finding the winning index and then reading it, instead of simply taking a maximum over two axes. The inference version does the latter, which is simpler; here the index is needed anyway, so the value is obtained by reading it.*

15.79 AdaptiveMaxPoolHeightForward

```
def AdaptiveMaxPoolHeightForward(x)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 160. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.19.

Purpose | What this contributes

Reduce spatial resolution or summarise a feature map into sequence columns and writer features.

Inputs

`x` the input

Output. a pair: the height collapsed to one row by taking the maximum, and the winning row indices.

Method | What is cached

Which row won in each column of each channel. The reasoning is the same as for ordinary max pooling: the gradient goes entirely to the element that was passed through, so its position must be known.

This is the layer that converts the image into a sequence, as described in the inference entry, so it is also the point where the gradient converts back: the sequence gradient coming from the recurrent layers becomes an image gradient here, placed into the rows that won.

15.80 AdaptiveAvgPoolWidthForward

```
def AdaptiveAvgPoolWidthForward(x)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 174. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.19.

Purpose | What this contributes

Reduce spatial resolution or summarise a feature map into sequence columns and writer features.

Inputs

`x` the input

Output. a pair: the average along axis 2, and the length averaged over.

Justification | Why the cache is a single number

An average needs nothing from its input to be differentiated, only how many values it was taken over, which is one integer. That is the cheapest cache in the file, and the contrast with max pooling is instructive: a maximum depends on *which* value won and so must remember it, whereas an average treats every value identically and so has nothing to remember.

Stage: Training dropout and expectation scaling

15.81 Method background: Training dropout and expectation scaling

Purpose | The role in this system

Randomly mask hidden responses during training and preserve their expected scale.

Justification | The reason for this construction

Inverted dropout divides retained responses by their keep probability. This discourages reliance on one particular activation combination; inference uses the unmasked network.

Alternatives | What changes if another method is used

No dropout removes this source of training noise. Weight decay penalises parameter magnitude and image augmentation changes inputs; these regularisers act differently and can be combined.

Library | Implementation speed and accuracy

A framework dropout layer handles train/eval switching and random masks. Match keep probability, mask shape, dtype and random seed for a numerical comparison. It adds no inference-time masking when disabled; it does not guarantee higher held-out accuracy. [43, 3].

15.82 DropoutForward

```
def DropoutForward(x, p, training, rng)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 203. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.81.

Purpose | What this contributes

Randomly mask hidden responses during training and preserve their expected scale.

Inputs

`x` the input

`p` the fraction to drop

`training`
whether this is a training pass

`rng` the seeded random generator

Output. a pair: the input with some entries zeroed and the rest scaled up, and the mask; or the input unchanged outside training.

Method | What dropout is for

Overfitting is when a network learns the training examples rather than the pattern in them, so it performs well on what it has seen and poorly on anything new. One way it happens is that the network comes to rely on a particular combination of features that happens to be reliable in the training data. **Dropout** [43] prevents that by randomly removing a fraction of the values at each training step, so no feature can be depended on: it may not be there. The network is forced to spread its evidence across several features, which generalises better.

Justification | Why the survivors are scaled up

Formula purpose and system use. Preserve expected activation magnitude while masking units during training. **Source basis:** [43, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{mask} = \frac{[\,u \geq p\,]}{1-p}, \qquad u \sim \text{Uniform}(0,1) \tag{15.8}$$

where

mask
what the input is multiplied by, elementwise

u a random number drawn uniformly between 0 and 1, one per element

p the drop probability

$[\,u \geq p\,]$
1 when the element survives and 0 when it is dropped

$\frac{1}{1-p}$ the compensating scale applied to the survivors

Dropping a fraction p of the values reduces the sum reaching the next layer by that fraction, so without compensation the layer below would see a smaller signal during training than at inference, and everything calibrated for one would be wrong for the other. Dividing the survivors by 1−p restores the expected total: a fraction 1−p of the values survive and each is multiplied by 1/(1−p), so the expectation is unchanged.

Justification | Why the scaling is done here rather than at inference

The compensation could instead be applied by scaling everything down by 1−p at inference, which is how dropout was originally described. Putting it in the training path instead is called **inverted**

dropout, and it is now standard for a practical reason: inference then requires no dropout-related code at all. The layer simply vanishes outside training, which is exactly what recognise.py does, where the gap in the parameter numbering is the only trace of it.

> *Outside training the input is returned unchanged with no mask. The backward pass accepts that absent mask and passes the gradient through untouched, which keeps the two halves consistent without either needing to know whether the other is in training mode.*

15.83 DropoutBackward

```
def DropoutBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 210. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.81.

Purpose | What this contributes

Randomly mask hidden responses during training and preserve their expected scale.

Inputs

`dout`
: the gradient with respect to the output

`cache`
: the mask, or nothing

Output. the gradient with respect to the input.

Method | The derivative

Dropout multiplies elementwise by a mask, and the derivative of multiplying by a constant is that constant, so the backward pass multiplies by the same mask. The mask includes the scaling factor, so the gradient is scaled by it too, which is necessary: the forward pass amplified the survivors, so their influence on the loss is amplified by the same factor.

A dropped element receives exactly zero gradient, which is correct rather than approximate. It genuinely had no effect on the output for this example, so the loss does not depend on it at all this step. It will receive gradient on steps where it survives, which is how a unit that is dropped half the time still trains, from half the examples.

Stage: LSTM and bidirectional recurrence

Method rationale, alternatives and library comparison: Section 6.26.

15.84 LstmForwardTrain

```
def LstmForwardTrain(x, weightIh, weightHh, biasIh, biasHh, hiddenSize)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 246. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.26.

Purpose | What this contributes

Use left and right image context to disambiguate sequential character features.

Inputs

`x` the input sequence, shaped (time, batch, features)

`weightIh`
: the input weights, all four gates stacked

`weightHh`
: the recurrent weights

`biasIh`
: the input bias

`biasHh`
: the recurrent bias

`hiddenSize`
: the width of the hidden and cell state

Output. a pair: the output at every step, and a cache holding every intermediate value at every step.

Method | What it adds to the inference version

The same gate arithmetic as LstmForward in recognise.py, with one addition that dominates its memory use: at every time step it records the input, the two incoming states, all four gate activations and the squashed cell value.

Justification | Why so much must be kept

The backward pass needs each of them, and the reason is that the LSTM's updates are products of the gates with the states. Differentiating a product requires both factors, so the gradient with respect to the forget gate needs the previous cell value and the gradient with respect to the previous cell needs the forget gate. None of it can be recomputed cheaply either, because recomputing the state at step t requires replaying every step before it.

Formula purpose and system use. Save the gated recurrent state needed by reverse differentiation. **Source basis:** [40, 41, 48]. Exact project constants remain implementation choices unless evidence is stated.

$$\text{memory} \;\approx\; T \times B \times \left(\,\text{inputSize} + 6H\,\right) \tag{15.9}$$

where

memory
: the number of values cached, per LSTM direction per layer

T the sequence length, 160 columns here

B the batch size

inputSize
: the width of the input at that layer

H the hidden size, 256

$6H$ the six vectors kept per step: two incoming states, four gates, and the squashed cell, less the one that is recomputed

With two layers and two directions that is eight such caches. At a hidden size of 256 and 160 steps it comes to a few million values per batch item, which is why the text model trains with small batches. This is the characteristic cost of **backpropagation through time**: the whole history of the sequence must be held in memory until the backward pass has consumed it.

> *The step caches are kept in a plain list indexed by time, built forwards and consumed backwards, which is the same arrangement as the convolution block's cache list and for the same reason.*

15.85 BidirectionalLstmForwardTrain

```
def BidirectionalLstmForwardTrain(x, paramsFwd, paramsBwd, hiddenSize)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 315. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.26.

Purpose | What this contributes

Use left and right image context to disambiguate sequential character features.

Inputs

`x` the input sequence

`paramsFwd`
: the forward direction's parameters

`paramsBwd`
: the backward direction's parameters

`hiddenSize`
: the width per direction

Output. a pair: the two directions concatenated, and a cache holding both directions' caches.

Method | What it does

Runs both directions as in the inference version and keeps both caches. Each direction is a complete independent LSTM with its own weights, so each has its own full set of per-step values, and the memory cost is doubled.

> *The backward direction is run on a reversed view of the input, which is free, and its output is reversed again to align positions. The reversals must be undone consistently in the backward pass too, which is the one real opportunity for error in an otherwise mechanical function.*

15.86 MultilayerBidirectionalLstmForwardTrain

```
def MultilayerBidirectionalLstmForwardTrain(x, layerParams, hiddenSize)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 341. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.26.

Purpose | What this contributes

Use left and right image context to disambiguate sequential character features.

Inputs

`x` the input sequence

`layerParams`
: a list of per-layer parameters

`hiddenSize`
: the width per direction

Output. a pair: the final output, and the list of per-layer caches.

Method | What it does

Applies each bidirectional layer in turn, collecting the caches. The same build-forwards, consume-backwards arrangement as the convolution block, one level up: a list of layer caches, each of which holds two direction caches, each of which holds a list of step caches.

Stage: Softmax and log sum exp

Method rationale, alternatives and library comparison: Section 6.23.

15.87 LogSumExp

```
def LogSumExp(a, b)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 360. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.23.

Purpose | What this contributes

Turn logits into a normalised class distribution and sum small probabilities stably.

Inputs

`a` one array of log-space values

`b` another of the same shape

Output. the logarithm of the sum of their exponentials, computed without overflow.

Purpose | What it is for

CTC adds probabilities together, and the probabilities are held as logarithms to avoid underflow, as explained under LogSoftmax. But a logarithm turns a *product* into a sum, not a *sum* into anything simple: adding two probabilities held as logarithms requires this function.

Formula purpose and system use. Sum exponentials in log space with a maximum subtraction for numerical stability. **Source basis:** [48, 51, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\log(e^a + e^b) \;=\; m + \log(e^{a-m} + e^{b-m}), \qquad m = \max(a, b) \tag{15.10}$$

where

a, b the two log-space values being combined

m the larger of the two, factored out

e^{a-m}
at most 1, since a is at most m, so it cannot overflow

The identity follows from factoring the larger exponential out of the sum and taking the logarithm. Its value is numerical: the exponentials in the rearranged form are at most 1, so nothing can overflow, while a direct computation would exponentiate a log probability of −800 and get zero, or of a large positive number and get infinity.

Justification | The infinity case, and why it needs handling

A forbidden transition is represented by negative infinity, meaning zero probability. When both arguments are that, the maximum is negative infinity and the rearranged expression computes infinity minus infinity, which is not a number. The answer should be negative infinity: the sum of two impossibilities is still impossible.

So the result is selected explicitly in that case. The code comment notes that numpy evaluates both branches of the selection eagerly, so the invalid arithmetic still happens and still warns even though its result is discarded, and the warning is suppressed for that reason. Suppressing a warning is usually a bad sign; here it is suppressing a warning about a value that has already been correctly replaced, and the suppression is scoped narrowly to this one expression.

> *Preserving negative infinity exactly, rather than using a large negative number, matters for correctness: an impossible path must contribute exactly nothing, and a finite stand-in contributes a tiny amount that accumulates over 160 steps and can make a forbidden alignment the winner.*

Stage: CTC decoding and forced alignment

Report equation 23 CTC occupancy and derivative

Method | Formula

$q_{t,k} = \sum_{s:\ell'_s=k} \exp(\alpha_{t,s} + \beta_{t,s} - \log Z)$ and $\partial L/\partial z_{t,k} = p_t(k) - q_{t,k}$.

Purpose | Why the system needs it

Propagate the sequence loss through logits. Alpha includes the current emission and beta excludes it in this convention, preventing double counting. State indices differ from class indices.

Evidence | Current location and source

CtcLoss; batch averaging and library reduction must also match. [34, 53].

Method rationale, alternatives and library comparison: Section 6.30.

15.88 CtcLoss

```
def CtcLoss(rawLogits, targets)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 373. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 6.30.

Purpose | What this contributes

The CTC loss, a forward and a backward sweep in logarithms add up the probability of every alignment of the label to the columns, and the occupation of each class per column gives the gradient.

Inputs

`rawLogits`
: the network's raw output, shaped (time, batch, classes)

`targets`
: the label sequences, one per batch item

Output. a pair: the mean loss over the batch, and the gradient with respect to the raw output.

Method | What is being computed

This is the heart of training the reader. The label says what was written and not where, so the loss cannot compare the output to a target column by column. Instead CTC [34] defines the probability of the target text as the total probability of *every* alignment of it to the columns, and the loss is the negative logarithm of that.

Formula purpose and system use. Marginalise all legal blank/repeat paths to the target transcript. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$L \;=\; -\log P(\mathbf{y} \mid \mathbf{x}) \;=\; -\log \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} p_t(\pi_t) \tag{15.11}$$

Source: [34]

where

L the loss for one example

$\mathbf{y}$ the target text

$\mathbf{x}$ the input image

π one alignment: a choice of label for each column

$\mathcal{B}^{-1}(\mathbf{y})$
: the set of every alignment that collapses to the target text, by the merge-then-remove-blanks rule described under DecodeCtc

$p_t(\pi_t)$
: the network's probability for the label this alignment chose at column t

T the number of columns

The negative logarithm is the standard form for a loss built from a probability: maximising the probability is minimising its negative logarithm, and the logarithm turns the product over columns into a sum, which is both numerically stable and what makes the gradient manageable.

Justification | Why the sum cannot be computed directly

The number of alignments is enormous. A 40-character label across 160 columns has on the order of 10 to the power 40 valid alignments, so enumerating them is as impossible as enumerating the seam paths in segment.py, and for the same reason. The remedy is also the same: dynamic programming over a quantity that can be built up column by column.

Method | The forward variable

Formula purpose and system use. Accumulate the forward valid-prefix probability for each target state. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$\alpha_t(s) \;=\; \Big(\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \alpha_{t-1}(s-2)\Big) \cdot p_t(\mathrm{ext}[s]) \tag{15.12}$$

Source: [34]

where

$\alpha_t(s)$
: the total probability of every alignment that reaches state s of the extended label sequence by column t

$\mathrm{ext}[s]$
: the label at position s of the extended sequence, built as blank, label, blank, label, … as described under CtcForcedAlign

$p_t(\cdot)$ the network's probability for that label at column t

$\alpha_{t-1}(s-2)$
: the skip term, excluded when the skip is forbidden

The recurrence is the same topology as the forced alignment: the same three transitions and the same restriction that a skip may not land on a blank or repeat the label two states back. The one crucial difference is that alignment took the maximum of the three arrivals and this takes the **sum**.

Justification | Why sum here and maximum there

This is the distinction worth being clear about. Alignment wants one particular layout, the most probable, so it takes the best arrival. Training wants the total probability of the text however it is laid out, so it adds the arrivals: every alignment that reaches this state contributes, and their probabilities add because they are mutually exclusive ways of the same thing happening.

Summing is also what makes CTC trainable at all. If training used the maximum it would be committing to one alignment and pushing the network toward it, which would be arbitrary early on when the network has no idea where anything is. Summing lets the network be rewarded for any layout that spells the right text, and the alignment emerges from training rather than being imposed.

In log space the sum is performed by LogSumExp, applied twice to combine the three terms, which is why that function exists.

Method | The backward variable

Formula purpose and system use. Combine forward and backward scores into conditional target occupancy. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$\beta_t(s) \;=\; \sum_{s' \in \{s,\, s+1,\, s+2\}} \beta_{t+1}(s') \cdot p_{t+1}(\mathrm{ext}[s']) \tag{15.13}$$

Source: [34]

where

$\beta_t(s)$
: the total probability of completing the label from state s at column t onwards

s'
: the states reachable from s, the mirror of the forward transitions

$p_{t+1}(\cdot)$
: the probability of the label emitted on arrival

The forward variable measures the probability of everything *before* a point and the backward variable the probability of everything *after* it. Both are needed because the gradient requires knowing, for each column and each label, the total probability of all the complete alignments passing through that point, and that is the product of the two.

Method | The total probability, and the loss

The total probability of the label is the forward variable at the final column, in either of the two states a valid alignment may end in, the last character or the trailing blank, summed. The loss is its negative logarithm.

Justification | The gradient, and why it takes this remarkably simple form

Formula purpose and system use. Subtract target occupancy from predicted probability to obtain the logit derivative. **Source basis:** [34, 42, 53]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial z_t(k)} \;=\; p_t(k) \;-\; \frac{1}{P(\mathbf{y} \mid \mathbf{x})} \sum_{s\,:\,\mathrm{ext}[s]=k} \alpha_t(s)\,\beta_t(s) \tag{15.14}$$

Source: [34]

where

$\frac{\partial L}{\partial z_t(k)}$
: the gradient with respect to the raw score for class k at column t

$p_t(k)$
: the network's current probability for that class, from the softmax

$\alpha_t(s)\beta_t(s)$
: the total probability of all complete alignments passing through state s at column t

$\sum_{s:\mathrm{ext}[s]=k}$
: summed over every state of the extended sequence that emits class k, since a character may appear several times

$P(\mathbf{y} \mid \mathbf{x})$
: the total probability of the label, which normalises the product into a proportion

The second term is called the **occupation**: among all the alignments that spell the target, the proportion that have class k at column t. So the gradient is the difference between what the network currently predicts for that class and what it *should* predict given the target. Where the network already agrees with the occupation the gradient is zero; where it predicts too much of a class the gradient is positive and that score is pushed down.

That this comes out as a simple difference is the same simplification that makes the cross-entropy gradient a difference, and it arises for the same reason: the softmax and the logarithm in the loss are inverse operations, so differentiating their composition cancels the awkward terms. It is why the gradient is computed with respect to the raw logits rather than with respect to the log probabilities, even though the loss was defined on the latter: taking the whole softmax and loss together is far simpler than either separately.

Justification | Why there is a loop over the batch

Each example has its own label and so its own length of extended sequence, so the dynamic programming tables are different shapes and cannot be computed as one array. The loop over the batch is therefore unavoidable, and inside it the loops over time and over states remain in Python too, which makes this the slowest function in the file. The small loop over states checking the skip restriction could be vectorised by precomputing the mask once per example instead of once per column, which is the obvious remaining optimisation.

> *Both the loss and the gradient are divided by the batch size, so the loss is a mean over examples rather than a total. That makes it comparable between batch sizes, which matters because the learning rate is tuned against the gradient's magnitude: without the division, doubling the batch size would double the gradient and effectively double the learning rate.*

Stage: Writer cross entropy

15.89 Method background: Writer cross entropy

Purpose | The role in this system

Penalise the network for assigning low probability to the labelled writer.

Justification | The reason for this construction

For a one-hot writer label, negative log probability is the categorical negative log likelihood. Its logit gradient is predicted probability minus the target indicator; it trains the mutually exclusive writer head.

Alternatives | What changes if another method is used

Squared probability error changes the derivative and optimisation behaviour. Class-weighted cross entropy changes the relative cost of examples and needs a justified weighting policy. This is distinct from CTC, which marginalises unknown character alignments.

Library | Implementation speed and accuracy

Framework cross-entropy losses normally accept raw logits and combine stable log softmax with target indexing. Feeding probabilities instead, or changing sum/mean reduction, changes the objective or gradient scale. A compiled loss may reduce overhead but is not inherently a different classifier. [48, 3].

15.90 CrossEntropyLoss

```
def CrossEntropyLoss(logits, targetIdx)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 434. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.89.

Purpose | What this contributes

Penalise the network for assigning low probability to the labelled writer.

Inputs

`logits`
: the raw scores, one row per example

`targetIdx`
: the correct class index for each example

Output. a pair: the mean loss, and the gradient with respect to the logits.

Purpose | What it is for

The loss for the writer namer, which is an ordinary classification: one correct answer per example, and its position is known, so none of the CTC machinery is needed.

Formula purpose and system use. Penalise low probability assigned to the labelled writer. **Source basis:** [48, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$L \;=\; -\frac{1}{B}\sum_{b=1}^{B}\log p_b(c_b) \tag{15.15}$$

where

L the mean loss over the batch

B the batch size

c_b the correct class for example b

$p_b(c_b)$
: the probability the network assigned to that class, from the softmax

Justification | Why the negative logarithm of the correct class

Cross-entropy for a single correct answer reduces to the negative logarithm of the probability assigned to it. Its shape is what makes it a good loss: a probability of 1 gives a loss of 0, a probability of 0.5 gives about 0.69, and as the probability approaches zero the loss grows without bound. So being confidently wrong is punished far more than being uncertain, which is the behaviour wanted from a classifier that reports confidences.

Method | The gradient

Formula purpose and system use. Compute the softmax-cross-entropy derivative for each writer logit. **Source basis:** [48, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial z_b(k)} \;=\; \frac{1}{B}\Big(p_b(k) - [\,k = c_b\,]\Big) \tag{15.16}$$

where

$\frac{\partial L}{\partial z_b(k)}$
: the gradient with respect to the raw score for class k of example b

$p_b(k)$
: the predicted probability for that class

$[\, k = c_b \,]$
: 1 for the correct class and 0 for every other

$\frac{1}{B}$
: the batch mean, matching the loss

The gradient is the predicted distribution minus the target distribution, the latter being 1 on the correct class and 0 elsewhere. The code expresses it exactly that way: copy the probabilities and subtract one from the correct class. Every wrong class is pushed down in proportion to how much it was predicted, and the right one is pushed up in proportion to how much it was missed.

The same cancellation as in CTC produces this. Differentiating the softmax alone gives a full matrix of terms, and differentiating the logarithm gives a reciprocal, and composing them cancels almost everything. This is the standard reason softmax and cross-entropy are implemented as one unit rather than two layers.

> *The log-sum-exp stabilisation is applied here too, the maximum being subtracted before exponentiating, for the reason given under LogSoftmax. It costs one pass and removes any possibility of overflow on a large logit.*

Stage: Analytic differentiation and training caches

15.91 Method background: Analytic differentiation and training caches

Purpose | The role in this system

Compute parameter derivatives without relying on framework automatic differentiation.

Justification | The reason for this construction

Saved forward intermediates let the chain rule propagate one loss derivative through convolution, normalisation, recurrence and pooling. Overlapping input patches must add their contributions.

Alternatives | What changes if another method is used

Finite differences independently check selected coordinates but cost two loss evaluations per coordinate and suffer cancellation. Automatic differentiation reduces manual algebra but still needs correct forward semantics.

Library | Implementation speed and accuracy

PyTorch autograd supplies reverse differentiation and can be a numerical reference. Match dtype, reduction, state and nonsmooth-point convention. A close gradient comparison validates checked operations, not every dataset or trained model. [48, 37, 52].

15.92 Conv2dBackward

```
def Conv2dBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 65. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

The weight gradient is the output gradient times those columns, and the input gradient is folded from columns back into image shape.

Inputs

`dout`
: the gradient of the loss with respect to this layer's output

`cache`
: what the forward pass saved

Output. three gradients: with respect to the input, the weights and the bias.

Method | Where the three formulas come from

The forward pass, in matrix form, is the flattened weights multiplied by the columns. Everything below follows from differentiating a matrix product, so it is worth stating that rule first: if Y = W X, then the gradient with respect to W is the output gradient times X transposed, and the gradient with respect to X is W transposed times the output gradient. Both the shapes and the transposes follow from requiring the result to have the same shape as the thing differentiated.

Method | The weight gradient

Formula purpose and system use. Accumulate convolution weight gradients from cached input patches and output derivatives. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial W} = \sum_b \frac{\partial L}{\partial Y_b} X_b^{\mathsf{T}} \tag{15.17}$$

where

$\frac{\partial L}{\partial W}$ the gradient for the filters, same shape as the filters

$\frac{\partial L}{\partial Y_b}$ the output gradient for batch item b, as a matrix of output channels by positions

X_b the cached columns for that item

T matrix transpose

$\sum_b$ summed over the batch

The sum over the batch is not an approximation or an average applied afterwards; it is what the derivative is. The same filter was applied to every item in the batch and at every position, so the loss depends on each weight through all of them, and the chain rule adds the contributions. This is the general rule for a shared parameter: wherever one value is used in several places, its gradient is the sum over those places.

Method | The input gradient, and the fold back into image shape

Formula purpose and system use. Propagate output derivatives into the patch representation through transposed weights. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial X} = W^{\mathsf{T}} \frac{\partial L}{\partial Y} \tag{15.18}$$

where

$\frac{\partial L}{\partial X}$ the gradient with respect to the columns

W the flattened filters

$\frac{\partial L}{\partial Y}$ the output gradient

That gives a gradient with respect to the *columns*, which is not what is wanted: the layer below needs a gradient with respect to the image. Converting one to the other is the only part of this function that is not a matrix product, and it is the subtle part.

The reason it is subtle is that the forward view was overlapping. One input pixel appears in up to nine patches, so it influenced up to nine output positions, and by the chain rule its gradient is the sum of the gradients from all of them. The loop over the nine filter offsets does exactly that: for each offset, the corresponding slice of the column gradient is *added* into the padded image gradient at the shifted position. Nine additions over whole arrays accumulate every contribution.

The accumulation is the whole point, and it is why the forward trick cannot be run in reverse. The forward pass could use a strided view because reading the same memory from several places is harmless; writing to it from several places is not, since the writes would overwrite rather than accumulate. So the backward direction must be an explicit addition, and the operation is known as **col2im**, the counterpart of im2col.

Method | Finally, removing the padding

The gradient was accumulated into an array the size of the padded input, and the layer below knows nothing of the padding, so the border is discarded. That is correct rather than merely convenient: the padding was constant zeros, not data, so the gradient with respect to it has nowhere to go.

Method | The bias gradient

Formula purpose and system use. Add overlapping patch derivatives back into the input image. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial b_o} = \sum_{b,p,q} \frac{\partial L}{\partial Y[b, o, p, q]} \tag{15.19}$$

where

$\frac{\partial L}{\partial b_o}$ the gradient for the bias of output channel o

$\sum_{b,p,q}$ summed over the batch and both spatial axes

One bias is added to every position of its channel, so by the shared-parameter rule its gradient is the sum of the output gradient over all those positions. No cache is needed because the derivative of adding a constant is one.

15.93 BatchNorm2dBackward

```
def BatchNorm2dBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 110. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Gradients through the normalising, where the mean and variance depend on every item in the batch.

Inputs

`dout`
: the gradient with respect to this layer's output

`cache`
: what the forward pass saved

Output. three gradients: with respect to the input, the scale and the shift.

Justification | Why this one is genuinely hard

Every other layer in this file has a local derivative: the output at one position depends on the input at one position, or a few. Batch normalisation does not. The mean and variance are computed from *every* value in the channel, so changing one input changes the mean, which changes the normalisation of every other value in the channel as well. Each input therefore affects its own output directly and every other output indirectly, and the gradient must account for all three routes.

Method | The three paths

- **Directly,** through its own standardised value.
- **Through the mean,** since the input contributed to the mean, and the mean is subtracted from every value in the channel.
- **Through the variance,** since the input contributed to the variance, and the variance divides every value in the channel.

Differentiating each path and collecting the terms gives an expression that simplifies considerably, which is the form the code uses.

Formula purpose and system use. Accumulate derivatives for the learned affine scale and offset. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial x} = \frac{1}{N\sigma}\left(N\frac{\partial L}{\partial \hat{x}} - \sum \frac{\partial L}{\partial \hat{x}} - \hat{x}\sum\left(\frac{\partial L}{\partial \hat{x}}\hat{x}\right)\right) \tag{15.20}$$

where

$\frac{\partial L}{\partial x}$ the gradient to pass down

$\frac{\partial L}{\partial \hat{x}}$ the gradient with respect to the standardised values, which is the incoming gradient multiplied by the learned scale

$\hat{x}$ the cached standardised values

σ the cached standard deviation

N the number of values per channel

$\sum$ summed over the batch and both spatial axes, per channel

The three terms correspond to the three paths, and each has a readable meaning. The first is the direct effect. The second subtracts the average gradient over the channel, which is the correction for the mean: it ensures the gradients sum to zero, so the layer below cannot be pushed in a direction that would merely shift the whole channel, since such a shift would be normalised away anyway. The third subtracts the component aligned with the standardised values themselves, which is the correction for the variance: it removes any push that would merely scale the channel, for the same reason.

So the layer passes down only the part of the gradient that is not a pure shift or a pure rescaling of the channel. That is the deeper statement of what batch normalisation does: it makes the network's behaviour independent of the scale and offset of each channel, and the backward pass correspondingly refuses to pass down gradients that would only change those.

Method | The scale and shift gradients

Formula purpose and system use. Differentiate through the batch-dependent mean and variance, not merely the fixed-statistics affine map. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial \gamma} = \sum \frac{\partial L}{\partial y}\,\hat{x}, \qquad \frac{\partial L}{\partial \beta} = \sum \frac{\partial L}{\partial y} \tag{15.21}$$

where

γ, β the learned scale and shift

$\frac{\partial L}{\partial y}$ the incoming gradient

$\hat{x}$ the standardised values

$\sum$ summed over everything but the channel axis

These two are straightforward by comparison. The output is the scale times the standardised value plus the shift, so the derivative with respect to the scale is the standardised value and with respect to the shift is one, each summed over every position the parameter was used at, by the shared-parameter rule.

> *The code comment attributes the simplified form to the standard derivation taught in the Stanford CS231n course notes, which is where this arrangement of the terms is usually given. The project's numerical gradient check exists largely to confirm that this function in particular is right, since it is the one most easily got wrong and an error in it produces slow or unstable training rather than an obvious failure.*

15.94 ReluBackward

```
def ReluBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 128. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dout`
: the gradient with respect to the output

`cache`
: the positivity mask

Output. the gradient with respect to the input.

Method | The derivative

Formula purpose and system use. Route the loss derivative through active positive responses. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases} \tag{15.22}$$

where

$\frac{\partial L}{\partial x}$ the gradient to pass down

$\frac{\partial L}{\partial y}$ the gradient arriving

x the original input, consulted only through the cached mask

ReLU is the identity where the input was positive and constant zero where it was not, so its derivative is one or zero accordingly. The backward pass is therefore the incoming gradient with the inactive positions zeroed: the gradient passes unchanged through the units that fired and stops dead at the ones that did not.

Justification | Why this is the property that makes deep networks trainable

A derivative of exactly 1 on the active path is what distinguishes ReLU from the smooth activations it replaced. The sigmoid's derivative is at most 0.25 and much smaller when saturated, so a gradient crossing twelve sigmoid layers is multiplied by twelve such factors and arrives at the first layer effectively zero. That is the vanishing gradient problem, and it is why networks of this depth were impractical before ReLU. With a factor of exactly 1 the gradient reaches the first layer at full strength.

> *The derivative at exactly zero does not exist, since the function has a corner there. Taking it as zero is conventional and the choice has no practical consequence: an input of exactly zero is a measure-zero event in floating point, and either choice gives a valid subgradient.*

15.95 MaxPool2dBackward

```
def MaxPool2dBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 148. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dout`
: the gradient with respect to the pooled output

`cache`
: the winning indices and the geometry

Output. the gradient with respect to the input.

Method | The routing rule

Max pooling passes one element of each window through unchanged, so its derivative with respect to that element is one and with respect to every other element of the window is zero. The gradient is therefore routed entirely to the winner and the rest of the window receives nothing.

Formula purpose and system use. Route a pooled derivative to its saved maximum location. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial x[b,c,i,j]} = \begin{cases} \dfrac{\partial L}{\partial y[b,c,p,q]} & (i,j) \text{ was the maximum of window } (p,q) \\ 0 & \text{otherwise} \end{cases} \tag{15.23}$$

where

$x[b,c,i,j]$
: an input element

$y[b,c,p,q]$
: the pooled output of the window containing it

(p,q)
: which window

An input that lost its window has no influence on the output at all, so changing it slightly changes nothing and its gradient is genuinely zero. That is a real consequence of pooling rather than an approximation: only a quarter of the inputs to a 2 by 2 pooling receive any gradient on a given example, which is part of why the layers below a pooling train more slowly.

Justification | Why no accumulation is needed here

The code comment notes that the stride equals the kernel, so the windows do not overlap and each input belongs to exactly one window. The gradient blocks can therefore be written straight into position rather than added. With overlapping windows an input could win several, and the contributions would have to be accumulated as they are in the convolution's backward pass.

> *The transpose before the final reshape interleaves the window axes back with the output axes, so that the blocks land in the right places. Getting this wrong produces a gradient that is correct in magnitude and scrambled in position, which trains to something, slowly and wrongly, without any error being raised. It is exactly the kind of fault the numerical gradient check exists to catch.*

15.96 AdaptiveMaxPoolHeightBackward

```
def AdaptiveMaxPoolHeightBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 167. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dout`
: the gradient with respect to the collapsed output

`cache`
: the input shape and the winning rows

Output. the gradient with respect to the full-height input.

Method | What it does

Creates a zero array of the original shape and writes the incoming gradient into the winning row of each column. Every other row gets nothing.

Method | What that means for training

Of the sixteen rows the feature map has at this point, exactly one per column receives any gradient on a given example. So each column of each channel trains from one row at a time, though which row varies between examples and over the course of training as the weights change. The effect is that the layer below learns to make its strongest response correct, rather than learning anything about the rows that lose, which is the behaviour a maximum should produce: the network is being asked whether a feature appears anywhere in the column, so only the best evidence matters.

15.97 AdaptiveAvgPoolWidthBackward

```
def AdaptiveAvgPoolWidthBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 179. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dout`
: the gradient with respect to the pooled output

`cache`
: how many values were averaged

Output. the gradient spread evenly back over those values.

Method | The derivative

Formula purpose and system use. Distribute a width-average derivative equally across its inputs. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$y = \frac{1}{N}\sum_{k} x_k \quad \Longrightarrow \quad \frac{\partial y}{\partial x_k} = \frac{1}{N} \quad \text{for every } k \tag{15.24}$$

where

y the pooled output

x_k the k-th input averaged over

N how many there were

$\Longrightarrow$ implies

Each input contributes one N-th of the average, so each receives one N-th of the gradient. The backward pass is therefore the incoming gradient repeated across the axis and divided by the length.

Method | The contrast with maximum pooling, and what it means for training

Where max pooling routes the whole gradient to one element and nothing to the rest, average pooling spreads it thinly over all of them. Every position therefore receives gradient on every example, which is why the writer namer's backbone trains more evenly than the reader's. It is also the right behaviour for what that layer means: the namer asks how prevalent a feature is across the line, so every position genuinely contributes to the answer and every position should be told how it contributed.

15.98 LinearBackward

```
def LinearBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 191. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dout`
: the gradient with respect to the output

`cache`
: the input and the weights

Output. three gradients: with respect to the input, the weights and the bias.

Method | The three gradients

Formula purpose and system use. Form weight, bias and input derivatives of the affine feature mapping. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Y}^{\mathsf{T}} X, \qquad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W, \qquad \frac{\partial L}{\partial b} = \sum_{\text{rows}} \frac{\partial L}{\partial Y} \tag{15.25}$$

where

$\frac{\partial L}{\partial Y}$ the incoming gradient, one row per example

X the cached input, one row per example

W the weight matrix, one row per output

T transpose

$\sum_{\mathtt{rows}}$ summed over the examples

All three follow from differentiating a matrix product, as in the convolution. The transposes are determined by the shapes: there is only one arrangement of each product that yields an array the same shape as the quantity being differentiated, which is a useful check when deriving such expressions by hand.

Justification | Why the leading axes are flattened

The code comment explains the one wrinkle. The input may carry more than a batch axis before its feature axis: the reader's text head applies this layer to a sequence, shaped time by batch by features, so there are two leading axes. The weight gradient must sum over every one of those positions, since the same weights were used at each.

Flattening every leading axis into one reduces any such case to the two-dimensional form, after which the single matrix product performs the sum automatically. The input gradient needs no flattening, because it has the same leading axes as the output and the product acts only on the last; it is reshaped back to the input's shape at the end. This is what lets one implementation serve both the namer's single vector and the reader's sequence.

15.99 ConvBlockBackwardTrain

```
def ConvBlockBackwardTrain(dout, caches, prefix, nLayers)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 231. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dout`
: the gradient with respect to the block's output

`caches`
: the per-group caches from the forward pass

`prefix`
: the block's name prefix

`nLayers`
: how many groups

Output. a pair: the gradient with respect to the block's input, and a dictionary of weight gradients keyed as the parameters are.

Method | The reversal

The groups are walked in reverse, and within each group the three layers are walked in reverse too: activation, then normalisation, then convolution. Each step takes the gradient from the step before it, so the gradient flows back through the block in exactly the opposite order to the way the data flowed forward.

Justification | Why the gradients are keyed by parameter name

The weight gradients are collected into a dictionary using the same names the parameters themselves use. That is what lets the optimiser be written without any knowledge of the network's structure: it iterates over the parameters and looks up each one's gradient by name. The names are therefore a contract between three separate things, the initialiser, the backward pass and the checkpoint format, and a mismatch in any of them fails with a missing key rather than silently training the wrong array.

> *The variable holding the gradient is reassigned at each step rather than being given a new name per layer. That reads oddly but is exactly right: there is only ever one gradient travelling backwards at a time, and naming its successive values separately would suggest they coexist.*

15.100 LstmBackward

```
def LstmBackward(dOutputs, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 274. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Walks back from the last column, carrying the gradient of the memory as well as of the output.

Inputs

`dOutputs`
: the gradient with respect to the output at every step

`cache`
: the per-step values and the weights

Output. five gradients: with respect to the input sequence, both weight matrices and both biases.

Method | Backpropagation through time

An LSTM applies the same weights at every step, so the loss depends on each weight through all 160 steps. By the shared-parameter rule the weight gradient is the sum over the steps, and the state gradients must be carried backwards through them, because the state at step t influenced the output

at every later step. The procedure is **backpropagation through time**: walk the steps in reverse, accumulating weight gradients and carrying state gradients.

Method | The two gradients carried backwards

At each step two quantities arrive from the step after it: the gradient with respect to the hidden state and the gradient with respect to the cell state. The hidden gradient is combined with the gradient arriving directly from the layer above, because the hidden state at step t is used twice, as that step's output and as the next step's input.

Formula purpose and system use. Combine loss contributions through the exposed hidden state and retained cell state. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\mathrm{d}h_t \;=\; \frac{\partial L}{\partial y_t} \;+\; \mathrm{d}h_{t+1}^{\text{carried}} \tag{15.26}$$

where

$\mathrm{d}h_t$ the total gradient with respect to the hidden state at step t

$\frac{\partial L}{\partial y_t}$ the gradient arriving from the layer above for this step's output

$\mathrm{d}h_{t+1}^{\text{carried}}$
: the gradient the following step computed with respect to its incoming hidden state

Adding the two is the single most important line of the function. Omitting the carried term computes the gradient as though each step's output were independent, which trains something that works tolerably on short sequences and never learns any long-range dependency, which is the entire reason an LSTM was chosen.

Method | Differentiating the two state updates

The updates are h = o · tanh(c) and c = f · c_prev + i · g, and everything else follows from differentiating them. Each gate's gradient is the gradient of whatever it multiplied, and then each is passed back through its own squashing function.

Formula purpose and system use. Propagate the gated cell derivative to previous time and the current affine gate inputs. **Source basis:** [48, 37, 52]. Exact project constants remain implementation choices unless evidence is stated.

$$\begin{aligned}
\mathrm{d}o &= \mathrm{d}h \odot \tanh(c_t) \\
\mathrm{d}c &= \mathrm{d}c^{\text{carried}} + \mathrm{d}h \odot o \odot \left(1 - \tanh^2(c_t)\right) \\
\mathrm{d}f &= \mathrm{d}c \odot c_{t-1} \\
\mathrm{d}i &= \mathrm{d}c \odot g \\
\mathrm{d}g &= \mathrm{d}c \odot i \\
\mathrm{d}c_{t-1} &= \mathrm{d}c \odot f
\end{aligned} \tag{15.27}$$

where

$\mathrm{d}o$, $\mathrm{d}f$, $\mathrm{d}i$, $\mathrm{d}g$
: the gradients with respect to the four gate outputs

$\mathrm{d}c$ the gradient with respect to this step's cell state

dc_{t-1}
: the gradient carried back to the previous cell state

c_t, c_{t-1}
: the cell states at this and the previous step

$\odot$
: elementwise multiplication

$1 - \tanh^2$
: the derivative of the hyperbolic tangent

Every line is the product rule: the gradient with respect to one factor is the incoming gradient times the other factor. The cell gradient has two contributions because the cell is used twice, revealed through the hidden state at this step and carried forward to the next, which is the same double-use argument as for the hidden state.

Library | The gradient path that makes long memory possible

The last line is worth isolating. The gradient carried back to the previous cell state is the current cell gradient multiplied by the forget gate, and by nothing else: no weight matrix is applied along this path. So if the forget gate is near 1, the gradient passes backwards through that step essentially unchanged, and it can do so for many steps in succession without shrinking.

That is the precise sense in which the LSTM solves the vanishing gradient problem. A plain recurrent layer multiplies the carried gradient by its recurrent weight matrix at every step, so after a hundred steps it has been multiplied by that matrix a hundred times and is either negligible or enormous. The LSTM's cell path multiplies only by gate values the network itself controls, and it can learn to set them near 1 where information must survive.

Method | Through the squashing functions

Each gate gradient is then passed back through its own activation to reach the pre-activation value, which is what the weights actually produced. For the three sigmoid gates the derivative is the output times one minus the output, and for the candidate it is one minus the output squared. Both are written in terms of the output rather than the input, which is why the cache holds the gate activations and not the pre-activations.

Method | The weight gradients

The four pre-activation gradients are concatenated back into the stacked layout the weights use, and the weight gradients are then matrix products of that stack with the step's input and previous hidden state. They are accumulated across steps with addition rather than assignment, which is the shared-parameter sum. Both biases receive the same gradient, since both were added to the same quantity, which is the backward counterpart of the inference note that only their sum has any effect.

> *The input gradient is written per step into a preallocated array rather than appended, because the steps are visited in reverse and the result must come back in forward order. Appending and reversing would also work; writing by index makes the ordering explicit at the point it matters.*

15.101 BidirectionalLstmBackward

```
def BidirectionalLstmBackward(dout, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 323. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dout`
: the gradient with respect to the concatenated output

`cache`
: both directions' caches and the hidden size

Output. the gradient with respect to the input, and both directions' weight gradients.

Method | Splitting the gradient

The forward pass concatenated the two directions along the feature axis, so the incoming gradient is split at the same point: the first half belongs to the forward direction and the second to the backward one. Concatenation and splitting are exact inverses, so no arithmetic is involved.

Justification | Why the two input gradients are added

Both directions read the *same* input sequence, so the input is a shared quantity and by the shared-parameter rule its gradient is the sum of the two directions' contributions. This is easy to get wrong by treating them as separate inputs and returning only one, which halves the gradient reaching the convolutional backbone and makes it train at half rate with no visible symptom.

Method | The reversal, undone

The backward direction's gradient must be reversed before being handed to its LSTM backward pass and reversed again afterwards, mirroring the two reversals of the forward pass. The invariant to hold on to is that every array crossing the boundary of this function is in forward time order, and the reversed order exists only inside it.

15.102 MultilayerBidirectionalLstmBackward

```
def MultilayerBidirectionalLstmBackward(dout, caches)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 350. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dout`
: the gradient with respect to the final output

`caches`
: the per-layer caches

Output. the gradient with respect to the input, and every layer's weight gradients.

Method | What it does

Walks the layers in reverse, passing each layer's input gradient to the layer below and collecting the weight gradients. There is no arithmetic of its own; it is the sequencing that makes the chain rule hold across the stack.

> *This completes the recurrent half. The gradient arriving here came from the text head, and the gradient leaving goes to the height-collapsing pool, which converts it back from a sequence gradient into an image gradient. Those two conversions, forward and backward, are what join the convolutional and recurrent halves of the reader.*

15.103 BackboneBackwardTrain

```
def BackboneBackwardTrain(dx, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1787. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dx` the gradient with respect to the backbone's output

`cache`
: what the forward pass saved

Output. the gradient with respect to the input and every backbone weight gradient.

Method | What it does

Walks the backbone in reverse: uncollapse the height, then the third block, then the second pooling, the second block, the first pooling and the first block, collecting weight gradients. It is the mirror of the forward sequence and contains no arithmetic beyond the sequencing.

> *The input gradient is computed and returned although nothing uses it, since the crops are data rather than parameters. It is produced because each layer's backward pass produces it as a matter of course, and discarding it earlier would mean a special case in the first convolution. It is also what the numerical gradient check compares against when verifying the backbone.*

15.104 TextModelBackward

```
def TextModelBackward(dRawLogits, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1869. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dRawLogits`
: the gradient from the loss

`cache`
: what the forward pass saved

Output. the gradient with respect to the input and every weight gradient.

Method | What it does

Reverses the forward sequence: the text head, the two recurrent layers, the transpose, and the backbone, collecting weight gradients keyed by parameter name.

Method | The transpose, reversed

The forward pass moved the width axis to the front to make a sequence; the backward pass moves it back, so the gradient leaving for the backbone is an image gradient again. A transpose is its own inverse given the right axis order, so nothing is computed, but the axes must be put back in exactly the order the backbone expects, and an error there produces a gradient that is correct in magnitude and scrambled in arrangement.

15.105 AuthorModelBackward

```
def AuthorModelBackward(dLogits, cache)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 2131. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.91.

Purpose | What this contributes

Compute parameter derivatives without relying on framework automatic differentiation.

Inputs

`dLogits`
: the gradient from the loss

`cache`
: what the forward pass saved

Output. the gradient with respect to the input and every weight gradient.

Method | What it does

Reverses the namer's forward sequence: the second linear layer, the dropout, the ReLU, the first linear layer, the average pooling, and the backbone. The average pooling's backward pass spreads the gradient evenly back over every position, which is why the namer's backbone receives gradient everywhere rather than only at the winning rows as the reader's does.

Stage: Optimisers schedules and gradient clipping

15.106 Method background: Optimisers schedules and gradient clipping

Purpose | The role in this system

Turn analytic gradients into stable parameter updates and manage training step sizes.

Justification | The reason for this construction

RMSprop adapts using recent squared gradients; AdamW separates weight shrinkage from adaptive gradient scaling; clipping limits gradient norm. These are training mechanisms, not live inference controls.

Alternatives | What changes if another method is used

SGD momentum is simpler but has different scaling. Coupled L2 and decoupled AdamW are not identical. Elementwise clipping changes direction unlike global norm clipping.

Library | Implementation speed and accuracy

PyTorch optimisers automate state and updates. Defaults, epsilon placement, moment bias correction and scheduler counters must match. Faster training is not proof of better final validation accuracy. [80, 81, 82, 79].

15.107 RMSprop

```
class RMSprop
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 446. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.106.

Purpose | What this contributes

Turn analytic gradients into stable parameter updates and manage training step sizes.

Method | Methods exposed by this current class

```
def __init__(self, params, lr=0.001, alpha=0.99, eps=1e-08, weightDecay=0.0)
def Step(self, grads)
```

Output. an optimiser object holding the parameters and a running average of squared gradients.

Purpose | The problem plain gradient descent has

A single learning rate must suit every weight, and it cannot. Different weights have gradients of wildly different magnitudes, by orders of magnitude between layers and between a bias and a convolution weight. A rate small enough to be stable for the largest gradients is far too small for the smallest, so some weights barely move while others oscillate.

Method | The idea: divide by the typical gradient size

RMSprop [89] keeps, for each weight, a running average of its squared gradient, and divides the step by the square root of that. A weight whose gradients are habitually large is divided by a large number and takes a small step; one with habitually small gradients takes a comparatively large one. The effect is that every weight moves at a comparable rate regardless of its gradient's scale.

Formula purpose and system use. Scale a gradient using a running estimate of its squared magnitude. **Source basis:** [80, 81, 82, 79]. Exact project constants remain implementation choices unless evidence is stated.

$$v \leftarrow \alpha\, v + (1-\alpha)\, g^2, \qquad w \leftarrow w - \eta\, \frac{g}{\sqrt{v}+\epsilon} \tag{15.28}$$

where

- v the running average of the squared gradient, per weight
- α how much of the old average to keep, 0.99 here
- g the current gradient for this weight
- w the weight
- η the learning rate
- ϵ a small constant, 1e-8, so a weight with no gradient history does not divide by zero

The square root is what makes the division dimensionally sensible: the running average is of squared gradients, so its square root is in the same units as a gradient, and the ratio is dimensionless. The name is from that root-mean-square. At 0.99 the average covers roughly the last hundred steps, so it describes the gradient's recent typical size rather than this step's value, which is what makes the scaling stable.

> *Weight decay here is added to the gradient, which is the classical formulation. The entry for AdamW explains why that interacts badly with the per-weight scaling and what the alternative is.*

15.108 AdamW

```
class AdamW
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 467. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.106.

Purpose | What this contributes

Momentum and that scaling together, with the weight decay applied separately from the gradient step.

Method | Methods exposed by this current class

```
def __init__(self, params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weightDecay=0.0)
def Step(self, grads)
```

Output. an optimiser object holding the parameters and two running averages per weight.

Method | What it adds to RMSprop

Adam [80] keeps a running average of the gradient itself as well as of its square, and uses the smoothed gradient for the step direction. The first is **momentum**: averaging successive gradients cancels the part that fluctuates between batches and keeps the part that is consistent, so the step follows the persistent downhill direction rather than the noise of whichever examples happened to be in this batch.

Formula purpose and system use. Update biased-corrected moments and apply decoupled parameter shrinkage. **Source basis:** [80, 81, 82, 79]. Exact project constants remain implementation choices unless evidence is stated.

$$
\begin{aligned}
m &\leftarrow \beta_1 m + (1-\beta_1)\, g \\
v &\leftarrow \beta_2 v + (1-\beta_2)\, g^2 \\
\hat{m} &= \frac{m}{1-\beta_1^t}, \qquad \hat{v} = \frac{v}{1-\beta_2^t} \\
w &\leftarrow w - \eta \left(\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} + \lambda w \right)
\end{aligned}
\tag{15.29}
$$

Source: [80]

where

m the running average of the gradient, the first moment

v the running average of its square, the second moment

β_1 the decay for the first, 0.9: an average over roughly ten steps

β_2 the decay for the second, 0.999: roughly a thousand steps

t the step number, counted from 1

$\hat{m}$, $\hat{v}$
the bias-corrected averages

λ the weight decay coefficient

η the learning rate

Justification | Why the averages must be bias-corrected

Both averages start at zero, which is not a neutral starting point but a strong claim that the gradient is zero. After one step the first moment is only one tenth of the observed gradient, and the second is one thousandth of its square, so early steps would be far too small and, worse, badly misproportioned between the two, since they are biased by different amounts.

The correction divides each by one minus its decay raised to the step number, which is exactly the total weight the average has accumulated so far. At step 1 the first moment is divided by 0.1, recovering the full gradient; by step 100 the divisor is nearly 1 and the correction fades away. So the optimiser behaves sensibly from its very first step, which matters because the first steps are taken from random weights where large corrections are most needed.

Justification | Why the W, and what it fixes

Weight decay shrinks every weight slightly at each step, which discourages large weights and reduces overfitting. Classically it is implemented by adding a multiple of the weight to its gradient, which for plain gradient descent is exactly equivalent.

With Adam it is not equivalent, and that is the point of AdamW [81]. A decay term added to the gradient passes through the division by the square-root average like any other gradient, so a weight with large gradients has its decay divided down and is barely regularised, while one with small gradients is decayed heavily. The strength of the regularisation therefore varies per weight in a way nobody intended, and it depends on the gradient history rather than on the weight.

The fix is to apply the decay to the weight directly, outside the scaled step, which is the separate term in the update above. Every weight is then shrunk by the same proportion, which is what weight decay is supposed to mean. This is called **decoupled** weight decay, and the W in the name stands for it.

Writing the decay inside the same parenthesis as the scaled step means it is multiplied by the learning rate, so the effective decay changes as the learning rate is annealed. The original formulation decouples it from the learning rate as well. The difference matters mainly late in training, when the rate has fallen and the decay falls with it.

15.109 ReduceLrOnPlateau

```
class ReduceLrOnPlateau
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 489. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.106.

Purpose | What this contributes

Turn analytic gradients into stable parameter updates and manage training step sizes.

Method | Methods exposed by this current class

```
def __init__(self, optimizer, factor=0.5, patience=5, minLr=1e-05)
def Step(self, metric)
```

Output. a scheduler object that lowers the learning rate when progress stalls.

Justification | Why the learning rate should change during training

A large rate is right early on, when the weights are far from any good configuration and large steps make rapid progress. It is wrong later, when the weights are close and a large step jumps past the minimum and bounces around it. So the rate should start large and end small, and a **scheduler** is what changes it.

Method | This scheduler's rule

It watches a metric, usually the validation loss, and remembers the best value seen. Each epoch that fails to beat the best counts as a bad epoch; once the bad epochs exceed the patience, the learning rate is multiplied by the factor and the count resets. A new best resets it too.

Justification | Why it reacts rather than following a schedule

The advantage over a fixed schedule is that it needs no estimate of how long training will take. It lowers the rate exactly when progress has stopped, which is the evidence that the current rate is now too large, whatever epoch that happens to be. The disadvantage is that it only reacts after the patience has elapsed, so several epochs are spent at a rate already known to be too large.

Justification | Why the patience is needed

Validation loss is noisy, because it is measured on a finite set and the weights change between epochs. A single epoch failing to improve means little. Requiring five consecutive failures distinguishes a genuine plateau from the ordinary fluctuation of a metric that is still improving overall. Too little patience lowers the rate on noise and ends training early at a rate too small to make progress.

The floor on the learning rate stops the repeated halving from reaching zero, at which point training would continue indefinitely without changing anything. It is a way of bounding wasted time rather than a property of the method.

15.110 CosineAnnealingLr

```
class CosineAnnealingLr
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 509. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.106.

Purpose | What this contributes

Turn analytic gradients into stable parameter updates and manage training step sizes.

Method | Methods exposed by this current class

```
def __init__(self, optimizer, tMax, baseLr=None)
def Step(self)
```

Output. a scheduler object that eases the learning rate down a cosine curve.

Method | The schedule

Formula purpose and system use. Decrease the training learning rate smoothly over the configured run. **Source basis:** [80, 81, 82, 79]. Exact project constants remain implementation choices unless evidence is stated.

$$\eta_t = \frac{\eta_0}{2}\left(1 + \cos\left(\frac{\pi t}{T}\right)\right) \tag{15.30}$$

where

η_t the learning rate at epoch t

η_0 the initial learning rate

t the current epoch

T the total number of epochs planned

cos the cosine, which runs from 1 at the start to -1 at the end over half a period

At epoch 0 the cosine is 1 and the rate is the full initial value; at the final epoch it is -1 and the rate is zero, with a smooth decrease between.

Justification | Why a cosine rather than a straight line

The shape is what recommends it. The cosine is flat at both ends and steepest in the middle, so training spends a while at nearly the full rate, then drops briskly, then spends a while at a very small rate. Both plateaux are useful: the first makes rapid early progress, and the second lets the weights settle precisely into a minimum rather than continuing to step past it. A linear decay spends equal time at every rate, including the middling ones that are neither fast nor precise.

Method | The trade-off against the plateau scheduler

This one requires knowing the total number of epochs in advance, since the schedule is defined against it. In exchange it never wastes epochs at a rate that has become too large, because it lowers the rate continuously rather than waiting for evidence. Having both available lets the choice be made per training run according to whether the length is known.

15.111 ClipGradNorm

```
def ClipGradNorm(grads, maxNorm=5.0)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 521. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 15.106.

Purpose | What this contributes

Turn analytic gradients into stable parameter updates and manage training step sizes.

Inputs

`grads`
: the gradient dictionary, modified in place

`maxNorm`
: the greatest allowed total magnitude, default 5.0

Output. the total gradient magnitude before any clipping.

Purpose | The problem it addresses

Recurrent networks occasionally produce an enormous gradient. A particular batch, often one with an unusually long or badly labelled line, can give gradients orders of magnitude larger than usual, and a single step with such a gradient moves the weights so far that everything learned so far is destroyed. Training then has to recover from near-random weights, and hours of work are lost to one batch. This is the **exploding gradient** problem.

Method | The remedy

Formula purpose and system use. Limit the global gradient norm while preserving its direction. **Source basis:** [80, 81, 82, 79]. Exact project constants remain implementation choices unless evidence is stated.

$$\|g\| \;=\; \sqrt{\sum_{k}\sum_{i} g_k[i]^2}, \qquad g \;\leftarrow\; g \cdot \min\!\left(1,\; \frac{\tau}{\|g\| + \delta}\right) \tag{15.31}$$

Source: [82]

where

$\|g\|$ the total gradient norm: every parameter's gradient treated as one long vector

$g_k[i]$ element i of the gradient for parameter k

τ the maximum norm allowed, 5.0

δ a tiny constant preventing a division by zero

$\min(1, \cdot)$
: so the scaling only ever shrinks

If the total magnitude exceeds the limit, every gradient is multiplied by the same factor so that the total comes to exactly the limit. The published remedy for exploding gradients is precisely this [82].

Justification | Why the whole network is scaled together

This is the part worth understanding. The norm is computed over every parameter at once, and one factor is applied to all of them, rather than each being clipped separately. The reason is that the

gradient's *direction* is the useful information: it says which way the loss decreases. Scaling everything by one factor preserves that direction exactly and changes only the distance travelled. Clipping each parameter independently would shorten some components and not others, which rotates the direction into one that is not a descent direction at all, so the step might increase the loss.

Justification | Why the norm is returned

The function returns the magnitude it measured, before any clipping, so the training loop can log it. That number is one of the most informative diagnostics available during training: a norm that is steady and comfortably below the limit means training is healthy, one that is clipped on most batches means the learning rate or the limit is wrong, and a sudden spike identifies the batch that caused it. Without the return value, clipping would silently conceal the very instability it is protecting against.

Stage: Checkpoint and log persistence

Method rationale, alternatives and library comparison: Section 3.1.

15.112 TrainableParams

```
def TrainableParams(params)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1739. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 3.1.

Purpose | What this contributes

Preserve weights, class mappings and evidence about a training run.

Inputs

`params`
: the full parameter dictionary

Output. the subset the optimiser may update.

Justification | What it excludes and why

The running mean and variance of every batch normalisation. They are stored alongside the parameters, because they must be saved in the checkpoint, but they are not parameters: no gradient is computed for them, and they are updated by observing the data rather than by descending a loss.

Method | What would happen if they were included

The optimiser would look up a gradient for them, find none, and fail with a missing key, which is the benign outcome. Had a zero gradient been supplied instead, the optimiser would apply weight decay to them and shrink them steadily toward zero, so the stored variances would fall and inference would divide by a smaller and smaller number, amplifying every activation. That is a silent corruption which would appear only at inference, as a network whose validation accuracy during training was fine and whose saved checkpoint performs badly.

> *The separation is by name suffix, which works but ties correctness to a naming convention. A parameter accidentally named with one of those suffixes would be silently frozen, and one of these accidentally named otherwise would be silently decayed.*

15.113 SaveCheckpoint

```
def SaveCheckpoint(path, params, extra=None)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1743. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 3.1.

Purpose | What this contributes

Preserve weights, class mappings and evidence about a training run.

Inputs

`path`
: where to write

`params`
: the parameters

`extra`
: additional values such as the epoch number

Output. nothing; the checkpoint file is written.

Method | What goes into a checkpoint

Every parameter, including the running statistics, plus metadata: which epoch it came from and the best validation loss seen so far. The metadata is what makes resuming possible.

Justification | Why the metadata must be in the file

Resuming a run requires knowing which epoch to continue from and what the best loss was, so that the scheduler and the best-checkpoint logic behave as though the run had never stopped. Keeping those in the checkpoint rather than in a separate file means they cannot be separated from the weights they describe, which is the failure a parallel log file invites: resuming from the wrong epoch number silently misschedules the learning rate.

> *The metadata keys begin with a double underscore, which is the same convention LoadStateDictAny uses to tell metadata from weights. The two functions are in different files and must agree on it, which is worth noting as another implicit contract.*

15.114 LoadCheckpointMeta

```
def LoadCheckpointMeta(path)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1751. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 3.1.

Purpose | What this contributes

Preserve weights, class mappings and evidence about a training run.

Inputs

`path`
: a checkpoint file

Output. the epoch it was saved at and the best loss recorded, where present.

Purpose | What it is for

Reads only the metadata, without the weights, so that a resume can establish where it is continuing from. Reading the metadata alone is cheap, where loading several hundred megabytes of weights to read two numbers is not.

> *Both values are optional, since a checkpoint written before the metadata convention existed has neither. Returning nothing rather than a default lets the caller distinguish "no record" from "epoch zero", and it starts the run at epoch 1 rather than claiming to resume.*

15.115 LoadParamsInto

```
def LoadParamsInto(params, checkpointPath)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1761. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 3.1.

Purpose | What this contributes

Loads saved weights into a built network, copying into the arrays the optimiser already holds.

Method | Current implementation

Preserve weights, class mappings and evidence about a training run. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
def LoadParamsInto(params,checkpointPath):
    sd,_ = CLS.LoadStateDictAny(checkpointPath)
    loaded = 0
    for k in params:
        if k in sd and sd[k].shape==params[k].shape:
            params[k][...] = sd[k].astype(np.float32)
            loaded+=1
    return loaded
```

Justification | Interpret the method rather than infer a benchmark

Named arrays and metadata make inference state inspectable and support interrupted work. A best checkpoint is chosen by a stated criterion rather than final epoch alone. [3, 4].

15.116 AppendLog

```
def AppendLog(path, line)
```

Evidence | Source and method link

`Software/DemoCode/train.py` line 1771. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 3.1.

Purpose | What this contributes

Preserve weights, class mappings and evidence about a training run.

Inputs

`path`
: the log file

`line`
: the line to add

Output. nothing; the line is appended.

Justification | Why a log file as well as printed output

Training runs for days, so the console scrollback is lost or truncated long before the run ends. A log file persists, and because each epoch writes one line of keyed values, the whole history of a run can be read or plotted afterwards.

Justification | Why appending rather than rewriting

Appending means a resumed run continues the same log rather than replacing it, so the record spans the interruptions. Appending is also safe against a crash: each line is complete when written, so an interrupted run leaves a readable log ending at the last finished epoch, where a rewritten file could be truncated mid-write.

16 Timing and allowance accounting

Evidence | Current implementation boundary

`Software/DemoCode/timing.py`. Exact file hashes are in the accompanying source manifest. Names, declarations and source locations below come from the captured source, not a runtime import.

Purpose | Input-to-output reading flow

Measure elapsed stage intervals and aggregate a request record. Compare those measurements with nominal motion allowances only after checking the stage boundary, host and completion status.

Evidence | Selected current module settings

These are declarations or expressions, not recovered environment values. Source comments do not establish a measured optimum. Larger glyph coordinate tables are designed data in the bundled source.

```
ALLOWANCE = {'classify': lambda chars: 2.0 + 0.025 * chars, 'reproduce': lambda chars: 4.0 * chars, '
    draw': lambda chars: 4.0 * chars}
ALLOWANCE_TEXT = {'classify': '2 s + 0.025 s per character', 'reproduce': '4 s per character', 'draw
    ': '4 s per character'}
```

Stage: Timing and requirement accounting

16.1 Method background: Timing and requirement accounting

Purpose | The role in this system

Measure request wall time and retain earlier generation work when drawing is requested later.

Justification | The reason for this construction

perf_counter differences measure elapsed time; Carry adds external work while Add only labels work already inside the wall interval. This avoids double counting stages.

Alternatives | What changes if another method is used

CPU time excludes waiting; summing nested stages can double-count; theoretical path time ignores overhead. A benchmark on another host is not an embedded qualification.

Library | Implementation speed and accuracy

Python perf_counter is already the timing primitive. Profilers can attribute cost more finely but add overhead. Multiprocessing stage wall time is not the sum of worker CPU times. [76, 3].

16.2 Stage

```
class Stage
```

Evidence | Source and method link

`Software/DemoCode/timing.py` line 47. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 16.1.

Purpose | What this contributes

Measure request wall time and retain earlier generation work when drawing is requested later.

Method | Methods exposed by this current class

```
def __init__(self, run, name, chars=0)
def __enter__(self)
def __exit__(self, *exc)
```

Method | Current implementation

Measure request wall time and retain earlier generation work when drawing is requested later. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class Stage:
    """See source snapshot for the original docstring."""

    def __init__(self, run, name, chars=0):
        self.run = run
        self.name = name
        self.chars = chars
        self.t0 = None

    def __enter__(self):
        self.t0 = time.perf_counter()
        return self

    def __exit__(self, *exc):
        wall = time.perf_counter()-self.t0
        self.run.Record(self.name, wall, self.chars)
        return False
```

Justification | Interpret the method rather than infer a benchmark

perf_counter differences measure elapsed time; Carry adds external work while Add only labels work already inside the wall interval. This avoids double counting stages. [76, 3].

16.3 Run

```
class Run
```

Evidence | Source and method link

`Software/DemoCode/timing.py` line 66. Implementation evidence: [3]. Related method, alternatives and library discussion: Section 16.1.

Purpose | What this contributes

Measure request wall time and retain earlier generation work when drawing is requested later.

Method | Methods exposed by this current class

```
def __init__(self, kind, chars=0, lines=0)
def Step(self, name, chars=None)
def Record(self, name, wall, chars)
def Add(self, name, seconds, chars=0)
def Carry(self, name, seconds, chars=0)
def Total(self)
def AsDict(self)
def AsJson(self)
def Report(self)
```

Method | Current implementation

Measure request wall time and retain earlier generation work when drawing is requested later. The excerpt below is from the current definition; the complete function is in the source snapshot. Its constants and branches, rather than an older signature, govern this path.

```
class Run:
    """See source snapshot for the original docstring."""

    def __init__(self, kind, chars=0, lines=0):
        self.kind = kind
        self.chars = chars
        self.lines = lines
        self.stages = []
        self.external = 0.0
        self.t0 = time.perf_counter()

    def Step(self, name, chars=None):
        return Stage(self, name, self.chars if chars is None else chars)

    def Record(self, name, wall, chars):
        self.stages.append(dict(stage=name, seconds=round(wall, 4),
                                characters=chars,
                                secondsPerChar=round(wall/chars, 5) if chars else None))

    def Add(self, name, seconds, chars=0):
        self.Record(name, float(seconds), chars)

    def Carry(self, name, seconds, chars=0):
        self.external += float(seconds)
        self.Record(name, float(seconds), chars)

    def Total(self):
        return (time.perf_counter()-self.t0)+self.external

    def AsDict(self):
        total = self.Total()
        accounted = sum(s["seconds"] for s in self.stages)
        out = dict(
            kind=self.kind,
            characters=self.chars,
            lines=self.lines,
            totalSeconds=round(total, 3),
            measuredSeconds=round(accounted, 3),
            overheadSeconds=round(max(0.0, total-accounted), 3),
            secondsPerCharacter=round(total/self.chars, 4) if self.chars else None,
            secondsPerLine=round(total/self.lines, 3) if self.lines else None,
            charactersPerSecond=round(self.chars/total, 2) if self.chars and total else None,
            stages=self.stages,
        )
        formula = ALLOWANCE.get(self.kind)
        if formula and self.chars:
            limit = formula(self.chars)
            out["limitSeconds"] = round(limit, 2)
            out["limitFormula"] = ALLOWANCE_TEXT.get(self.kind, "")
```

```
            out["withinLimit"] = bool(total <= limit)
            out["fractionOfLimit"] = round(total/limit, 3)
            out["secondsSpare"] = round(limit-total, 2)
        return out

    def AsJson(self):
        return json.dumps(self.AsDict(), indent=1)

    def Report(self):
# Excerpt ends; full definition continues in bundled source.
```

Justification | Interpret the method rather than infer a benchmark

perf_counter differences measure elapsed time; Carry adds external work while Add only labels work already inside the wall interval. This avoids double counting stages. [76, 3].

17 Expanded measurement, verification and coefficient evidence

Report equation 35 Geometric agreement and natural baseline

Method | Formula

$F = 2PR/(P + R)$ and $Q_k = 100F_{\text{robot},k}/F_{\text{genuine},k}$.

Purpose | Why the system needs it

Compare thresholded centreline precision/recall and then contextualise robot agreement using genuine writing variation. F is not character accuracy or IoU; Q is project-defined and can exceed 100 percent. Mean writer ratios differ from a ratio of pooled means.

Evidence | Current location and source

Historical geometric evaluation, not a loaded classifier confidence. [56, 10].

Evidence | Appendix calculations plots and recorded checks

The two-crop page vote, finite-difference derivative, patch-storage calculation, 12 ms switch confirmation, stroke-width sweep, confusion matrices, seam/capture counts and training epoch logs are all retained in the verification chapter. For a confusion matrix, row normalisation gives prediction fractions conditional on actual writer; it is not class precision. Wilson limits describe binomial decision counts subject to dependence caveats. A recognition score used to select a candidate is not an independent physical test of that candidate. Every retained historical number stays attached to its population and source.

Purpose | The named procedures and their current homes

The pen-toggle procedure is `SetPen`; debounced switch reads are `TriggeredEndstop`; dynamic axis calibration uses `_CalibrateAxis` and `Calibrate`; coordinated integer movement is `BresenhamMove`. The report's line-ownership refinement maps to `AssignOwnership` and its correction helpers. Labelled handwriting to a writer profile maps to `ExtractLineGlyphs`, `ExtractAuthorRaw` and `BuildAuthorProfile`. Joint search/accepted repair is `SynthesizeJointBestOf`. The extended training algorithm maps to the train-mode forward/cache/backward functions, loss routines, optimiser step, validation and checkpoint logic. These names are linked through the function index; unrelated helpers do not outrank them.

Method | What each report plot can and cannot show

The integral/ratio example demonstrates arithmetic for one pixel, not all lighting conditions. A seam/cost overlay shows the route preferred by the chosen objective; a labelled crop establishes whether that route preserves the intended ink. A convolution feature plot shows activation responses, not fixed semantic labels for channels, and independent colour scales cannot be compared as absolute strengths. A CTC plot places symbols against feature frames, not physical pen time. Forced alignment is conditional on the supplied transcript. A Bresenham staircase represents integer commanded positions, not an observed vibration trace. Stroke-normalisation panels show mask, skeleton and restored width rather than guaranteed shape preservation.

Variant clouds, medoids and prototypes have different provenance: a medoid is a real selected sample, whereas a prototype is constructed. Candidate-score panels show optimisation against the judging networks. Confusion matrices require the actual per-class counts. Word overlays depend

on registration, normalisation and distance tolerance; they are not automatic proof of character correctness. Timing bars require host, stage boundaries and completed-job status. Training curves require the selected checkpoint and holdout definition. The verification chapter explains the exact reported calculations and their qualifications rather than rerunning them.

17.1 Verification, validation, selection and qualification are different

Verification asks whether an implementation performs the stated calculation. A finite-difference derivative, an integral-image hand sum and comparison with a separately implemented operation are examples. Validation asks whether the method represents the intended task: correct line ink, readable letters or actual carriage distances. Model selection chooses settings using observed performance; qualification assesses the specified requirement under the documented test conditions. A high validation score used to choose a width is evidence for selection, not an untouched qualification result.

The additions explain how calculations can be checked; none was rerun as an experiment. New arithmetic is labelled as constructed or derived. No coefficient sweep, inference, robot motion, training, lighting experiment, old-segmenter comparison, pressure repair or demonstration preparation belongs to this work.

17.2 How the recorded NumPy–Torch comparisons should be understood

The final report Table 7 records reader/writer maximum-logit differences normalised by maximum magnitude of 2.12×10^{-14} and 3.04×10^{-15}; absolute loss differences of 1.14×10^{-13} and 1.78×10^{-15}; worst-ranked gradient relative L2 differences of 2.80×10^{-13} and 6.01×10^{-14} [10]. Its prose saying all values are below 10^{-13} is inconsistent with two displayed reader entries. The table values are retained rather than replaced with that blanket description.

For corresponding tensors a, b, useful measures are

Formula purpose and system use. Purpose in this explanation: How the recorded NumPy–Torch comparisons should be understood **Source basis:** [10, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$
\begin{aligned}
e_{\max} &= \max_i |a_i - b_i|, \\
e_{\mathrm{rel,max}} &= e_{\max} / \max(\epsilon, \max_i |b_i|), \\
e_{\mathrm{rel},2} &= \|a - b\|_2 / \max(\epsilon, \|b\|_2).
\end{aligned}
$$

Other symmetric denominators are possible and give different numbers. The exact archived checker defines which was used; a descriptive table heading alone does not recover every denominator/floor. Relative metrics near zero can be misleading, so retain an absolute metric too. “Worst ranked” must identify the compared parameter tensors, not imply all possible states and inputs were exhausted.

A meaningful comparison holds fixed input arrays, parameters, precision, layout, padding, activation conventions, training/evaluation mode, batchnorm statistics, LSTM gate order, blank index, sequence lengths and loss reduction. Dropout is disabled or its mask is shared. Compare intermediate operations before relying on final logits. Float64 agreement near these scales supports consistency for the checked cases; it does not validate the model’s learned accuracy or guarantee a whole training trajectory matches. Floating-point operations are order-dependent [52]. Independently trained checkpoints can differ for many reasons beyond accumulated rounding error.

Table 6’s performance comparison is a different experiment: reader pooled 94.06% versus 94.39%, writer 98.08% versus 97.51% on common material. These are trained-route evaluation scores, not

numerical roundoff measures. Matching an operation to Torch and achieving a character score answer different questions. The writer QTP2 score 153/156 should not be silently assigned to the alternative first-principles checkpoint.

17.3 Finite differences and all Appendix A.1 arithmetic

For scalar parameter θ_i, central difference is

Formula purpose and system use. Purpose in this explanation: Finite differences and all Appendix A.1 arithmetic **Source basis:** [10, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$g_i^{\mathrm{FD}} = \frac{L(\theta + he_i) - L(\theta - he_i)}{2h}.$$

Taylor expansion cancels the first-order error terms and gives an $O(h^2)$ truncation error when L is sufficiently smooth. Making h too small causes subtraction cancellation and roundoff; making it too large measures curvature rather than the local derivative. ReLU zeros and max-pool ties are nondifferentiable, so a central difference crossing them need not equal the implementation's selected subgradient. These observations follow from calculus and floating-point arithmetic, not new network tests.

The report records losses 0.4703364611173033 and 0.4703403997759956, $h = 10^{-4}$. In the recorded plus/minus order,

Formula purpose and system use. Purpose in this explanation: Finite differences and all Appendix A.1 arithmetic **Source basis:** [10, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{0.4703364611173033 - 0.4703403997759956}{0.0002} \simeq -0.0196932934615.$$

The analytic derivative is -0.0196932934309, absolute discrepancy about 3.06×10^{-11}. This is one recorded coordinate check, not a worst-case bound for all derivatives. Recovering it operationally would require the exact frozen input, target, parameter coordinate, dtype, weights, loss reduction and perturbation order; the two scalar losses alone do not identify those objects. No checker was executed in this edition.

The patch-storage example is $368640 \times 4/2^{20} = 1.40625$ MiB. The factor four assumes four bytes per element (float32); float64 would double the value. MiB uses 2^{20} bytes, while MB uses 10^6. An im2col view can become a materialised array during reshape/multiplication; count actual storage, not only logical view shape. This arithmetic does not establish peak process memory. The switch-confirmation example is $(3 - 1) \times 6 = 12$ ms; three reads contain two intervals.

17.4 The mysterious "Appendix calculation 1": the page vote

The final report labels Appendix A.0.1 "System integration calculation". It combines line writer probabilities using ink-count weights:

Formula purpose and system use. Purpose in this explanation: The mysterious "Appendix calculation 1": the page vote **Source basis:** [10, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\bar{p}_k = \frac{\sum_i w_i p_{i,k}}{\sum_i w_i}, \quad w_i \geq 0.$$

For two archived Author O01 crops,

Formula purpose and system use. Purpose in this explanation: The mysterious "Appendix calculation 1": the page vote **Source basis:** [10, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\bar{p}_O = \frac{20204(0.9996674) + 21750(0.9999279)}{41954} \simeq 0.9998024.$$

Weights sum to 41954; normalised weights sum to one, so the pooled vector remains a probability vector when the inputs are. Weighting by ink count assumes more ink supplies more useful evidence; a dark artefact or a long misclassified line can instead dominate. This is not a Bayesian product of independent likelihoods, and near-one softmax values do not prove near-certain real-world identity. The example uses two crops and cannot substitute for the full-page result [10].

17.5 Levenshtein edits, character accuracy and exact text measures

For reference prefix length i and predicted prefix length j,

Formula purpose and system use. Purpose in this explanation: Levenshtein edits, character accuracy and exact text measures **Source basis:** [10, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$D_{i,j} = \min\{D_{i-1,j} + 1, D_{i,j-1} + 1, D_{i-1,j-1} + \mathbf{1}[t_i \neq \hat{t}_j]\},$$

with $D_{i,0} = i, D_{0,j} = j$. The three branches represent deletion, insertion and substitution/match. This dynamic-programming calculation gives minimum edit count [55]. For "cat" versus "cut", one substitution gives CER 1/3 and character score 2/3. A missing space counts if spaces are included. Case folding and punctuation removal must be specified before comparison; changing them changes the task.

Pooled CER is $\sum_i D_i / \sum_i |t_i|$, not the unweighted mean of line CERs. For lengths 2 and 98 with errors 1 and 0, pooled accuracy is 99%, whereas mean line accuracy is 75%. The final report uses the pooled formula. Its raw accuracy can become negative if insertions exceed the reference length; the candidate `CharAcc` clamps at zero. These are different conventions at that extreme.

Exact-line match counts a line only if the entire chosen text agrees. Word error rate, fraction of exact words and longest-common-subsequence word recall are also different. The inspected server `WordAcc` computes case-insensitive word-level LCS recall, despite its short descriptive comment. It should not automatically be interpreted as standard WER complement or strict positional exact-word accuracy. Reported numerical evidence must be read with the actual scorer and text normalisation.

17.6 Wilson intervals: derivation, numerical meaning and dependence

For x correct binary decisions out of n and $\hat{p} = x/n$, invert the score inequality $(\hat{p} - p)^2 \leq z^2 p(1-p)/n$. Solving the resulting quadratic gives

Formula purpose and system use. Purpose in this explanation: Wilson intervals: derivation, numerical meaning and dependence **Source basis:** [10, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$\frac{\hat{p} + z^2/(2n) \;\pm\; z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}.$$

Use $z \simeq 1.96$ for a nominal two-sided 95% interval [90, 57]. For 153/156, $\hat{p} \simeq 0.98077$ and the interval is about 94.5–99.3%; for 159/184, about 80.7–90.6%. These are reported line-decision examples. Even 25/25 gives a lower bound about 86.7%, not certainty of universal success.

Wilson avoids the zero-width interval that a naive normal approximation can give at all successes. Its binomial interpretation still assumes suitable independent trials with a shared success probability. Lines sharing page, writer, lighting and model are dependent. The report therefore treats its intervals descriptively. It is incorrect to claim a 95% posterior probability that a fixed unknown p lies in a frequentist interval without an additional Bayesian model. Character edit scores are not simple independent Bernoulli trials and cannot be inserted as fractional "successes" without justification.

17.7 Direct geometry: registration before comparison

The direct word-shape check is independent of a reader's text labels at scoring time, but depends on correct crop/transcript matching and image processing. Binary masks are thinned to centreline point sets. Normalisation and registration define which differences are ignored. Translation removes position; isotropic scaling removes uniform size; independent X/Y scaling also removes aspect ratio; shear correction removes slant. A deformable warp can erase precisely the writer features being assessed. Therefore allowable transforms are part of the metric, not cosmetic preparation.

The archived natural-baseline protocol documents two alternatives: line x-height scaled to 48 pixels, or tightly cropped whole-word ink height scaled isotropically to 48 pixels; centring and at most five pixels translation per axis; point tolerance 4.8 pixels. Whole-word height includes ascenders and descenders, unlike body x-height. No width-only stretch, slant correction or deformable warp belongs to that protocol. A 4.8-pixel tolerance is one tenth of the selected 48-pixel reference height; changing that reference changes the physical meaning of tolerance. The local evidence source is `Report/NaturalVariationBaseline_2026-11-02/PROTOCOL.md` and `FINDINGS.md`. Its historical aggregate is not substituted for the final report's later aggregation.

Optimising translation for maximum overlap already introduces an alignment selection step. The same policy must be applied to genuine and robot pairs. Reference crops must be exact-token matches and self-matches excluded. Nearest-reference selection gives a higher or equal score than averaging every reference and becomes easier with more references. Matching reference banks controls that opportunity. A plot of beautifully aligned skeletons must retain the raw word photographs and transformation policy for context.

17.8 Point-set precision, recall and harmonic F-score

Let A be robot skeleton points and B genuine-reference points. With distance tolerance τ, define

Formula purpose and system use. Purpose in this explanation: Point-set precision, recall and harmonic F-score **Source basis:** [10, 3]. Exact project constants remain implementation choices unless evidence is stated.

$$P = \frac{|\{a \in A : \min_{b \in B} \|a - b\| \le \tau\}|}{|A|}, \qquad R = \frac{|\{b \in B : \min_{a \in A} \|b - a\| \le \tau\}|}{|B|}, \quad F = \frac{2PR}{P + R}.$$

The thresholded point-set definition is used by reconstruction benchmarks; its application to 2D handwriting and the chosen normalisation are project choices [56]. Each point asks whether any nearby point exists; this is not one-to-one matching. A distance transform gives distance to the nearest foreground grid point; a spatial tree can independently compute nearest-point distances for the same sets. Agreement checks arithmetic, not semantic validity of the chosen crops.

Constructed example: 90 of 100 robot points have a reference neighbour, while 80 of 100 reference points have a robot neighbour. Then $P = 0.9, R = 0.8, F = 1.44/1.7 \simeq 0.8471$. Missing strokes lower recall; extra strokes lower precision. F is not intersection-over-union, character correctness or Pearson correlation. Tolerance can make shifted strokes match despite zero exact pixel overlap. If both sets are empty, or one is empty, a scoring convention is required; silent division by zero is not a definition. Scores can depend on point density unless sampling is controlled.

17.9 Natural-variation normalisation and why 100% is not a ceiling

For writer k, let $F_{r,k}$ and $F_{g,k}$ be robot and genuine agreement against corresponding reference banks, using the same protocol. The final report defines $Q_k = 100F_{r,k}/F_{g,k}$ and averages eleven writer ratios. For writer 150 the displayed 87.73/90.01 gives about 97.47%. For Owen 70.22/76.03 gives about 92.36%, subject to rounded table inputs. The final result is about 86.7% relative agreement, not 86.7% correctly written characters [10].

Equal-writer mean $\frac{1}{K}\sum_k F_{r,k}/F_{g,k}$ differs from ratio of means $\sum_k F_{r,k}/\sum_k F_{g,k}$. For two writers with $(F_g, F_r) = (0.9, 0.9), (0.5, 0.25)$, mean ratio is 0.75 but ratio of means is $1.15/1.4 \simeq 0.8214$. This explains why an older evidence note can quote a different aggregate without either arithmetic necessarily being wrong. State the exact rule rather than mixing versions.

If $F_{r,k} > F_{g,k}$, Q_k exceeds 100%; it means closer agreement to the available bank than the measured genuine baseline, not superhuman style. A small denominator destabilises the ratio. A baseline below one does not make an absolute 85% target impossible; the ratio is a newly defined comparison, not a logical replacement for the proposal's original requirement. Repeated words are a selected subset, often shorter/common words. Four or five words for a writer do not establish full-page fidelity. A nearest-reference figure can have a higher F than the corresponding ordinary-reference writer mean. Both policies should be named.

17.10 Direct length checks and completion-time statistics

For commanded length L_0 and measured L_i, signed error is $e_i = L_i - L_0$; absolute error $|e_i|$; mean signed error indicates bias; mean absolute error describes typical magnitude; maximum absolute error addresses a worst observed limit. The report's 70 readings give mean absolute error 0.297 mm and maximum 0.49 mm. Neither is an uncertainty bound for all future drawings. A ruler/caliper's resolution, endpoint definition, paper motion and repeated measurements affect interpretation. A sloped line length may combine errors in both axes and squareness. Scaling calibration and independent acceptance measurements should not be confused.

For completed job j, normalised completion time is $r_j = t_j/n_j$, where the counting convention for n_j must include/exclude spaces consistently. The mean of job ratios $\frac{1}{J}\sum_j r_j$ gives every job equal weight; $\sum_j t_j/\sum_j n_j$ weights by characters. For 10 characters in 30 s and 100 in 100 s, these give 2 s/character and about 1.182 s/character. The report's 25 confirmed completed jobs and recorded stages must be kept distinct from unconfirmed/pending rows and ideal trajectory budgets. Full request time includes generation and motion when the record says so; classifier-only latency does not. Host names and laptop/ODROID evidence must remain attached to the timings.

The conditional 2.76 network-speed factor from a separate benchmark is not a measured system acceleration. If image time is t_I and selected network time t_N, the conditional estimate is $t_I + t_N/2.76$ plus all unchanged stages. Dividing the entire request by 2.76 would overstate the gain. This is Amdahl-style accounting of unchanged work, not a new passing qualification.

17.11 Capture records, plots and the old-segmenter question

The report's Table 18 lists expected/reduced/gradient/+3-degree/90-degree line counts: Owen 20/19/20/11/19, Dylan 24/24/24/25/24, and Yeukita 26/26/26/26/26. These are digital perturbation observations, not an illumination robustness guarantee. Equal counts do not establish correct labels, ownership or readable crops. Reduced resolution can merge thin strokes; a synthetic gradient is not every physical shadow, glare or phone-light configuration.

Seam-sensitivity plots vary a cost coefficient and show line/ownership behaviour. A visually cleaner seam is evidence for that example, not a universal coefficient optimum. CNN channel plots show activations under one checkpoint/input; independent colour scales prevent comparing absolute strength across panels. CTC plots show frame symbols and constrained paths; Bresenham plots show integer command locations. Joint-score plots show model-based selection, not independent physical style accuracy.

The requested old-versus-new segmentation accuracy comparison is a meaningful paired-ablation question, but running it is explicitly outside this task. To interpret a future result, the same frozen pages, labels, classifier, preparation and scoring would be needed. Failed or unmatched crops must remain accounted for; evaluating only surviving easy crops can inflate the score. A report saying segmentation "affects accuracy" is plausible causal reasoning, but a quantified benefit requires that controlled evidence.

17.12 Coefficient evidence register and questions left open

Derived from stated geometry/electronics.
Forty mm/rev, nominal 80 steps/mm, 12.5 micrometre spacing, nearest-rounding bounds, RPM conversion, 12 ms confirmation span, conditional reference voltage and resistor currents. These validate arithmetic only under their stated inputs.

Published method, project constants.
Otsu and integral sums; deskew objective; seam dynamic programming; Zhang–Suen; CTC and LSTM; dropout; Douglas–Peucker; optimiser families; edit distance and score interval. Their publications do not establish the project's exact parameter choices.

Reported selection evidence.
Stroke-width 0.17 versus the other three displayed widths. Keep the unresolved page/line wording and selected validation status visible. Do not promote it to an independent result.

Documented local rationale, not newly verified.
0.45 connection suppression between commented print/joined examples; 15%/threefold solid-crop guard with commented crop-density observations; visual settling dwell. These are stronger than no explanation, weaker than independently audited coefficient optima.

Heuristics still lacking an established optimum.
Rotation margin 1.35; 1.25/1.4-pitch run guards; 6% peaks; $0.6P$ suppression; seam weights 4, 0.012, 0.02 and closing $0.07P$; 15% ownership; darkness 0.55; 64 by 640 canvas and network widths/depths; 15% split and 20% sampling; dropout 0.2; body threshold 45%; cut weight 1.15, clip 3 and minimum $0.08h_x$; 75-degree continuation; 0.6-pixel simplification; 2.1/2.9-width caps; 12-grid descriptor; drift $0.85/\pm0.25$; clearance 0.05; maximum join 0.80 x-heights; physical join 0.6 mm; anchoring endpoints 0.24/0.32; search/repair budgets; pen overrun, debounce and timeout settings. Their operational roles are explained in the relevant chapters. No numerical optimum is fabricated in place of the other agent's sweeps.

Unresolved physical/source facts.
Actual curve underlying torque interpolation and assumed friction; exact installed driver sense

resistors and reference voltage; GPIO current-limit source; full rail/ground map; base pull-down placement and transistor saturation under real load. Calculation can expose these gaps but cannot replace missing measurements.

17.13 Source audit and how to trace assertions

The named final report, local function bodies, cited original papers and manufacturer/framework documentation serve different roles. The report is the source for historical numbers; code is the source for present formulas and defaults; papers establish method origins; component datasheets establish conditional electrical relations. A generic citation cannot authenticate an installed board or as-run checkpoint.

New bibliography entries provide links for the A4988 and carrier current relation, low-side transistor reasoning, GPIO API, PyTorch conventions, Wilson/NIST, CTC exposition, RMSprop and source snapshots. Original entries for Otsu, summed-area tables, thinning, seam methods, LSTM, CRNN, optimisers, initialisation and edit distance are retained. Sources are cited near the definitions they support. The substantial worked arithmetic and sensitivity reasoning in these additions are explicit derivations, not quotations.

Automated full-text access was blocked for some publisher DOI pages and the ODROID/onsemi pages. Their bibliographic links are retained with access limitations, rather than claiming they were read successfully. Some older references have no persistent URL in the original bibliography. This manual does not certify all forty-four report references or every historical claim. Current implementation explanations have been revised against the captured DemoCode tree. In particular the torque paragraph and GPIO current claim still contain missing-source issues. A later complete reference audit would need each cited edition/page and the specific claim it supports. This is deliberately distinct from asserting the report has been repaired.

For a future numerical claim, record input identity, source revision, checkpoint identity, host, preparation, dtype, units, partition, count, aggregation and raw evidence location. A bare rounded number cannot supply those facts. The present expansion adds understanding and traceability while leaving measurements, source code and the final report unchanged.

17.14 Additional vocabulary needed to defend the calculations

Dimensionless coefficient.
A pure ratio or multiplier without units. Its numerical effect still depends on the scale of the quantity it multiplies. Changing cost normalisation can change a coefficient's meaning.

Heuristic versus hyperparameter.
A heuristic is a practical rule without a general optimality guarantee. A hyperparameter is a setting chosen outside ordinary weight learning. A setting can be both. A learned convolution weight is a model parameter; dropout probability is a hyperparameter.

Guard, floor, cap and clamp.
A guard handles an invalid or exceptional case; a floor enforces a minimum; a cap enforces a maximum; clipping enforces both. A numerical guard prevents arithmetic failure, not necessarily bad input.

Quantile and percentile.
A quantile q_f marks fraction f of a distribution; a percentile expresses $100f$. The 60th percentile is not 60% of the maximum. Interpolation conventions can differ on small samples.

Argmax and argmin.
They return the location/index attaining an extreme, not the extreme value. Ties require a rule, often first occurrence.

Indicator.
$\mathbf{1}[E]$ is one if statement E is true and zero otherwise. It encodes a decision without implying a probability.

Recurrence, state and backtracking.
A recurrence computes new entries from previous ones. A state summarises what a prefix needs for future choices. Backtracking follows saved choices to recover a path. A recurrence can sum paths, as CTC does, or optimise one path, as a seam/alignment does.

Dynamic programming versus greedy choice.
Dynamic programming compares prefix solutions for each state; a greedy rule takes an immediate best choice. Both can be fast, but they solve different optimisation problems.

Prior and posterior.
A prior expresses information before a particular observation; a posterior conditions on it. The project's glyph "prior" is a pooled geometric reference, not a fully specified Bayesian probability distribution. CTC state occupancy is a conditional path posterior under the model and target.

Likelihood versus confidence.
Likelihood evaluates observations/labels under a model. A softmax output is normalised model confidence; its match to real frequencies requires calibration evidence. High confidence can be wrong.

Calibration, two meanings.
Mechanical calibration relates pulse counts to reference positions/lengths. Probability calibration relates predicted confidence to observed frequency. The procedures and units are unrelated.

Bias and variance, two contexts.
Measurement bias is systematic offset; sampling variance is variation across repeated estimates. A neural bias is an additive learned parameter. An unbiased variance estimator is a statistical formula, not a guarantee a classifier is fair or accurate.

Regularisation and weight decay.
Regularisation constrains learning to reduce overfitting. Weight decay shrinks parameters. Dropout, augmentation, early selection and penalties influence generalisation by different mechanisms; their strengths cannot simply be added as one scalar percentage.

Epoch, update and iteration.
An update changes parameters after a batch; an iteration may mean one such processing step; an epoch follows the sampler's definition of a pass. Replacement/mixture sampling can revisit images within one epoch. A checkpoint's epoch index is not elapsed time.

Logit and log probability.
A logit is an unrestricted score. Log-softmax subtracts log-sum-exp to produce log probabilities. Log probabilities are nonpositive; logits need not be. Exponentiating an arbitrary logit alone is not a normalised probability.

Jacobian, gradient and chain rule.
A Jacobian contains derivatives of vector outputs with respect to vector inputs. A scalar loss gradient is the corresponding vector derivative. Backpropagation efficiently composes these derivatives without storing every full Jacobian.

Subgradient.
A selected derivative-like value at a nonsmooth point. ReLU at zero and max ties require conventions; disagreement there is not necessarily evidence of an incorrect smooth derivative elsewhere.

Float32, float64 and tolerance.
Floating-point types have finite precision, not exact real arithmetic. Comparison tolerance defines an allowed discrepancy for a particular scale. A tolerance is neither a p-value nor a physical instrument uncertainty.

Isotropic versus anisotropic.
Isotropic scaling uses one factor in both axes; anisotropic scaling uses different factors and changes aspect ratio. Normalisation describes a transformation, not proof that remaining differences are exclusively writer style.

Aliasing and interpolation.
Aliasing occurs when sampled detail becomes indistinguishable from another pattern. Interpolation estimates intermediate values but cannot reliably recreate information lost by undersampling. Antialiasing before reduction limits high-frequency content to reduce this risk.

Topology versus geometry.
Topology concerns connections, components and holes; geometry concerns positions, lengths and angles. Small geometric error does not necessarily preserve a tiny loop, and a connected skeleton can still have poor geometric fidelity.

Open-loop versus feedback.
Open-loop axis control estimates position from commands. Feedback observes a quantity, such as an endstop or pen-up switch. Having some switches does not make every interior carriage position closed-loop or verify pen pressure.

Resolution, repeatability, accuracy and uncertainty.
Resolution is the smallest represented/readable increment. Repeatability describes consistency under repeated conditions. Accuracy concerns agreement with a reference. Uncertainty characterises the range attributable to measurement knowledge. A fine step lattice alone establishes none of the other three.

Macro and micro aggregation.
Macro gives each group equal weight; micro pools elementary counts. Equal-writer ratios, word means and pooled character counts therefore answer different questions even on the same files.

Ablation and sensitivity.
Ablation removes a mechanism to assess its contribution under controlled conditions. Sensitivity varies a setting and observes response. A sensitivity plot can reveal fragility without proving a global optimum. No such new execution is part of this edition.

Confusion matrix.
Rows/columns count actual/predicted class pairs. Row normalisation divides by actual-class counts, showing conditional prediction fractions. It does not equal class precision, which uses predicted-class totals. An overall average without counts can conceal class imbalance.

Selection leakage and provenance.
Selection leakage occurs when a test set influences tuning and is later presented as untouched. Provenance records where an input, model or number came from. It prevents an old result from being silently attributed to a newer code revision or different host.

17.15 Persistent source links for the retained method references

These links supplement the original bibliography. Publisher access restrictions are not treated as successful full-text review. Method citations establish origins; project arithmetic examples and parameter trade-offs remain explicit derivations and engineering reasoning.

Otsu; integral images.
https://doi.org/10.1109/TSMC.1979.4310076
https://doi.org/10.1145/800031.808600.

Thinning; polyline simplification.
https://doi.org/10.1145/357994.358023
https://doi.org/10.3138/FM57-6770-U75U-7727.

CTC; CRNN; batch normalisation.
https://www.cs.toronto.edu/~graves/icml_2006.pdf
https://arxiv.org/abs/1507.05717
https://arxiv.org/abs/1502.03167.

LSTM original and derivative context.
https://doi.org/10.1162/neco.1997.9.8.1735
https://www.cs.toronto.edu/~graves/preprint.pdf.

Dropout; Adam; AdamW.
https://www.jmlr.org/papers/v15/srivastava14a.html
https://arxiv.org/abs/1412.6980
https://arxiv.org/abs/1711.05101.

Initialisation and clipping.
https://proceedings.mlr.press/v9/glorot10a.html
https://arxiv.org/abs/1502.01852
https://proceedings.mlr.press/v28/pascanu13.html.

Bresenham; Wilson.
https://doi.org/10.1147/sj.41.0025
https://www.jstor.org/stable/2276774.

Seam methods and document threshold alternative.
https://doi.org/10.1109/ICFHR.2014.127
https://doi.org/10.1145/1276377.1276390
https://doi.org/10.1016/S0031-3203(99)00055-2.

18 Glossary

Terms used in this manual, in plain language. They are grouped by subject rather than alphabetically, because reading a group in order explains more than reading a definition on its own.

18.1 Arrays and arithmetic

Array
A grid of numbers. A one-dimensional array is a list, a two-dimensional array is a table, and a greyscale image is a two-dimensional array where each number is the brightness of one pixel, usually from 0 for black to 255 for white.

Element-wise
An operation applied to every number in an array independently. Adding 10 to an image, element-wise, makes every pixel ten levels brighter.

Vectorised
Written so that an operation is applied to a whole array at once rather than by looping over the elements one at a time in Python. This matters for speed: the loop runs inside compiled code instead of in the interpreter, which is typically between ten and a hundred times faster.

Cumulative sum, or prefix sum
An array where each entry is the total of everything up to that point. For the input $(2, 3, 1, 4)$ the cumulative sum is $(2, 5, 6, 10)$. Its usefulness is that the total of any stretch of the original can then be found by one subtraction, because the total from i to j equals the entry at j minus the entry at i.

Mask
An array of true and false values, used to mark which pixels belong to something. An "ink mask" is true at every pixel judged to be ink and false elsewhere.

Dot product
For two lists of equal length, the sum of the products of corresponding entries. If both lists are first scaled so that their own length is one, the dot product equals the cosine of the angle between them considered as arrows, which is why it is used in this project to measure how similar two shapes are.

Norm
The length of a list of numbers considered as an arrow from the origin, which is the square root of the sum of the squares of its entries.

Interpolation
Estimating a value between known values. If a transformed image needs the brightness at a position halfway between two pixels, interpolation supplies it as a blend of the neighbours.

Median
The middle value when a set of numbers is sorted. It is used throughout this project in preference to the mean (the ordinary average) because it is insensitive to a small number of extreme values: one badly wrong measurement among eleven good ones moves the mean but not the median.

Percentile
The value below which a given share of the data falls. The 40th percentile is the value that 40 per cent of the data lies below. The median is the 50th percentile.

Variance and standard deviation
Measures of how spread out a set of numbers is. The variance is the average of the squared distances from the mean; the standard deviation is its square root, which has the same units as the data.

18.2 Images and shapes

Threshold
A brightness level used to divide an image into two groups, typically ink and paper. "Thresholding" is applying it.

Histogram
A count of how many pixels fall at each brightness level. It describes an image's distribution of tones without reference to where anything is.

Morphology
A family of operations that change the shapes in a mask rather than the values in an image: growing shapes, shrinking them, filling small holes, removing small specks.

Structuring element, or kernel
The small window a morphological or filtering operation looks through. A square kernel of radius 2 is a five-by-five window centred on each pixel.

Dilation and erosion
Growing and shrinking shapes. Dilation turns a pixel on if anything nearby is on; erosion turns it off unless everything nearby is on.

Closing
A dilation followed by an erosion by the same amount. The net effect is that small gaps are bridged while the overall thickness of a shape is preserved.

Connected component
A group of pixels that touch one another, treated as a single object. A letter, a dot over an `i` and a speck of dust are each a connected component.

Connectivity
Which neighbours count as touching. Four-connectivity counts only the pixels above, below, left and right; eight-connectivity also counts the four diagonals, so two pixels meeting at a corner belong to the same component.

Skeleton, or centreline
A one-pixel-wide line running down the middle of a thick stroke. Thinning a pen stroke to its skeleton recovers approximately the path the pen travelled.

Projection profile
The result of adding up the ink in each row, or in each column, producing a one-dimensional summary of a two-dimensional image. Lines of text appear as peaks in the row profile.

Polyline
A path described by an ordered list of points, understood to be joined by straight segments.

Aspect ratio
The ratio of width to height.

Bounding box
The smallest upright rectangle containing a shape.

18.3 Networks and training

Neural network
A function with a very large number of adjustable numbers, called weights, arranged in layers. Each layer takes the output of the previous one and performs a simple operation; the composition can represent complicated relationships. Training is the process of adjusting the weights so the function gives the right answers on examples.

Weight and bias
The adjustable numbers. A weight multiplies an input; a bias is added afterwards. Together they are the parameters.

Layer
One stage of a network.

Feature map
The output of a convolutional layer: an array showing, for each position in the image, how strongly some learned pattern was detected there.

Channel
One such map. A layer producing 64 channels is detecting 64 different patterns.

Convolution
Sliding a small grid of weights over an image and recording the response at every position, which detects the same pattern wherever it occurs.

Activation function
A simple non-linear function applied after a layer, such as setting negatives to zero. Without one, stacking layers would be pointless, because a chain of purely linear operations is equivalent to a single linear operation.

Pooling
Reducing the size of a feature map by summarising each small window, typically by its largest value, which saves work and makes the result tolerant to small shifts.

Logit
A raw score produced by the final layer, before it is converted into a probability. Logits may be any real number, positive or negative.

Softmax
The conversion from logits to probabilities: exponentiate each score and divide by the total, which forces them to be positive and to sum to one.

Loss function
A single number measuring how wrong the network's output is on a training example. Training consists of reducing it.

Gradient
The collection of derivatives of the loss with respect to every weight: for each weight, the rate at which the loss would change if that weight changed. It is what tells training which direction to move each weight.

Chain rule
The rule of calculus for differentiating a composition of functions, which is what makes it possible to compute the gradient of a loss with respect to a weight many layers away from it.

Backpropagation
Applying the chain rule layer by layer from the loss back to the first layer, so that every weight's gradient is obtained in one backward pass.

Forward pass, backward pass
Computing the output from the input, and computing the gradients from the loss, respectively.

Batch
A group of examples processed together, so that one weight update is based on their average gradient rather than on a single example, which is both faster and less noisy.

Epoch
One complete pass through the training data.

Learning rate
How far each weight is moved per update.

Optimiser
The rule that turns gradients into weight updates.

Overfitting
Learning the training examples rather than the general pattern, so that performance on new data is poor. It is detected by measuring on data the network was not trained on.

Holdout, or validation set
Data deliberately excluded from training and kept for measurement, which is the only way an accuracy figure means anything.

Checkpoint
A saved copy of all the weights, so a trained network can be reloaded later.

Recurrent layer
A layer that processes a sequence one position at a time while carrying information forward, so that a decision at one position can depend on earlier ones.

Inference
Running a trained network forward to get an answer, as opposed to training it.

18.4 This project's own terms

Glyph
One stored example of one character in one writer's hand: a set of pen strokes, in x-height units, with measurements describing how much space it occupies.

Variant
One of several stored glyphs for the same character by the same writer, since nobody writes a letter identically twice.

Prior distance
A number measuring how far a stored glyph's shape sits from the typical shape of that character pooled across all writers. Low means a well-formed letter; high means the shape was probably cut out wrongly.

Anchor, or fallback letterform
A clean, readable shape used in place of a stored variant when the stored variants are too damaged to be legible. Preferably the writer's own most typical version of the letter.

Legibility lambda
A number between 0 and 1 controlling how often the anchor is used in place of the writer's own shapes, and therefore where the output sits between being readable and being recognisably that person's hand.

Connectedness
The measured fraction of neighbouring letter pairs within a word that the writer joins without lifting the pen.

Ligature
The joining stroke between two letters written without lifting the pen.

Trajectory
The pipeline's representation of a drawing: a list of pen strokes in millimetres.

G-code
The text format understood by the machine, consisting of lines such as `G1 X10 Y20 F900`, meaning "move in a straight line to this position at this feed rate".

Endstop
A switch at the end of an axis's travel, used to find a known position and to prevent the machine driving into its own frame.

19 Function index and source provenance

Evidence | Coverage and scope

Every top-level function/class in the captured fourteen Python modules is listed below; methods are listed inside their class entries. Nested local helpers belong to their enclosing function. The source ZIP includes the exact captured Python and startup files for cross-checking; the manual does not execute them. The report equation cards cover numbered equations 1 through 35. A library comparison is grouped by method family so small helpers need not repeat it.

Module	Function or class	Source line
paths.py	PersonalWriterPages	22
web/camera.py	_GetLock	39
web/camera.py	PreviewStart	52
web/camera.py	PreviewStop	68
web/camera.py	PreviewActive	76
web/camera.py	PreviewFrameJpeg	83
web/camera.py	CaptureImage	105
segment.py	Integral	12
segment.py	BoxSum	18
segment.py	BoxCount	27
segment.py	BoxMean	34
segment.py	Dilate	41
segment.py	Erode	45
segment.py	Close	50
segment.py	AnyWithinRadius	54
segment.py	LongRunMask	61
segment.py	RowRunLength	74
segment.py	Otsu	89
segment.py	BinarizeLine	108
segment.py	InkMask	427
segment.py	PaperRegionMask	438
segment.py	LabelComponents	142
segment.py	PieceBounds	180
segment.py	Blur	340
segment.py	Elastic	364
segment.py	DrawPolyline	377
segment.py	_Weights	224
segment.py	Resize	243
segment.py	Halve	266
segment.py	FitFixedCanvas	274
segment.py	_Sample	288
segment.py	_Like	303
segment.py	Rotate	309
segment.py	Warp	322
segment.py	CoreBand	130
segment.py	_LetterHeight	464
segment.py	LinePitch	484
segment.py	LineCentres	632
segment.py	EstimateSlantDeg	405
segment.py	SkewScore	544
segment.py	Deskew	560
segment.py	_SheetCrop	944
segment.py	LineAxisScore	957
segment.py	UprightPage	974
segment.py	LeastCostSeam	515

Module	Function or class	Source line
segment.py	SeamCuts	656
segment.py	RemovePrintedStructure	577
segment.py	DropArtifactPieces	592
segment.py	_BandOwnership	680
segment.py	_DotsJoinedAbove	755
segment.py	_RingsToStem	784
segment.py	_FragmentsToNeighbour	837
segment.py	AssignOwnership	873
segment.py	_BuildLineCrop	886
segment.py	DropSolidCrops	986
segment.py	SegmentLines	1004
segment.py	Skeletonize	194
segment.py	PolylineLength	420
recognise.py	LoadStateDictAny	31
recognise.py	ResizeLineImageFixed	157
recognise.py	TensorFromResized	161
recognise.py	FrameXToPixel	251
recognise.py	StrokeNormalize	171
recognise.py	Conv2d	41
recognise.py	ConvBlockForward	113
recognise.py	PaperCRNNNumpy	259
recognise.py	AuthorClassifierCNNNumpy	325
recognise.py	ClassifyImage	348
recognise.py	BatchNorm2d	62
recognise.py	Relu	68
recognise.py	Sigmoid	72
recognise.py	Linear	95
recognise.py	MaxPool2d	76
recognise.py	AdaptiveMaxPoolHeight	87
recognise.py	AdaptiveAvgPoolWidth	91
recognise.py	LogSoftmax	100
recognise.py	Softmax	107
recognise.py	LstmForward	123
recognise.py	BidirectionalLstmForward	143
recognise.py	MultilayerBidirectionalLstm	150
recognise.py	DecodeCtc	187
recognise.py	CtcForcedAlign	204
recognise.py	ReadTextScored	294
recognise.py	ReadText	301
recognise.py	Levenshtein	305
recognise.py	CharAcc	319
style.py	TracePolylines	56
style.py	_Dir	115
style.py	_PruneSpurs	126
style.py	_JoinAtJunctions	146
style.py	SimplifyPolyline	180
style.py	_TrimLigatureTails	205
style.py	ExtractLineGlyphs	254
style.py	_ClassOk	505
style.py	_PenLength	515
style.py	_LongestStroke	524
style.py	RenderRefStats	724
style.py	_MedianWordGap	763
style.py	ExtractAuthorRaw	774
style.py	BuildAuthorProfile	789
style.py	_ShapeGrid	536
style.py	BuildLetterPrior	570
style.py	_PriorScore	588

Module	Function or class	Source line
style.py	PriorFilter	596
style.py	_ConsensusFilter	604
style.py	_ResampleN	620
style.py	_AlignStroke	632
style.py	_DenoisedPrototype	647
style.py	SmoothStroke	703
curate.py	ParseSpec	42
curate.py	LoadProfile	55
curate.py	Listing	132
curate.py	Variants	198
curate.py	Drop	219
curate.py	Apply	224
curate.py	Main	303
curate.py	Medoid	153
curate.py	Prototype	173
curate.py	DrawGlyph	67
curate.py	Sheet	95
synthesise.py	_BaselineGlyph	182
synthesise.py	Trajectory	255
synthesise.py	_SwapProb	290
synthesise.py	AutoLegibility	314
synthesise.py	_GapRange	327
synthesise.py	_Advance	346
synthesise.py	_CoreGlyph	357
synthesise.py	_GlyphSource	374
synthesise.py	_Jitter	445
synthesise.py	_Resample	462
synthesise.py	StyleCalibration	484
synthesise.py	_ApplyCal	524
synthesise.py	_Ligature	544
synthesise.py	_BodyStroke	564
synthesise.py	_SplitWords	579
synthesise.py	_WordWidth	587
synthesise.py	SynthesizeText	596
synthesise.py	SynthesizeJointBestOf	813
synthesise.py	_Draw	752
synthesise.py	RenderTrajectory	771
synthesise.py	_LineLogProbs	783
synthesise.py	_WeakChars	788
retrace.py	PageStrokes	49
retrace.py	LinePitchMm	77
retrace.py	PruneSpecks	83
retrace.py	RetracePage	100
retrace.py	Main	141
reproduce.py	GantryConfig	18
reproduce.py	XhForRuling	91
reproduce.py	_TallestLine	120
reproduce.py	ToMachine	151
reproduce.py	_Decimate	198
reproduce.py	OptimizeStrokeOrder	212
reproduce.py	_Clamp	252
reproduce.py	_ClampToUsableArea	819
reproduce.py	AxisDirValue	564
reproduce.py	_RetractOffSwitch	569
reproduce.py	_StepAxis	738
reproduce.py	_CalibrateAxis	756
reproduce.py	Calibrate	775
reproduce.py	NewPenState	269

Module	Function or class	Source line
reproduce.py	SetPenLines	277
reproduce.py	TriggeredEndstop	545
reproduce.py	_AllTriggeredEndstops	560
reproduce.py	PenIsUp	703
reproduce.py	SetPen	710
reproduce.py	BresenhamMove	663
reproduce.py	_GcodeLinearMove	827
reproduce.py	_GcodeArcMove	849
reproduce.py	WriteGcode	294
reproduce.py	GantryNotConnectedError	406
reproduce.py	NotCalibratedError	410
reproduce.py	Connect	501
reproduce.py	Disconnect	529
reproduce.py	_TriggerEmergencyStop	598
reproduce.py	RequestAbort	613
reproduce.py	EmergencyStop	620
reproduce.py	ParseGcodeLine	803
reproduce.py	RunGcodeFile	875
reproduce.py	_RunGcodeLines	900
web/server.py	OpenUpright	125
web/server.py	IngestImage	149
web/server.py	StandUp	388
web/server.py	ReadAllLines	394
web/server.py	NameWriter	439
web/server.py	SplitExpected	468
web/server.py	WordAcc	517
web/server.py	_Clamp	715
web/server.py	BaseAuthor	242
web/server.py	LoadProfiles	247
web/server.py	Remember2	265
web/server.py	Remember	273
web/server.py	CheckApiKey	283
web/server.py	Index	292
web/server.py	Static	297
web/server.py	ApiAuthors	307
web/server.py	ApiGetImage	319
web/server.py	ApiCameraCapture	331
web/server.py	ApiPreviewStart	342
web/server.py	ApiPreviewStop	351
web/server.py	ApiPreviewStream	357
web/server.py	WriteReport	479
web/server.py	ApiReport	504
web/server.py	ApiClassify	535
web/server.py	ReadBackText	656
web/server.py	JobResponse	696
web/server.py	ApiGenerate	727
web/server.py	ApiRetrace	839
web/server.py	IsConnected	889
web/server.py	WriteGcodeFile	896
web/server.py	ParseGcode	910
web/server.py	ApiGcodeUpload	946
web/server.py	ApiGetGcode	976
web/server.py	ApiGantryStatus	992
web/server.py	ApiGantryCalibrate	1004
web/server.py	ApiGantryStop	1025
web/server.py	ApiWriteGcode	1044
web/server.py	ApiModelInfo	1096
web/server.py	ApiTimings	1167

Module	Function or class	Source line
web/server.py	ApiSamples	1192
web/server.py	ApiSampleFile	1205
web/server.py	ApiGlyphs	1214
web/server.py	ApiCurateGet	1233
web/server.py	ApiCuratePost	1247
web/server.py	ApiCurateDelete	1272
web/lineworkers.py	Configure	30
web/lineworkers.py	CanPool	37
web/lineworkers.py	_Workers	43
web/lineworkers.py	_CapThreads	58
web/lineworkers.py	_Init	91
web/lineworkers.py	ReadOne	96
web/lineworkers.py	NameOne	108
web/lineworkers.py	Map	124
web/render.py	_font	31
web/render.py	_RenderOneGlyph	44
web/render.py	GlyphGridPath	72
web/render.py	SampleCrops	105
train.py	IdctMatrix	536
train.py	UnfilterPng	550
train.py	DecodePng	592
train.py	BuildHuffTable	632
train.py	JpegBitReader	659
train.py	DecodeHuffValue	715
train.py	Extend	729
train.py	DecodeJpeg	740
train.py	ReadImageFile	865
train.py	EncodeCrop	878
train.py	DecodeCrop	887
train.py	ReadLabelLines	894
train.py	ClusterRows	902
train.py	FindRuleLineClusters	910
train.py	RemoveStraightFormLines	919
train.py	DetectPageBounds	930
train.py	MergeUndersizedFragments	942
train.py	MergeOrSplitToExpectedLineCount	975
train.py	EstimateLinePitch	1010
train.py	SplitLinesByPeriodicity	1023
train.py	ExtractLinePatches	1051
train.py	EncodeText	1169
train.py	FindPeaks	1177
train.py	LineImageCache	1294
train.py	IAMLineDatasetRaw	1413
train.py	TrainEvalView	1518
train.py	TextCollateFn	1536
train.py	AuthorLabeledView	1547
train.py	AuthorCollateFn	1571
train.py	AddPersonalSamples	1580
train.py	TekliaLineDataset	1646
train.py	IterateBatches	1808
train.py	TextNumpyBatch	1905
train.py	ValidateTextModel	1918
train.py	BuildPagesSplit	1939
train.py	BuildTekliaSplit	1949
train.py	PersonalLineView	1955
train.py	CollectPersonalLines	1971
train.py	BuildJointSplit	1983
train.py	IterateMixedBatches	1995

Module	Function or class	Source line
train.py	TrainTextModel	2015
train.py	ValidateAuthorModel	2162
train.py	TrainAuthorModel	2179
train.py	Main	2280
train.py	MyAffine	1207
train.py	AugmentShear	1220
train.py	AugmentRotate	1226
train.py	FindPerspectiveCoeffs	1231
train.py	AugmentPerspective	1241
train.py	PerspectiveJitterPoints	1250
train.py	AugmentElastic	1275
train.py	AugmentLineImage	1282
train.py	InitConv	1689
train.py	InitBatchNorm	1697
train.py	InitLinear	1702
train.py	InitLstmDirection	1709
train.py	InitConvBlock	1718
train.py	InitTextModelParams	1819
train.py	BuildLstmLayerParams	1843
train.py	InitAuthorModelParams	2098
train.py	Conv2dForward	39
train.py	ConvBlockForwardTrain	217
train.py	InitBackboneParams	1733
train.py	BackboneForwardTrain	1777
train.py	EvalBackboneForward	1798
train.py	TextModelForward	1854
train.py	EvalTextForward	1895
train.py	AuthorModelForward	2114
train.py	EvalAuthorForward	2152
train.py	BatchNorm2dForwardTrain	90
train.py	ReluForward	123
train.py	LinearForward	184
train.py	MaxPool2dForward	133
train.py	AdaptiveMaxPoolHeightForward	160
train.py	AdaptiveAvgPoolWidthForward	174
train.py	DropoutForward	203
train.py	DropoutBackward	210
train.py	LstmForwardTrain	246
train.py	BidirectionalLstmForwardTrain	315
train.py	MultilayerBidirectionalLstmForwardTrain	341
train.py	LogSumExp	360
train.py	CtcLoss	373
train.py	CrossEntropyLoss	434
train.py	Conv2dBackward	65
train.py	BatchNorm2dBackward	110
train.py	ReluBackward	128
train.py	MaxPool2dBackward	148
train.py	AdaptiveMaxPoolHeightBackward	167
train.py	AdaptiveAvgPoolWidthBackward	179
train.py	LinearBackward	191
train.py	ConvBlockBackwardTrain	231
train.py	LstmBackward	274
train.py	BidirectionalLstmBackward	323
train.py	MultilayerBidirectionalLstmBackward	350
train.py	BackboneBackwardTrain	1787
train.py	TextModelBackward	1869
train.py	AuthorModelBackward	2131
train.py	RMSprop	446

Module	Function or class	Source line
train.py	AdamW	467
train.py	ReduceLrOnPlateau	489
train.py	CosineAnnealingLr	509
train.py	ClipGradNorm	521
train.py	TrainableParams	1739
train.py	SaveCheckpoint	1743
train.py	LoadCheckpointMeta	1751
train.py	LoadParamsInto	1761
train.py	AppendLog	1771
timing.py	Stage	47
timing.py	Run	66

Evidence | Unresolved sources and comparison limits

The exact motor torque-curve points, the assumed friction coefficient, installed sense resistance/reference voltage and GPIO current-limit provenance remain conditional. Published algorithms do not supply optima for the project seam, glyph-cut, drift or joining constants. Historical sweeps are identified as such; this edition does not absorb another agent's unreviewed new experiments. Some publisher/manufacturer pages blocked automated full-text access; those limitations are retained in bibliography notes rather than described as a completed reference certification. The reader should use the exact component edition and operating conditions before adopting a hardware numerical claim.

Bibliography

[1] Python Software Foundation, "multiprocessing process based parallelism." https://docs.python.org/3/library/multiprocessing.html. Explicit start contexts, pools, ordering and process semantics.

[2] NumPy contributors, "matmul." https://numpy.org/doc/stable/reference/generated/numpy.matmul.html. Matrix multiplication and BLAS.

[3] D. R. Braune, "Democode source snapshot captured for the november 2026 documentation revision," 2026. Local Software/DemoCode including core, training, web server, lineworkers, camera, curation, retracing and timing. Exact source hashes and static coverage manifest accompany this edition. No inference or hardware execution.

[4] NumPy contributors, "savez." https://numpy.org/doc/stable/reference/generated/numpy.savez.html.

[5] OpenCV contributors, "Videocapture class reference." https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html.

[6] Pillow contributors, "Imageops." https://pillow.readthedocs.io/en/stable/reference/ImageOps.html. EXIF transpose.

[7] J. Sauvola and M. Pietikäinen, "Adaptive document image binarization." https://doi.org/10.1016/S0031-3203(99)00055-2, 2000. Alternative local thresholding method.

[8] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[9] F. C. Crow, "Summed-area tables for texture mapping," *ACM SIGGRAPH Computer Graphics*, vol. 18, no. 3, pp. 207–212, 1984.

[10] D. R. Braune, "23539195 braunedr finalreport," 2026. User-supplied Prism PDF, unsuffixed Downloads copy, creation date 3 November 2026, 108 PDF pages. Local project evidence; not independently repeated.

[11] W. Postl, "Detection of linear oblique structures and skew scan in digitized documents," in *Proc. 8th International Conference on Pattern Recognition*, pp. 687–689, 1986.

[12] R. Bellman, "The theory of dynamic programming," *Bulletin of the American Mathematical Society*, vol. 60, no. 6, pp. 503–515, 1954.

[13] N. Arvanitopoulos and S. Süsstrunk, "Seam carving for text line extraction on color and grayscale historical manuscripts." https://doi.org/10.1109/ICFHR.2014.127, 2014. Method family, not validation of the project's coefficients.

[14] L. He, Y. Chao, and K. Suzuki, "A run-based two-scan labeling algorithm," *IEEE Transactions on Image Processing*, vol. 17, no. 5, pp. 749–756, 2008.

[15] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.

[16] SciPy contributors, "ndimage uniform filter." https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.uniform_filter.html. Window, dtype and boundary conventions.

[17] OpenCV contributors, "Miscellaneous image transformations." https://docs.opencv.org/4.x/d7/d1b/group___imgproc___misc.html. Integral images and threshold operations.

[18] SciPy contributors, “ndimage binary closing.” https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.binary_closing.html. Footprint and border conventions.

[19] R. M. Haralick, S. R. Sternberg, and X. Zhuang, “Image analysis using mathematical morphology,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 9, no. 4, pp. 532–550, 1987. https://www.haralick.org/journals/04767941.pdf.

[20] OpenCV contributors, “Structural analysis and shape descriptors.” https://docs.opencv.org/4.x/d3/dc0/group___imgproc___shape.html. Connected components and approxPolyDP.

[21] SciPy contributors, “ndimage label.” https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.label.html.

[22] P. Y. Simard, D. Steinkraus, and J. C. Platt, “Best practices for convolutional neural networks applied to visual document analysis,” in *Proc. 7th International Conference on Document Analysis and Recognition*, pp. 958–963, 2003.

[23] SciPy contributors, “ndimage gaussian filter.” https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.gaussian_filter.html.

[24] OpenCV contributors, “Geometric image transformations.” https://docs.opencv.org/4.x/da/d54/group___imgproc___transform.html.

[25] G. Wolberg, *Digital Image Warping*. IEEE Computer Society Press, 1990.

[26] Pillow contributors, “Image module.” https://pillow.readthedocs.io/en/stable/reference/Image.html.

[27] SciPy contributors, “signal find peaks.” https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.find_peaks.html.

[28] A. Vinciarelli and J. Luettin, “A new normalization technique for cursive handwritten words,” *Pattern Recognition Letters*, vol. 22, no. 9, pp. 1043–1050, 2001.

[29] N. Arvanitopoulos and S. Süsstrunk, “Seam carving for text line extraction on color and grayscale historical manuscripts,” in *Proc. 14th International Conference on Frontiers in Handwriting Recognition*, pp. 726–731, 2014.

[30] S. Avidan and A. Shamir, “Seam carving for content-aware image resizing,” *ACM Transactions on Graphics*, vol. 26, no. 3, 2007.

[31] T. Y. Zhang and C. Y. Suen, “A fast parallel algorithm for thinning digital patterns,” *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984.

[32] scikit-image contributors, “Morphology and skeletonisation.” https://scikit-image.org/docs/stable/api/skimage.morphology.html.

[33] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,” *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[34] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” in *Proc. 23rd International Conference on Machine Learning*, pp. 369–376, 2006.

[35] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[36] PyTorch contributors, “Conv2d api documentation.” https://docs.pytorch.org/docs/2.15/generated/torch.nn.Conv2d.html. Cross-correlation and output shape conventions.

[37] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd International Conference on Machine Learning*, pp. 448–456, 2015.

[38] PyTorch contributors, “Batchnorm2d api documentation.” https://docs.pytorch.org/docs/2.15/generated/torch.nn.BatchNorm2d.html. Running variance convention and momentum.

[39] X. Glorot, A. Bordes, and Y. Bengio, “Deep sparse rectifier neural networks.” https://proceedings.mlr.press/v15/glorot11a.html, 2011.

[40] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[41] PyTorch contributors, “Lstm api documentation.” https://docs.pytorch.org/docs/2.15/generated/torch.nn.LSTM.html. Gate ordering, shapes and bidirectionality.

[42] A. Hannun, “Sequence modeling with ctc.” https://distill.pub/2017/ctc/, 2017. Author’s mathematical exposition and visual explanation.

[43] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[44] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. http://yann.lecun.com/exdb/publis/pdf/lecun-98.pdf.

[45] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition.” https://arxiv.org/abs/1512.03385, 2016.

[46] NumPy contributors, “as strided.” https://numpy.org/doc/stable/reference/generated/numpy.lib.stride_tricks.as_strided.html. Overlapping views and memory layout.

[47] K. Chellapilla, S. Puri, and P. Simard, “High performance convolutional neural networks for document processing,” in *Proc. 10th International Workshop on Frontiers in Handwriting Recognition*, 2006.

[48] A. Graves, “Supervised sequence labelling with recurrent neural networks.” https://www.cs.toronto.edu/~graves/preprint.pdf, 2012. Recurrent derivatives and sequence-labelling background.

[49] PyTorch contributors, “Linear layer.” https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html. Affine operation and initialisation convention.

[50] PyTorch contributors, “Maxpool2d.” https://docs.pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html.

[51] SciPy contributors, “special logsumexp.” https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.logsumexp.html.

[52] PyTorch contributors, “Numerical accuracy.” https://docs.pytorch.org/docs/2.15/notes/numerical_accuracy.html. Floating-point comparisons; accessed 10 November 2026.

[53] PyTorch contributors, “Ctcloss api documentation.” https://docs.pytorch.org/docs/2.15/generated/torch.nn.CTCLoss.html. Reduction and tensor conventions; accessed 10 November 2026.

[54] A. J. Viterbi, “Error bounds for convolutional codes and an asymptotically optimum decoding algorithm,” *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[55] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[56] Tanks and Temples benchmark, “Evaluation tutorial, appendix b.” https://tanksandtemples.org/tutorial/. Precision/recall and distance-threshold F-score; dimensional setting differs from this project’s 2D handwriting.

[57] NIST/SEMATECH, “e-handbook of statistical methods: Confidence limits for a binomial proportion.” https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm. Score interval and limitations.

[58] T. Y. Zhang and C. Y. Suen, “A fast parallel algorithm for thinning digital patterns.” https://doi.org/10.1145/357994.358023, 1984. Publisher full text inaccessible during this edition’s audit; original citation retained.

[59] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature.” https://doi.org/10.3138/FM57-6770-U75U-7727, 1973. Publisher full text inaccessible during this edition’s audit.

[60] M. Bulacu and L. Schomaker, “Text independent writer identification and verification using textural and allographic features,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 4, pp. 701–717, 2007. https://doi.org/10.1109/TPAMI.2007.1009.

[61] A. Brink, J. Smit, M. Bulacu, and L. Schomaker, “Writer identification using directional ink trace width measurements,” *Pattern Recognition*, vol. 45, no. 1, pp. 162–171, 2012. https://www.ai.rug.nl/~mbulacu/brink-quill-pr2012.pdf.

[62] T. Haines, O. Mac Aodha, and G. Brostow, “My text in your handwriting,” *ACM Transactions on Graphics*, vol. 35, no. 3, pp. 26:1–26:18, 2016. https://reality.cs.ucl.ac.uk/projects/haines/haines16handwriting.pdf. Original full PDF exceeded automated-reader size limit; report citation and public project reference retained.

[63] Pillow contributors, “Imagedraw.” https://pillow.readthedocs.io/en/stable/reference/ImageDraw.html.

[64] OpenCV contributors, “Drawing functions.” https://docs.opencv.org/4.x/d6/d6e/group__imgproc___draw.html.

[65] A. Graves, “Generating sequences with recurrent neural networks.” https://arxiv.org/abs/1308.0850, 2013.

[66] L. Kang *et al.*, “Ganwriting content conditioned generation of styled handwritten word images.” https://arxiv.org/abs/2003.02567, 2020.

[67] G. Dai *et al.*, “One dm one shot diffusion mimicker for handwritten text generation.” https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/07598.pdf, 2024.

[68] S. J. Ling, J. Sanny, and W. Moebs, “University physics volume 1: Torque.” https://openstax.org/books/university-physics-volume-1/pages/10-6-torque, 2016.

[69] Allegro MicroSystems, “A4988: Dmos microstepping driver with translator and overcurrent protection.” https://www.allegromicro.com/-/media/files/datasheets/a4988-datasheet.pdf, 2022. 4988-DS Rev. 8, especially timing table, step modes and current control.

[70] Pololu, “A4988 stepper motor driver carrier, black edition.” https://www.pololu.com/product/2128. Current-limiting explanation; accessed 10 November 2026.

[71] ROHM Semiconductor, “Npn transistor: Low-side switch fundamentals.” https://techweb.rohm.com/product/transistors-diodes/transistors/27354/, 2026. Accessed 10 November 2026.

[72] onsemi, "Bc337, bc337-16, bc337-25, bc337-40: Amplifier transistors." https://www.onsemi.com/pdf/datasheet/bc337-d.pdf. Manufacturer datasheet; automated full-text access blocked during this edition's source audit.

[73] Linux kernel contributors, "Gpio character device userspace api." https://www.kernel.org/doc/html/latest/userspace-api/gpio/chardev.html. v2 API; accessed 10 November 2026.

[74] Hardkernel, "Odroid-n2 expansion connectors." https://wiki.odroid.com/odroid-n2/hardware/expansion_connectors. Manufacturer pin reference; automated access blocked. Installed mappings in this manual are checked against local source and final-report Table 17, not electrically verified.

[75] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.

[76] Python Software Foundation, "time perf counter." https://docs.python.org/3/library/time.html#time.perf_counter.

[77] LinuxCNC contributors, "G code overview." https://linuxcnc.org/docs/html/gcode/overview.html. Reference dialect, not proof of this project's command support.

[78] Pallets, "Flask quickstart." https://flask.palletsprojects.com/en/stable/quickstart/.

[79] G. Hinton, "Neural networks for machine learning: Lecture 6e, rmsprop." https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf, 2012.

[80] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd International Conference on Learning Representations*, 2015.

[81] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. 7th International Conference on Learning Representations*, 2019.

[82] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. 30th International Conference on Machine Learning*, pp. 1310–1318, 2013.

[83] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proc. IEEE International Conference on Computer Vision*, pp. 1026–1034, 2015.

[84] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th International Conference on Artificial Intelligence and Statistics*, pp. 249–256, 2010.

[85] W3C, "Portable network graphics specification." https://www.w3.org/TR/png/. Filter reconstruction and datastream semantics.

[86] ITU T, "Recommendation t 81 digital compression and coding of continuous tone still images." https://www.w3.org/Graphics/JPEG/itu-t81.pdf, 1992.

[87] U.-V. Marti and H. Bunke, "The iam-database: An english sentence database for offline handwriting recognition," *International Journal on Document Analysis and Recognition*, vol. 5, pp. 39–46, 2002.

[88] PyTorch contributors, "Data loading utilities." https://docs.pytorch.org/docs/stable/data.html.

[89] T. Tieleman and G. Hinton, "Lecture 6.5 — rmsprop: Divide the gradient by a running average of its recent magnitude," 2012. COURSERA: Neural Networks for Machine Learning.

[90] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927. Original article; https://www.jstor.org/stable/2276774.